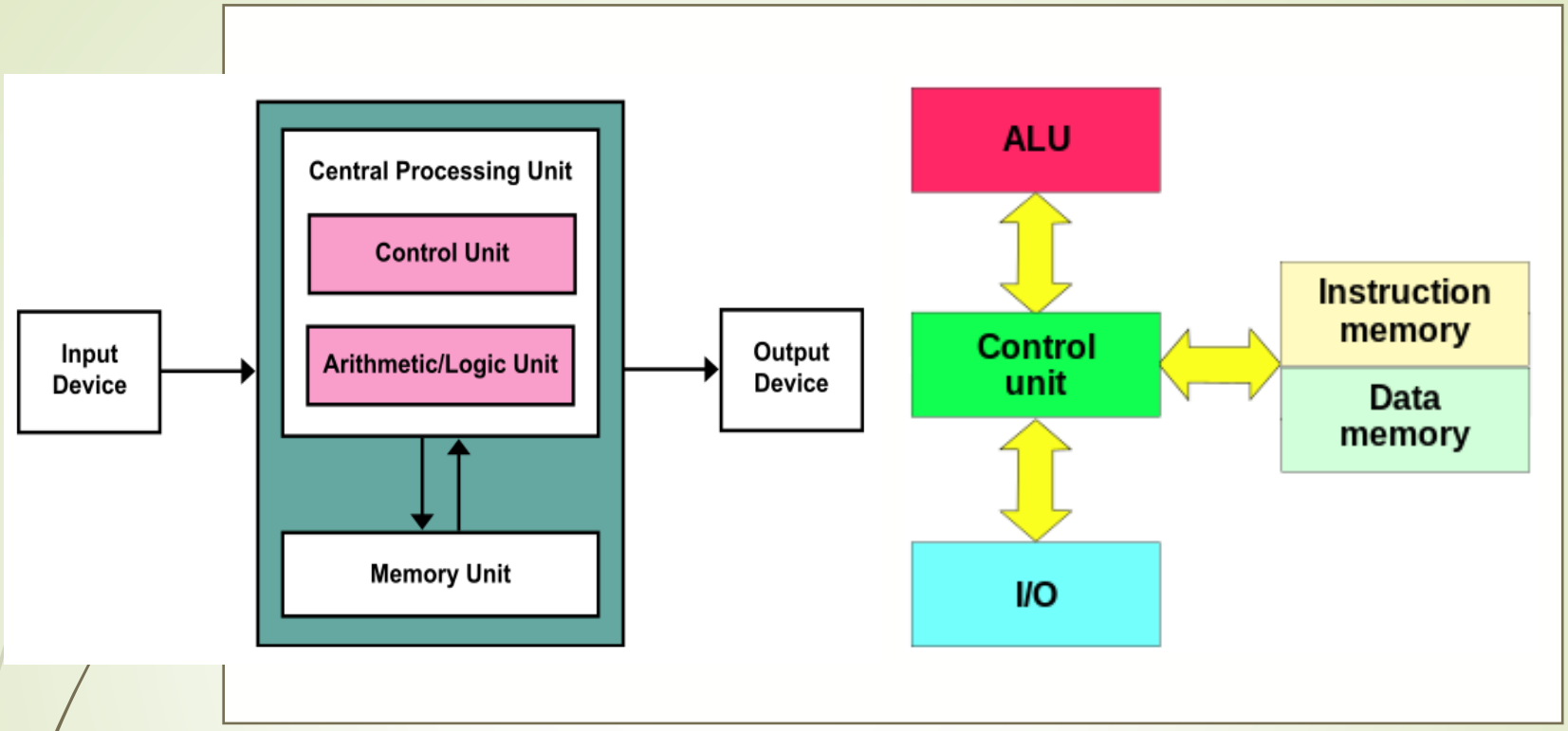
Arhitecturi si Prelucrari Paralele APP

Nicolae Tapus

Arhitectura Von Neumann

- Arhitectura a fost proiectată de matematicianul și fizicianul renumit John Von Neumann în 1945.
- Arhitectura Von Neumann este un model teoretic al calculatorului bazat pe conceptul de program stocat unde programele și datele sunt stocate în aceeași memorie.
- Conceptul Von Neumann a dominat arhitecturile calculatoarelor.
- Acest concept a stat la baza sistemelor secventiale, monoprocesor.
- Reamintim ca o structura Von Neumann este caracterizata de:
  - un singur element de procesare CPU care contine: unitatea de aritmetică și logică (ALU), unitatea de comandă (CU) și registrele generale;
  - memoria este formata din locatii de dimensiune fixa cu organizare liniara si adresabila pe un singur nivel;
  - memoria principală utilizată pentru a stoca datele și instrucțiunile programului
  - reprezentarea interna a datelor si instructiunilor se face sub aceeasi forma (binara).
  - executa operatiile elementare in mod secvential;
  - limbajul masinii este in general de nivel scazut avand instructiuni ce controleaza operatii simple si actioneaza asupra unor operanzi elementari;
  - capabilitatile de intrare iesire sunt reduse;



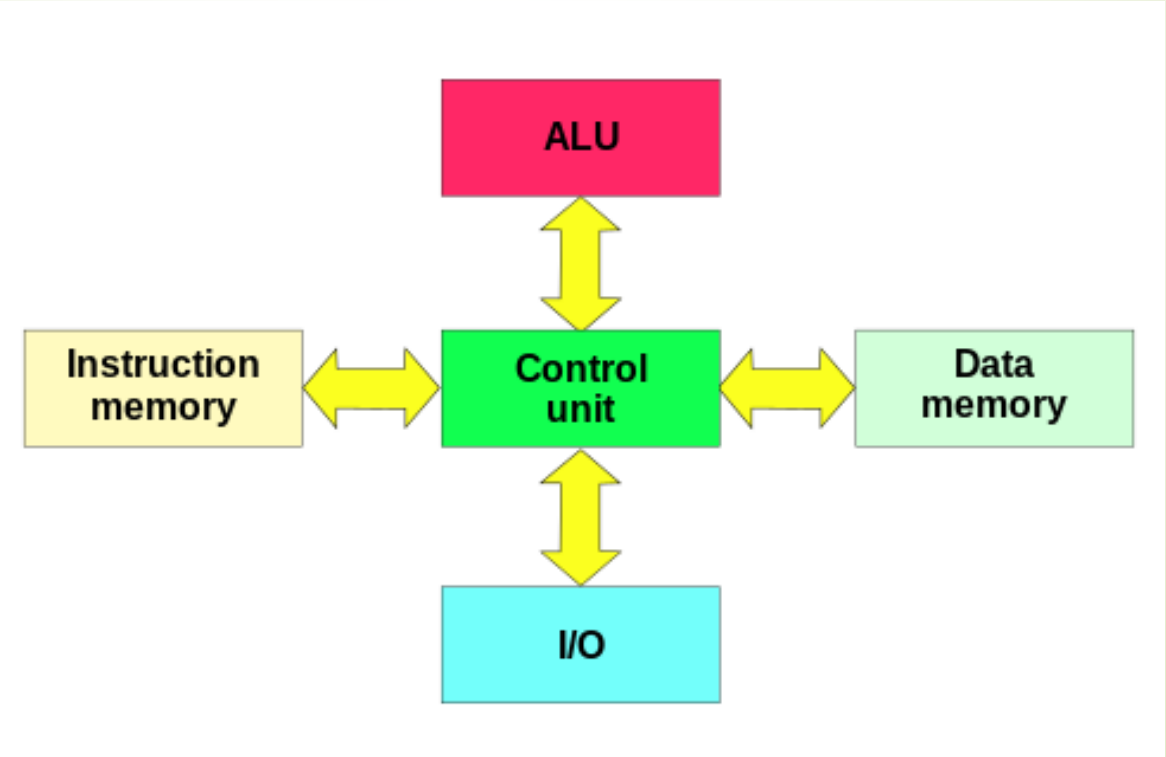
Arhitectura Von Newmann

Arhitectura Harvard

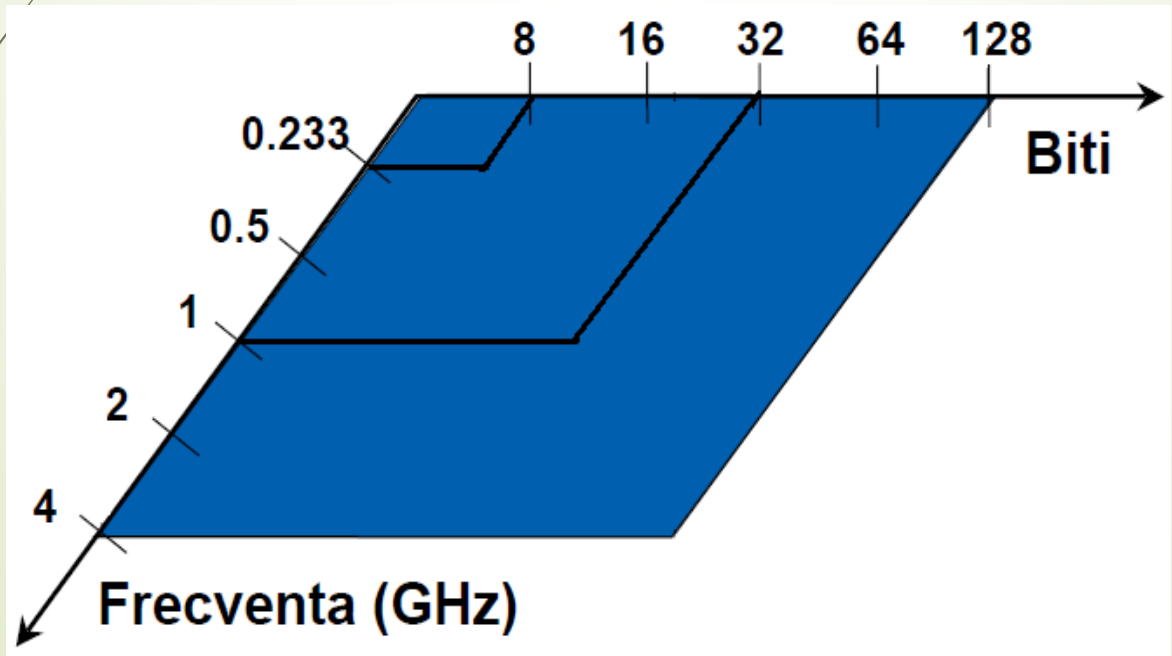
- Este o arhitectură a calculatoarelor cu stocare separată fizic și căi de semnal pentru datele și instrucțiunile de program.
- Spre deosebire de arhitectura Von Neumann care utilizează o sngura magistrala atât pentru preluarea instrucțiunilor din memorie, cât și pentru transferul datelor, arhitectura Harvard are un spațiu de memorie separat pentru date și instrucțiuni.

Arhitectura Harvard

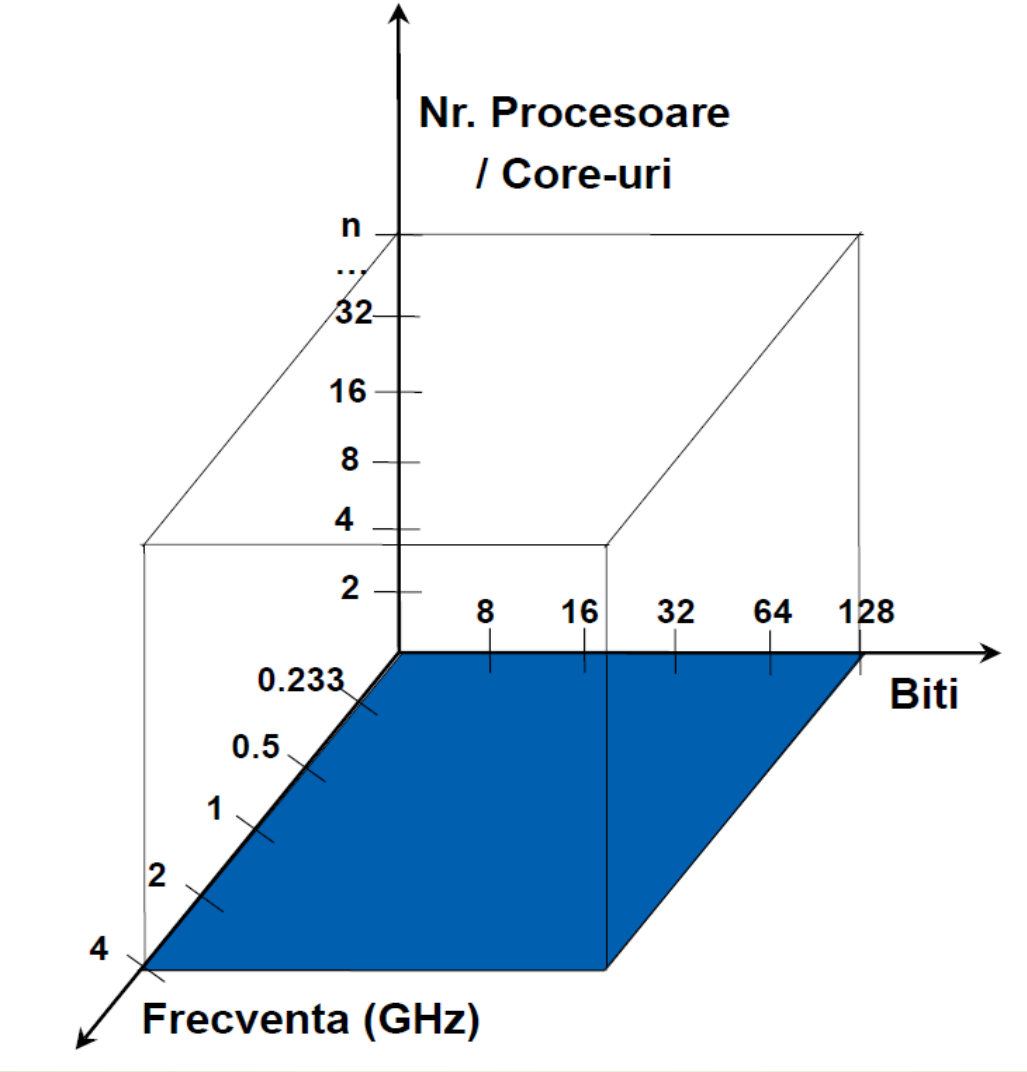
- Arhitectura modificată Harvard este utilizata în mod obișnuit în microcontrolere și DSP (Digital Signal Processing).
- În arhitectura Harvard, unitatea de procesare poate finaliza o instrucțiune într-un singur ciclu dacă există strategii de implementare adecvate.
- https://ro.sawakinome.com/articles/technology/unassigned-2474.html



Evaluare primara a puterii de calcul



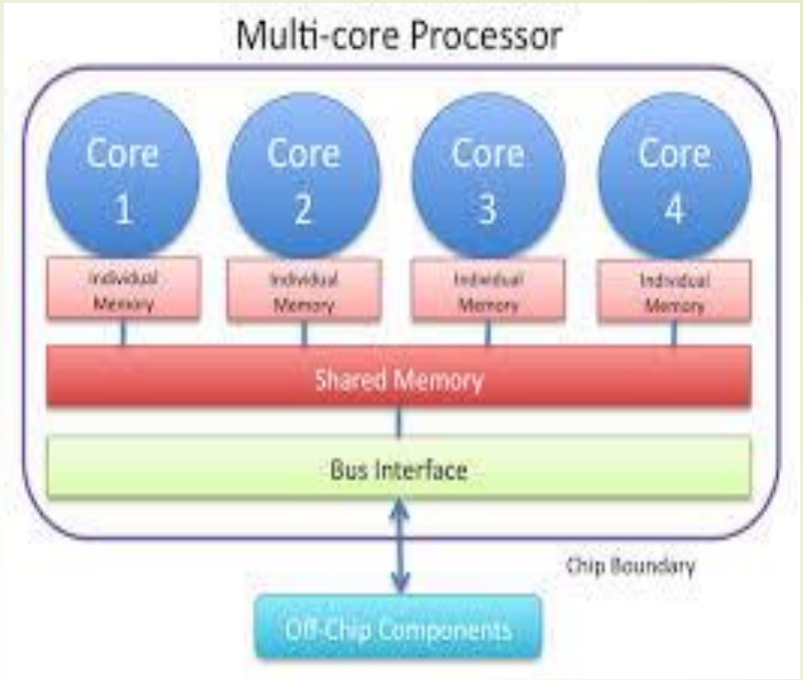
Evaluare putere de calculprimara system multiprocessor

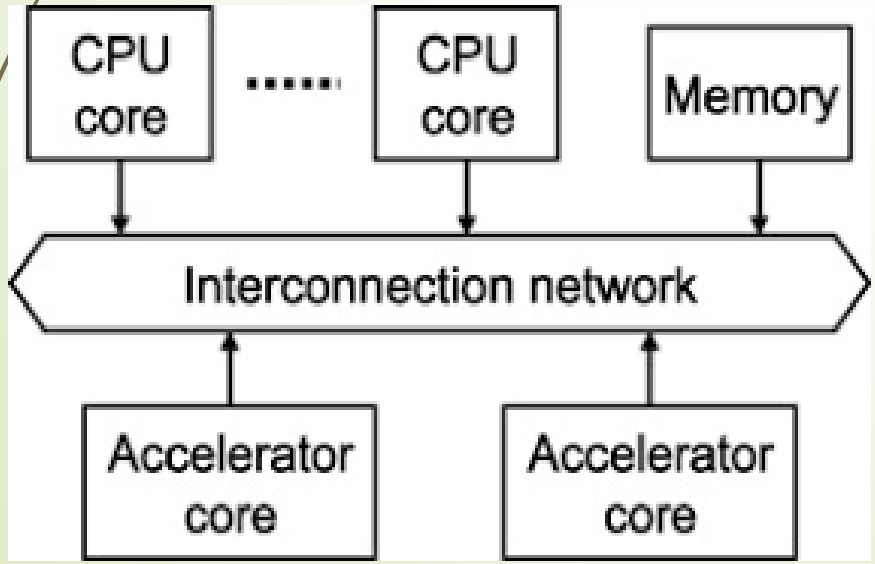


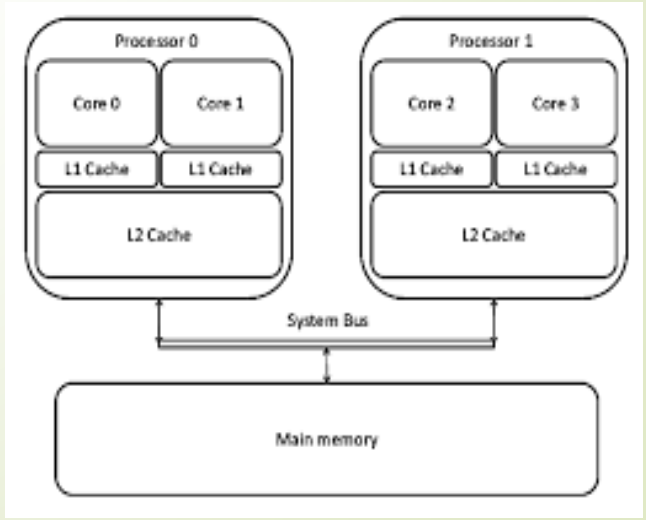

ZILOG
Z80
CPU
DALLAS
7623
1


intel
Core™ i7

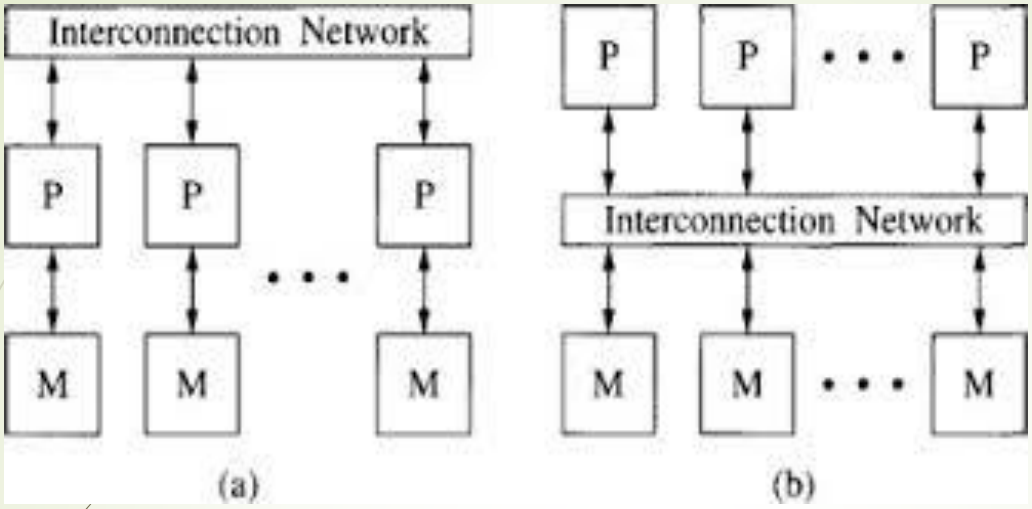
Structuri multicore

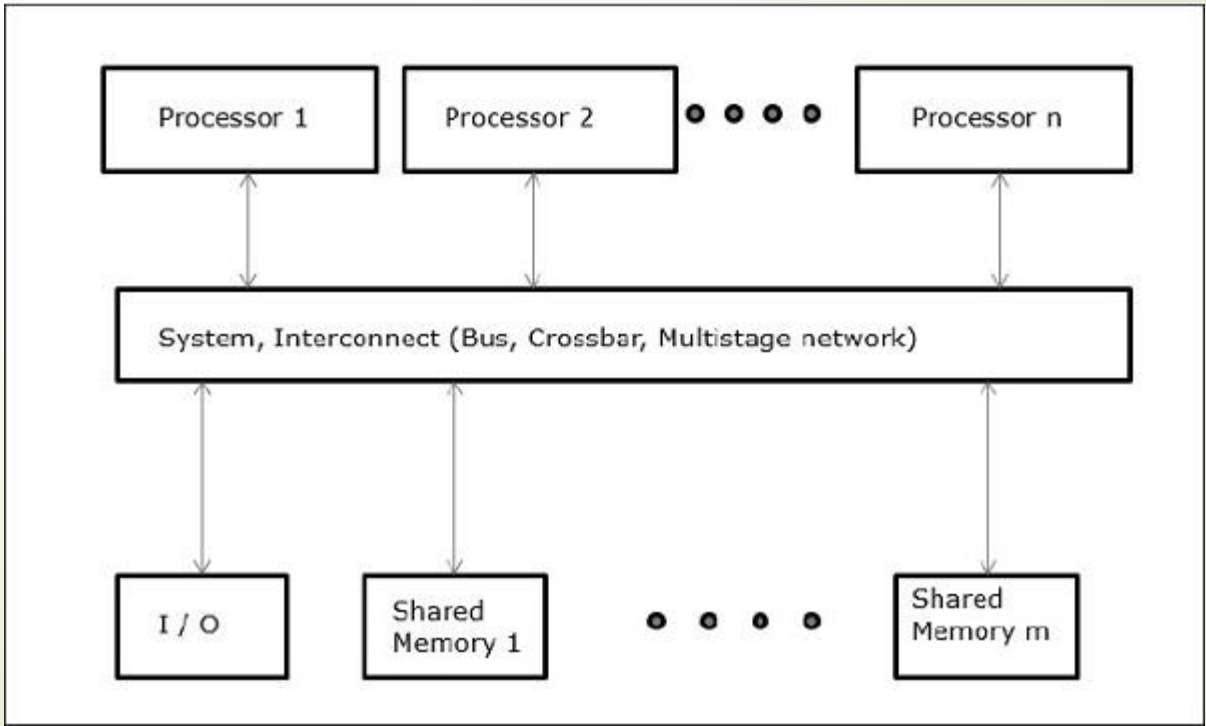




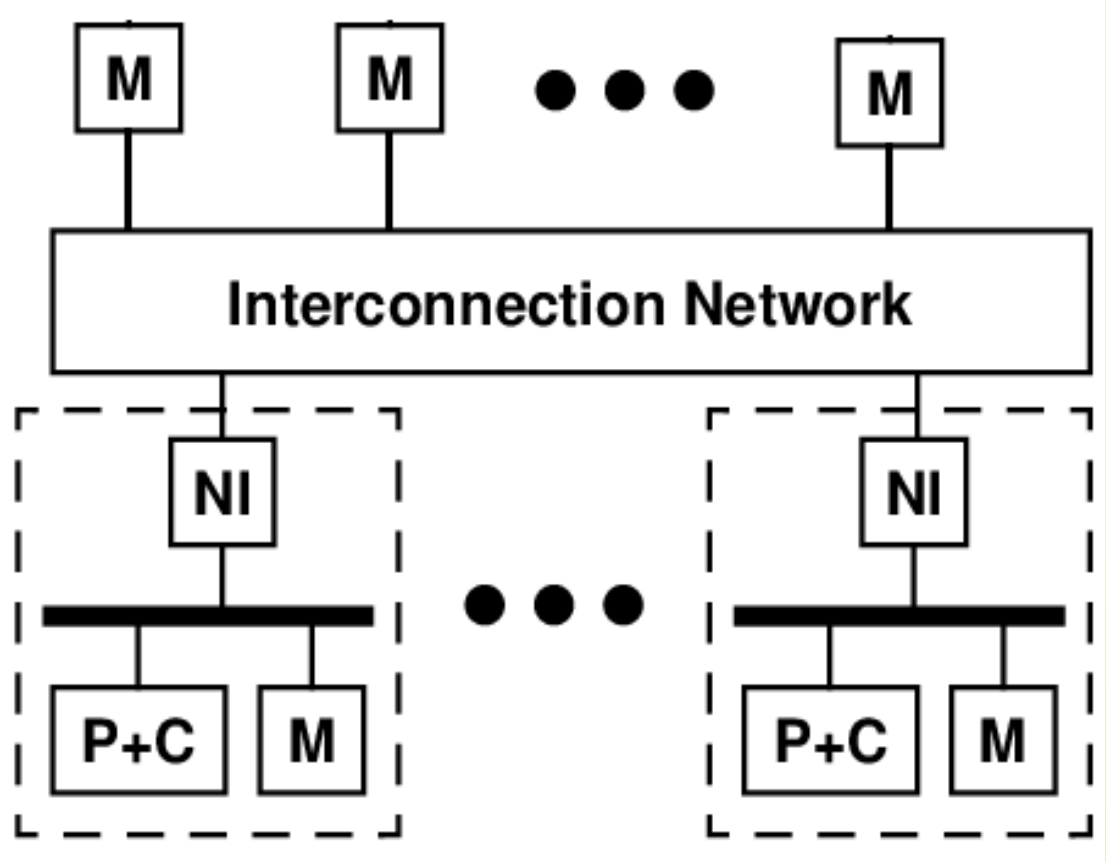


Sisteme UMA

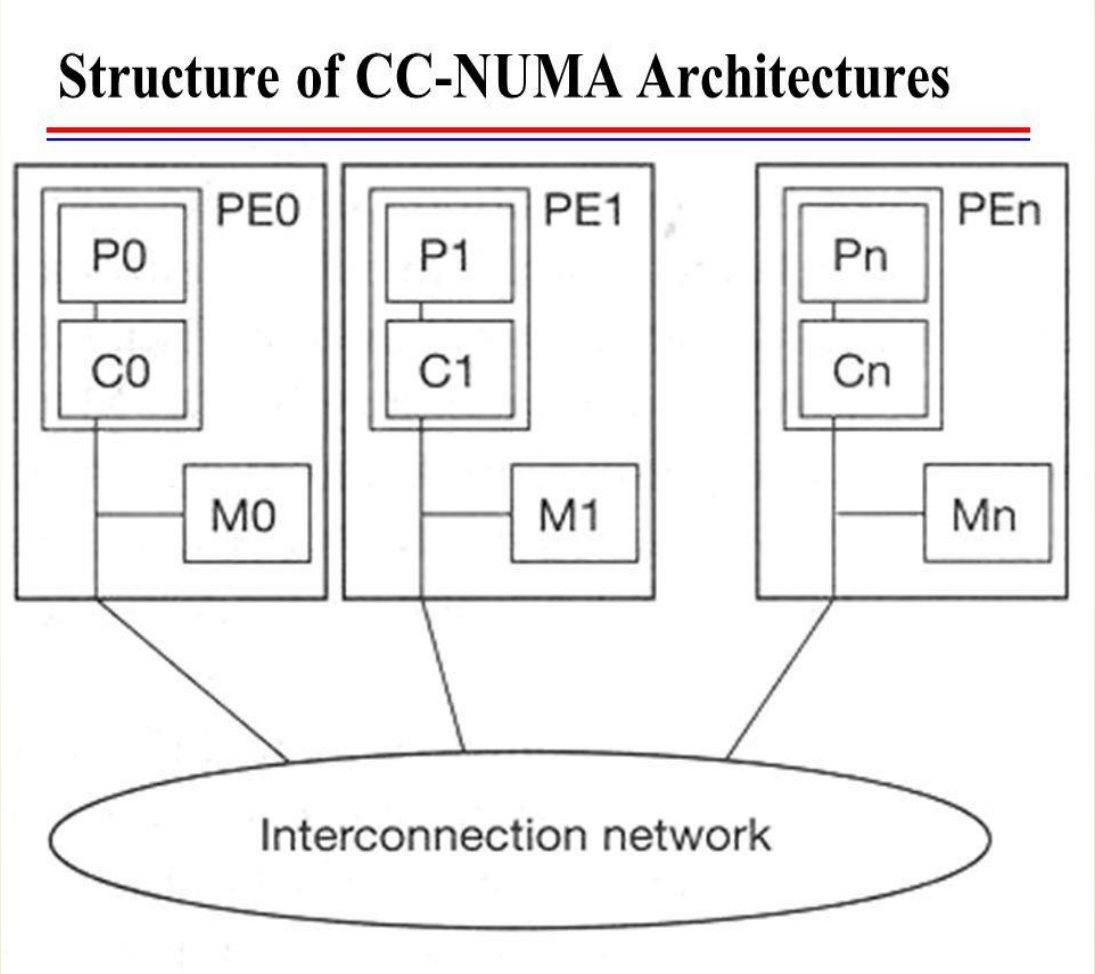

Interconnection Network
P
P
P
M
M
M
(a)
P
P
P
Interconnection Network
M
M
M
(b)


Processor 1
Processor 2
Processor n
System, Interconnect (Bus, Crossbar, Multistage network)
I / O
Shared Memory 1
Shared Memory m

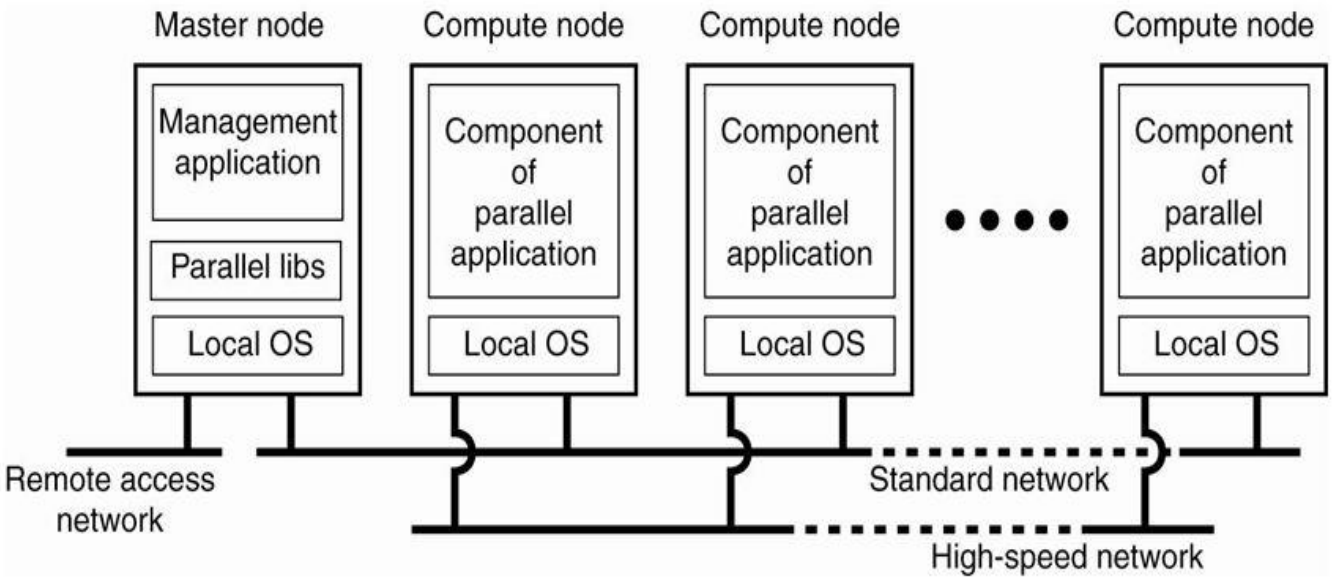
Sisteme NUMA



CC NUMA


Structure of CC-NUMA Architectures
PE0
P0
C0
M0
PE1
P1
C1
M1
PEn
Pn
Cn
Mn
Interconnection network

Cluster Computing Systems



An example of a cluster computing system.

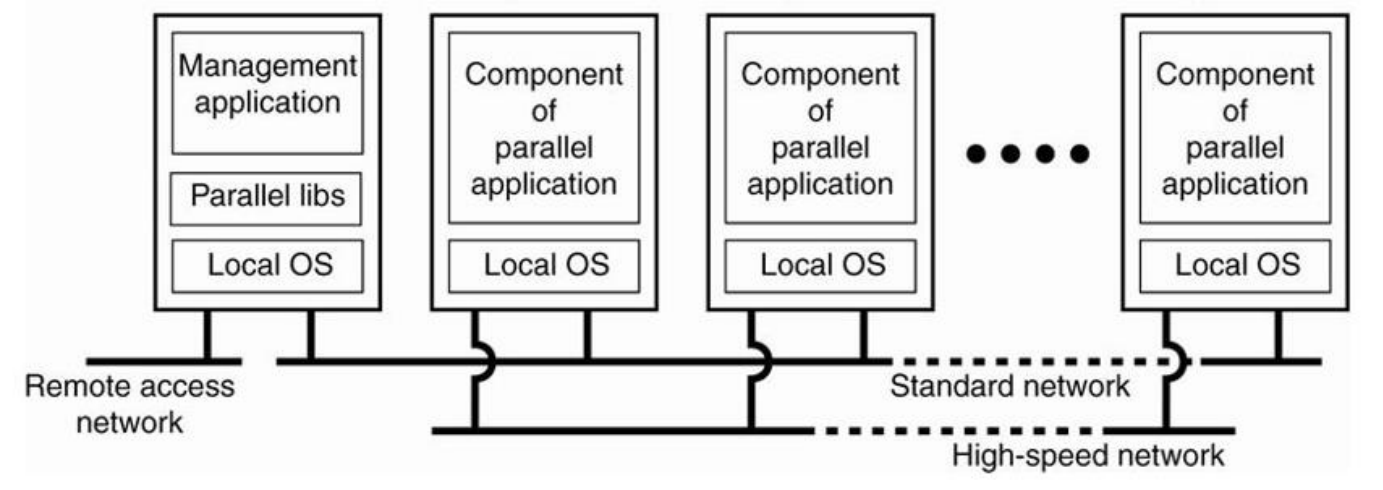
Cluster Computing





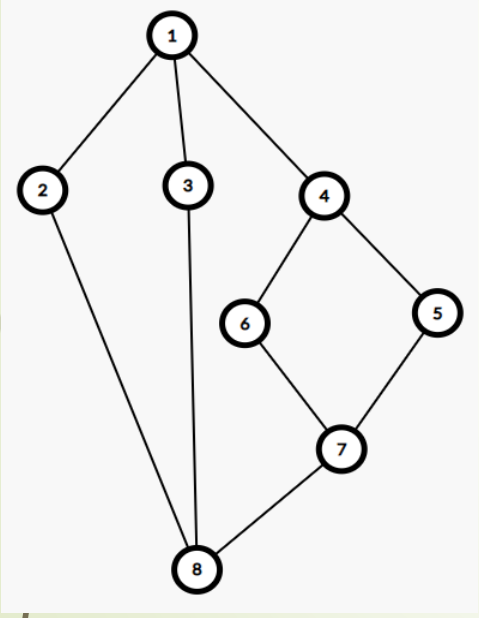
Tunnel Vision by Experts

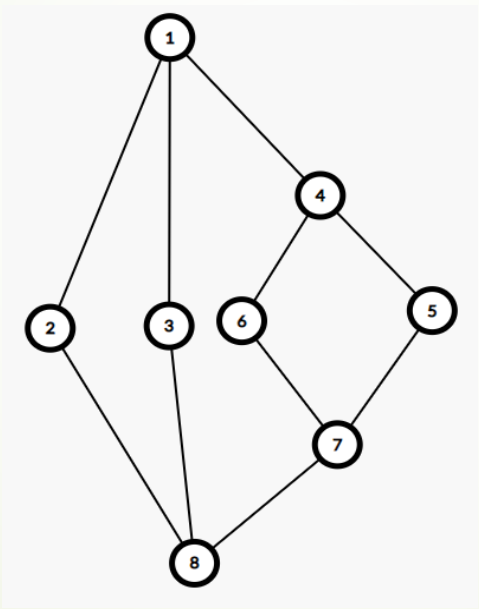
- "I think there is a world market for maybe five computers."
  - Thomas Watson, chairman of IBM, 1943.
- "There is no reason for any individual to have a computer in their home"
  - Ken Olson, president and founder of digital equipment corporation, 1977.
- "640K [of memory] ought to be enough for anybody."
  - Bill Gates, chairman of Microsoft,1981.

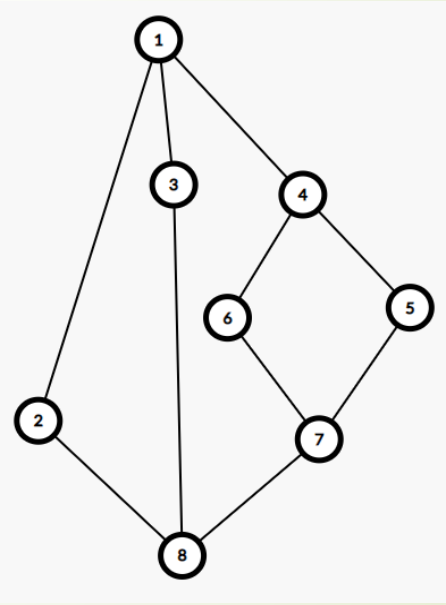

Management application
Parallel libs
Local OS
Component of parallel application
Local OS
Component of parallel application
Local OS
Component of parallel application
Local OS
Remote access network
Standard network
High-speed network

Posibilitati executie sarcini

1 2 3 4 5 6 7 8 Paralel







1
2
3
4
5
6
7
8

Dependenta de date intre Sarcini

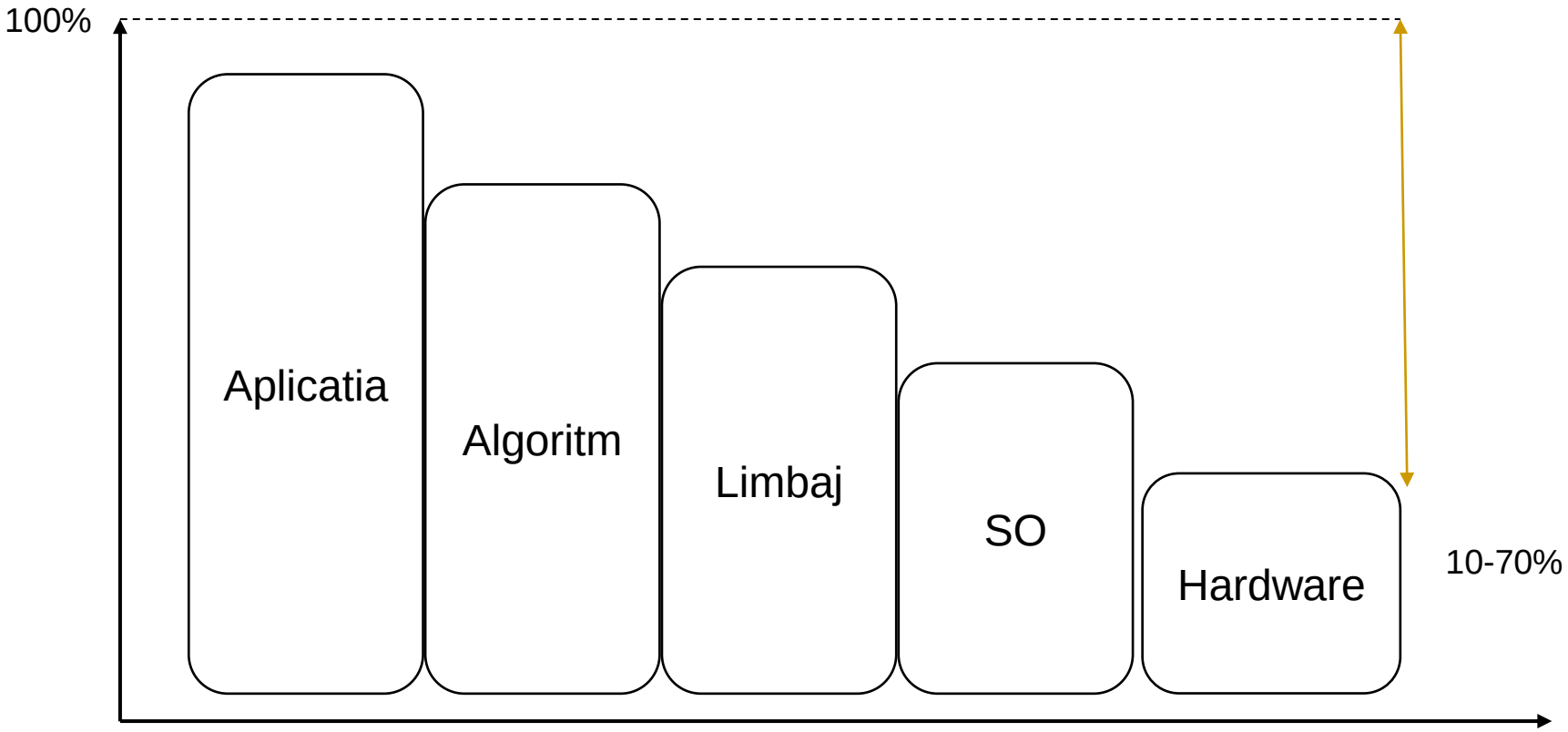
Serial

Problematici asociate calculului paralel

Aspecte ale prelucrarii paralele

- Problemele de baza in calculul parallel se refera la urmatoarele aspecte:
- Stabilirea granularitatii task-urilor
- Elaborarea algoritmilor cu paralelism intrinsec
- Proiectarea limbajelor de programare
- Proiectarea unor arhitecturi hardware adecvate
- Proiectarea unor sisteme de operare adecvate

Grad de paralelism



Nivelurile de paralelism

- Paralelismul poate fi examinat la multe niveluri in functie de complexitatea dorita. Frecvent gasim descrierea paralelismului la nivelurile:

* job $JOB = \{ JOB_1, \ldots, JOB_i, \ldots\ldots, JOB_m \}$
* task $JOB_i = \{ Ti_1, \ldots, Ti_j, \ldots..., Ti_n \}$
* proces $Ti_j = \{Pi_{j1}, \ldots., Pi_{jk}, ..., Pi_{jp}\}$
* thread $Pi_{jk} = \{THi_{jk1} \ldots THi_{jkt}\}$
* variabila
* instructiune
* bit

Nivelurile de paralelism

Nivel de job-uri

Se executa doua sau mai multe programe independente pe resurse de procesare distincte.

- $JOB = \{ JOB_1, \ldots, JOB_i, \ldots\ldots, JOB_m \}$

Nivel de task-uri

- $JOB_i = \{ Ti_1, \ldots, Ti_j, \ldots\ldots, Ti_n \}$

Fiecare Ti se executa pe cate un procesor distinct insa exista o relatie intre Ti si Tj in ceea ce priveste transferul de date sau completarea functiilor de prelucrare realizate in cadrul job-ului.

Exemplu:

Consideram un robot care are mai multe grade de libertate. Pentru fiecare grad de libertate exista un procesor. Programul de conducere a robotului este partitionat in task-uri care se ocupa de cite un grad de libertate al robotului.

Taskurile se executa in paralel, dar interactioneaza pentru miscarea robotului.

Nivel de proces

* $Ti_j = \{Pi_{j1}, ….,Pi_{jk}, ..., Pi_{jp}\}$

Task-urile sunt alcatuite din mai multe procese care in general sunt identificate de utilizator sau de catre compilator daca acesta este destinat pentru structuri multiprocesor.
Sa consideram task-ul

```
for i=1 to n
        x(i)=x(i-2)+y(i-2)
        y(i)=x(i-2) * y(i-1)
        suma(i)=x(i) +y(i)
        if s(i) > a then c=c+1
                    else d=d+1
 end for
```

In cadrul acestui task identificam doua procese:

```
P1(i)
        x(i)=x(i-2) + y(i-2)
        y(i)=x(i-2) * y(i-1)
P2(i)
        suma(i)=x(i) +y(i)
        if s(i) > a then c=c+1
                    else d=d+1
```

Intre P1(i) si P2(i) exista o dependenta de date, deci nu pot fi efectuate in paralel
Insa P1(i) si P2(i-1) pot fi efectuate in paralel

Nivel de proces

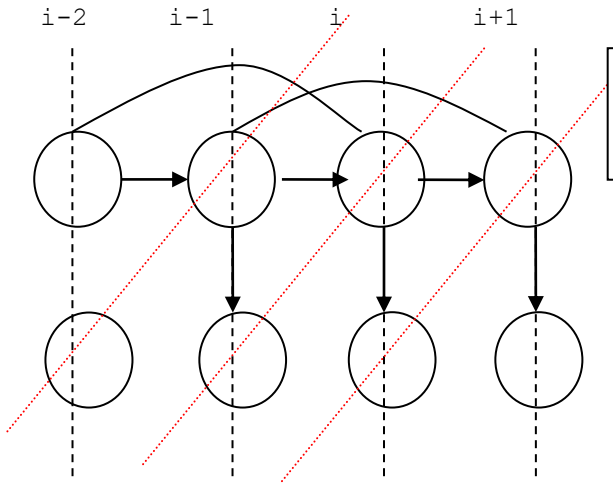
- Interdependenta intre aceste procese este urmatoarea

P1
x(i)=x(i-2)+y(i-2)

y(i)=x(i-2) * y(i-1)

P2
suma(i)=x(i) +y(i)

if sum a(i) > a then c=c+1
else d=d+1



Se pot executa simultan:
P1(i) si P2(i-1)

Nivel de variabila

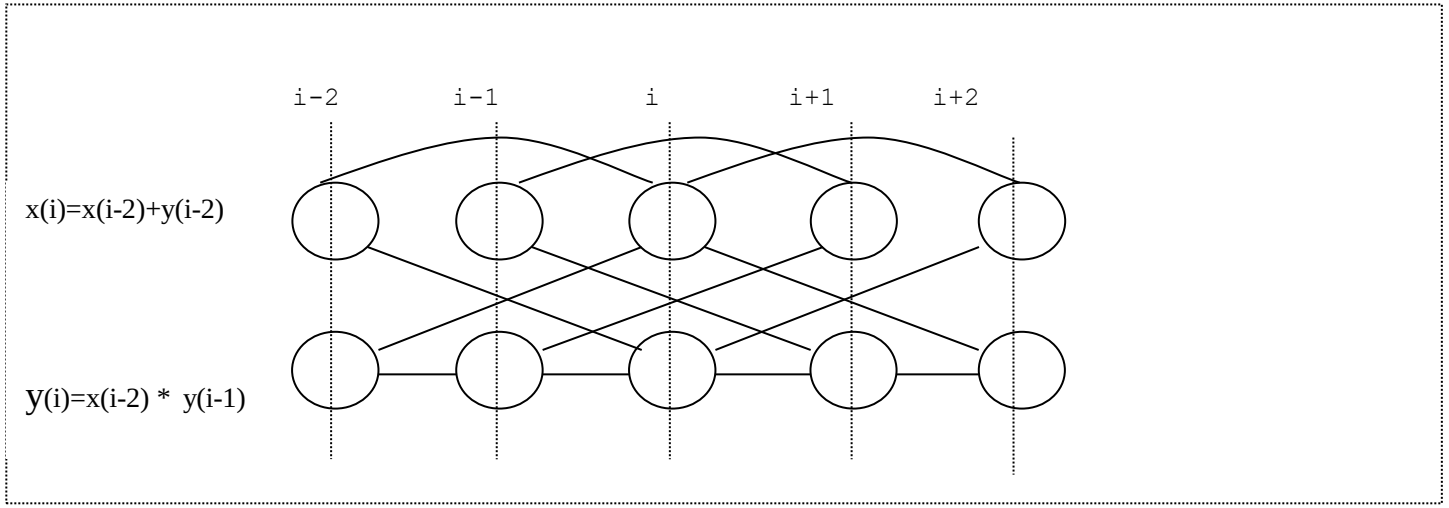
Pi= { Ii1, ….Iik}

Fiecare proces este format dintr-uo multime de instructiuni care pot calcula variabilele de iesire in functie de variabilele de intrare.

Paralelismul in cadrul unui proces se poate realiza prin calculul simultan a mai multe variabile de iesire.

P1
x(i)=x(i-2)+y(i-2)
y(i)=x(i-2) * y(i-1)

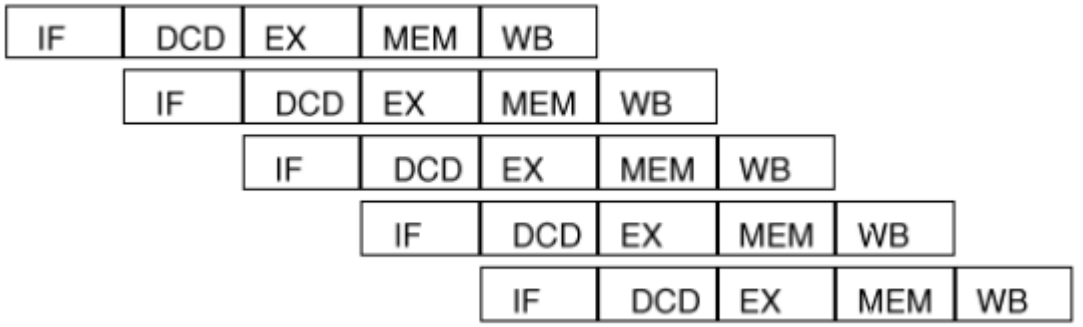
In exemplul anterior putem considera ca in cadrul procesului P1 variabilele x(i) si y(i) se pot calcula in paralel
sau
x(i) si y(i+1) se pot calcula in paralel



Nivel de Instructiune

Generarea codului, pentru calculul expresiei z =x + y, in structura pipeline

lw r1, x	IF	ID	EX	MEM	WB								
lw r2, y		IF	ID	EX	MEM	WB							
add r3, r1, r2			IF	ID	stall	EX	MEM	WB					
sw z, r3				IF	stall	ID	EX	MEM	WB				



Exemplu

- Să luăm în considerare un procesor care implementează o structura paralela pipeline de citire-interpretare-executie pentru procesare suprascalară.
- Arătați îmbunătățirea performanței față de procesarea scalară cu pipeline și procesarea fără pipeline, presupunând un ciclu de instrucțiuni contine:
  - • citire care necesita o singura perioada de ceas
  - • decodarea instructiunii necesita două perioade de ceas
  - • executia instructiunii necesita trei perioade de ceas
- și avem o secvență de 200 de instrucțiuni:

Exemplu solutie

- O structure fara pipeline necesita 1200 de cicluri de ceas: (nx6)

200 * (1 + 2 + 3) = 1200

n * (1 + 2 + 3)

F	D1	D2	E1	E2	E3																						
						F	D1	D2	E1	E2	E3																
												F	D1	D2	E1	E2	E3										
																		F	D1	D2	E1	E2	E3				
																								F	D1	D2	E1	E2	E3

- O structura pipeline scalară necesita 603 cicluri de ceas: ((n-1) * 3 +6)

1 + 2 + (200 * 3) = 603

1 + 2 + (n * 3)

F	D1	D2	E1	E2	E3																		
			F	D1	D2	E1	E2	E3															
						F	D1	D2	E1	E2	E3												
									F	D1	D2	E1	E2	E3									
												F	D1	D2	E1	E2	E3						
															F	D1	D2	E1	E2	E3			

pipeline superscalară cu două unități

- O structura pipeline superscalară cu două unități paralele ar necesita 303 cicluri de ceas (n / 2 + 1) * 3)

1 + 2 + ((200/2) * 3) = 303

1 + 2 + ((n / 2) * 3)

F	D1	D2	E1	E2	E3						
F	D1	D2	E1	E2	E3						
		F	D1	D2	E1	E2	E3				
		F	D1	D2	E1	E2	E3				
				F	D1	D2	E1	E2	E3		
				F	D1	D2	E1	E2	E3		
						F	D1	D2	E1	E2	E3
						F	D1	D2	E1	E2	E3

Sau daca fetch-ul se realizeaza secvential

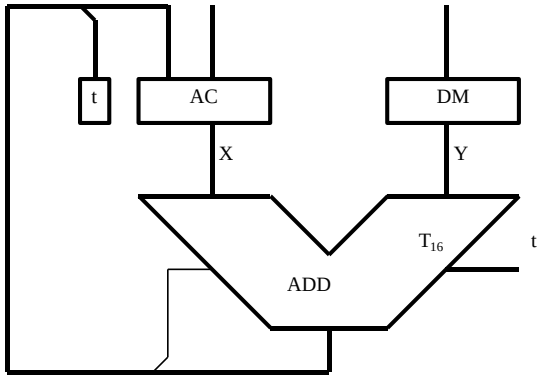
1+2+((200/2)*4= 403

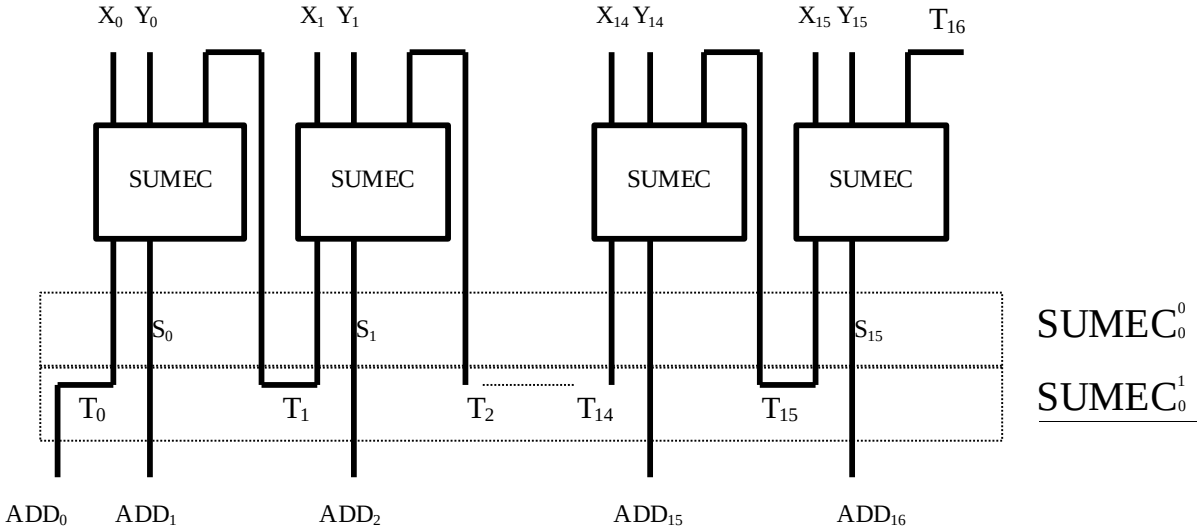
1 + 2 + ((n / 2) * 4)

F	D1	D2	E1	E2	E3									
	F	D1	D2	E1	E2	E3								
			F	D1	D2	E1	E2	E3						
				F	D1	D2	E1	E2	E3					
						F	D1	D2	E1	E2	E3			
							F	D1	D2	E1	E2	E3		
									F	D1	D2	E1	E2	E3
										F	D1	D2	E1	E2	E3

Nivel bit

Toate calculatoarele, cu foarte putine exceptii, utilizeaza unitati aritmetice paralele cu sau fara anticiparea transportului si cu structura pipeline.





Ce este performanța?

- În calcul, performanța este definită de 2 factori
  - Cerințe de calcul (ce trebuie făcut)
  - Resurse de calcul (cat costă să o faci)
- Problemele de calcul se traduc în cerințe
- Resurse de calcul interacțiune și compromise
- Performanța în sine este o măsură a cât de bine cerințele de calcul pot fi satisfăcute
- Evaluăm performanța pentru a înțelege relațiile dintre cerințe și resurse
- Decideți cum să schimbați „soluțiile” pentru a atinge obiectivele
- Măsurile de performanță reflectă deciziile despre cum și cât de bine „soluțiile” sunt capabile să satisfacă cerințe de calcul

problemele de performanță

Aici ne preocupă problemele de performanță când folosind un mediu de calcul paralel

- Performanță în raport cu calculul paralel
- Performanța este rațiunea de a fi pentru paralelism

Performanță paralelă versus performanță secvențială

Dacă „performanța” nu este mai bună, paralelismul nu este necesar

Prelucrarea în paralel include tehnici și tehnologii necesare pentru a calcula în paralel

- Hardware, rețele, sisteme de operare, biblioteci paralele,
- Limbajele de proramare, compilatoare, algoritmi, instrumente, ...

Paralelismul trebuie să ofere performanță

Cum? Cât de bine?

Așteptarea performanței

- Dacă fiecare procesor este evaluat la k MFLOPS și există p procesoare, ar trebui să vedem performanța k * p MFLOPS?
  - Dacă durează 100 de secunde la un procesor, nu ar trebui sa dureze 10 secunde pe 10 procesoare?
- Mai multi factori afectează performanța
- Fiecare trebuie înțelesi separat
- Dar ei interacționează între ei în moduri complexe
  - Soluția la o problemă poate crea alta
  - O problemă poate masca alta problema, etc.
- Scalarea (sistem, dimensiunea problemei) poate schimba condițiile
- Trebuie să înțelegeți limitele de performanță

Calcule paralele

- Un calcul paralel “jenant” este unul care poate fi evident împărțit în task-uri independete complet care pot fi executate simultan
  - Într-un calcul paralel cu **adevărat jenant**, nu există interacțiunea între procese separate
  - Într-un calcul paralel **aproape jenant** rezultă calcule ce trebuie distribuite și colectate / combinate într-un fel
- Calculele paralel jenante au potențial pentru a atinge viteza maximă pe platforme paralele
  - Dacă este nevoie de timp T secvențial, există potențialul de a realiza timpul T / P care rulează în paralel cu P procesoare
  - De ce acest lucru nu este întotdeauna adevarat?

Relatia intre algoritmi paraleli si arhitecturi paralele

- Prelucrarea paralela include atat algoritmi paraleli cat si arhitecturi paralele.
- Un algoritm paralel poate fi considerat ca o colectie de procese independente care se executa simultan, procesele comunicand in timpul executiei.
- Astfel un algoritm se executa pe unitati functionale hardware care in general constau din
  - elemente de procesare;
  - module de transfer date.

- Ceea ce intereseaza este cum se transpun procesele pe unitatile functionale hardware.

Algoritmi paraleli / Arhitecturi paralele

- H.T.Kung a fost unul din primii care au studiat relatia intre algoritmi si arhitectura. A stabilit citeva caracteristici si a prezentat corelarea intre ele:

Algoritmi paraleli	Arhitecturi paralele
granularitate modul	complexitate procesor
control concurent	mod de operare
mecanismul datelor	structura memoriei
geometria comunicatiei	retele de comutare
.complexitate algoritm	numar de procesoare; dimensiune memorie

Algoritmi - Arhitectura

- **Granularitate modul (obiect)** - se refera la complexitatea modulului care poate fi job, task, proces sau instructiune.
  - De obicei exista posibilitati de paralelism la o granularitae mare dar care nu este exploatat deoarece implica o comunicatie intensa si o crestere a complexitatii software-ului.
  - La acest nivel se face o analiza intre granularitate si comunicatie pentru a stabili solutia cea mai buna.
- **Control concurent** se refera la schema de selectie a modulelor pentru executie.
  - Aceasta trebuie sa satisfaca dependenta de date si dependenta de control (asigurarea excluderii mutuale a accesului la aceeasi resursa).

  Citeva scheme de control sunt bazate pe:
  - disponibilitatea datelor (data flow)
  - control centralizat (sinchronized)
  - cereri (demand-driven).
- exemplu algoritmii care prelucreaza matrice se potrivesc foarte bine pe procesoarele sistolice sau pe masive de procesoare iar algoritmii care au transferuri conditionate si alte iregularitati se potrivesc foarte bine pe arhitecturi asincrone cum ar fi multiprocesoare si data-flow.

- **Mecanismul datelor** se refera la faptul cum sunt utilizati operanzii.
  - Datele furnizate de instructiuni pot fi utilizate ca "date pure" in structurile data-flow sau pot fi depuse in locatii adresabile in masinile Von Neumann.
- **Geometria comunicatiei -** se refera la sablonul de interactiune intre module.
  - Geometria comunicatiei poate fi regulata sau neregulata.
- **Complexitate algoritm** se refera la numarul de operatii necesare pentru implementarea algoritmului.
  - Are influenta asupra numarului de procesoare si asupra dimensiunii memoriei.

Performanță și scalabilitate

- Evaluare
  - Runtime secvențial (Tseq) este o funcție de
    - dimensiunea problemei și arhitectura
  - Runtime paralel (Tpar) este o funcție de
    - dimensiunea problemei și arhitectura paralelă
    - numarul de procesoare utilizate în execuție
- Performanță paralelă afectată de
  - algoritm + arhitectură
- Scalabilitate
  - Abilitatea algoritmului paralel de a atinge performanța câștigă proporțional cu numărul de procesoare și cu dimensiunea problemei

Scalabilitate

- Un program se poate utiliza mai multe procesoare
  - Cum evaluezi scalabilitatea?
- Cum evaluezi performantele scalabilității?
- Evaluare comparativă
  - Dacă se dublează numărul de procesoare, la ce să ne așteptăm?
  - Scalabilitatea este liniară?
- Utilizați o măsură de eficiență paralelă
  - Este menținută eficiența pe măsură ce crește dimensiunea problemei?
- Aplicați valori de performanță

Indicatori de performantele ai calculului paralel

Datorita complexitatii calculului paralel este greu de a stabili o masura a performantelor care sa stabileasca real si absolut performantele unui sistem cu arhitectura paralela.
Totusi sunt utilizati cativa indicatori care masoara diferite aspecte globale.

Rata de executie

masoara rata de producere a unor rezultate in unitatea de timp.

Uzual se folosesc :

- **MIPS / GIPS / TIPS** – milioane /giga/tera de instructiuni pe secunda care masoara numarul de instructiuni pe care le executa pe secunda o unitate de prelucrare mono sau multiprocesor. Un astfel de indicator este inadecvat pentru o masina SIMD care executa aceeasi instructiune pe mai multe fluxuri de date.
- **MOPS** / **GOPS / TOPS** - milioane /giga/tera de operatii pe secunda care masoara operatiile efectuate de unitatile de prelucrare. O astfel de unitate de masura nu tine seama de lungimea cuvintului si nici de natura operatiilor.
- **MFLOPS** / **GFLOPS / TFLOPS / PFLOPS-** milioane /giga/tera/peta de operatii cu virgula mobila pe secunda (Giga) care masoara operatiile cu virgula mobila efectuate (Tera) de unitatile de prelucrare.O astfel de masura este adecvata numai pentru aplicatii numerice dar nu reprezinta nici un fel de masura pentru prelucrari alfanumerice sau pentru aplicatii bazate pe inteligenta artificiala.
- **MLIPS** / **GLIPS/ TLIPS**– milioane /giga/tera de inferente logice pe secunda care masoara numarul de inferente logice realizate in aplicatii de IA

Indicatori

Presupunem ca avem o structura cu p procesoare, care participa la rezolvarea unei probleme.

Viteza de prelucrare - V_p

$$V_p = \frac{T_1}{T_p}$$ unde

T_1 - timpul necesar pentru prelucrare utilizind un singur procesor

T_p - timpul necesar pentru prelucrare utilizind **p** procesoare

Cu alte cuvinte, V_p reprezinta raportul intre prelucrarea secventiala si cea paralela care scoate in evidenta cresterea in viteza datorita paralelismului.

deoarece se consuma timp cu sincronizarea, comunicarea si "overhead" cerut de interactiunea intre procesoare.

$1 \leq V_p < p$

Eficienta E_p

Este definita ca raportul intre viteza de prelucrare si numarul de procesoare.

$$E_p = \frac{V_p}{p} = \frac{T_1}{p * Tp} < 1$$ Eficienta este o masura a eficientei costului

Cost $p * T_p$ este $p * T_p >> T_1$

Indicatori

Redundanta Rp

Este definita ca raportul intre numarul total al operatiilor Op necesar efectuarii calculului cu p procesoare si numarul de operatii O1 necesar efectuarii calculului cu un singur procesor.

$$R_p = \frac{O_p}{O_1} = \frac{\textbf{nt total de operatii efectuate pe cele p procesoare}}{\textbf{nr de operatii necesare efectuarii pe 1 procesor}}$$

reflecta timpul pierdut cu overhead-ul.

Utilizarea Up

Este definita ca raportul dintre numarul de operatii O_p necesar efectuarii calculului cu p procesoare si numarul de operatii care ar fi putut fi efectuate cu p procesoare in timpul T_p

$$U_p = \frac{O_p}{p * Tp} \leq 1$$

Limite ale calculului paralel

Este foarte important a stabili limita calculului paralel.
Fie Tn timpul necesar executiei a n taskuri de k tipuri diferite.
Fiecare tip consta din ni taskuri necesitind ti secunde fiecare

$$Tn = \sum_{i=1}^{k} (ni*ti) \qquad \text{iar} \qquad n = \sum_{i=1}^{k} ni$$

Tn timpul necesar executarii celor n task-uri

Prin definitie, rata de executie R este numarul de operatii efectuate in unitatea de timp

$$Rn = \frac{n}{Tn} = \frac{n}{\sum_{i=1}^{k} (ni*ti)}$$

Fie $fi=ni/n$ $\quad \sum_{i=1}^{k} fi = 1 \quad$ $Ri = \frac{1}{ti}$

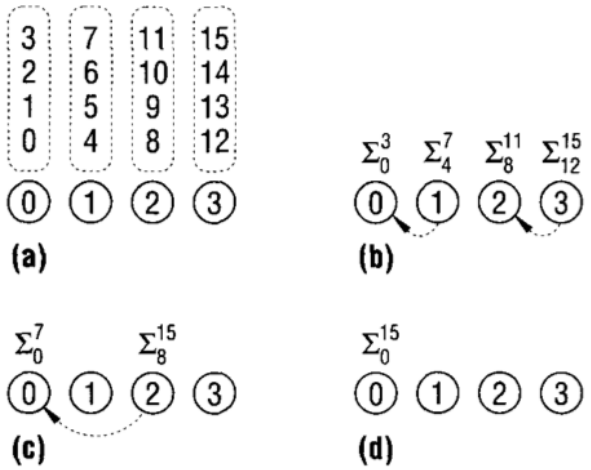
In acest caz rezulta

$$Rn = \frac{n}{\sum_{i=1}^{k} (ni*ti)} = \frac{1}{\sum_{i=1}^{k} (ni/n*ti)} = \frac{\mathbf{1}}{\mathbf{\sum_{i=1}^{k} (fi/Ri)}}$$

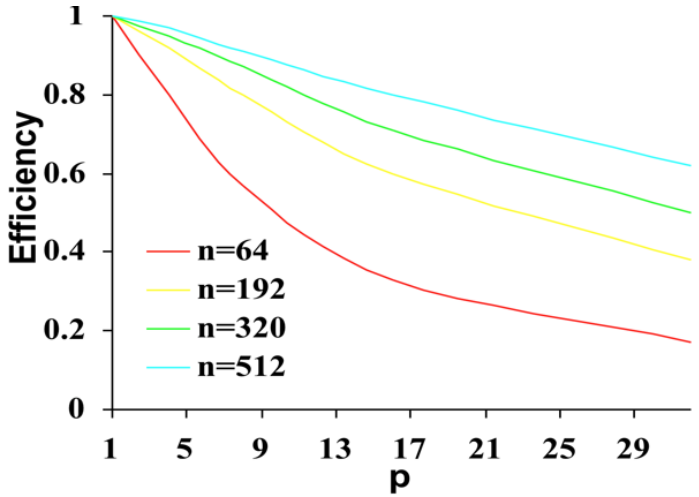
Aceasta relatie reprezinta "bottleneck" - limitarea in structurile paralele.

Scalabilitatea in a aduna n numere

- Scalabilitatea unui sistem paralel este masura capacitatii de a creste viteza de prelucrare utilizand mai multe procesoare
- Adunarea a n numere utilizand p procesoare conduce la:



- $T_p = \frac{n}{p} + 2 * \log p$
- $V_p = \frac{n}{\frac{n}{p} + 2 * \log p}$
- $\boldsymbol{E_p = \frac{V_p}{p} = \frac{T_1}{p * Tp} = \frac{n}{n + 2 * p * \log p}}$



Legea lui Amdahl

Sa consideram

f = fseq procentul de program care se executa secvential

1-f = fpar procentul de program care poate fi paralelizat

Fie T_1 *timpul de executie pe o structura cu 1 procesor*

Fie T_p *timpul de executie pe o structura cu p procesoare*

Vp viteza de prelucrare

$Vp = T1 / Tp = = T1 / (f*T1 + (1-f)*T1 / p)) = 1 / (f + (1-f)/p))$

Sau

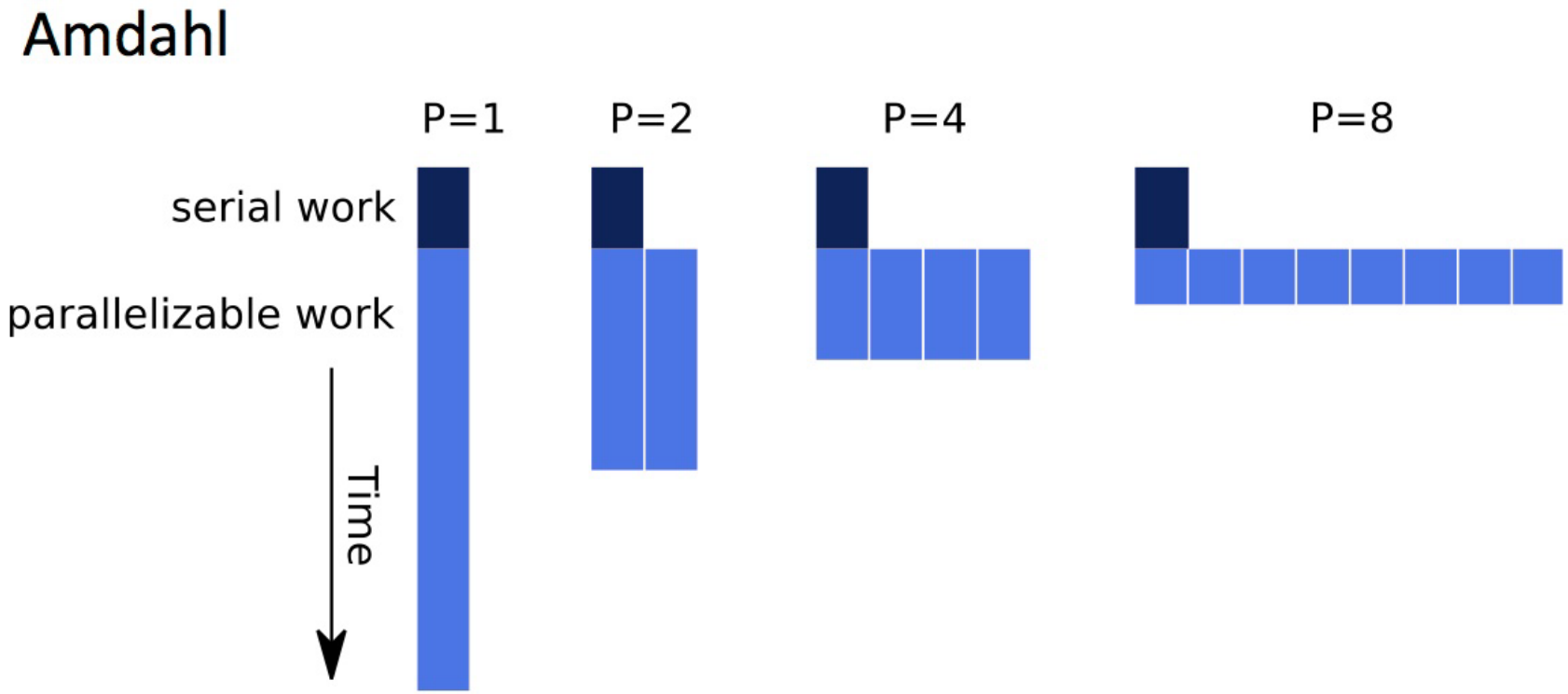
$Vp = 1 / (fseq + fpar/p)$

$Vp = 1 / (fseq + (1-fseq)/p)$

Cand $p \rightarrow \infty$

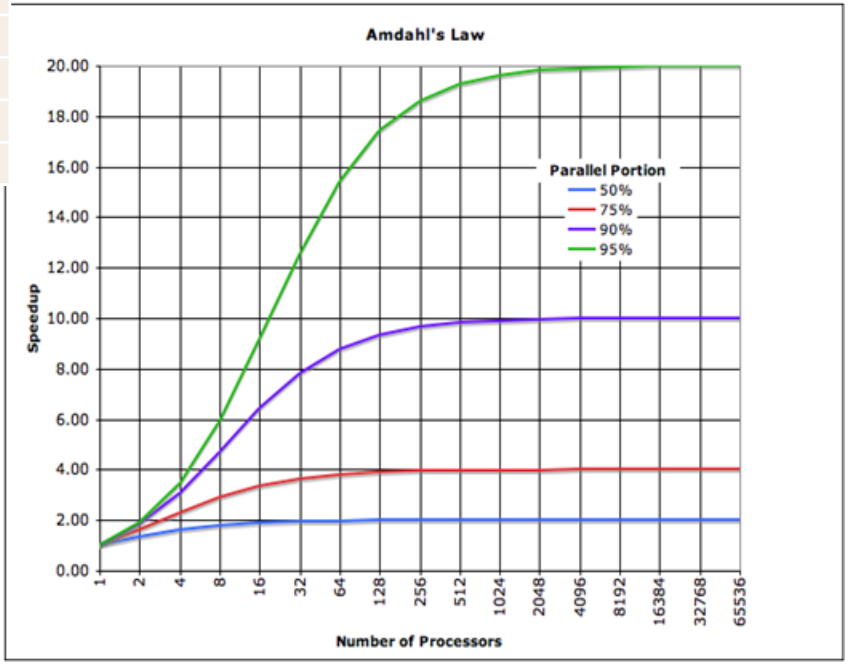
$Vp = 1 / f$ *sau* $Vp = 1 / fseq$

Amdahl



Cresterea in Viteza

	Crestere viteza			
		Paralelism		
	f=100%	*f=50%*	*f=10%*	*f=1%*
Numar procesoare	*(1-f)=0%*	*(1-f)=50%*	*(1-f)=90%*	*(1-f)=99%*
1	1	1.00	1.00	1.00
2	1	1.33	1.82	1.98
5	1	1.67	3.57	4.81
10	1	1.82	5.26	9.17
100	1	1.98	9.17	50.25
1,000	1	2.00	9.91	90.99
10,000	1	2.00	9.99	99.02
100,000	1	2.00	10.00	99.90
1,000,000	1	2.00	10.00	99.99
10,000,000	1	2.00	10.00	100.00



Scalabilitate

- *Capacitatea algoritmului paralel de a obține câștiguri de performanță proporțional cu numărul de procesoare și dimensiunea problemei*

Când se aplică Legea lui Amdahl?

- *Când dimensiunea problemei este fixa*
- *Scalare puternică ($p \to \infty$, $Vp = V_\infty \to 1 / f$)*

Limita de accelerare este determinată de gradul de secvențialitate, nu de numarul de procesoare !!!

- *Eficiența perfectă este greu de realizat*

Legea lui Gustafson-Barsis (accelerare scalată)

Un Data center este interesat , cand se pune problema scalarii, de probleme mai mari

- Cât de mare poate fi o problemă (HPC Linpack)
- Care este contrangerea problemei care se executa in paralel

Să presupunem că timpul de executie este menținut constant

- $Tp = C = (f + (1-f)) * C = (fseq + fpar) * C$
- $fseq$ este fracțiunea de Tp cheltuită în execuția secvențială
- $fpar$ este fracțiunea de Tp cheltuită în execuție paralelă

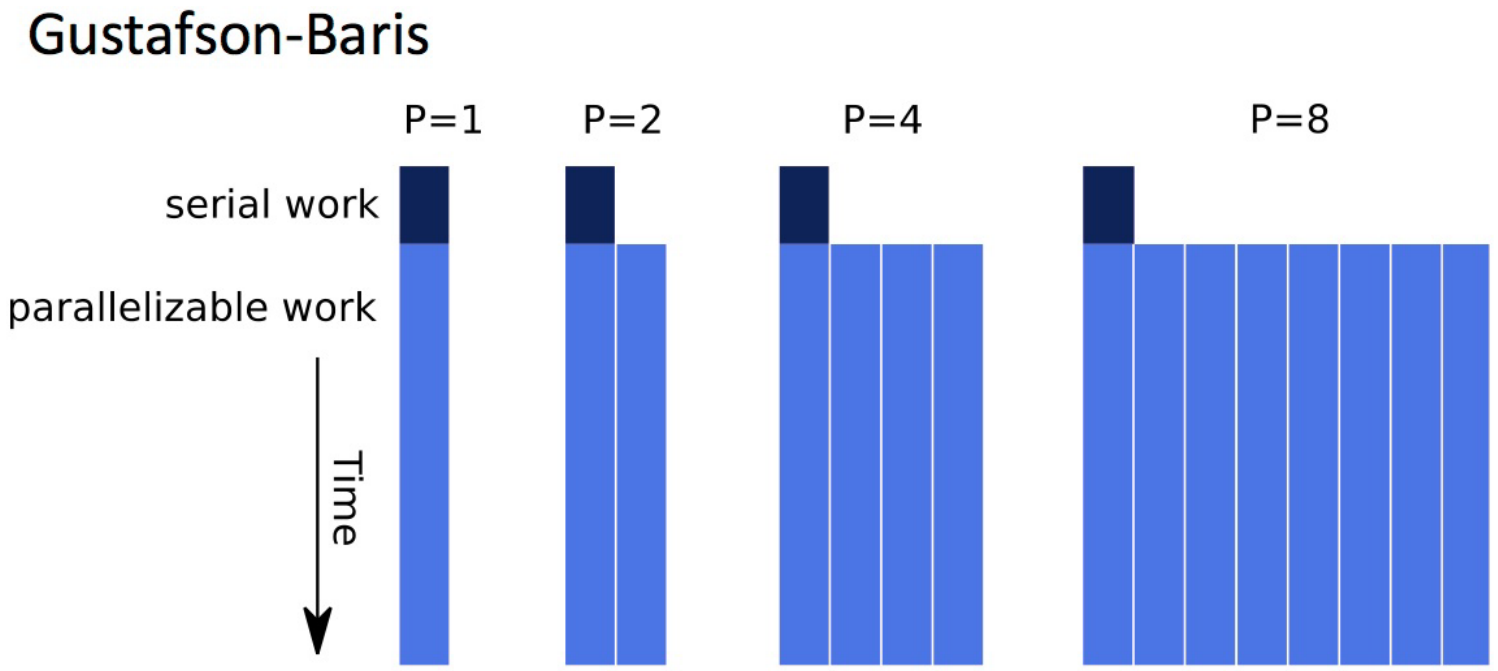
Scalara pe p procesoare conduce la

- $Ts = fseq + p* (1- fseq) = fseq + p* fpar$, *deoarece* $fpar = 1- fseq$
- $p*fpar$ *ne arata cat putem incarca structura din centrul de date*
- $Ts = 1- fpar + p*fpar = 1 + (p-1)fpar$
- $Tn = fseq + (1-fseq)*p / p = 1$

Care este viteza în acest caz?

$Vp = Ts / Tp = Ts / 1 = fseq + p(1 – fseq) = 1 + (p-1)\, fpar$

Gustafson - Baris



Gustafson-Barsis' / Scalabilitate

Scalabilitate

- Capacitatea algoritmului paralel de a atinge performanța proporțional cu numărul de procesoare și cu dimensiunea problemei

Când se aplică Legea lui Gustafson?

- Când dimensiunea problemei poate crește ca număr
- Scalare slabă $Vp = 1 + (p\text{-}1)fpar$
- Funcția de accelerare include numărul de procesoare !!!
- Poate menține sau crește eficiența paralelă prin scalare problema / probleme

Sun and Ni

- Introduce modelul bazat pe limita de memoriei
- $Ts = fseq + g(p)(1- fseq)$
  - *unde G(n) este cresterea de memorie a unui procesor*
- $Tp = fseq + g(p)(1- fseq) /p$
- $Vp=Ts/Tp= (fseq + g(p)(1- fseq)) /(fseq + g(p)(1- fseq) /p)$

- *Caz 1 : g(n)=1*
- $Vp= (fseq + 1- fseq) /(fseq + (1- fseq) /p)= \mathbf{1 /(fseq + (1- fseq) /p)} \sim Amdahl$

- *Caz 2 : g(n)=p*
- $Vp=Ts/Tp= (fseq + p(1- fseq)) /(fseq + p(1- fseq) /p)=(fseq + p(1- fseq)) /1 =$ $\mathbf{fseq + p(1- fseq)} \sim Gustafson$

- *Caz 3: g(p)= m > p*
- $Vp=Ts/Tp= (fseq + m(1- fseq)) /(fseq + m(1- fseq) /p)=$
- $\mathbf{(fseq + m(1- fseq))/(m/p+ (1-m/p)fseq)}$

- *Workload-ul creste mai repede decat cererea de memorie.*

Limita lui Worlton

Jack Worlton a studiat limitele calculului paralel utilizind un model care aproximeaza operarea unui multiprocesor.

El presupune ca un program paralel consta din :

- sectiuni de prelucrare distribuite pe diverse procesoare;
- sectiuni de sincronizare;
- sectiuni care se executa secvential si constituie un "overhead".

Fie

ts timpul de sincronizare;
to sectiune de overhead
t media timpului de executie a unui task
N numarul de task-uri (intre puncte de sincronizare)
P numarul de procesoare

Timpul de executie secvential al celor N task-uri este :

$T1 = N*t$

Timpul de executie a celor N task-uri executate pe p procesoare este :

$Tn,p = ts + (\lceil N / p \rceil)*(t+to)$

Viteza de prelucrare

$$Vn,p = T1 / Tn,p = \frac{N*t}{ts + (\lceil N / p \rceil)*(t+to)} = \frac{1}{ts / (N*t) + (1/N)*(\lceil N / p \rceil)*(1+(to/t))}$$

Comentarii Worlton

Observatii:

Pentru a creste viteza de prelucrare trebuie sa avem in vedere reducerea efectului sincronizarii, overhead-ului si a numarul de pasi $\lceil N / p \rceil$

- ***efectul sincronizarii*** cauzat de ts/(N*t) poate fi redus fie prin micsorarea timpului de sincronizare ts sau prin marirea intervalului intre sincronizari N*t
- ***efectul overhead-ului*** cauzat de to/t poate fi redus prin reducerea timpului de overhead to sau prin cresterea granularitatii task-urilor.

Cresterea granularitatii task-urilor ajuta la reducerea atat a efectului sincronizarii cat si a overhead-ului.

- ***numarul pasilor de calcul*** $\lceil N / p \rceil$ poate fi redus prin cresterea numarului de procesoare si avand un numar de task-uri multiplu de procesoare.

$$V_{n,p} = T1 / T_{n,p} = \frac{N*t}{ts + (\lceil N / p \rceil)*(t+to)} = \frac{1}{ts / (N*t) + (1/N)*(\lceil N / p \rceil)*(1+(to/t))}$$

Eficienta

$$Vn,p = T1 / Tn,p = \frac{N*t}{ts + (\lceil N / p \rceil)*(t+to)} = \frac{1}{ts / (N*t) + (1/N)*(\lceil N / p \rceil)*(1+(to/t))}$$

Eficienta

$$En,p = \frac{Vn,p}{p}$$

Presupunind ca avem un numar mare de task-uri si ca overhead-ul este neglijabil relativ la granularitate avem :

p >>>> N fix

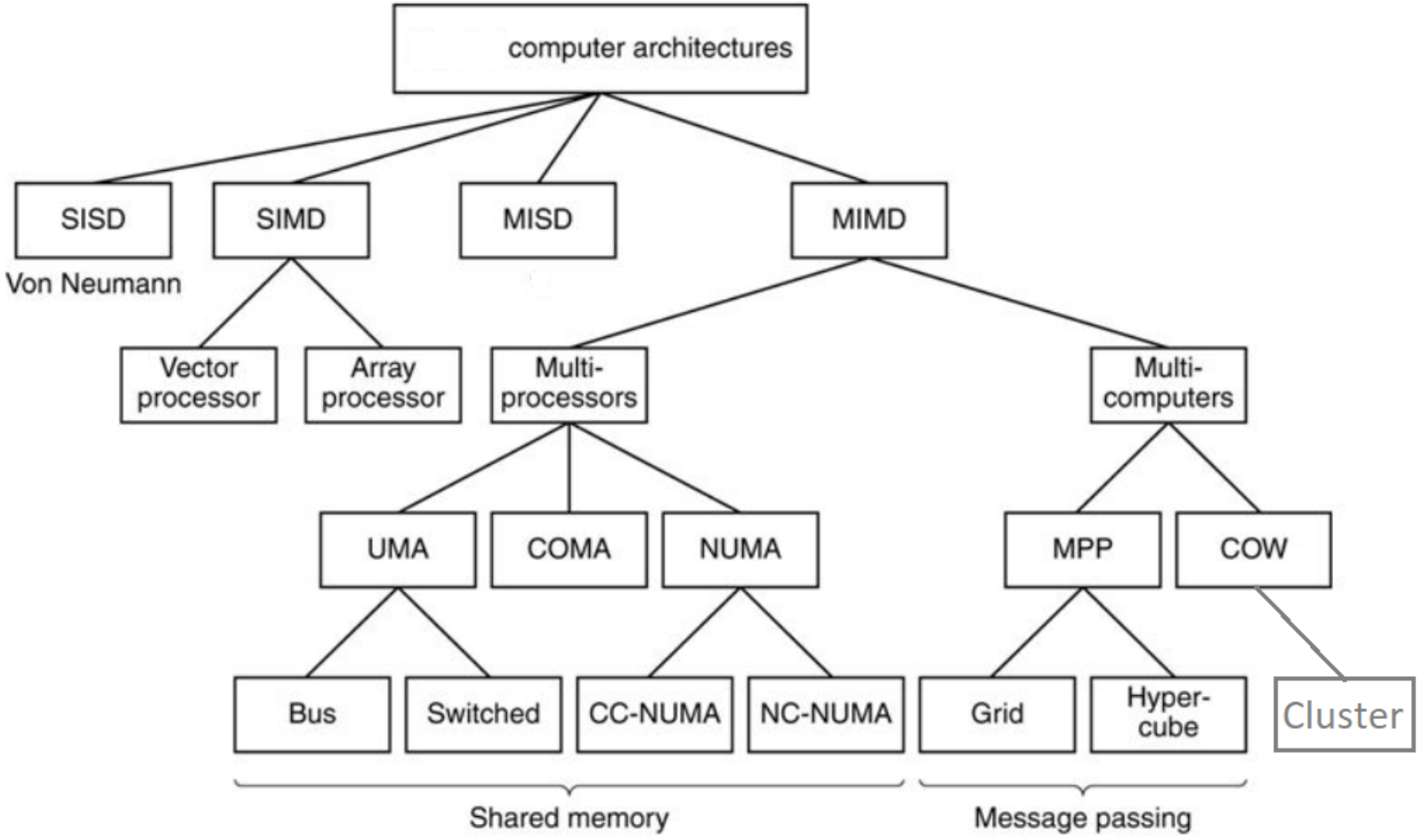
$$Vn,p = \frac{N}{ts/t} \quad \text{si} \quad En,p = 0$$

N >>>> p fix

$$Vn,p = \frac{P}{1 + to/t} \quad \text{si} \quad En,p = \frac{1}{1 + to/t}$$

Taxomomii
Modele de calcul Paralel

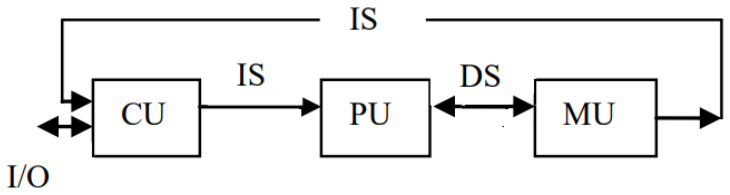
Arhitecturi de calculatoare



Taxonomii

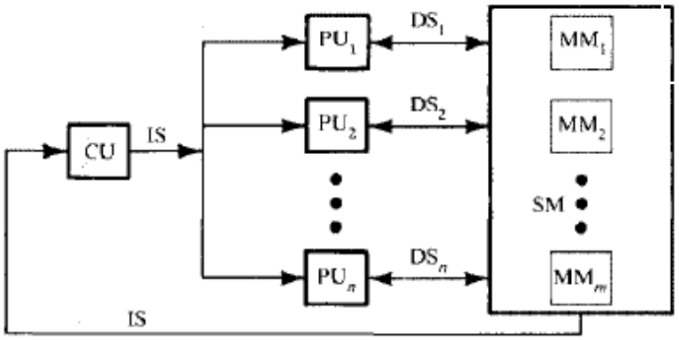
1. ***Taxonomia lui Flynn (bazata pe relatia dintre fluxul de instructiuni si fluxul de date:***

- SISD (single instruction single data flow)
- SIMD (single instruction multiple data flow)
- MISD (multiple instruction single data flow)
- MIMD (multiple instruction multiple data flow)

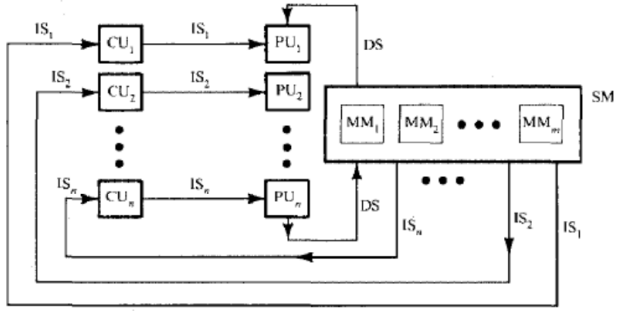


(a) SISD Uniprocessor Architecture

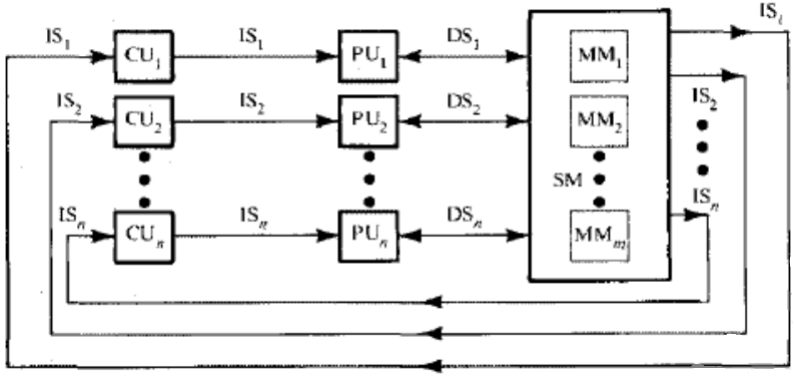
(b) SIMD Architecture



(c) MISD Architecture (the Systolic Array)



(d)MIMD architecture



Taxonomii

Taxonomia lui Shore:

- masina seriala pe cuvant paralela pe biti
- masina paralela pe cuvant seriala pe biti
- masina ortogonala
- masina masiv neconectat
- masina masiv conectat

Structurala

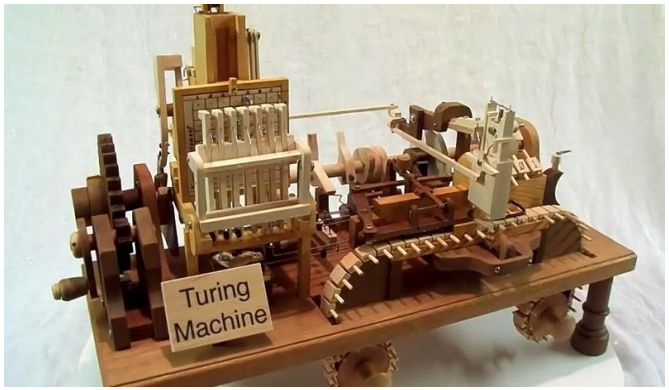
- uniprocesoare seriale
- unicalculatoare paralele bazata pe paralelism functional si pipeline
- masive de procesoare

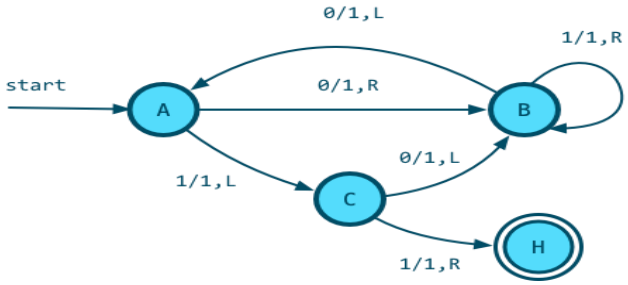
Modele ale calculului paralel

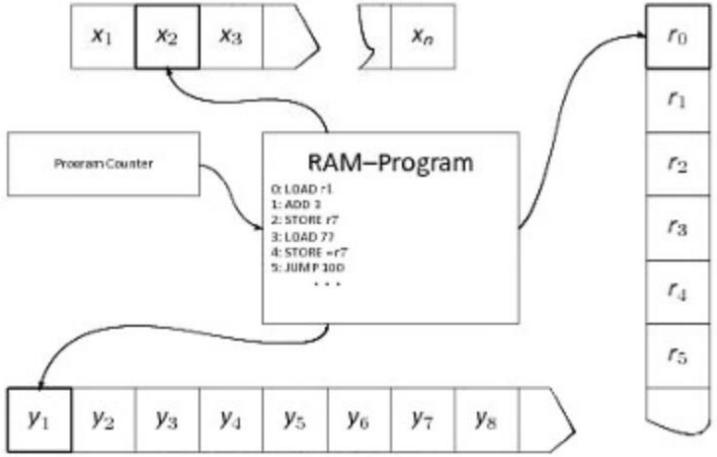
1. Model RAM (Random Acces Machine)
2. PIPELINE
3. PROCESOARE DE VECTORI
4. MULTIPROCESOARE cu memorie partajata
5. MULTIPROCESOARE cu transfer prin mesaje
6. PRAM (Parallel Random Acces Machine)
7. PROCESARE SISTOLICA
8. PROCESARE DATA FLOW

O mașină cu acces aleatoriu (RAM):

- o bandă de intrare infinită X cu poziția capului citit i ∈ N,
- o bandă de ieșire infinită Y cu poziția capului de scriere o ∈ N,
- un registru contor de program PC
- un program format dintr-un număr finit de instrucțiuni P [1], ..., P [m],
- o memorie constând dintr-o succesiune infinită de registre r0,r1,r2







RAM = (X ,Y, S, s0, f, Z)

S multimea finita a starilor automatului;
s0 starea initiala;

X alfabetul finit al benzii de intrare;

Y alfabetul finit al benzii de iesire;

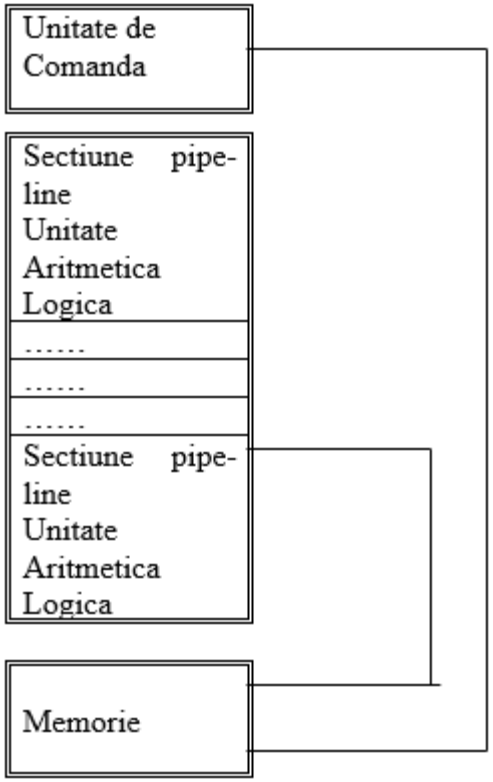
f functia de tranzitie, eventual partial definite
f : S x X -->S x Y

Z inclus in S - multimea starilor finale

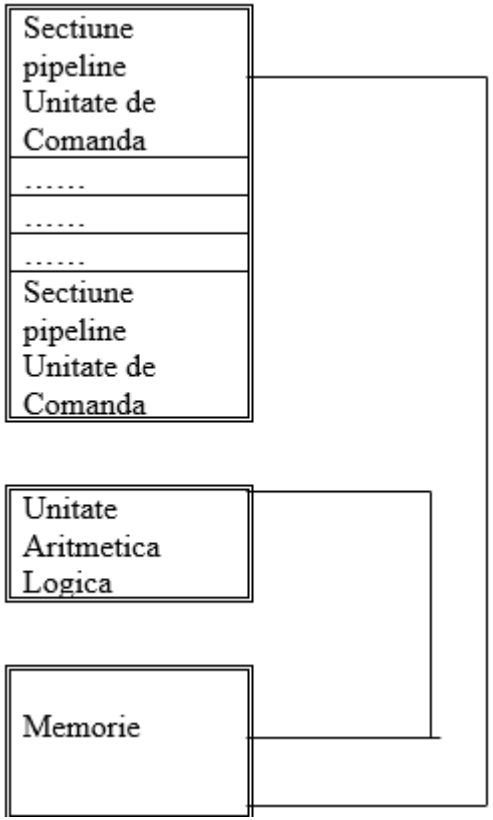
Structura pipeline.

Conceptul de pipeline consta in a imparti un task T in subtask-uri T_1 , T_2 , ...,T_k si de a le atribui unor elemente de procesare.
Paralelismul se poate realiza fie la nivel de:

- prelucrare aritmetica;



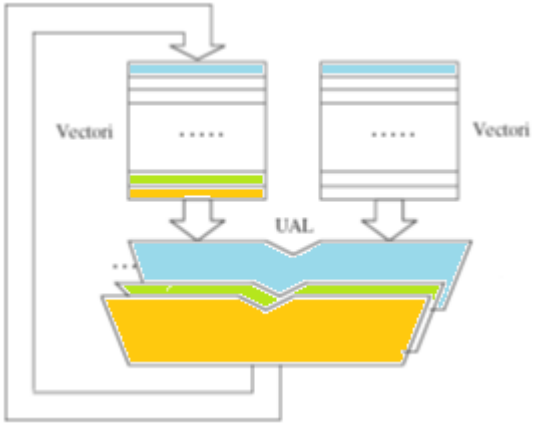
citire, interpretare, executie instructiuni



Procesoare de vectori.

Multiprocesoare cu memorie partajata.

Procesoare de vectori.



In cadrul acestui model, exista un set de instructiuni care trateaza vectorul ca operand simplu. (SIMD) sau
elemente de procesare vectoriala.(UAL de tip pipeline si registre de vectori).

O astfel de structura o gasim in sisteme monoprocesor care au ca extensie procesare de vectori.

Acopera o gama redusa de probleme cu caracter vectorial.

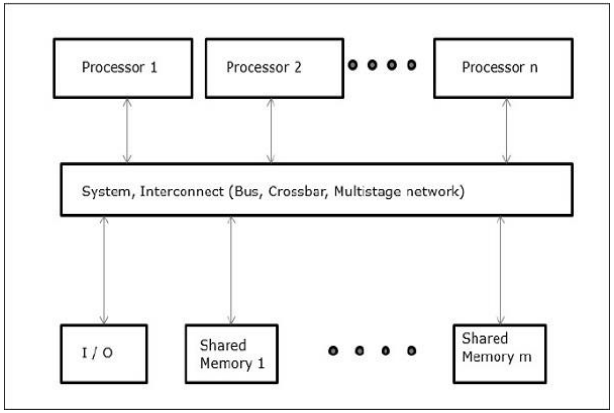
Multiprocesoare cu memorie partajata.

Acest model descrie structurile in care fiecare procesor contine propria unitate de prelucrare, propria memorie si propria unitate de comanda si comunica prin intermediul unei retele de comutare cu module de memorie partajate.

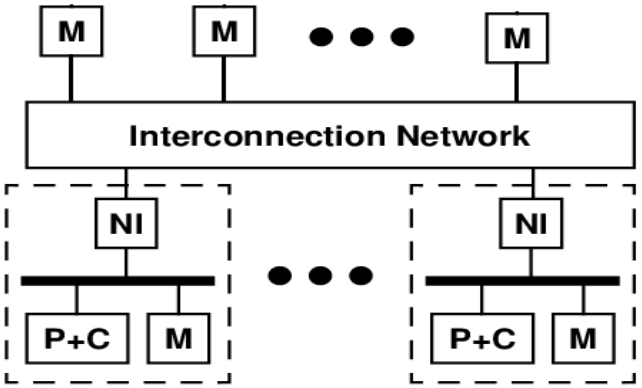
Fiecare procesor executa propriul set de instructiuni din memoria locala sau memoria partajata.

Reteaua de comutare permite schimbul de informatii intre procesoare si intre procesoare si memoria partajata.

UMA



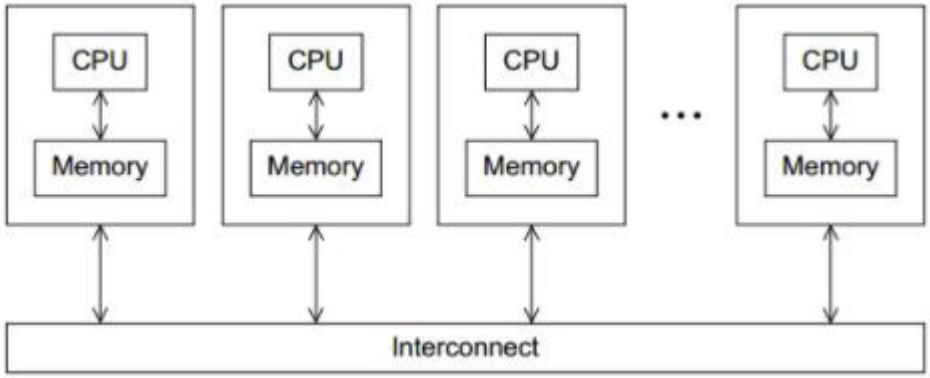
NUMA



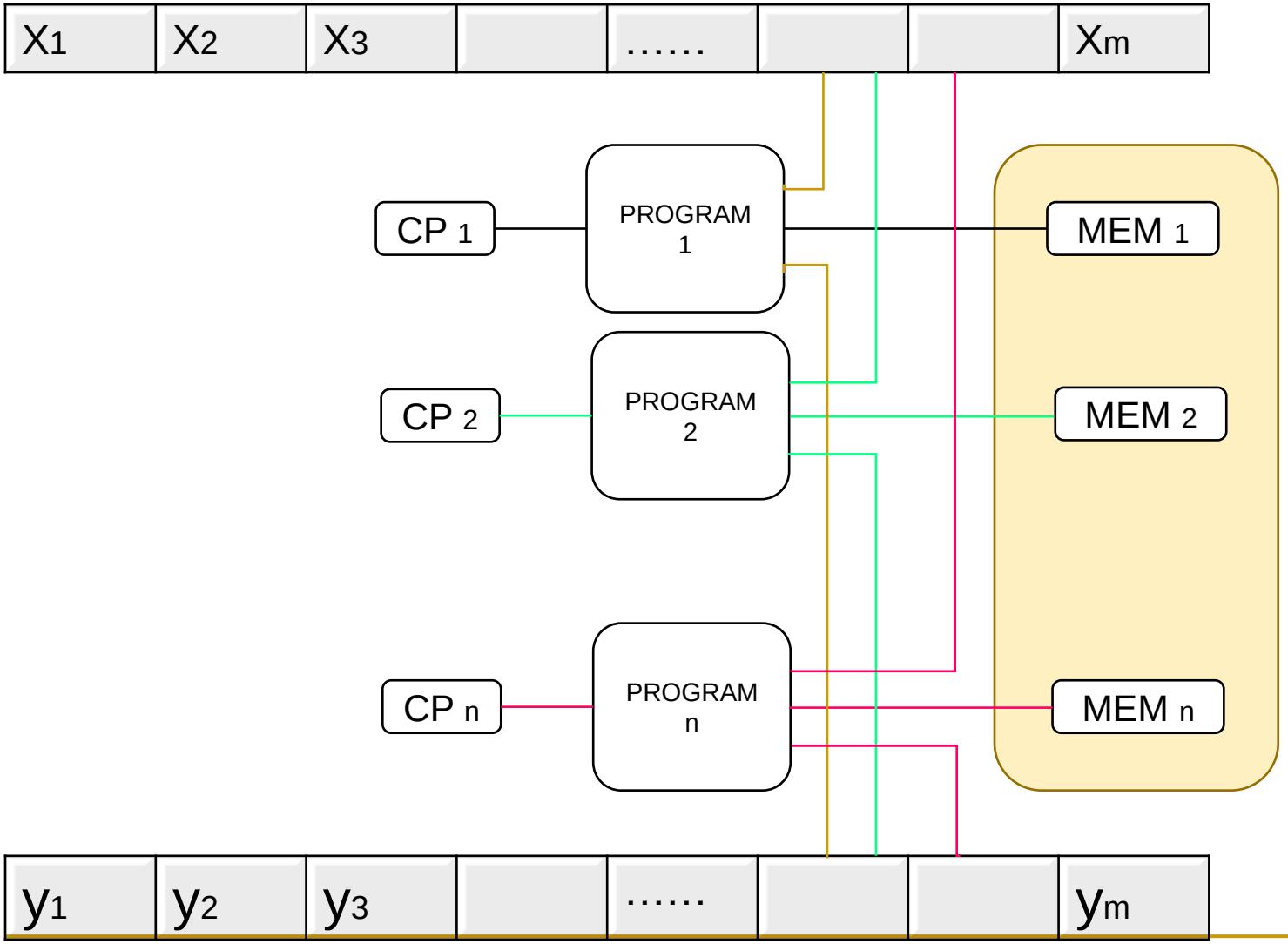
Multiprocesoare cu transfer prin mesaje

Multiprocesoare cu transfer prin mesaje.

- In acest model memoria este distribuita fiecarui procesor astfel incit fiecare procesor dispune de propriul program si propria memorie de date.
- Comunicarea datelor partajate este realizata prin schimburi de mesaje prin intermediul unei retele de comutare mesaje.
- Acest model este diferit de modelul cu memorie partajata deoarece interconexiunea se realizeaza prin mesaje si nu prin acces direct (comutare de circuite).



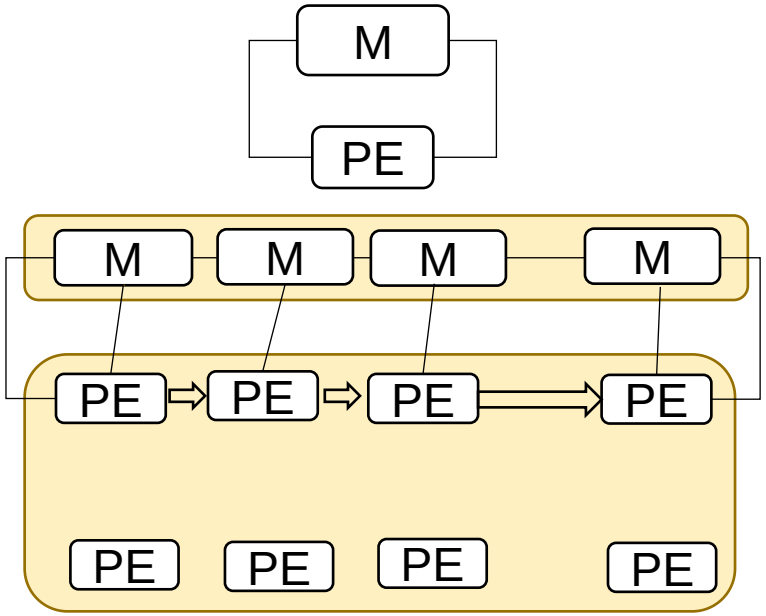
Modelul PRAM -parallel RAM

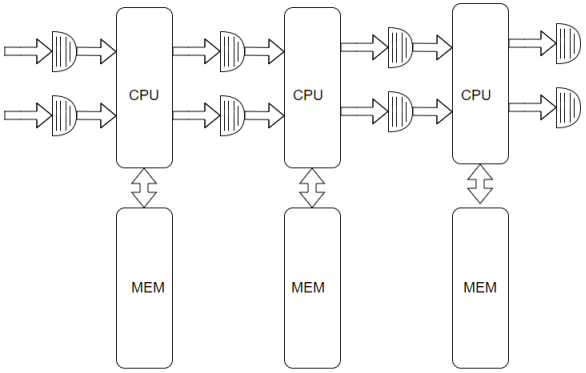


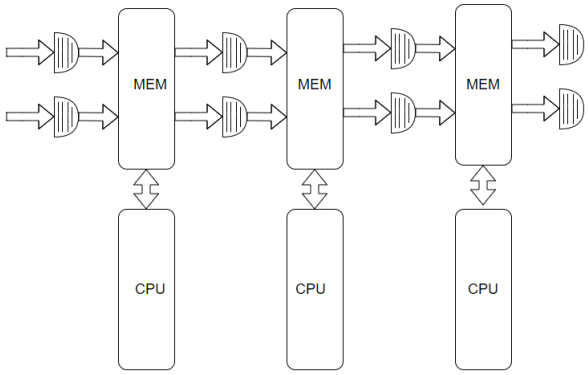
Procesare sistolica

Procesare sistolica.

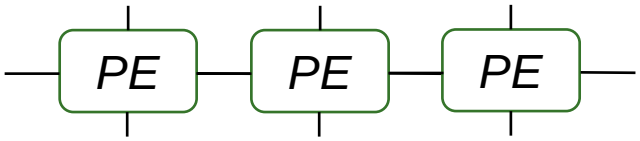
Modelul procesarii sistolice consta din elemente de procesare identice aranjate intr-o structura pipeline.
Procesarea sistolica este caracterizata prin interconexiuni simple si regulate ceea ce permite integrarea VLSI.

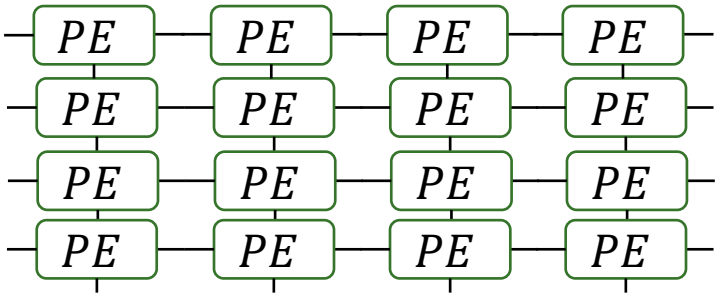


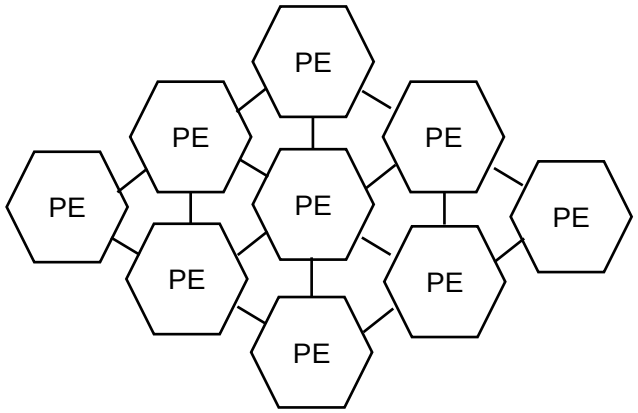


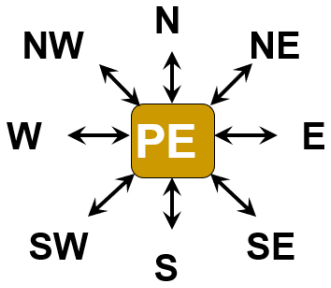


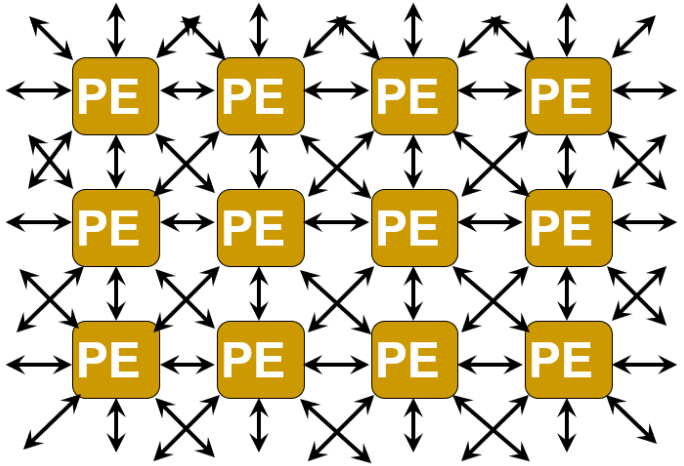
Modalitati de conexiune Sistolica





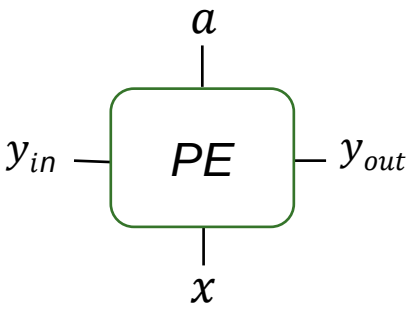






Exemplu

- Fie
- $A=\begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & & \ddots & \vdots \\ a_{n1} & an_2 & \cdots & a_{nn} \end{bmatrix}$ $Y=\begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix}$ $X=\begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}$ $Y_0=\begin{bmatrix} y_{01} \\ y_{02} \\ \vdots \\ y_{0n} \end{bmatrix}$
- Dorim sa calculam
- $Y=A*X+Y_0$
- Presupunem ca avem un element de procesare sistolica

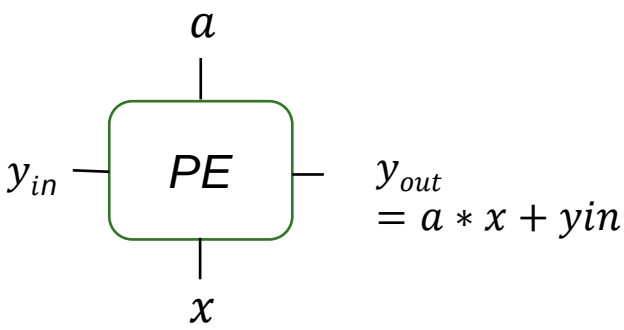


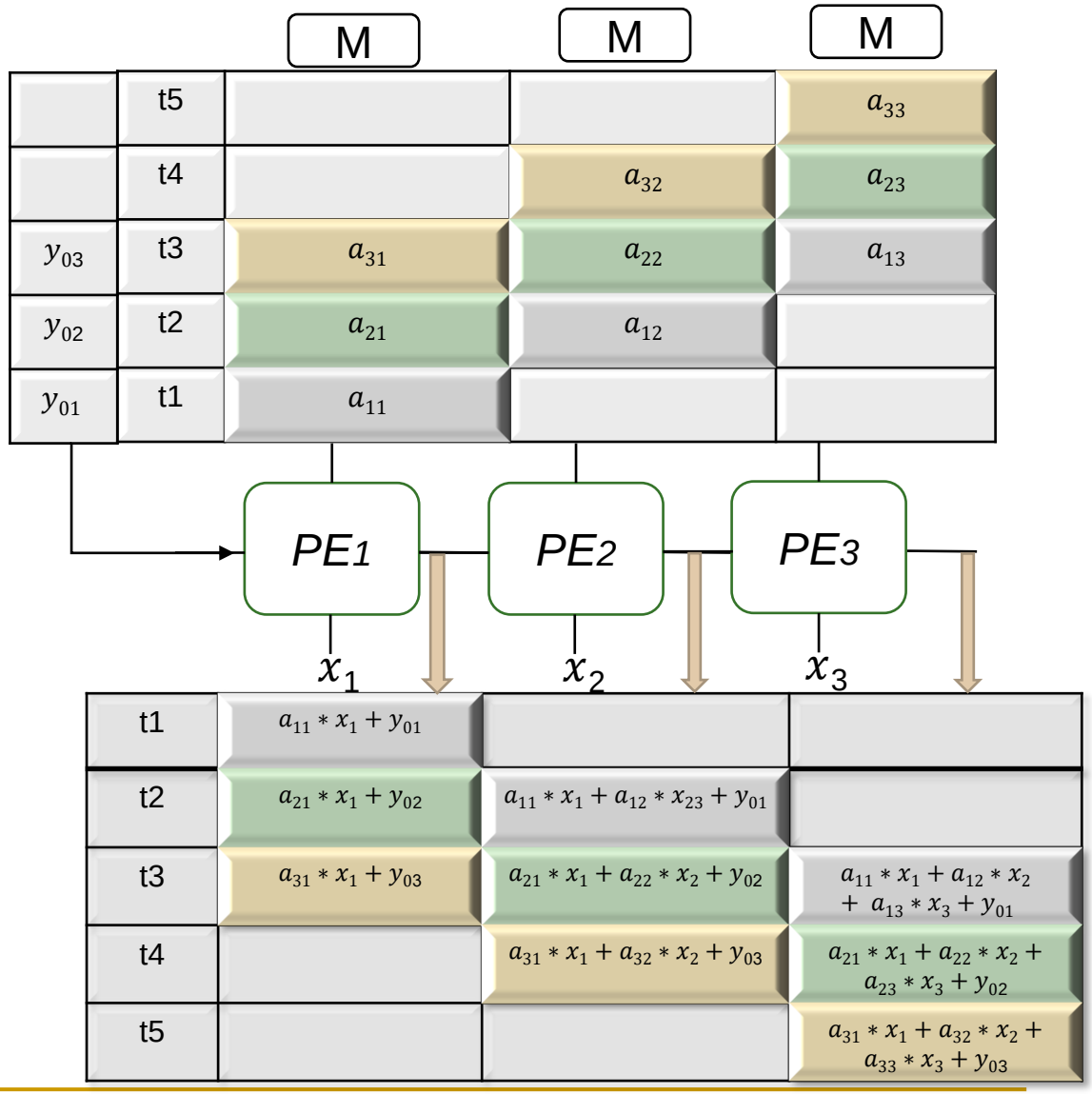
$$y_{out} = a * x + yin$$

Y	A					X	Y_0
y_1	a_{11}	a_{12}	a_{13}		a_{1n}	x_1	y_{01}
y_2	a_{21}	a_{22}	a_{23}		a_{2n}	x_2	y_{02}
y_3	a_{31}	a_{32}	a_{33}		a_{3n}	x_3	y_{03}
y_n	a_{n1}	a_{n2}	a_{n3}		a_{nn}	x_n	y_{0n}

Exemplu pentru n=3, structura unidimensionala

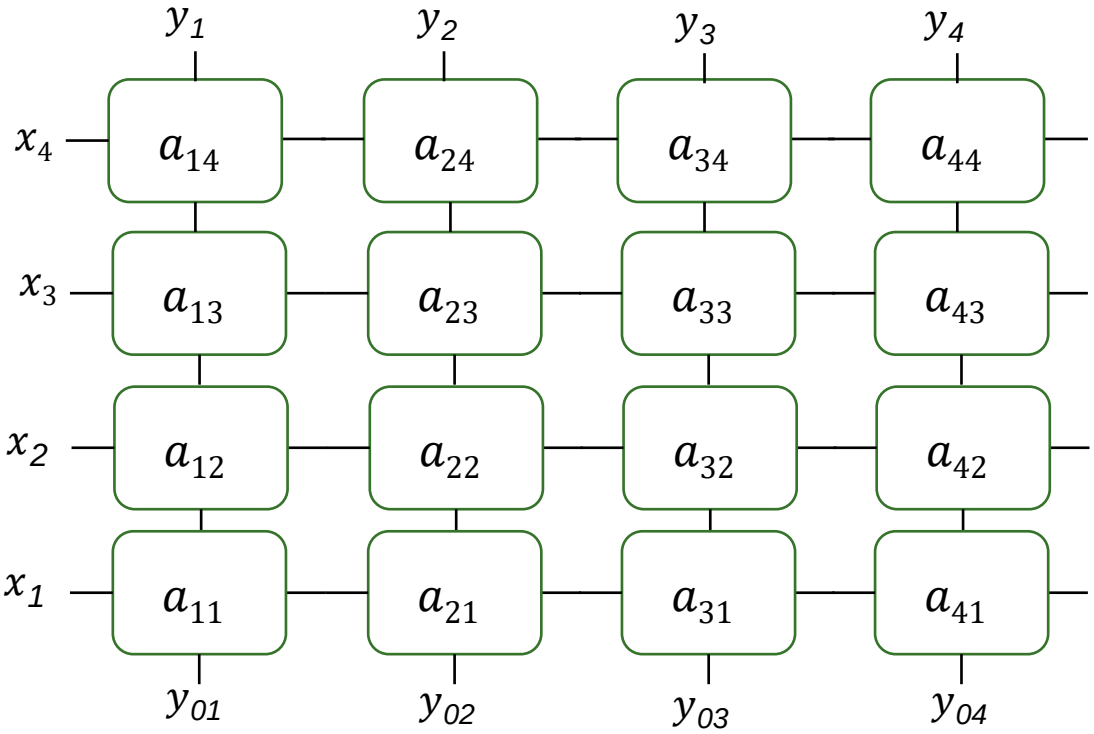
- $y_i = \sum_{j=1}^{n} a_{ij} * xj + y_{0i}$
- $y_1 = a_{11} * x_1 + a_{12} * x_2 + a_{13} * x_3 + y_{01}$
- $y_2 = a_{21} * x_1 + a_{22} * x_2 + a_{23} * x_3 + y_{02}$
- $y_3 = a_{31} * x_1 + a_{32} * x_2 + a_{33} * x_3 + y_{03}$

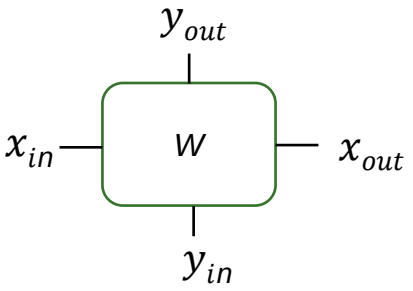




Exemplu pentru n=4, structura bidimensionala

- $y_i = \sum_{j=1}^{n} a_{ij} * xj + y_{0i}$
- $y_1 = a_{11} * x_1 + a_{12} * x_2 + a_{13} * x_3 + a_{14} * x_4 + y_{01}$
- $y_2 = a_{21} * x_1 + a_{22} * x_2 + a_{23} * x_3 + a_{24} * x_4 + y_{02}$
- $y_3 = a_{31} * x_1 + a_{32} * x_2 + a_{33} * x_3 + a_{34} * x_4 + y_{03}$
- $y_4 = a_{41} * x_1 + a_{42} * x_2 + a_{43} * x_3 + a_{44} * x_4 + y_{04}$

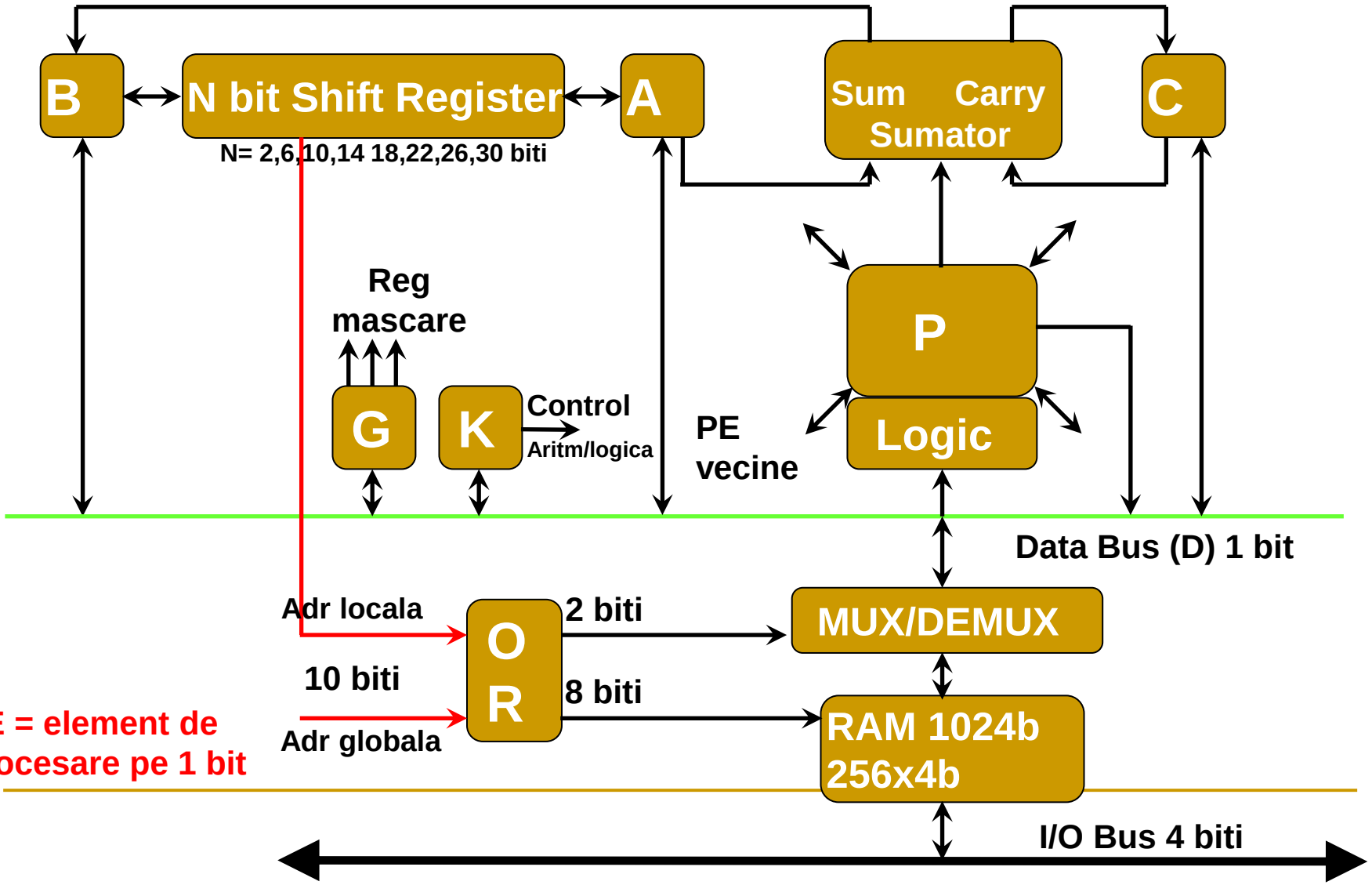




$$y_{out} = w * xin + yin$$

$$x_{out} = xin$$

Exemplu de masina sistolica, Arhitectura Masinii Blitzen – PE



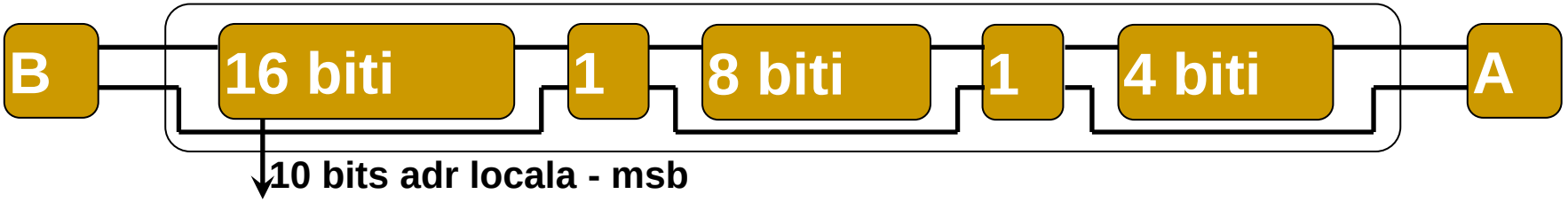
Elementele PE

- P – registru pe 1 bit
  - Operanzi
  - Operatori in functiile aritmetice
  - Utilizat la rutare: primeste ops de la PE-uri adiacente
- K – registru de control aritmetico-logic
  - Faciliteaza aparitia ops aritmetice inverse (+/-)
  - Algoritmi de impartiri (non-returning-division)
- G – registru de mascare
  - Se indica daca operatia se executa sau nu
- A & B – registru pe 1 bit
  - Preiau rezultatele de la sumator
  - Contribuie la suma

Elementele PE

A	P	C	B	C
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

- Sumator
  - ADD: A + P + C → B, C
  - HADD: A + C → B, C
- Shift Register (SR) – Registru de deplasare



  - N stagii = 2, 6, 10, 14, 18, 22, 26, 30
  - 30 biti + A & B = 32 biti
  - Bitul obtinut prin deplasare nu e neaparat salvat
  - La fiecare pas SR se deplaseaza cu o pozitie R/L
  - SR-ul poate fi sters

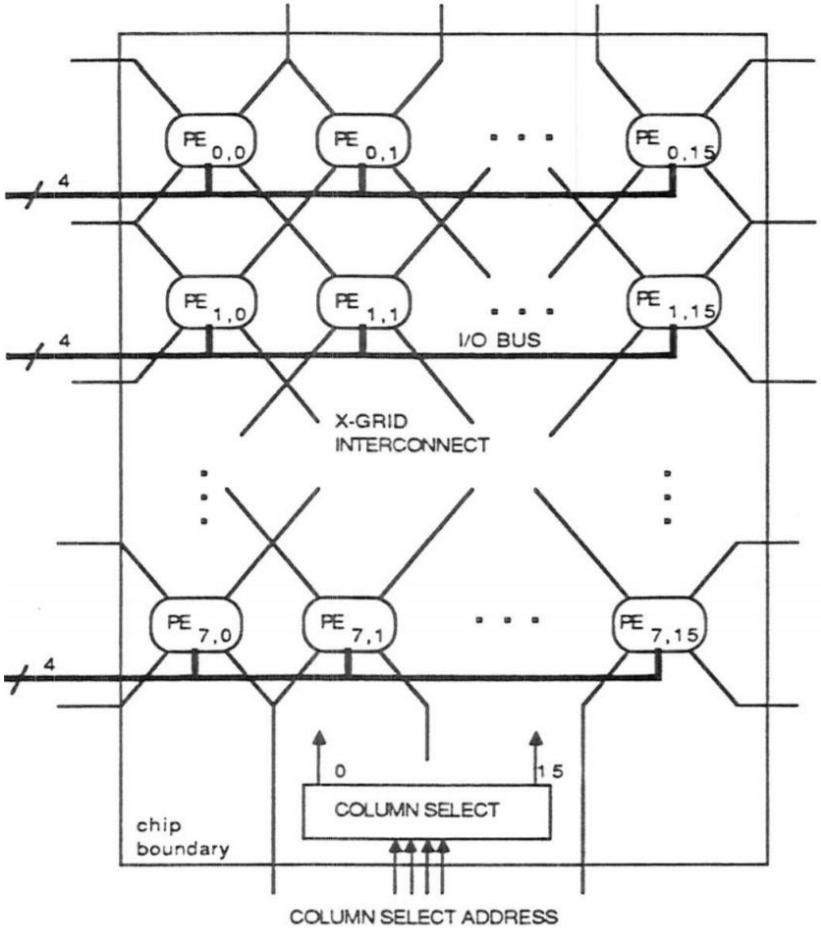
Memoria & I/O Bus

- Accesul la memoria RAM
  - Cu o adresa globala de forma 2^p (p e multiplu de 2)
  - Cu o adresa locala → specific masinii Blitzen (10 biti)
- I/O Bus
  - Formata din 4 biti si utilizata pentru conectarea PE-urilor la resurse externe de stocare a datelor
  - O magistrala I/O este folosita in comun de catre 16 PE-uri

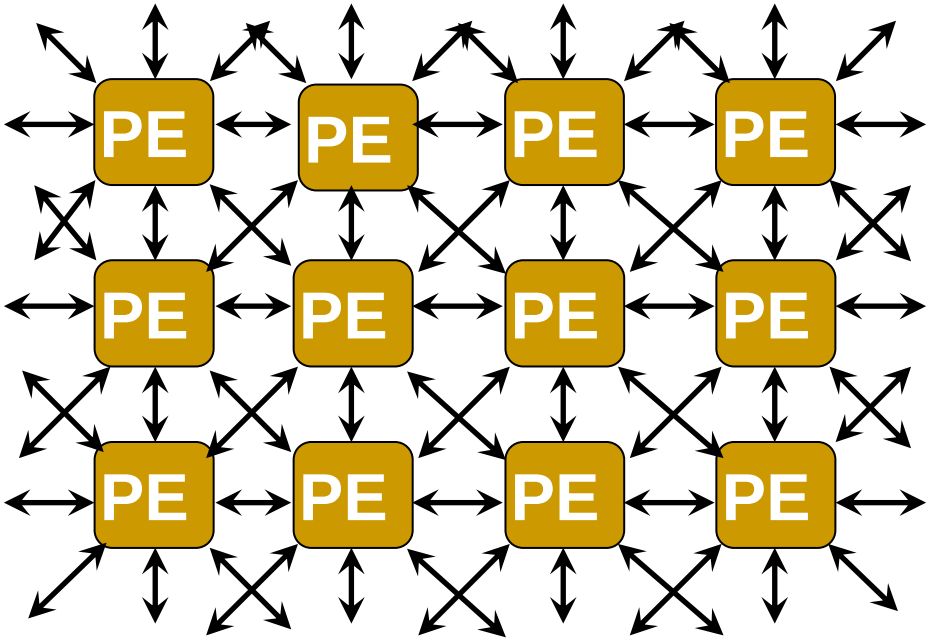


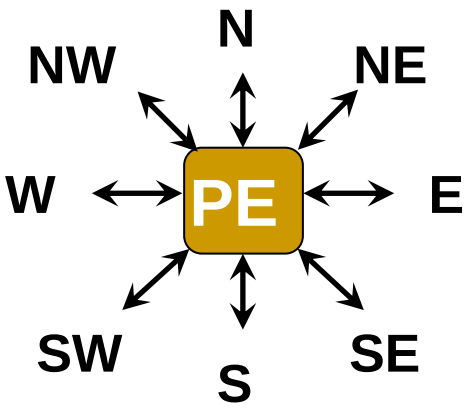
Chip-ul Blitzen

- Un Chip Biltzen este format din
  - 8 magistrale
  - 8 linii a cate 16 procesoare fiecare → 128 PE/chip, fiecare cu cate 1K de RAM
  - Fiecare PE functioneaza la 20MHz cu 8x4=32 biti/ciclu



Un Sistem Blitzen



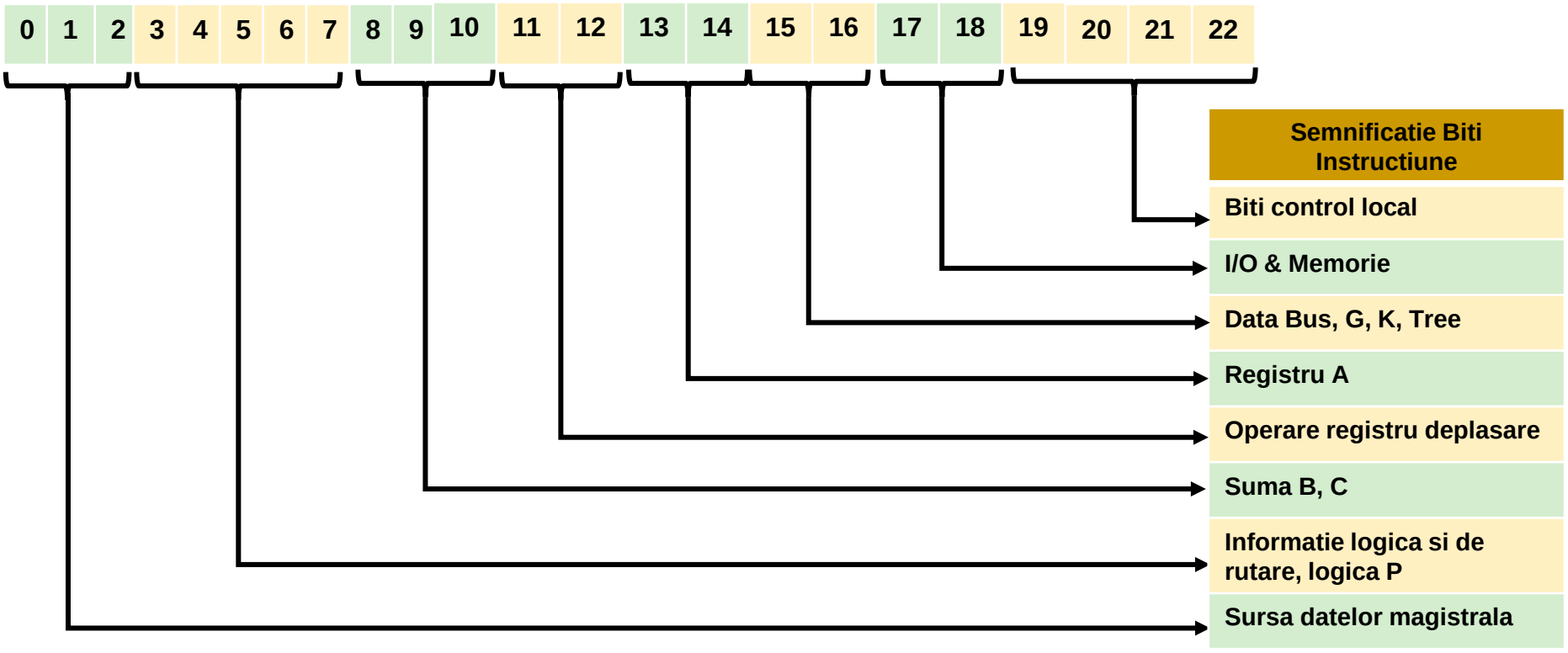


Structura X-Grid

- Sunt 8 directii de rutare
  - N, S, E, W, NE, SE, SW, NW
- Chip-urile Blitzen se grupeaza pe linii de 128 chip-uri → 128 x 128 PE = 16384PE (sistem Blitzen)

Formatul Instructiunilor

- La fiecare ciclu se executa o instructiune intr-un pipeline cu 3 stagii: Decodare, Broadcast, Executie
- Instructiunile sunt pe 23 de biti organizati astfel:



Semnificatia Bitilor Instructiune

- Bitii 0, 1, 2: sursa datelor de pe magistrala
  - Registrii A, B, C
  - Registrii G sau K
  - Registrul P
  - Memorie (cand este adresata memoria se executa un read 1b)
- Bitii 3, 4, 5, 6, 7:
  - Operatia logica ce trebuie operata in P
  - Operatia de rutare catre unul din cei 8 vecini
  - Configurarea registrului de deplasare (no assigns to P)
- Bitii 8, 9, 10: operatii executate de sumator
  - Suma & Carry din sumator catre registrii B si C
  - Pot fi utilizate si pentru incarcarea lui B & C

Semnificatia Bitilor Instructiune

- Bitii 11,12: operatii de deplasare/shiftare
  - Stanga (L)/Dreapta (R)/Stergere (D)
- Bitii 13,14: operatii cu registrul A
  - Setare directa
  - Setare cu continului registrului de deplasare/shiftare
- Bitii 15,16: rezultatul e transferat din magistrala de date
  - Catre registrii G, K
  - Catre OR-ul ierarhic (Tree)
- Bitii 17,18: prelucrarea informatiilor din
  - Memoria locala (R/W operations)
  - I/O Bus (I/O operations)

Semnificatia Bitilor Instructiune

- Bitii 19,20,21,22: control local
- Bit 19: operatii mascate & nemascate ale registrului P
  - Bit 19 = 0: operatiile P nemascate
  - Bit 19 = 1: operatiile P mascate;
- Bit 20 : identifica operatii mascate efectuate de memorie, sumator, SR, & registrii A, B sau C
  - Bit 20 = 0: operatiile nemascate
  - Bit 20 = 1: operatiile mascate;
- Bit 21 : poate identifica daca operatiile sunt transmise sau daca a avut loc un broadcast. Este folosit pt:
  - Bit 21 = 0: operatiile curente
  - Bit 21 = 1: op complementate
- Bit 22 : operatii cu
  - Bit 22 = 0: adresa globala
  - Bit 22 = 1: cu cei 10 biti din registrul de deplasare (adr locala)

Particularitati Blitzen

- PE-urile lucreaza la nivel de bit
- Registrul de Shiftare/Deplasare este bidirectional
- Blitzen-ul are memorie on-chip de 1k (1024 b)
- Este posibila mascarea separata a diverselor tipuri de operatii cu memoria sau cu SR (de deplasare)
- Exista un control local al adresei de memorie (Local Address Modifier)
- Bitul K de la Blitzen permite o executie a datelor de tip MIMD
- Masina Blitzen in anumite aplicatii este mult mai performanta ca MPP

Exemple

- Mai multe instructiuni elementare distincte se pot executa in acelasi timp:
  - ADD + preluare date din memorie + preluare de pe magistrala de date
- Exemple de microinstructiuni:
  - SET_C // C ← 1
  - ADD // A + P + C → B, C
  - MOV_BD // pune B pe magistrala de date D
  - MOV_MD(ADDR) // pune de la ADDR pe magistrala D
  - MOV_DP // pune in P continutul de pe D
  - ROUTE_E // trimite P catre E si incarca in P din W

Adunarea a doua Numere (8b)

```
#include "blitzen.h"     /*contine setul de instructiuni */
#define XADDR   100
#define YADDR   200
#define WADDR   300
#define NUMBITS 8
main (){
// suma W = X + Y;
// valori cum ar fi "X0" fac referinta la bitul 0 din X
set_route(GRID);
  load_file("in1.ism",XADDR,XADDR,8);   //citesc X si Y
  load_file("in2.ism",YADDR,YADDR,8);
   CLR_C;                  // clear C,
for (int i=0; i< NUMBITS; i++){
        MOV_MD(XADDR+i); //* A <- X(i), data de la addresa XADDR pe mag da date D
        MOV_DA;           // pune in A continutul de pe D
   END;
        MOV_MD(YADDR+i); //* P <- Y(i)
        MOV_DP;
   END;
        ADD;             //* adunare, rezultat in B
   END;
        MOV_BD;          //* W(i) <- B
        MOV_DM(WADDR+i);
   END;

     }

     save_file("sum.osm",WADDR,WADDR,8);  //salvare rezultat
     zyg_end();
}
```

Categorii de Algoritmi

- Algoritmii ce se preteaza la utilizarea masinii Blitzen fac parte din urmatoarele categorii
- 1. Embarrassingly Parallel Algorithms (algoritmi “rusinosi” paraleli)
  - Fiecare PE actioneaza independent (paralelism maxim);
  - Exemplu: Valoarea de prag (apartenenta la domeniu)
- 2. Near Embarrassingly Parallel Algorithms
  - Un PE are nevoie de informatii pe care le va primi de la alte PE
  - Exemplu: tratarea imaginilor (filtre etc); detectia contururilor

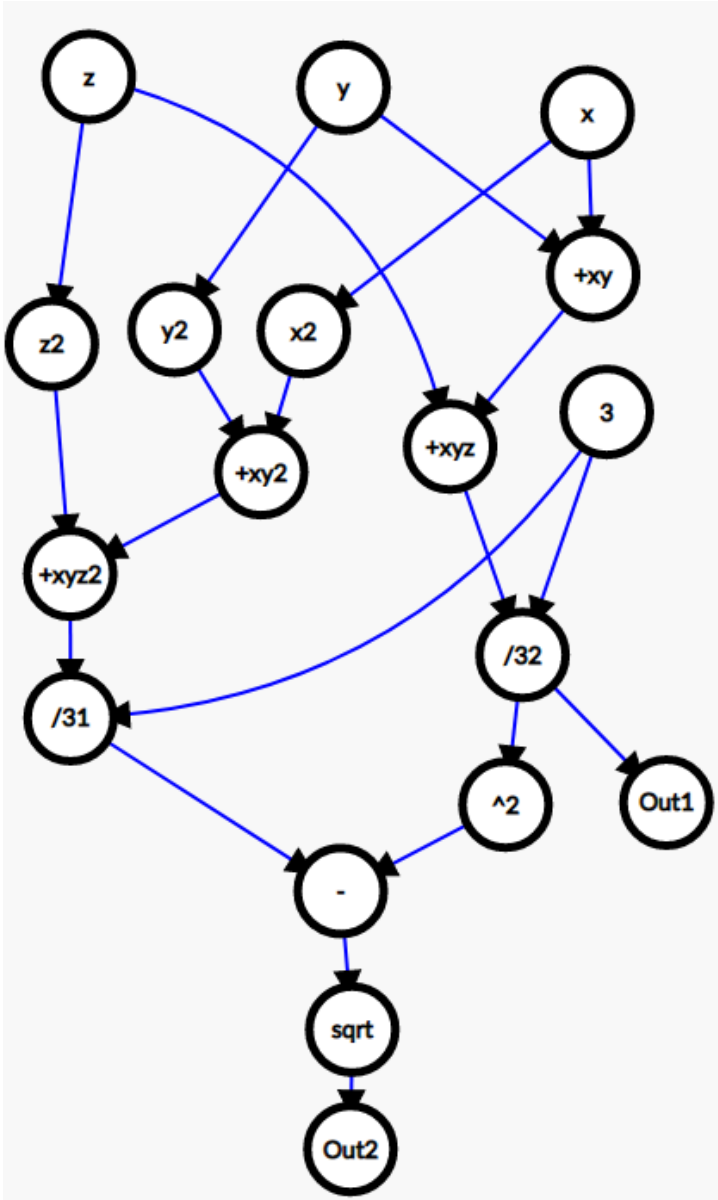
Regula de activare:

- O instrucțiune este activată (adică executabilă) dacă sunt disponibili toți operanzii.
- Observați că în modelul von Neumann, o instrucțiune este activată dacă este indicată de PC.
- Regula de calcul sau regula de declanșare, specifică când se execută efectiv o instrucțiune activată.
- O instrucțiune este declanșată (adică executată) când devine activată.
- Efectul declanșării unei instrucțiuni este prelucrarea datelor sale de intrare (operanzi) și generarea datelor de ieșire (rezultate).

Exemplu

- function Med_Dev:
- Calculeaza media si deviatia standard a trei variabile

```
function Med_Dev(x,y,z: real returns
   real,real);
 let
  Out1 := (x + y + z)/3;
  Out2 := SQRT((x**2 + y**2 + z**2)/3 - Out1**2);
 in
  Out1, Out2
 endlet
endfun
```



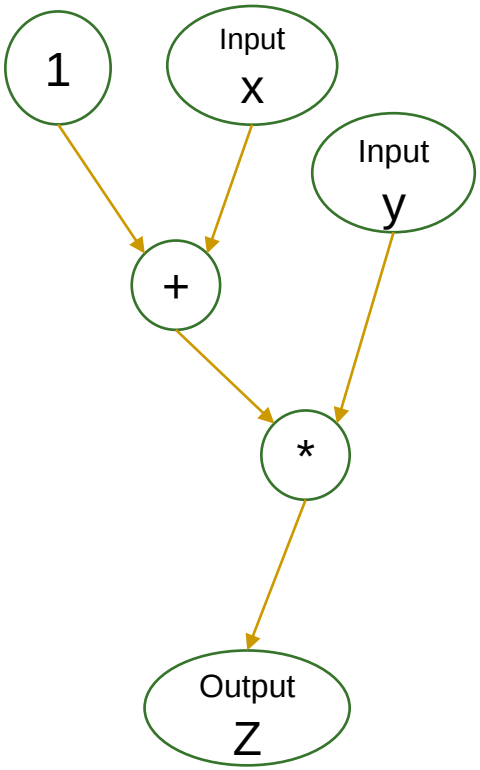
Modelul Data Flow

- In Model bazat pe flux de date (data flow):
- execuția unei instructiuni de prelucarea este efectuata doar de disponibilitatea operandului!
- Nu dispune de PC
- lipsesc cele două caracteristici ale modelului von Neumann care devin blocaje în exploatarea paralelismului

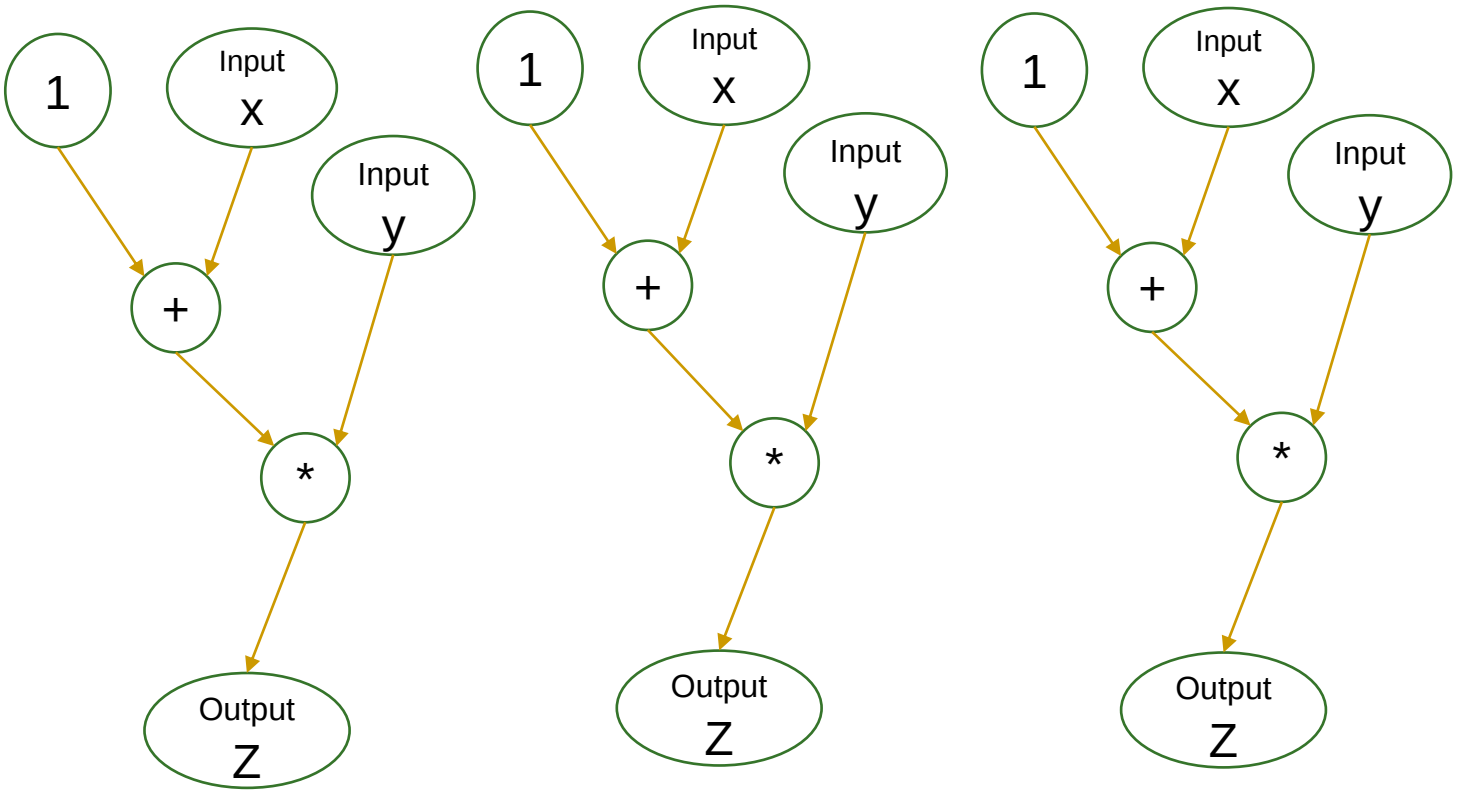
Exemplu

- Un program de flux de date este compilat într-un graf de flux de date care este un graf direcționat format din noduri, care reprezintă instrucțiuni și arce, care reprezintă dependențe de date între instrucțiuni.
- Grafiul fluxului de date este similar cu un graf de dependență utilizat în reprezentările intermediare ale compilatoarelor.
- În timpul executării programului, datele se propagă de-a lungul arcurilor în pachete de date, numite jetoane (token-uri).
- Acest flux de jetoane permite unele dintre noduri (instrucțiuni) sa inceapa executia și le declanșează.

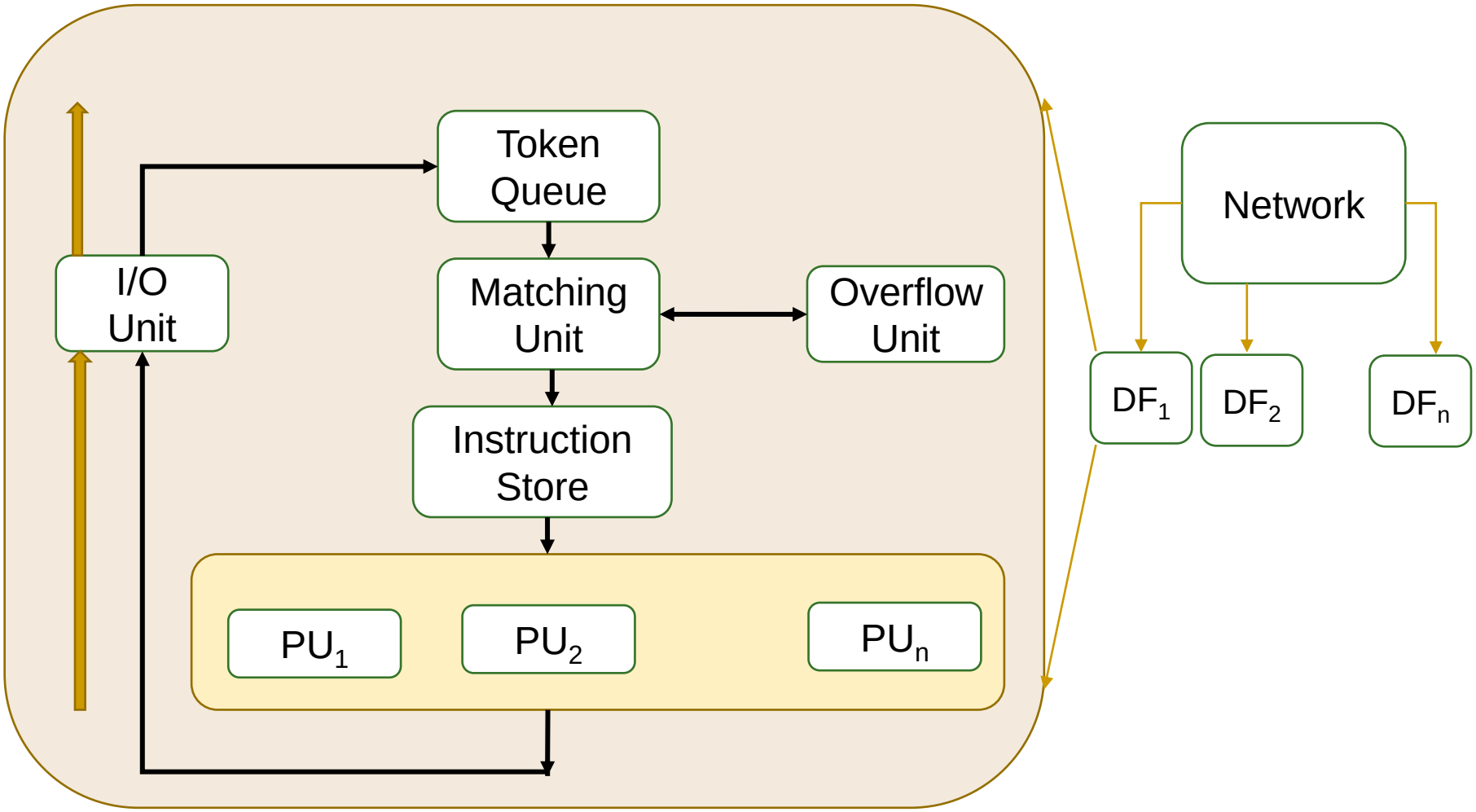
Z=(x+1)*y



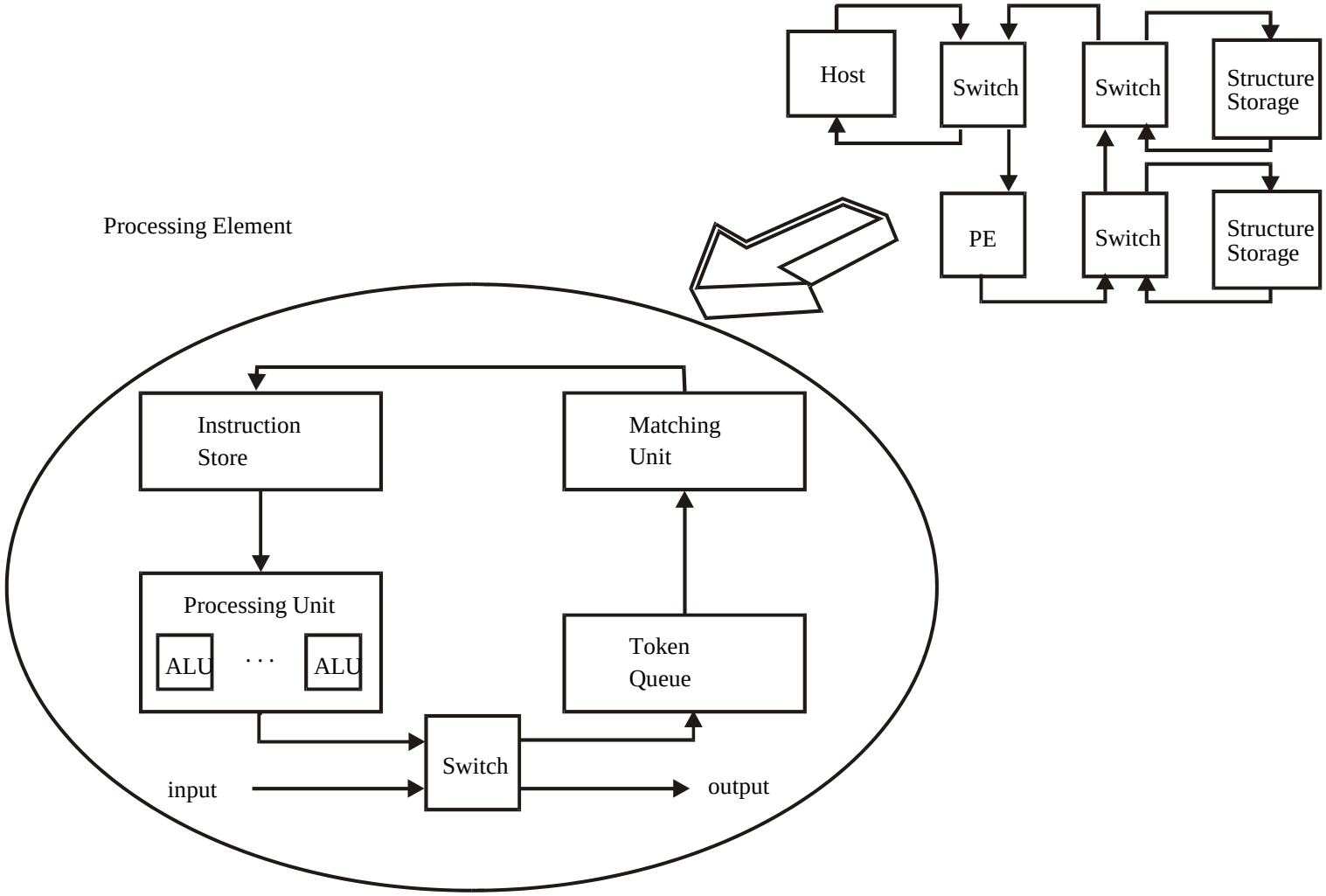
Executie


1
Input
x
Input
y
+
*
Output
Z
1
Input
x
Input
y
+
*
Output
Z
1
Input
x
Input
y
+
*
Output
Z

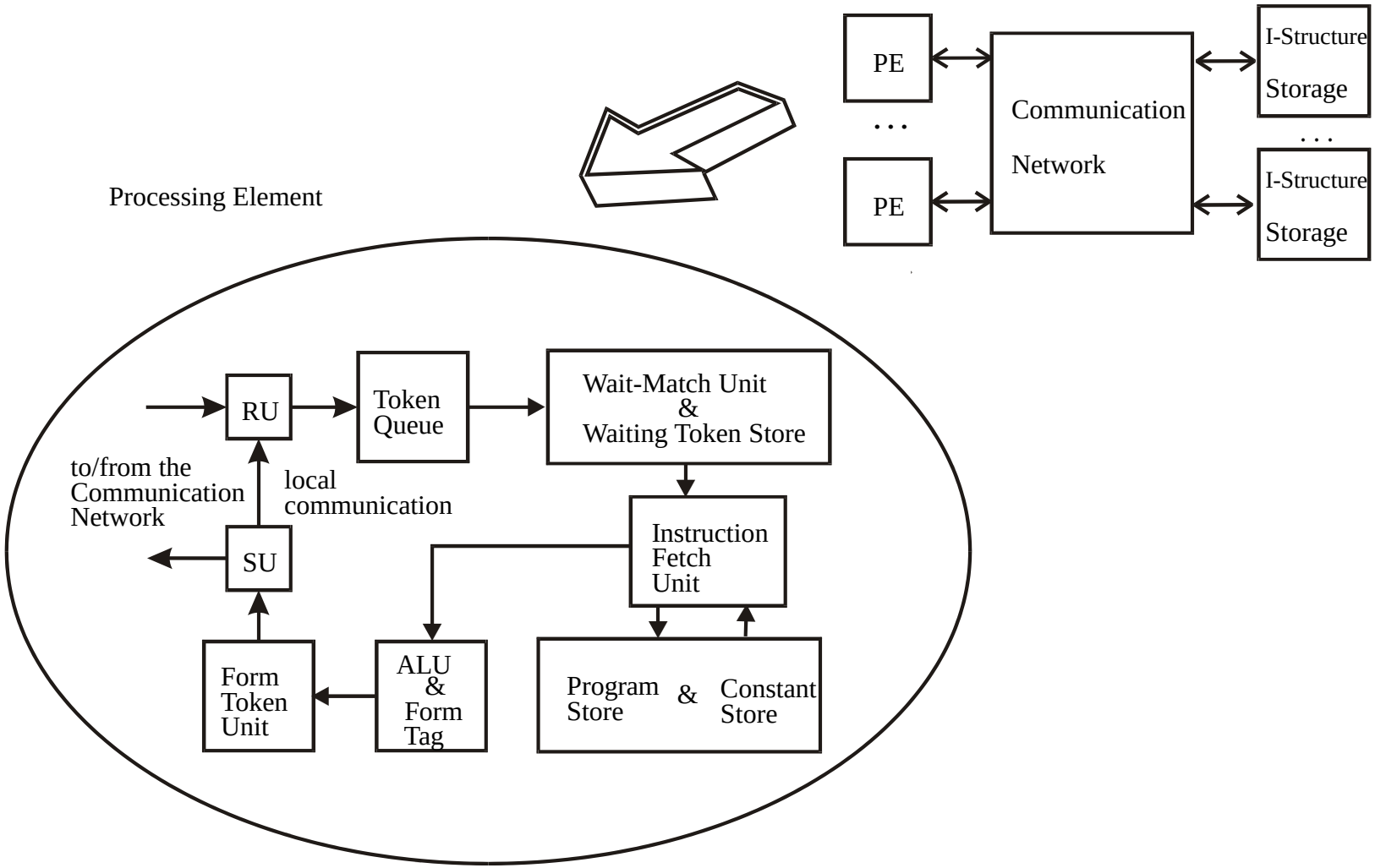
Manchester Dataflow Computer



Manchester Dataflow Machine



MIT Tagged-Token Dataflow Architecture



Caracteristici importante ale grafului asociat fluxului de date

- Funcționalitate:
  - *Evaluarea unui graf de flux de date este echivalentă cu evaluarea funcției matematice corespunzătoare.*
- Compozibilitate:
  - *Grafurile fluxului de date pot fi combinate pentru a forma grafuri noi.*

Dataflow Static

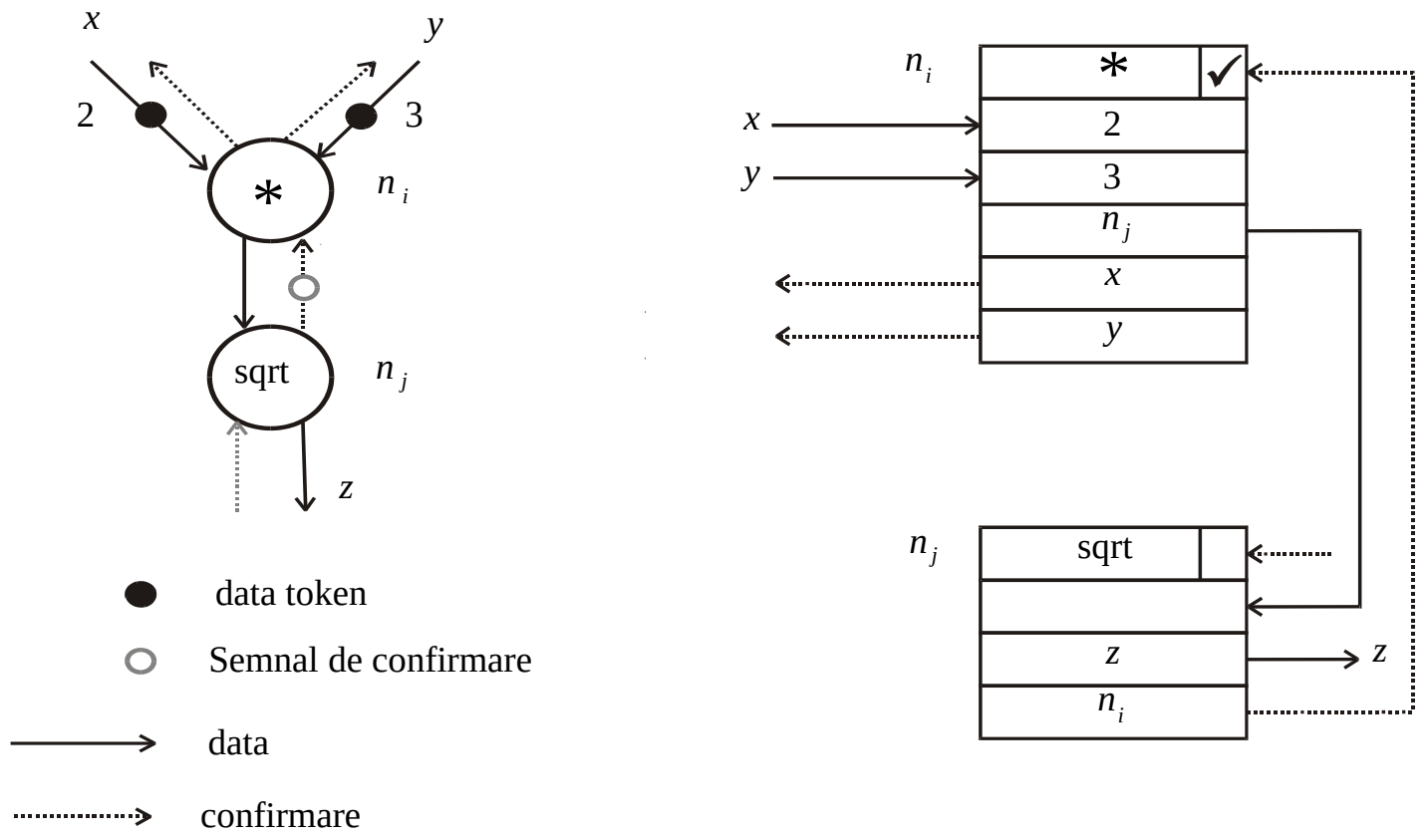
- Un graf de flux de date este reprezentat ca o colecție de șabloane de activitate, fiecare conținând:
  - codul instrucțiunii reprezentate,
  - sloturi de operand pentru păstrarea valorilor de operand,
  - câmpurile de adresă de destinație, referindu-se la sloturile de operand din șabloanele de activitate din secvente care trebuie să primească valoarea rezultatului.

Deficiente dataflow static:

- Iterațiile consecutive ale unei bucle pot fi doar executate sub forma de pipeline.
- Datorită token-urilor (jetoanelor) de confirmare, traficul de token-uri este dublat.
- Lipsa suportului de programare care este esențial pentru limbajul de programare modern
  - fără apeluri de procedură,
  - fără recursivitate.

Avantaj: model simplu de implementat

Dataflow : Template-ul de activitate



Semale de confirmare

- Abordarea statică a fluxului de date permite cel mult un simbol pe orice arc.
- Nu pot fi token-uri (jetoane) diferite destinate aceleiași destinații.
- Extinderea regulii de bază pentru declansare se face astfel:
  - Un nod activat este declanșat dacă nu există nici un indicativ pe oricare dintre arcurile sale de ieșire.
- Implementarea restricției prin semnale de confirmare (jetoane suplimentare), transmise de-a lungul arcurilor suplimentare de la nodurile consumatoare catre cele producătoare.
- Regula de declansare poate fi modificată la forma sa originală:
  - Un nod este declanșat în momentul în care devine activat.

- *Atentie: restrictiile structurale sunt ignorate presupunând resurse nelimitate!*

Dataflow Dinamic

- Fiecare iterație de buclă sau invocare de subprogram ar trebui să poată executa în paralel ca o instanță separată a unui subgraf reentrant.
- Replicarea este doar conceptuală.
- Fiecare *token* (jeton) are un *tag (o etichetă):*
  - adresa instrucțiunii pentru care este destinată valoarea particulară a datelor
  - informații de context
- Fiecare arc poate fi văzut ca o *multime* care poate conține un număr arbitrar de Token-uri cu tag-uri diferite.
- Regula de activare și declanșare este:
  - Un nod este activat și declanșat de îndată ce jetoanele cu etichete identice sunt prezente pe toate arcele de intrare.

- Restrictiile structurale sunt ignorate!

Interpretare de tip U

- Fiecare *token* constă dintr-un nume de activitate și date
  - numele activității cuprinde tag-ul.
- Tag (eticheta) are:
  - o adresă de instrucțiune **n**
  - câmpul contextual **c** care identifică în mod unic contextul în care urmează să fie invocată instrucțiunea,
  - numărul de inițiere **i** care identifică iterația de buclă în care are loc această activitate.
- De mentionat faptul ca **c** este el însuși un nume de activitate.
- Deoarece instrucțiunea de destinație poate necesita mai multe intrări, fiecare token poartă și numărul portului său de *destinație* ***p***.

- Token-ul se poate reprezenta prin $\langle c.i.n, data \rangle_p$

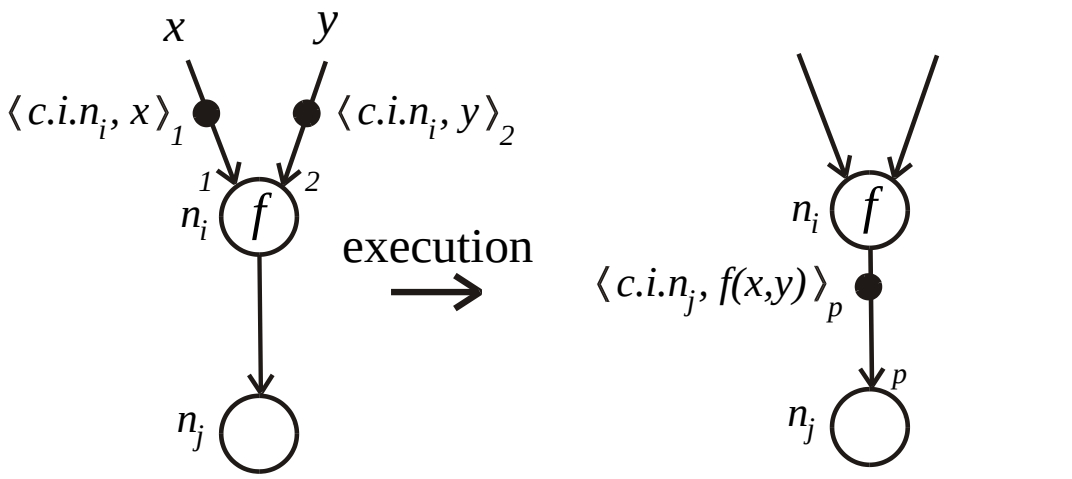
< c-context . i-iteratia . n-adresa , data >

Interpretare de tip U

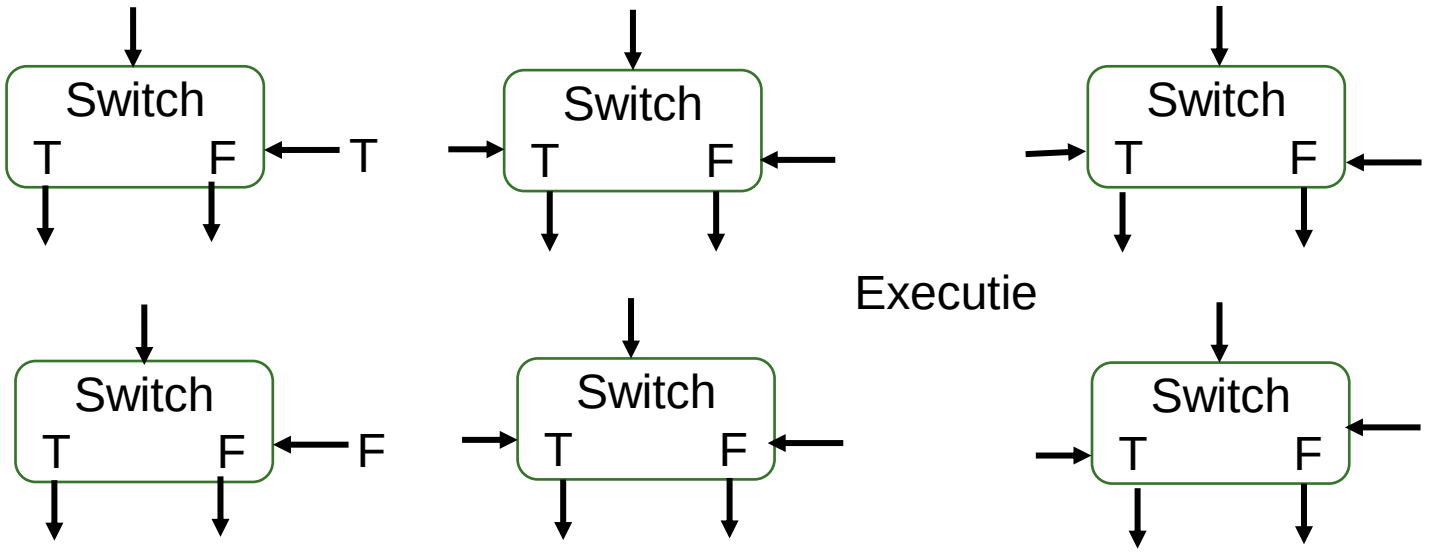
$\langle c.i.n, data \rangle_p$ *< c-context . i-iteratia . n-adresa , data >*

- dacă nodul n_i îndeplinește o funcție f și
- dacă portul p al lui n_j este destinația lui n_i,
- atunci avem

$$in: \{\langle c.i.n_i, x\rangle_1, \langle c.i.n_i, y\rangle_2\} \qquad out: \{\langle c.i.n_j, f(x,y)\rangle_p\}$$



Implementare decizie

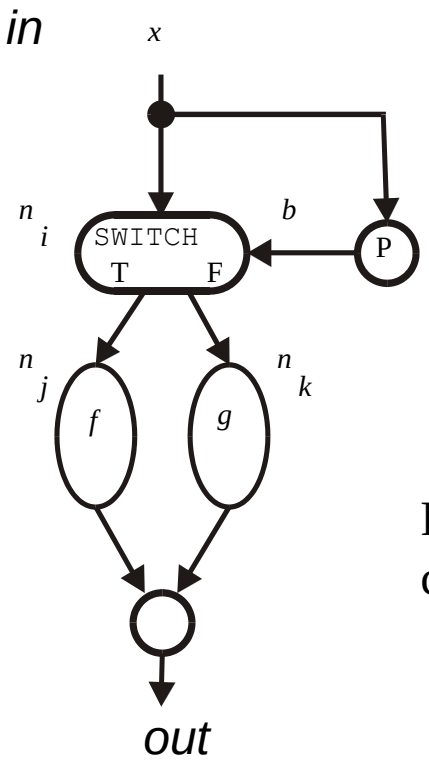


T/F sunt complementare
0/1 sau 1/0

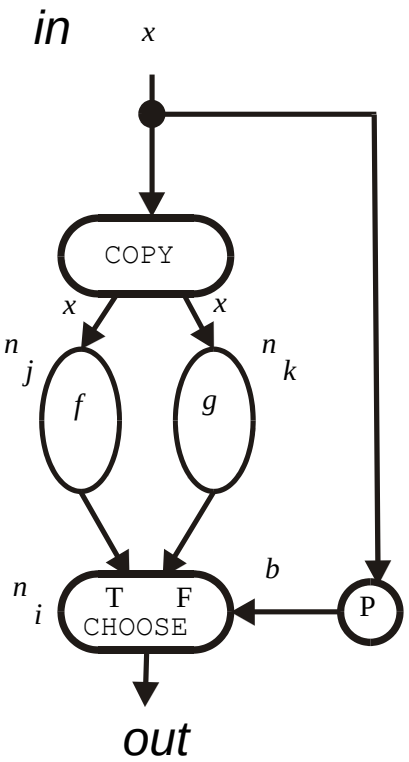
Implementare ramificatie (decizie)

$\langle c.i.n, data \rangle_p$ *< c-context . i-iteratia . n-adresa , data >*

$$in : \{\langle c.i.n_i, x\rangle_{data}; \langle c.i.n_i; b\rangle_{control}\} \qquad out : \begin{cases} \{\langle c.i.n_j, x\rangle\} if & b = T \\ \{\langle c.i.n_k, x\rangle\} if & b = F \end{cases}$$



Ramificatie clasica



Evaluare speculativa a ramificatiei

L, L^{-1}, D, and D^{-1} Operatori pentru impementarea Buclelor

$\langle c.i.n, data \rangle_p$ *< c-context . i-iteratia . n-adresa , data >*

$L: \quad in: \{\langle c.i.n_i, x \rangle\}$ $\qquad out: \{\langle c'.1.n_k, x \rangle\}, \quad unde \quad c' = \langle c.i.n_i \rangle$

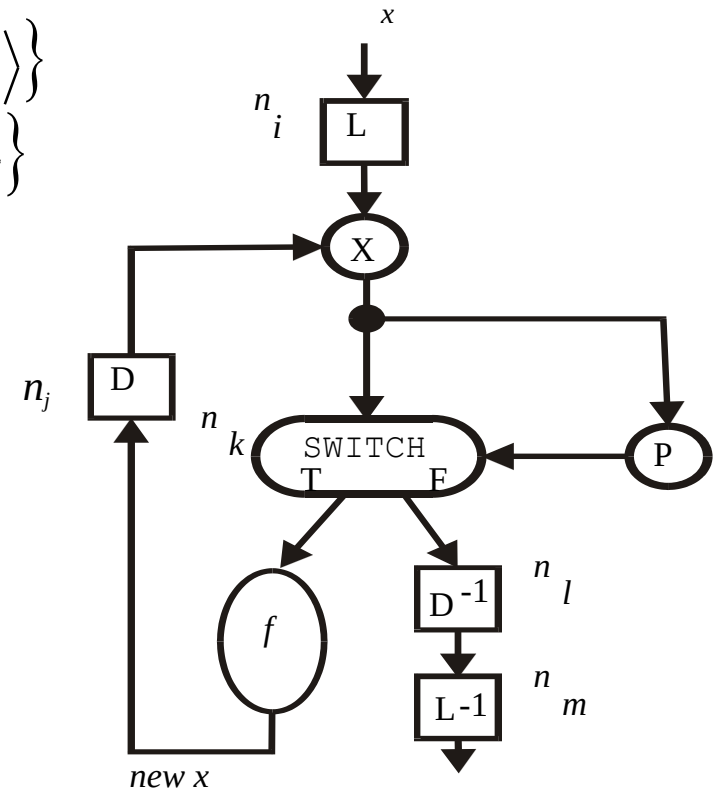
$D: \quad in: \{\langle c'.j.n_j, x \rangle\}$ $\qquad out: \{\langle c'.j+1.n_k, x \rangle\}$

$D^{-1}: \quad in: \{\langle c'.k.n_l, x \rangle\}$ $\qquad out: \{\langle c'.1.n_m, x \rangle\}$

$L^{-1}: \quad in: \{\langle c'.1.n_m, x \rangle\}$ $\qquad out: \{\langle c'.i.n_n, x \rangle\}$

- L: Initalizare, context bucla
- D: incrementeaza contor bucla
- D^{-1}: initializeaza contor la 1
- L^{-1}: reface contextul original

Nota: **c** este el însuși un nume de activitate



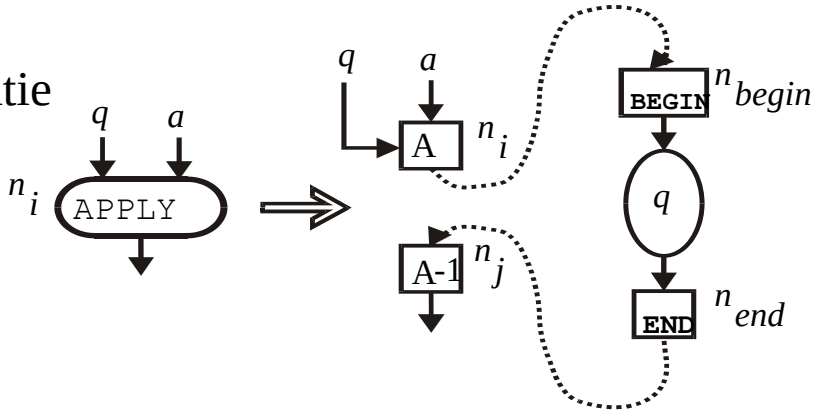
A, A^{-1}, BEGIN, and END Operatori pentru functii

- A: $in:\{\langle c.i.n_i, q\rangle_{func}, \langle c.i.n_i, a\rangle_{\arg}\}$ $out:\{\langle c'.1.n_{begin}, a\rangle\}$

  unde $c' = \langle c.i.n_j \rangle$ si n_j este adresa operatorului A^{-1}

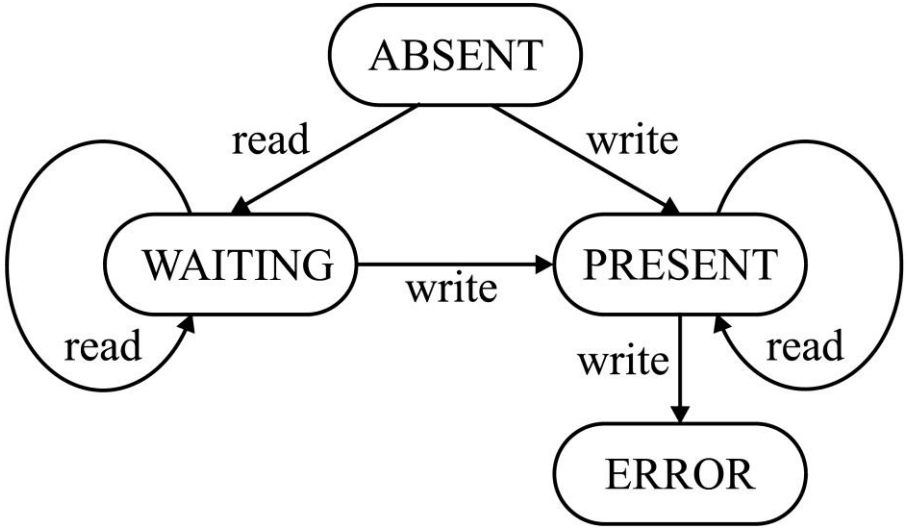
- END: $in:\{\langle c'.i.n_{end}, q(a)\rangle\}$ $out:\{\langle c.i.n_j, q(a)\rangle\}$

- A: creaza un nou context
- BEGIN: replica token-ul pentru fiecare ramificatie
- END: returneaza rezultatul
- A^{-1}: replica iesirea pentru succesori



Structura I

- Statutul fiecărui element al structurii I poate fi:
  - ***prezent**:* elementul poate fi citit, dar nu scris,
  - ***absent***: o cerere de citire trebuie amânată, dar este permisă o operație de scriere în acest element,
  - ***așteptare***: cel puțin o cerere de citire a elementului a fost amânată.



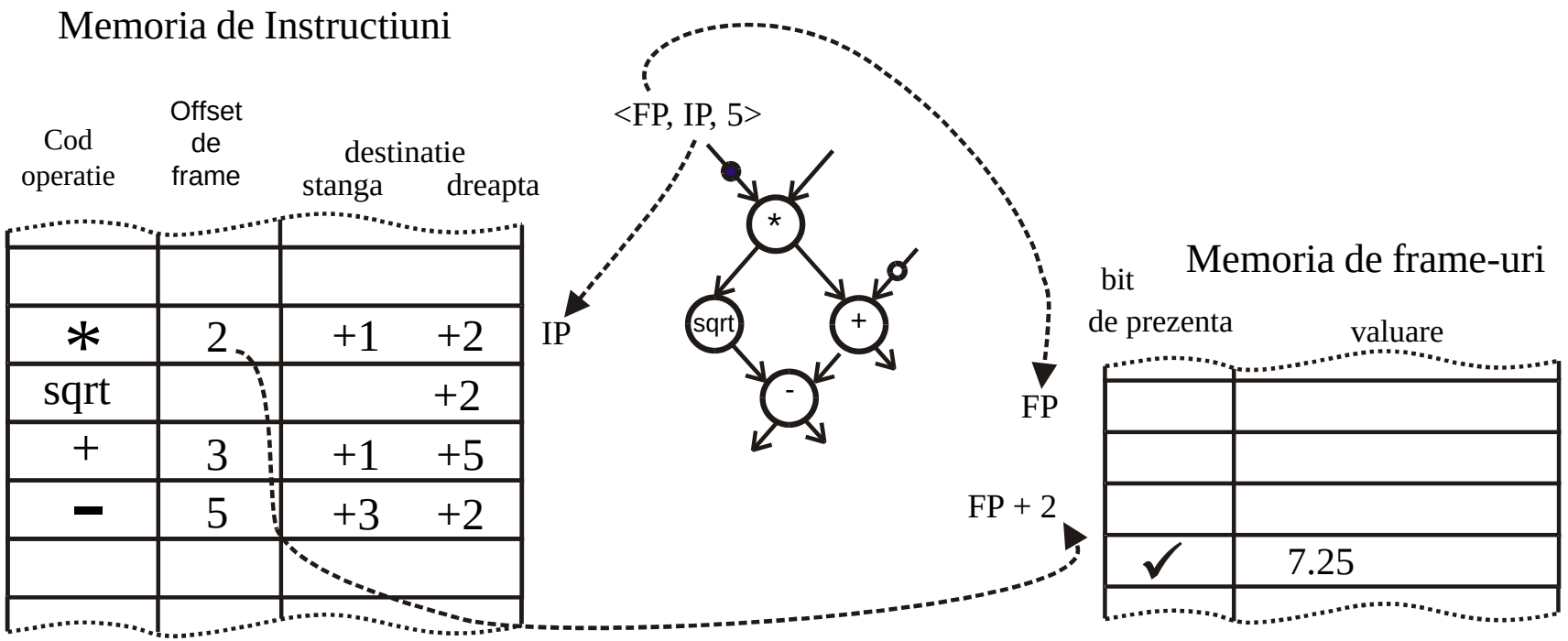
structura I

- Sunt definite trei operații elementare pe structurile I:
  - ***alocare:*** rezervă un număr specificat de elemente pentru o nouă structură I,
  - ***I-fetch:*** recuperează conținutul elementului structura - I specificat (dacă elementul nu a fost încă scris, atunci această operațiune este amânată automat),
  - ***I-store:*** scrie o valoare în elementul structura - I specificat (dacă acel element nu este gol, se raportează o condiție de eroare).
- Aceste operații elementare sunt utilizate pentru a construi noduri SELECT și ASSIGN.
- ***I-fetch*** este implementată ca operație de memorie în fază divizată:
  - o cerere de citire emisă unei structuri I este independentă în timp de răspunsul primit și, prin urmare, nu provoacă o așteptare de către PE emitent.

Implementare cu token explicit

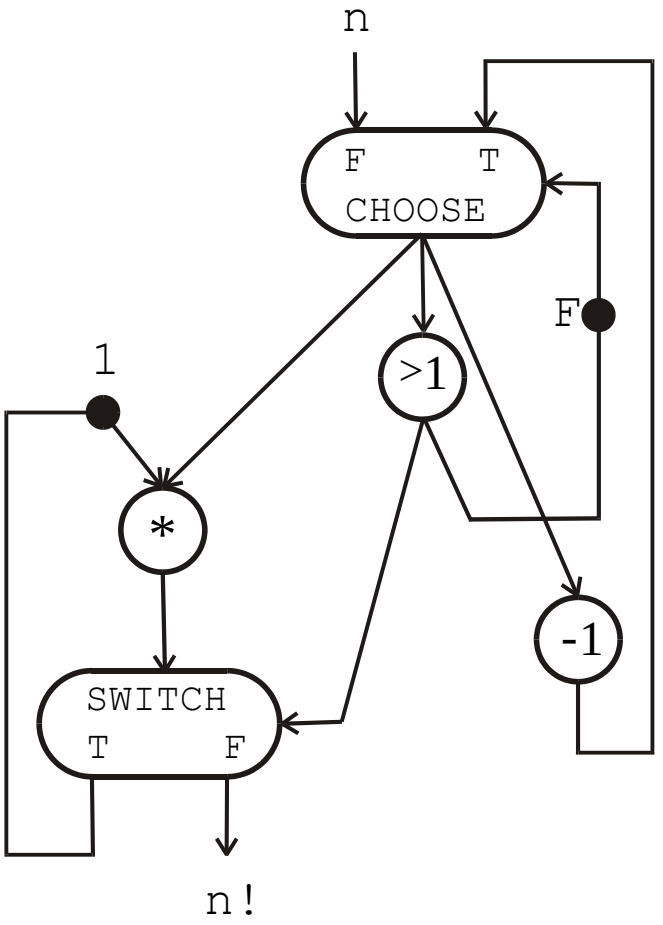
- **Scop**: implementarea eficientă a potrivirii token-urilor.
- **Ideea de bază**: se aloca un frame separat în memoria de frame-uri pentru fiecare iterație de buclă activă sau invocare de subprogram.
- Un ***frame*** este format din sloturi în care fiecare slot deține un operand care este utilizat în activitatea corespunzătoare.
- Deoarece accesul la sloturi este direct nu este necesară nici o căutare asociativă.

Explicit Token



Exemplu calcul factorial n!

```
initial j = n; k = 1
 while j > 1 do
   j = j - 1;
   k = k * j;
return k
```



Exemplu de prelucrare vectori

Input d,e,f

//a,b,c,d,e,f

//vectori de date

$C_0=0$

For i=1 to n

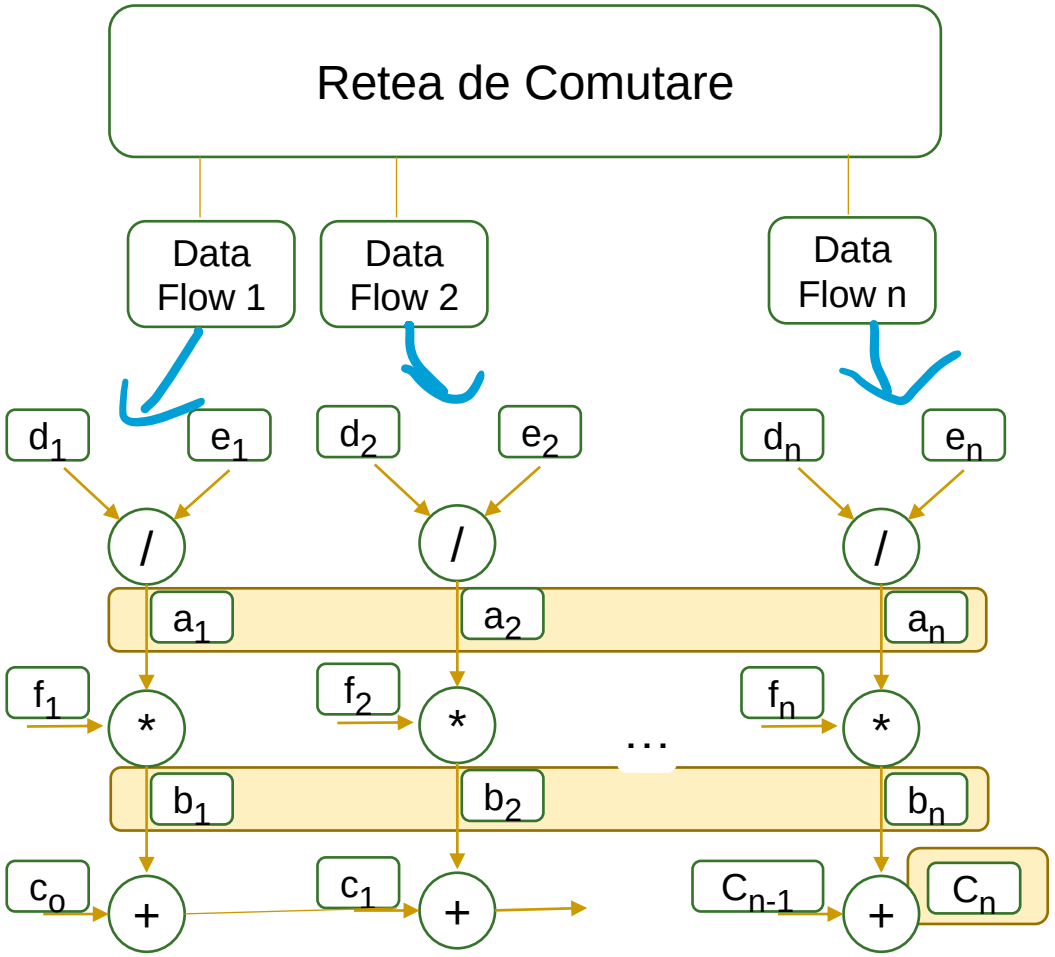
$a_i=d_i / e_i$

$b_i=a_i*f_i$

$c_i=b_i+c_{i-1}$

End for

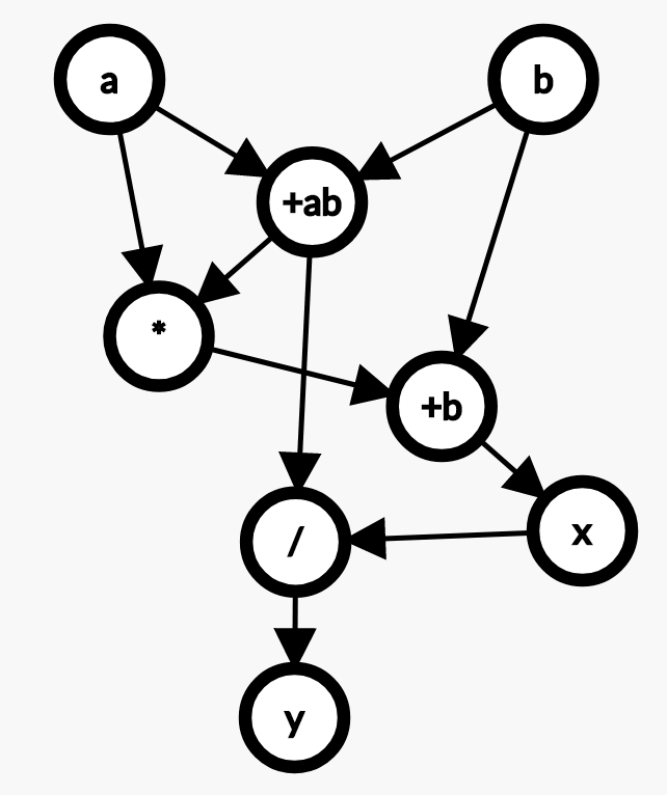
Output a,b,c_n



Ordinea instructiunilor irelevanta

Exemplu:

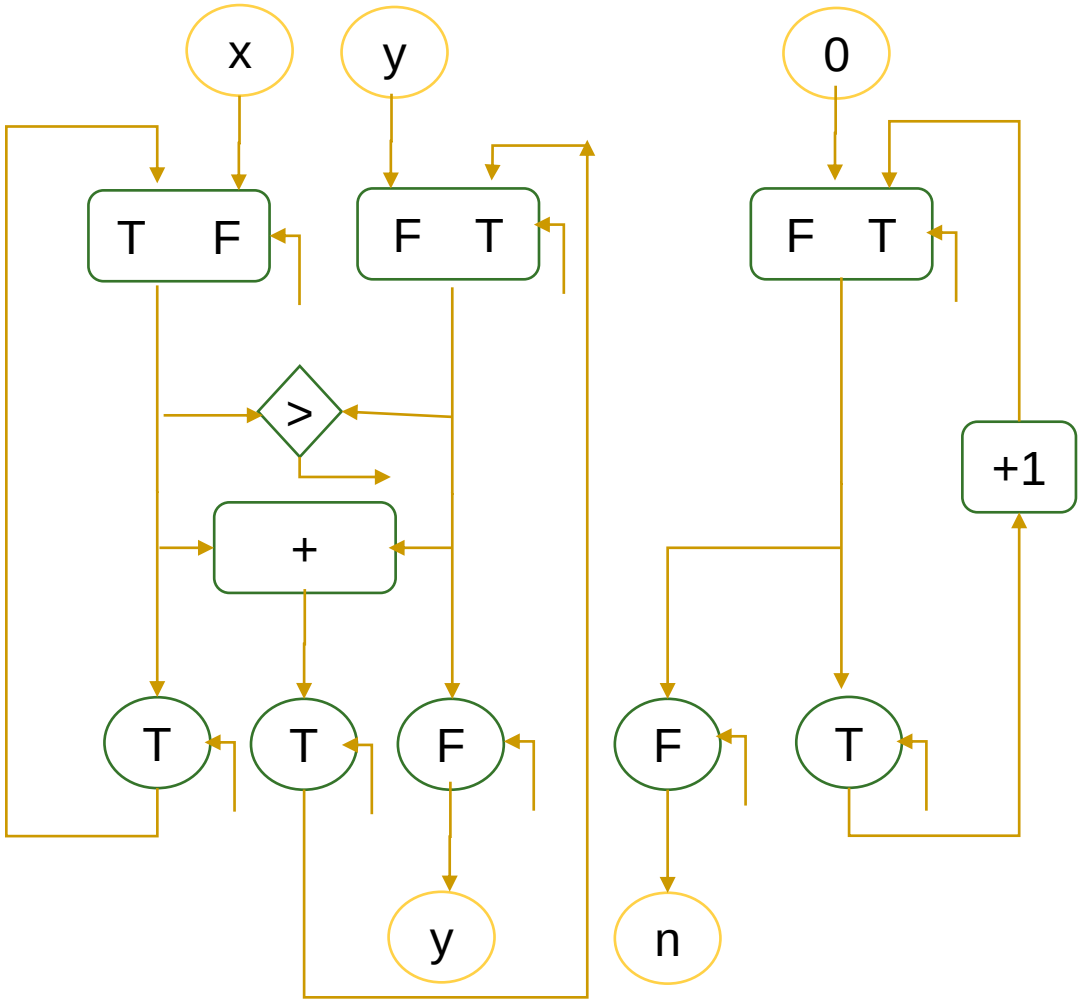
```
input a,b
  y = (a+b)/x
  x = (a*(a+b))+b
output y,x
```



Exemplu de bucla

```
input y,x
 n = 0
 while y<x do
    y =  y + x
    n =  n + 1
 end
output y,n
```

Nu s-a folosit array



Caracteristici de baza

- Executia instructiunilor este bazata pe disponibilitatea datelor
- Datele sunt pastrate in cadrul instructiunii
- Disponibilitatea datelor este verificata de o unitate specializata
- Tokenul associat datelor este verificat de o unitate specializata
- Executia instructiunilor este asincrona

Avantaje/ Dezavantaje

Avantaje

- Potential ridicat de parallelism
- Viteza crescuta de executie
- Usor de integrat comunicatia si sincronizarea

Dezavantaje

- Overhead-ul mare pentru control
- Manipularea dificila a structurilor de date

Unde se foloseste data flow in microprocesoarele actuale

Cea mai recentă generație de microprocesoare superscalare afișează o execuție dinamică în afara ordinii, denumită flux de date local sau flux de date micro.
Colwell și Steck 1995, în prima lucrare despre PentiumPro:
Fluxul instrucțiunilor in Arhitectura Intel este prevăzut cu mecanismul prin care instrucțiunile sunt decodificate în micro-operații (μops), sau serie de μops, iar aceste μops sunt plasate într-un set de microperatii executabile speculative, care se vor executa în afara ordinii de operații (out of order),
Vor fi executate pe principiul data flow (când operanzii sunt gata) și utilizate în ordinea programului sursă.

Fiecare instrucțiune este gata să fie executată imediat ce toți operanzii sunt disponibili.

Exemplu care evidentiaza diferite structuri elementare

Consideram adunarea a doi vectori

$X = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}$ $Y = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix}$

Z= X+Y

$z_i = x_i + y_i$

Fiecare componenta are mantisa si exponent (caracteristica)

$x_i = x_{ic}\ x_{im}$

Operatia de adunare are 4 operatii elementare organizate in pipeline

- Compararea caracteristicilor
- Deplasarea mantisei (daca este cazul)
- Adunarea mantiselor
- Normalizarea rezultatului
- Aceste operatii sunt independente – structura pipeline
- Consideram aceste operatii dureaza ~t (pentru simplificare)

CONTROLUL PROCESELOR – SARCINILOR CONCURENTE

*Obiective

- Acest capitol este dedicat tratării separate a celor mai reprezentative aspecte legate de controlul şi gestiunea sarcinilor concurente.
- Se pune un accent deosebit pe prezentarea
  - unor algoritmi de excludere mutuala
    - context centralizat
    - context distribuit
  - analiza situaţiilor de blocare
  - Sincronizarea sarcinilor.
- Problemele tratate în acest capitol asigură baza teoretică pentru analiza sau dezvoltarea unor sisteme software care sa lucreze
  - în regim de multiprogramare
  - multiprelucrare
- atât pentru sisteme monoprocesor, cât şi pentru sisteme cu prelucrare paralelă

Definiţii şi concepte de bază

- Activităţile paralele se pot desfăşura atât în sisteme de tip MIMD, SIMD cât şi în cadrul sistemelor convenţionale, de tip SISD.
- Un exemplu de activităţi paralele într-un sistem de calcul conventional de tip SISD, Von Neumann îl constituie efectuarea simultană a operaţiilor din UCP, cu cele din subsistemul de intrări/ieşiri prin DMA sau canal de I/E.
- De asemenea, activităţi paralele se întâlnesc la nivelul microoperaţiilor care constituie o microinstrucţiune în cadrul unei unităţi de comandă microprogramată.

În general pentru creşterea performanţelor unui sistem de calcul se prevăd mai multe procesoare de diferite tipuri, cum ar fi:

- procesoare centrale de prelucrare;
- procesoare de intrări/ieşiri;
- procesoare specializate (coprocesoare matematice, coprocesoare neurale, etc).

- Pe de altă parte mai multe programe sau părţi ale unui aceluiaşi program pot fi executate în paralel.
- Comunicaţia între acestea poate fi asigurată printr-un schimb de mesaje.
- Chiar dacă există un singur procesor (nu o structură MIMD), care este partajat între mai multe programe, vom admite că, din punct de vedere logic aceste programe se execută în paralel disputându-şi accesul la diferite resurse fizice ale sistemului.

Sarcina -proces secvenţial

Sarcina - proces secvenţial este o activitate care constă din execuţia, pe un procesor, a unui grup de instructiuni (de obicei indivizibil) asociat unui set de date specificat.

- Deşi pare că fiecare sarcina are procesorul şi datele sale, în realitate mai multe sarcini pot utiliza în comun
  - un procesor
  - o secvenţă de cod
  - o structură de date.

O *sarcină* poate fi specificată prin relaţia sa cu exteriorul:

- intrările necesare;
- funcţia executată;
- ieşirile generate;
- starea la un moment dat;
- timpul de execuţie.

- Comportarea intrinsecă, internă, nu va fi specificată decât în situaţii excepţionale, pentru a facilita înţelegerea interacţiunii cu celelalte sarcini.
- Une sarcini i se asociază două evenimente:
  - SI - iniţiere sarcină;
  - SF - terminare sarcină.
- Dacă notăm cu t(·) timpul de apariţie a unui eveniment, vom considera că:

  $t(SF) - t(SI) \neq 0$ şi finit

cu condiţia ca toate resursele necesare execuţiei lui S să fie disponibile în momentul t(SI)

Vom considera sarcinile neinterpretate, ceea ce este echivalent cu faptul că pentru o mulţime dată de sarcini să ne intereseze secvenţele evenimentelor de iniţiere şi terminare fără să ne preocupe particularităţile de execuţie ale sarcinilor.

În ceea ce priveşte granularitatea unei sarcini, aceasta poate fi:

- program -JOB;
- modul de program - task;
- procedură - sarcina;
- secventa - thread
- instrucţiune;
- microinstrucţiune;
- microoperaţie.

Sistemul de calcul în care se execută sarcinile constă dintr-o mulţime de resurse:

- procesoare;
- dispozitive de I/E;
- biblioteci de programe;
- proceduri;
- variabile;
- fişiere de date, etc.

şi este caracterizat de o mulţime Σ de stări.

În funcţie de granularitatea sarcinilor, procesorul poate fi:

- calculator universal;
- UCP într-un sistem multiprocesor;
- UAL;
- unitate de comandă;
- microsecvenţiator într-o unitate de comandă.

Atât posibilităţile sistemului, cât şi funcţiile sarcinilor date sunt reprezentate prin tranziţii permise în Σ : $\sigma_0\sigma_1\sigma_2$..... σ_n–spatiul starilor,

de forma $\sigma_i \rightarrow \sigma_j$, (σ_i, σ_j Є Σ) definite pentru iniţierea şi terminarea sarcinilor.

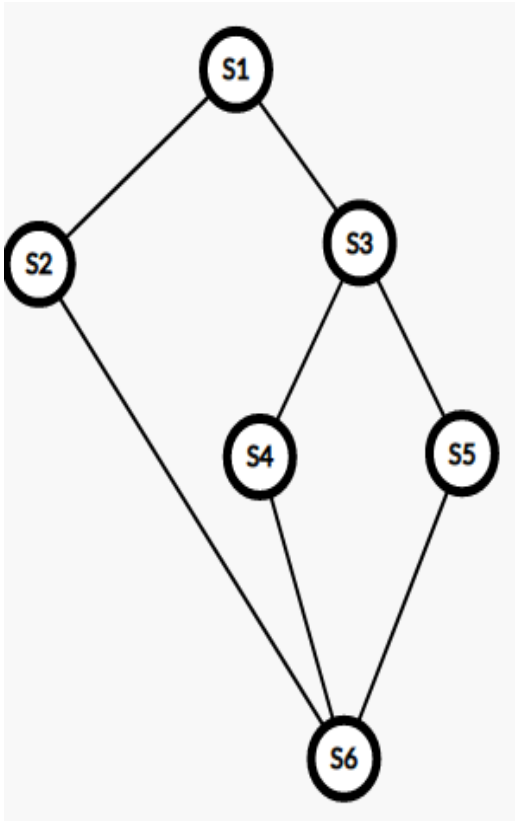
Iniţierea unui sarcina corespunde unei tranziţii de stări care reflectă:

- preluarea resurselor necesare;
- iniţializarea stării sistemului (a resurselor);
- citirea parametrilor de intrare.

Terminarea unei sarcini corespunde unei tranziţii de stări care reflectă:

- eliberarea sau suspendarea temporară a utilizării resurselor;
- scrierea parametrilor de ieşire;
- dacă valori ale parametrilor sau stări interne sunt asociate cu resursele, terminarea unei sarcina trebuie însoţită de valoarea stării resurselor.

Fie $\boldsymbol{S} = \{ S_1, S_2, ..., S_n \}$ o mulţime de sarcini iar **<** o relaţie de ordine parţială, nereflexivă, pe ***S*** (denumită şi relaţie de precedenţă pe mulţimea de sarcini ***S***).
Perechea **C** = (***S*** , **<**) este denumită **sistem de sarcini.**



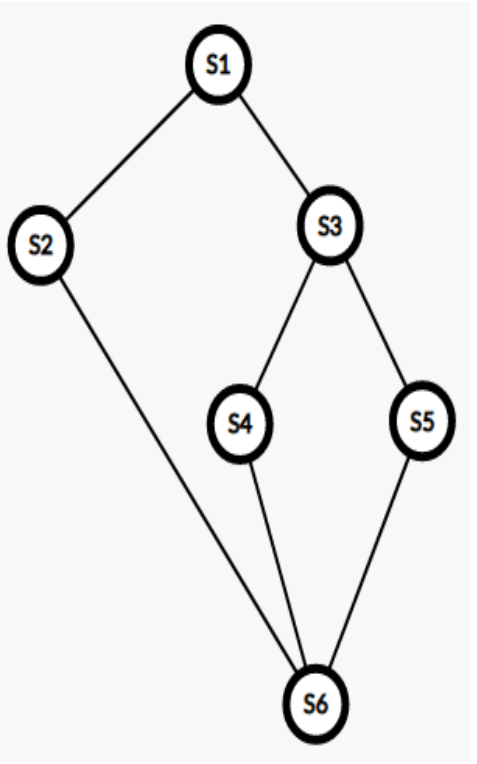
- Relaţia **<** reprezintă precedenţele în execuţia sarcinilor.

Astfel, S_i **<** S_j

- înseamnă că S_i trebuie terminată înainte de iniţierea lui S_j.

Sarcinile din ***S*** sunt independente dacă pentru **C** = (***S*** , **<**)
avem **<** = Φ; unde Φ este mulţimea vidă.

- O reprezentare grafică convenabilă a unui sistem de sarcini se obţine utilizând grafuri aciclice direcţionate, *GAD*.
- Sracinile sunt reprezentate prin noduri iar relaţia de precedenţă prin mulţimea arcelor.
- Relaţia de precedenţă dată de mulţimea arcelor poate fi definită ca fiind cea mai mică relaţie în ***S*** a cărui închidere prin tranzitivitate este **<**.
- Deoarece **<** reprezintă o ordonare în timp a sarcinilor, vom spune că două sarcini S_i şi S_j, pot fi concurente dacă şi numai dacă ele sunt independente, adică între ele nu există o relaţie de precedenţă.

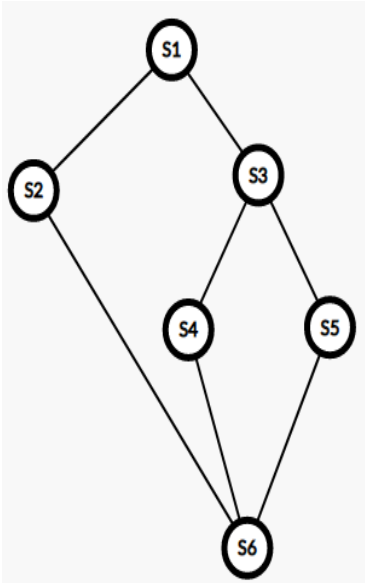


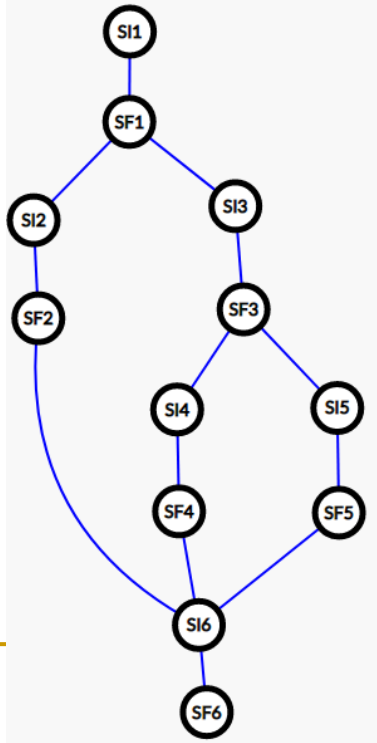
*Exemplu:

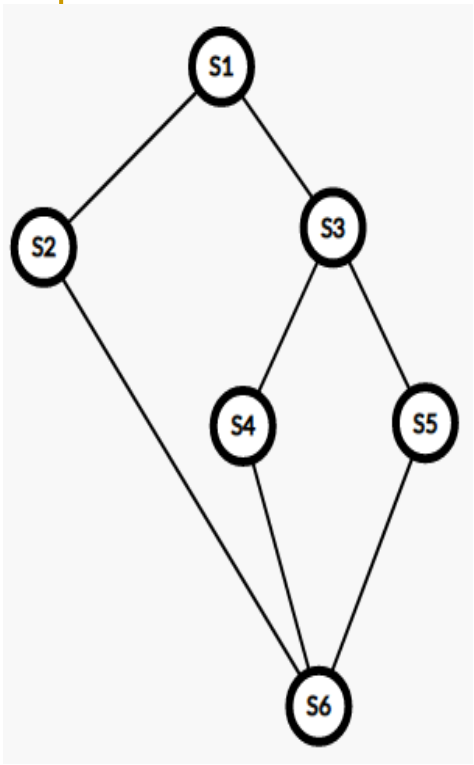
- Două sarcini S_i si S_j sunt independente dacă S_i nu este nici predecesor, nici succesor al lui S_j.
- O sarcină S din sistemul de sarcini **C** este pe nivelul k, sau are nivelul k, dacă cea mai lungă cale de la S la un nod terminal are lungimea k.
- Un nod terminal are nivelul 1.

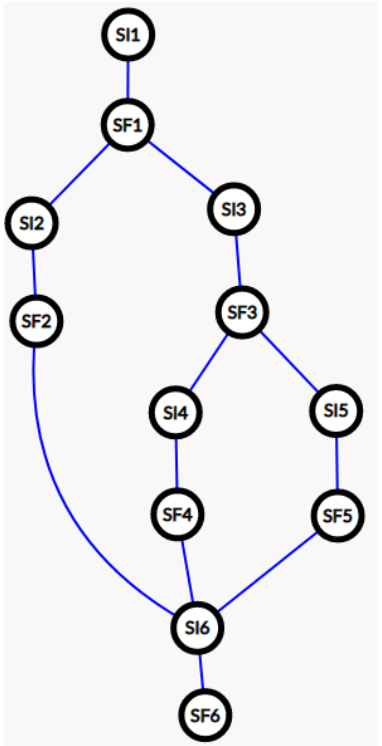
O secvenţă de execuţie a unui sistem de n sarcini, **C** = (***S***, **<**) este orice şir $\alpha = a_1\ a_2 \ldots\ldots a_{2n}$ de evenimente de iniţiere şi terminare a sarcinilor cu respectarea relaţiilor de precedenţă impuse de **<**.

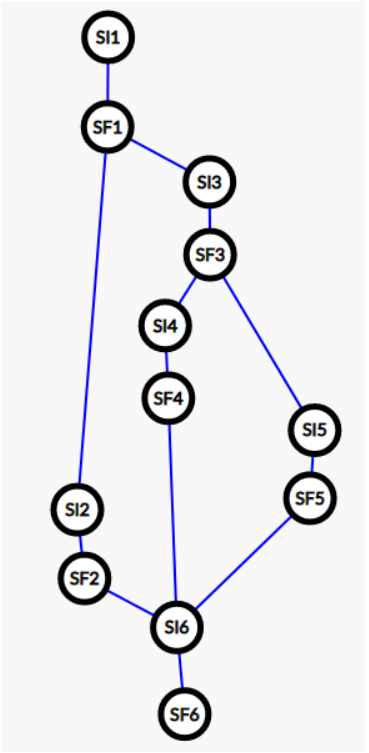
- Altfel spus:
  - pentru orice S Є ***S***, SI – S initiat şi SF - S terminat apar o singură dată în α ;
  - dacă a_i= SI şi a_j= SF, atunci i < j ;
  - dacă a_i= SF_k şi a_j= SI_l ,cu S_k **<** S_l atunci i < j ;

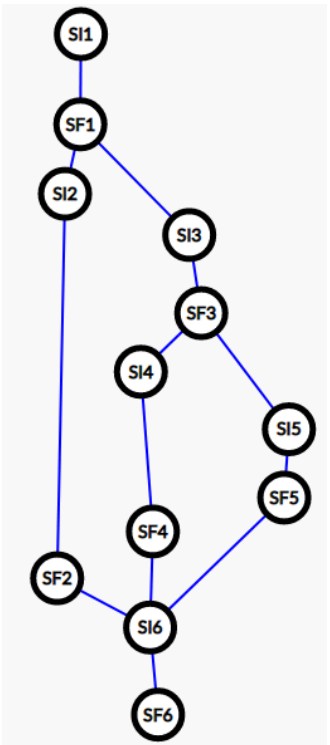










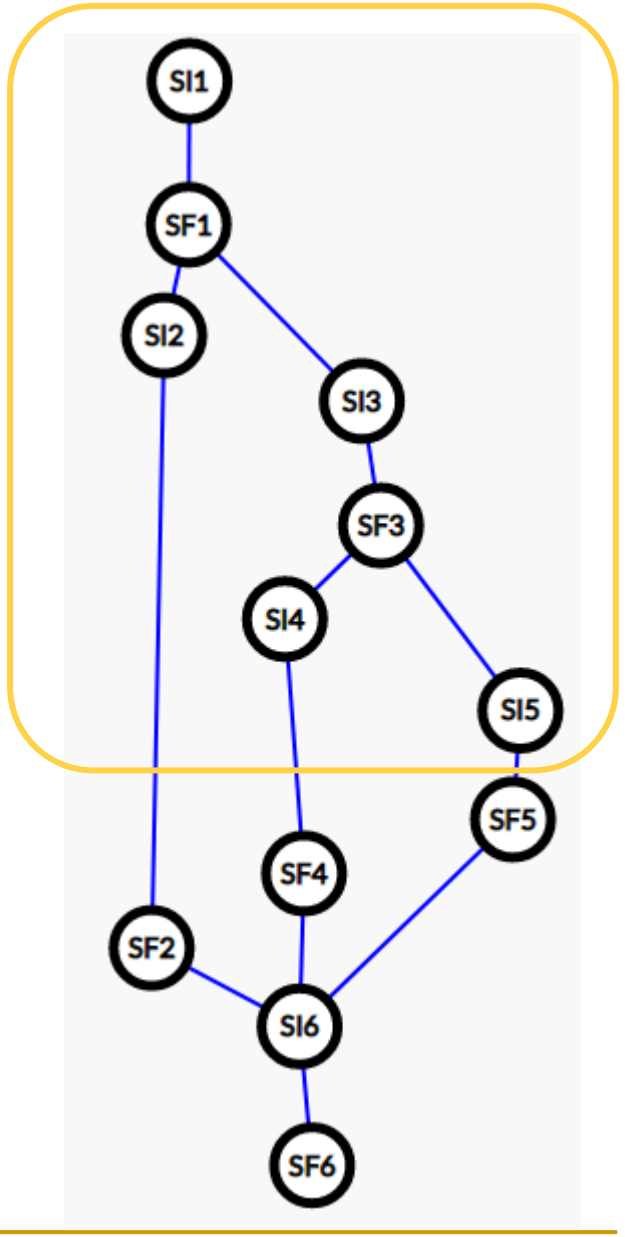


Sistem de sarcini reprezentat prin graf

Exemple de secvenţe de execuţie pentru sistemul de sarcini

- C=(S, **<**)
- S={S1,S2, S3,S4,S5,S6}
- reprezentat prin graful
- $\alpha_1 = SI_1\ SF_1\ SI_3\ SF_3\ SI_4\ SF_4\ SI_5\ SF_5\ SI_2\ SF_2\ SI_6\ SF_6$
- $\alpha_2 = SI_1\ SF_1\ SI_2\ SI_3\ SF_3\ SI_4\ SI_5\ SF_5\ SF_4\ SF_2\ SI_6\ SF_6$
- $\alpha_3 =$

secvență de execuție parțială



O secvenţă de execuţie parţială este orice prefix al unei secvenţe de execuţie.

Mulţimea secvenţelor de execuţie reprezintă mulţimea tuturor secvenţelor de evenimente (de iniţiere şi terminare) care conduc la finalizarea lui **C** respectându-se relaţia **<**.

Un sarcină S este activă după o secvenţă de execuţie parţială

$\alpha_p = a_1 a_2 \ldots\ldots a_k$, dacă există $i \leq k$ astfel că

$a_i = SI$, dar pentru orice $j \leq k$, $a_j \neq SF$

Ex

$\alpha_p = SI_1\ SF_1\ SI_2\ SI_3\ SF_3\ SI_4\ SI_5$

- $\alpha_2 = SI_1\ SF_1\ SI_2\ SI_3\ SF_3\ SI_4\ SI_5\ SF_5\ SF_4\ SF_2\ SI_6\ SF_6$

Fie Σ spaţiul stărilor pentru un sistem de sarcini.

- Secvenţa stărilor corespunzătoare secvenţei de execuţie

$$\alpha = a_1\ a_2 \ldots\ldots a_{2n}$$

este dată prin:

$$\sigma = \sigma_0\sigma_1\sigma_2 \ldots\ldots\ldots \sigma_{2n}, \quad \sigma_j \in \Sigma,\ 0 \leq j \leq 2n\ ,$$

iar σ_0 este starea iniţială dată.

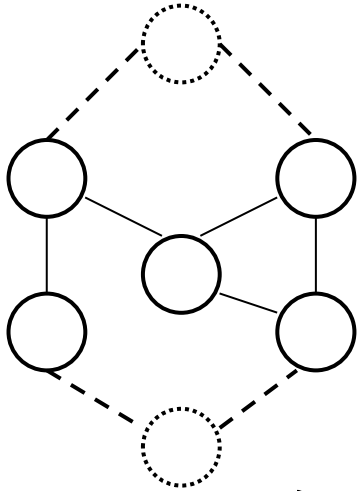
- Tranziţia stării definită de evenimentul a_i este reprezentată de:

$$\sigma_{i-1} \longrightarrow \sigma_i$$

- O tranziţie pentru o pereche de stări-eveniment (σ,a) va fi definită numai dacă evenimentul a poate să apară când sistemul este în starea σ.

Un sistem de sarcini reprezentat printr-un graf cu un sigur nod iniţial şi un singur nod terminal se numeşte închis.

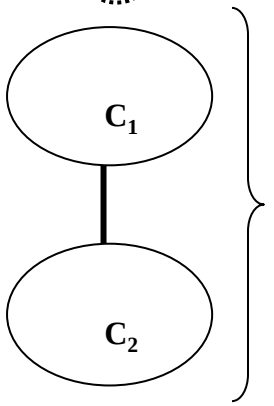
- Dacă sistemul de n sarcini nu este închis se pot prevedea sarcinile S_0 şi S_{n+1} inoperante pentru a obţine un sistem închis.



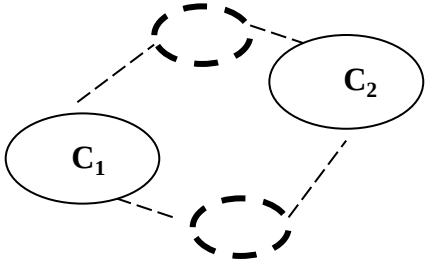
- Două sisteme de sarcini închise $\mathbf{C}_1$ şi $\mathbf{C}_2$ pot fi concatenate ($\mathbf{C}_1.\mathbf{C}_2$), graful asociat obţinându-se prin introducerea unui arc de la nodul terminal al lui $\mathbf{C}_1$ la nodul iniţial al lui $\mathbf{C}_2$.
- O secvenţă de execuţie a lui ($\mathbf{C}_1.\mathbf{C}_2$) este orice şir

$$\alpha = \alpha_1.\alpha_2$$

de secvenţe de execuţie α_1 a lui $\mathbf{C}_1$, α_2 a lui $\mathbf{C}_2$.



Combinarea paralelă a două sisteme de sarcini $\mathbf{C}_1$ şi $\mathbf{C}_2$, $\mathbf{C}_1 \mid \mathbf{C}_2$, care nu au sarcini în comun,($\forall S_i \in \mathbf{C}_1$ şi $\forall S_j \in \mathbf{C}_2$, $S_i \neq S_j$) constă în simpla lor reuniune.



- Graful lui $\mathbf{C}_1 \mid \mathbf{C}_2$ are ca subgrafuri disjuncte grafurile lui $\mathbf{C}_1$ şi $\mathbf{C}_2$.

Orice sarcină din $\mathbf{C}_1$ este independentă de orice sarcina din $\mathbf{C}_2$.
Dacă $a = a_1\ a_2.....a_{2m}$ şi $b = b_1\ b_2.....b_{2n}$ sunt secvenţe de execuţie pentru $\mathbf{C}_1$ respectiv pentru $\mathbf{C}_2$, o secvenţă de execuţie a lui $\mathbf{C}_1 \mid \mathbf{C}_2$ se formează astfel:

$$c = c_1c_2.....c_{2(m+n)}$$

unde :

$c_1 = a_1 \sim b_1$ (fie a_1 fie b_1)

dacă $a_1\ a_2.....a_i$ şi $b_1\ b_2.....b_j$ sunt elemente ale lui

$c_1\ c_2.....c_{i+j}$ cu $i+j < 2(m+n)$

atunci :

$$c_{i+j+1} = a_{i+1} \sim b_{j+1} \quad (\text{ fie } a_{i+1} \text{ fie } b_{j+1})$$

Combinarea paralelă a secvenţelor de execuţie apare frecvent în proiectarea sistemelor, în special a sistemelor cu prelucrare paralelă, uneori sub o formă diferită de cea prezentată anterior în sensul că unele sisteme implică secvenţe repetitive.

- Sistemele care conţin secvenţe repetitive pot fi modelate fie prin grafuri aciclice fie prin grafuri ciclice.
- Astfel, se pot modela k cicli ai unui sistem **C** prin concatenarea

$$\mathbf{C}^k = \mathbf{C}_1 \cdot \mathbf{C}_2 \cdot \ldots\ldots \cdot \mathbf{C}_k$$

- O secvenţă de execuţie α a lui $\mathbf{C}^k$ este de forma:

$$\alpha = \alpha_1 \alpha_2 \ldots\ldots \alpha_k$$

unde α_i este o secvenţă de execuţie a lui **C** pentru iteraţia i.

Un sistem de sarcini se numeşte ciclic dacă este de forma $\mathbf{C}^k$ pentru $k > 1$, sau dacă este o combinare paralelă de sisteme închise în care cel puţin unul este ciclic.

Proprietatea de determinare a sistemelor de sarcini

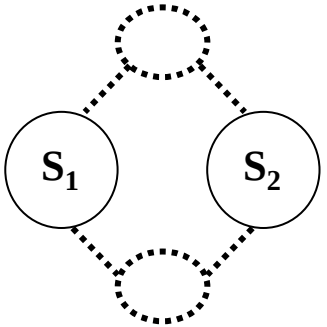
Definiţie:
Sistem determinat (sistem de sarcini functional) este un sistem format din sarcini care se execută în paralel şi cooperează pentru realizarea unor operaţii logice şi de calcul, care produce acelaşi rezultat indiferent de durata de execuţie a fiecărei sarcini **independente**, sau de ordinea în care acestea se executa.

Definiţie:
Un sistem de sarcini este nedeterminat dacă rezultatele produse de sarcini independente depind de ordinea în care acestea se executa.

- Exemplu de sistem nedeterminat:

S_1: $R_1 \leftarrow$ BUSFN (M; DCD (ADR))

S_2: M * DCD (ADR) $\leftarrow R_2$



S_1 - citeşte din locaţia de memorie de la adresa ADR;

S_2 - scrie în locaţia de memorie de la adresa ADR.

$\alpha_1 = Sl_1\, SF_1\, Sl_2\, SF_2$

se va citi valoarea R_1 si in celula de memorie va ramane valoarea R_2

$\alpha_2 = Sl_2\, SF_2\, Sl_1\, SF_1$

in celula de memorie va ramane valoarea R_2 si se va **citi valoarea R2**

Produc rezultate diferite

- Nedeterminarea se poate rezolva introducând o relaţie de precedenţă adecvată între sarcinile care erau independente.

- Pentru a stabili condiţiile necesare şi suficiente ca un sistem să fie determinat, vom considera un model simplificat în care sistemul fizic este privit ca o mulţime ordonată de locaţii de memorie:

$$\mathbf{M} = (M_1, M_2, ..., M_m)$$

ce conţin orice valoare dintr-o mulţime de valori **V**.

- Stările sistemului vor fi definite de valorile care se găsesc în memorie la un moment dat:

De exemplu dacă:

$$\alpha = a_1 a_2 ..... a_{2n}$$ şi

$$\sigma = \sigma_0 \sigma_1 \sigma_2 ..... \sigma_{2n}$$

reprezintă o secvenţă de execuţie, respectiv secvenţa de stări corespunzătoare,

iar $M_i(k)$reprezintă valoarea din celula M_i imediat după evenimentul a_k

starea sistemului va fi:

$$\sigma_k = [M_1(k), M_2(k), ..., M_m(k)]$$

Mulţimea tuturor stărilor poate fi definită astfel:

$\mathbf{\Sigma} = \mathbf{V}^m$ $\quad\quad$ $\mathbf{V}^m = [\ \mathbf{V} \times \mathbf{V} \times \ldots\ldots \times \mathbf{V}\ ]$, produs cartezian.

- Pentru a formaliza efectul execuţiei unei sarcini asupra celulelor de memorie vom considera că fiecărei sarcini S i se asociază funcţia:

$$f_S : V^d \longrightarrow V^r$$

unde:

$d = |D_S|$ cardinalul domeniului valorilor de intrare, D_S;

$r = |R_S|$ cardinalul domeniului valorilor de ieşire, R_S.

- Pentru o stare iniţială σ_0 şi o secvenţă de execuţie α, secvenţa corespunzătoare de stări: $\sigma = \sigma_0\sigma_1 \ldots\ldots \sigma_{2n}$ este definită în felul următor :

Fie $D_S = (M_{x1}, M_{x2}, \ldots\ldots, M_{xd})$ domeniul valorilor de intrare;

$R_S = (M_{y1}, M_{y2}, \ldots\ldots, M_{yr})$ domeniul valorilor de ieşire;

dacă $a_{k+1} =$ SI, atunci $M_i(k+1) = M_i(k)$, $\quad 1 \le i \le m$;
dacă $a_{k+1} =$ SF, şi $a_l =$ SI , $l \le k$, atunci stările locaţiilor de memorie din domeniul de valori al lui S la momentul k+1 sunt:

$$[M_{y1}(k+1), M_{y2}(k+1), \ldots, M_{yr}(k+1)] = f_S\,(M_{x1}(l), M_{x2}(l), \ldots, M_{xd}(l))$$ şi

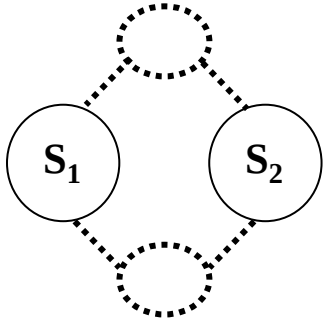
$$M_i(k+1) = M_i(k) \quad \text{pentru } (\forall)\ M_i \notin R_S$$

- Altfel spus, dacă a_{k+1} este un eveniment de iniţiere nu apare o schimbare a stării, adică $\sigma_{k+1} = \sigma_k$.
- Dacă însă a_{k+1} este un eveniment de terminare a lui S, $\sigma_{k+1} \neq \sigma_k$ doar în domeniul R**s**, noile valori din R**s** fiind determinate de f_S pe baza valorilor din domeniul de definiţie din momentul imediat precedent iniţierii lui S .
- Secvenţa de stări $\sigma = \sigma_0\sigma_1 \ldots\ldots \sigma_{2n}$ ce rezultă dintr-o secvenţă de execuţie, poate fi reprezentată sub forma unui tablou de dimensiuni m * (2n+1) având
- pe linii celulele de memorie M, iar
- pe coloane stările.

Sistem de sarcini nedeterminat

*Exemplu:

Să considerăm un sistem format din două sarcini, iar relaţia de precedenţă fiind mulţimea vidă



$$\mathbf{C} = (\{S_1, S_2\}, \Phi)$$

$$\mathbf{M} = (M_1, M_2)$$

$$D_{S1} = D_{S2} = R_{S1} = R_{S2} = \mathbf{M}$$

Sistemului i se atribuie două interpretări:

Sarcini	Interpretare 1	Interpretare 2
S_1	$(M_1,M_2) \leftarrow (1,M_2)$	$(M_1,M_2) \leftarrow (M_1+M_2,M_2)$
S_2	$(M_1,M_2) \leftarrow (2,M_2)$	$(M_1,M_2) \leftarrow (M_1,M_1+M_2)$

Valorile iniţiale corespunzătoare stării iniţiale s_0: $M \leftarrow (1,2)$

Fie două secvenţe de execuţie α_1 şi α_2

$$\alpha_1 = SI_1\, SF_1\, SI_2\, SF_2 \text{ şi}$$

$$\alpha_2 = SI_2\, SF_2\, SI_1\, SF_1$$

Comportarea celor două secvențe de execuție

Să analizăm comportarea celor două secvenţe de execuţie conform cu cele două interpretări ale sistemului:

	$\alpha_1 = SI_1\ SF_1\ SI_2\ SF_2$	$\alpha_2 = SI_2\ SF_2\ SI_1\ SF_1$
	$\sigma = \delta_0\ \delta_1\ \delta_2\ \delta_3\ \delta_4$	$\sigma = \delta_0\ \delta_1\ \delta_2\ \delta_3\ \delta_4$
Interpretare 1	M_1 1 1 1 1 2	M_1 1 1 2 2 1
	M_2 2 2 2 2 2	M_2 2 2 2 2 2
Interpretare 2	M_1 1 1 3 3 3	M_1 1 1 1 1 4
	M_2 2 2 2 2 5	M_2 2 2 3 3 3

Se observă că sistemul de sarcini C nu este determinat pentru nici una din cele două interpretări .

Reprezentare prin secventa de valori

- O reprezentare mai convenabilă constă din secvenţa de valori pe care un sistem de sarcini dat C o înscrie într-o celulă de memorie Mi, în timpul unei secvenţe de execuţie α .
- Această reprezentare se notează sub formă vectorială:

$V(M_i,\alpha) = (V_1,V_2,.....,V_p)$

Considerând

$\alpha = a_1a_2.....a_{2n}$ secvenţa de execuţie

$\sigma = \sigma_0\sigma_1\ldots\ldots \sigma_{2n}$, secvenţa de stări corespunzătoare,

ε= Φ şirul vid, secvenţa de valori corespunzătoare se defineşte astfel:

$V(M_i,\varepsilon) = M_i(\Phi)$ în lipsa unei secvenţe de execuţie, iar

$V(M_i, a_1a_2 \ldots a_k)=$ <u>dacă</u> a_k= SF şi $M_i \in R_S$ (domeniul unde S produce valori)

<u>Atunci</u> = $(V(M_i, a_1a_2 \ldots a_{k-1}), M_i(k))$

<u>Altfel</u> nu se modifică deci este $V(M_i, a_1 \ldots a_{k-1})$

nu se mai adaugă o nouă valoare

- Se iau în considerare numai valorile înscrise în celulele de memorie din domeniul de valori şi numai la terminarea S.

Obs. $V(M_i, \alpha)$= ..... $V(M_j, \alpha)$= ..... pot avea dimensiuni diferite

- Trebuie notat că $V(M_i,\alpha)$ şi $V(M_j,\alpha)$ nu trebuie să fie de aceeaşi lungime deoarece M_i poate fi în R pentru anumite sarcini
- iar M_j poate fi în R pentru altele (în număr diferit).

Pentru secvenţa de valori: $V(M_i,\alpha) = (v_1,v_2,.....,v_p)$ se defineşte *valoarea finală* $F(M_i,\alpha) = v_p$.

Rezultă că pentru secvenţa de execuţie partiala

$\alpha = a_1a_2...a_k$

$\delta_k = [F(M_1,\alpha),F(M_2,\alpha),...,F(M_m,\alpha)]$

	α	a1	a2	a3	a4	a5	….	ak	….	a2n
	δ0	δ1	δ2	δ3	δ4	δ5	….	δk	….	δ2n
M1			■							
M2					■					
M3						■				
….										
Mi				■						
….										
Mj			■							
….										
Mm					■					

Exemplu: Să reconsiderăm exemplul precedent:

* Secvenţa de stări poate fi reprezentată astfel printr-un tablou:

	$\alpha_1 = SI_1\ SF_1\ SI_2\ SF_2$			$\alpha_2 = SI_2\ SF_2\ SI_1\ SF_1$		
	V(M,ε)	V(M,a_1a_2)	V(M,$a_1..a_4$)	V(M,ε)	V(M,a_1a_2)	V(M,$a_1..a_4$)
Interpretare 1	M_1 1	1	2	M_1 1	2	1
	M_2 2	2	2	M_2 2	2	2
Interpretare 2	M_1 1	3	3	M_1 1	1	4
	M_2 2	2	5	M_2 2	3	3

Interpretarea 1 este nedeterminată deoarece S_1 şi S_2 sunt în dispută pentru a scrie în M_1, iar
în interpretarea 2 o sarcina scrie într-o celulă citită de alta sarcina.

	$\alpha_1 = SI_1\ SF_1\ SI_2\ SF_2$	$\alpha_2 = SI_2\ SF_2\ SI_1\ SF_1$
	$\sigma = a_0\ a_1\ a_2\ a_3\ a_4$	$\sigma = a_0\ a_1\ a_2\ a_3\ a_4$
Interpretare 1	M_1 1 1 1 1 2	M_1 1 1 2 2 1
	M_2 2 2 2 2 2	M_2 2 2 2 2 2
Interpretare 2	M_1 1 1 3 3 3	M_1 1 1 1 1 4
	M_2 2 2 2 2 5	M_2 2 2 3 3 3

- Un sistem de sarcini **C** = (***S***, **<**) este neinterpretat dacă se cunosc doar relatia de precedenta **<** , domeniile de definitie D şi de valori R pentru fiecare sarcina.
- O interpretare pentru **C** constă din specificarea funcţiei f_S pentru fiecare S_i Є ***S*** .

În continuare vom considera sisteme de sarcini **neinterpretate.**

Definiţie:
Un sistem de sarcini ***C*** *este determinat* dacă pentru orice stare iniţială $\mathcal{A}_0$ dată, **$V(M_i,\alpha) = V(M_i,\alpha')$,** $1 \leq i \leq m$, pentru toate secvenţele de execuţie α ,….α' din **C**.

(Secvenţele de valori depind numai de valorile iniţiale în $\mathcal{A}_0$.)

Definiţie:
Sarcinile S şi S' sunt neinterferente dacă:

S este succesor sau predecesor a lui S', sau

$R_S \cap R_{S'} = R_S \cap D_{S'} = D_S \cap R_{S'} = \Phi$

- Mulţimea ***S*** = { $S_1, S_2, ... S_n$ } se spune că este formată din sarcini mutual neinterferente dacă pentru $\forall$ i , j (i $\neq$ j) S_i si S_j sunt neinterferente.

- Pentru a arăta că un sistem de sarcini mutual neinterferente este determinat vom prezenta *teorema de suficienţă şi necesitate.*

- Înainte de a prezenta această teoremă vom stabili o lemă utilizată în cadrul teoremei de necesitate şi suficienţă.

Lemă:
Fie **C** = (**<** , α) un sistem de n sarcini mutual neinterferente iar S o sarcina terminala a lui **C**. ($R_S \cap R_{S'} = R_S \cap D_{S'} = D_S \cap R_{S'} = \Phi$)
Dacă $\alpha = \beta_1$ SI β_2 SF β_3 este o secvenţă de execuţie validă a lui **C**, atunci
$\alpha' = \beta_1 \ \beta_2 \beta_3$ SI SF este de asemenea o secvenţă de execuţie a lui **C** pentru $\mathcal{S}_0$ dată,

$$V(M_i,\alpha) = V(M_i,\alpha'), \ \forall \ 1 \le i \le m$$

Justificare :

- S nu are succesori în **C** conform cu (**<** , α') satisface relaţiile de precedenţă din **C** şi deci trebuie să fie o secvenţă de execuţie validă.
- S scrie numai în celulele ce aparţin lui R_S şi deoarece pentru orice S' iniţiată în β_3, după terminarea lui S în α, $R_S \cap D_{S'} = \Phi$ fiind neinterferente, orice astfel de sarcini S' găsesc aceleaşi valori în $D_{S'}$ atât pentru α cât şi pentru α'.
- Deci pentru $M_i \notin R_S$ rezultă $V(M_i,\alpha)=V(M_i,\alpha')$

- Pentru orice $M_j \in D_S$, $V(M_j,\beta_1)=V(M_j , \beta_1\beta_2\beta_3)$ deoarece nici o sarcina S' din $\beta_1\beta_2\beta_3$ nu scrie în D_S deoarece $R_{S'} \cap D_S = \Phi$ fiind mutual neinterferente.
- Deci $F(M_j ,\beta_1) = F(M_j ,\beta_1\beta_2\beta_3)$ pentru $\forall M_j \in D_S$, iar S scrie aceeaşi valoare în $\forall\ M_i \in R_S$ atât pentru α cât şi pentru α'.
- Considerând că v este valoarea înscrisă de S în $M_i \in R_S$ pentru α, rezultă că:

$V(M_i,\alpha) = V(M_i , \beta_1 \text{ SI } \beta_2 \text{ SF } \beta_3)$

$V(M_i,\alpha) = V(M_i , \beta_1 \text{ SI } \beta_2 \text{ SF})$ nu există S' $\in$ β_3 care scrie în R_S

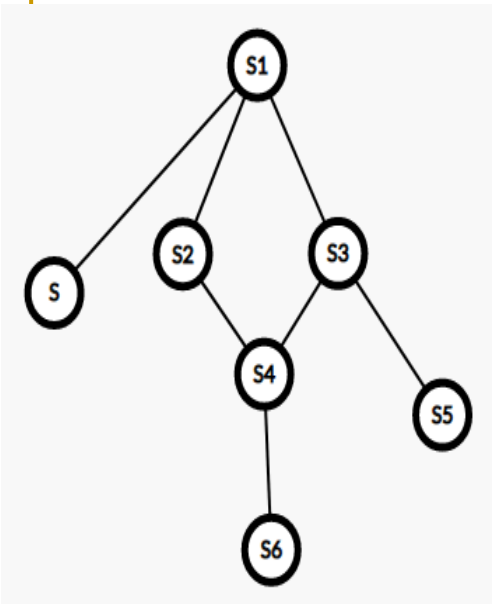
$V(M_i,\alpha) = (V(M_i , \beta_1 \text{ SI } \beta_2) ,v)$ S scrie valoarea v în

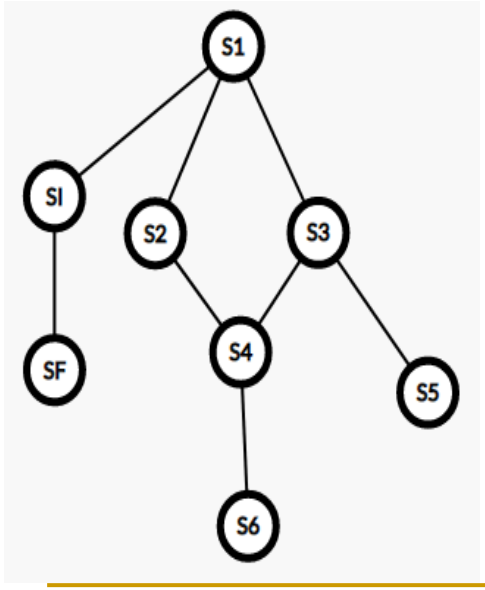
$V(M_i,\alpha) = (V(M_i , \beta_1) , v)$ nu există S' $\in$ β_2 care scrie în R_S

$V(M_i,\alpha) = (V(M_i , \beta_1 \beta_2 \beta_3),v)$ nu există S' $\in$ $\beta_2\beta_3$ care scrie în R_S

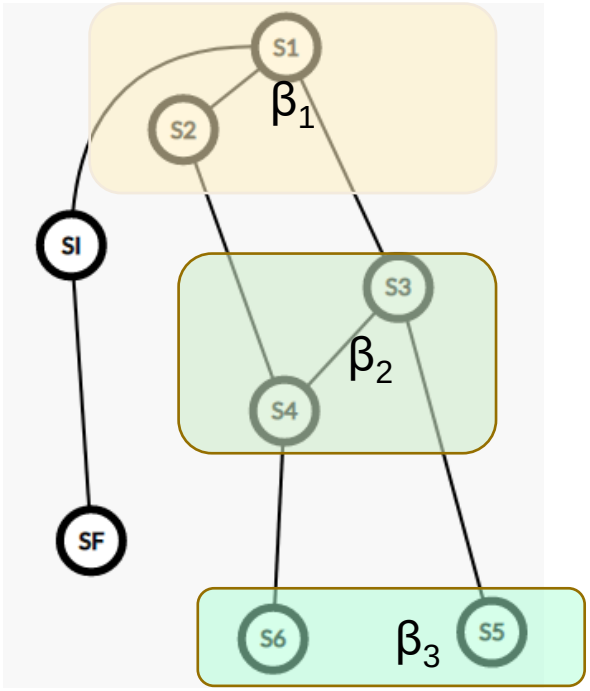
$V(M_i,\alpha) = (V(M_i , \beta_1 \beta_2 \beta_3 \text{ SI SF})$ S scrie v în M_i

$V(M_i,\alpha) = V(M_i , \alpha')$

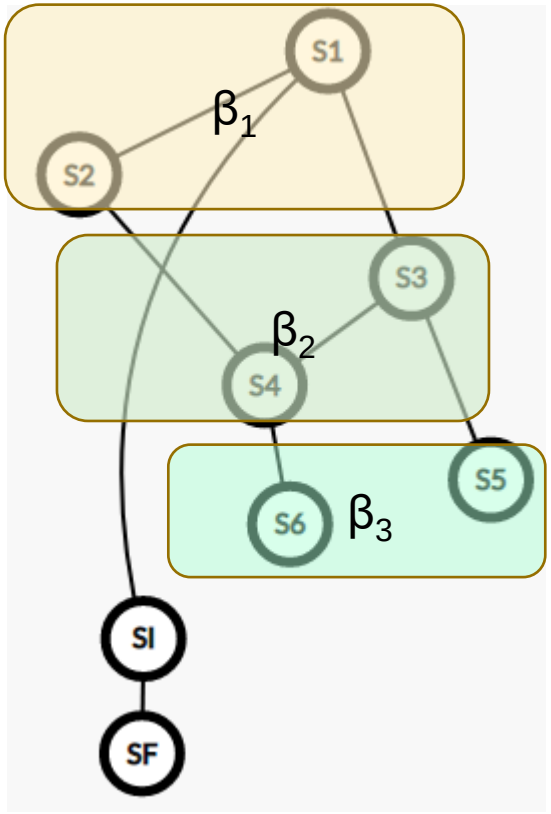




$\alpha = \beta_1$ SI β_2 SF β_3



$\alpha' = \beta_1 \ \beta_2 \beta_3$ SI SF



Lema

Teorema de suficienţă:
Sistemele de sarcini formate din sarcini mutual neinterferente sunt determinate.

Justificare (prin inducţie)

- Pentru un sistem de sarcini format dintr-o singura sarcina este evident.
- Presupunem că afirmaţia este adevărată pentru sisteme cu n-1 sarcini, (α'_1 şi α'_2 sunt secvenţe de execuţie pentru un sistem cu n-1 sarcini)
- Presupunem sistemul **C** = (***S***, **<**) format din n sarcini.

- Dacă sistemul **C** are o singură secvenţă de execuţie validă, el este determinat.
- Presupunem 2 secvenţe de execuţie α_1, α_2 în **C** şi S sarcina terminala în ***S***.

Conform lemei putem forma 2 secvenţe de execuţie α'_1 şi α'_2 astfel:

$\alpha_1 = \alpha'_1$ SI SF; $\alpha_2 = \alpha'_2$ SI SF,

în condiţiile în care

$$V(M_i, \alpha_1) = V(M_i, \alpha'_1), \quad 1 \le i \le m$$

$$V(M_i, \alpha_2) = V(M_i, \alpha'_2), \quad 1 \le i \le m$$

α'_1 şi α'_2 sunt secvenţe de execuţie pentru un sistem cu n-1 sarcini: **C'** = (***S*** \ S, **<'**), relaţia de precedenţă **<'** fiind obţinută din **<** eliminând relaţia de precedenţă ce implică pe S.

$V(M_i, \alpha'_1) = V(M_i, \alpha'_2)$ din ipoteza de inducţie.

- Deci se poate presupune că valorile din D_S sunt aceleaşi, atât pentru α'_1 cât şi pentru α'_2,
- respectiv $F(M_i, \alpha'_1) = F(M_i, \alpha'_2)$ pentru $M_i \in D_S$.
- Rezultă astfel că pentru α'_1 şi α'_2, sarcina S scrie aceeaşi valoare v pentru orice celulă $M_i \in R_S$.

Pentru $\mathbf{M_i \notin R_S}$:

$\mathbf{V(M_i, \alpha_1)}$	$= V(Mi\ ,\ \alpha'1\ SI\ SF)$	Lemă
	$= V(M_i\ ,\ \alpha'_1)$	$M_i \notin R_S$ si ip. de inducţie
	$= V(M_i\ ,\ \alpha'_2)$	$M_i \notin R_S$
	$= V(M_i\ ,\ \alpha'_2\ SI\ SF)$	Lemă
	$\mathbf{= V(M_i\ ,\ \alpha_2)}$	

Pentru $M_i \notin R_S$ $\mathbf{V(M_i, \alpha_1) = V(M_i, \alpha_2)}$

Pentru $\mathbf{M_i \in R_S}$:

$\mathbf{V(M_i, \alpha_1)}$	$= V(Mi\ ,\ \alpha'1\ SI\ SF)$	Lemă
	$= (V(M_i\ ,\ \alpha'_1),v)$	ip. de inducţie
	$= (V(M_i\ ,\ \alpha'_2),v)$	
	$= V(M_i\ ,\ \alpha'_2\ SI\ SF)$	Lemă
	$\mathbf{= V(M_i\ ,\ \alpha_2)}$	

Pentru Mi ∈ RS $\mathbf{V(Mi, \alpha1) = V(Mi, \alpha2)}$

- *Sistemul **C** este determinat dacă sarcinile sunt mutual neinterferente.*

Teorema de necesitate:
Fie **C** un sistem de sarcini astfel că pentru fiecare $S \in \boldsymbol{S}$, f_S nu este specificată, dar D_S şi $R_S \neq \Phi$ sunt date.
C este determinat pentru orice interpretare a sarcinilor sale numai dacă acestea sunt mutual neinterferente.

Justificare (reducere la absurd**)**

- Presupunem că S, S' sunt interferente, independente (adică nu există nici o relaţie de ordine între ele).
- Atunci există două secvenţe de execuţie valide:

  $\alpha = \beta 1$ SI SF SI' SF' $\beta 2$

  $\alpha' = \beta 1$ SI' SF' SI SF $\beta 2$

- Presupunem că există o celulă de memorie $M_i \in R_S \cap R_{S'}$ (sunt considerate interferente) şi putem alege f_S şi $f_{S'}$ (două interpretări, pentru S şi S'), astfel încât pentru:

  f_S: S să scrie în M_i valoarea u;

  $f_{S'}$: S' să scrie în M_i valoarea v; $u \neq v$

În ceea ce priveşte valorile din R_S şi $R_{S'}$ pentru secvenţele α şi α’ putem spune că:

$$V(M_i, \alpha) = V(M_i, \beta 1 \text{ SI SF } \text{SI'} \text{ SF}') = (V(M_i, \beta_1), u, v);$$

$$u \neq v$$

$$V(M_i,\alpha') = V(M_i, \beta_1 \text{ SI' SF' SI SF}) = (V(M_i, \beta_1), v, u);$$

Deci în celula M_i, secvenţele α si α' înscriu secvenţe de valori diferite

$$(V(M_i, \beta_1), u, v); \text{ respectiv } (V(M_i, \beta_1), v, u);$$

$$V(M_i, \alpha) \neq V(M_i, \alpha')$$

ceea ce înseamnă că sistemul de sarcini **C** nu este determinat.

Deci este necesar ca $R_S \cap R_{S'} = \Phi$ pentru că altfel rezultă că **C** este nedeterminat.

- Presupunem că există o celulă $M_j \in D_S \cap R_{S'}$.
- Presupunem că există o celulă $M_i \in R_S$.
- Atunci, putem alege o interpretare a lui S', o funcţie $f_{S'}$ astfel încât $F(M_j, \beta_1) \neq F(M_j, \beta_1$ SI' SF').
- Rezultă în acest caz că S citeşte valori diferite pentru α şi α' (având în vedere că $M_j \in D_S \cap R_{S'}$).

- Putem alege în acest caz o funcţie f_S astfel încât sarcina S să scrie în M_i, valoarea u pentru secvenţa α şi valoarea v pentru secvenţe α', cu $u \neq v$.

În acest caz putem spune că:

$V(M_i, \alpha) = V(M_i, \beta_1$ SI SF SI' SF')

$= V(M_i, \beta_1$ SI SF) $\quad R_S \cap R_{S'} = \Phi$

$= (V(M_i, \beta_1), u)$ S scrie u pentru α

$V(M_i, \alpha') = V(M_i, \beta_1\ SI'\ SF'\ SI\ SF)$

$= (V(M_i, \beta_1\ SI'\ SF'), v)$

$= (V(M_i, \beta_1), v)$ S scrie v pentru α'

deci $V(M_i, \alpha) \neq V(M_i, \alpha')$ deci **C** nu este determinat.

Rezultă că trebuie ca : $D_S \cap R_{S'} = \Phi$

Similar se poate arăta că trebuie ca : $D_{S'} \cap R_S = \Phi$

O aplicaţie foarte importantă a teoremelor arătate anterior se referă la paralelismul maxim în sisteme de sarcini în care constrângerile de precedenţă sunt impuse numai de cerinţele de determinare.

- Din definiţia dată pentru determinare rezultă că fiecare sarcina produce o singură secvenţă de valori pentru o celulă de memorie M_i în condiţiile unei stări iniţiale date.

- Două sisteme cu aceeaşi mulţime de sarcini S sunt echivalente dacă sunt determinate şi dacă pentru aceeaşi stare iniţială produc aceeaşi secvenţă de valori.

Un sistem de sarcini **C** şi graful asociat G (definit de relatia de precedenta **<**) implică un paralelism maxim dacă **C** este determinat şi dacă **eliminarea** unui arc oarecare (S, S') din G, face ca sarcinile S şi S' să devină **interferente**.

- Astfel dacă (S , S') este un arc într-un graf cu maximum de paralelism atunci:

$$(R_S \cap R_{S'}) \cup (R_S \cap D_{S'}) \cup (D_S \cap R_{S'}) \neq \Phi$$

- Având un sistem de sarcini **C**, determinat, este util să construim sistemul echivalent **C'** cu maximum de paralelism.

Sistem de sarcini cu maximum de paralelism

Teoremă :
Avand un sistem de sarcini **C**= (***S***, **<**) se poate construi un sistem
C' = (***S***, **<'**) unde
relaţia <' este închiderea prin tranzitivitate a relaţiei:
$Z=\{(S,S') \in < \mid (R_S \cap R_{S'}) \cup (R_S \cap D_{S'}) \cup (D_S \cap R_{S'}) \neq \Phi\}$
C' este unicul sistem echivalent cu **C** care implică
maximum de paralelism.

Exemplu :
Fie sistemul de sarcini C = (S , **<**) unde
S = {S1,S2,S3,S4,S5,S6,S7,S8 }
Să se construiască sistemul de sarcini C'=(S, **<'**) echivalent, care realizează maxim de paralelism tinand seama de cerinţele de determinare (care are la baza conflictul de resurse)

- Sistemul este caracterizat prin:
- $M = \{ M_1, M_2, M_3, M_4, M_5 \}$

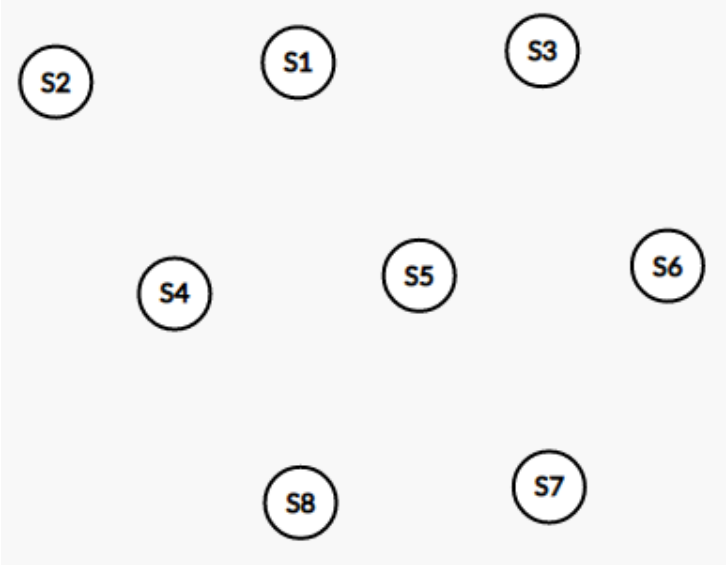
iar domeniile D_S şi R_S sunt specificate prin tabelul :

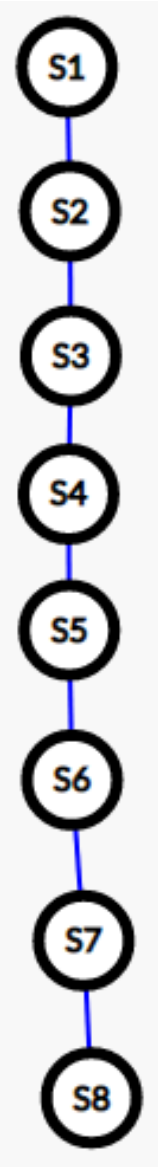
M	Domeniu de definiţie D_S	Domeniu de valori R_S
M1	S1,S2,S7,S8	S3
M2	S1,S7	S5
M3	S3,S4,S8	S1
M4	S3,S4,S5,S7	S2,7
M5	S6	S4,S6,S8

Posibillitati de executie a sistemului de sarcini



maxim de paralelism tinand seama de cerinţele de determinare bazate pe eliminarea conflictului de resurse





$$Z=\{(S,S') \in < \mid (R_S \cap R_{S'}) \cup (R_S \cap D_{S'}) \cup (D_S \cap R_{S'}) \neq \Phi\}$$

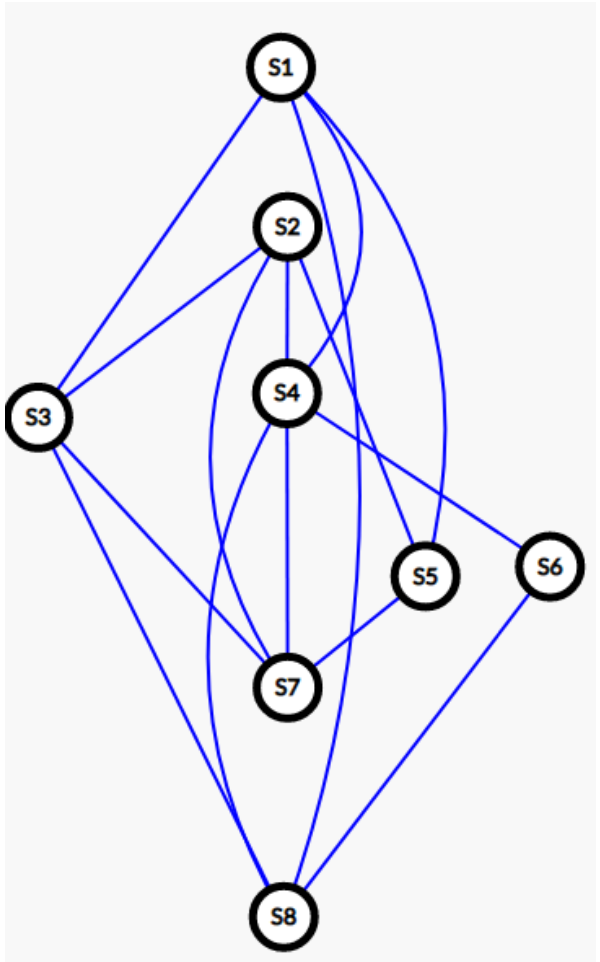
Relaţia Z, conform definiţiei de mai sus va fi dată de mulţimea arcelor:

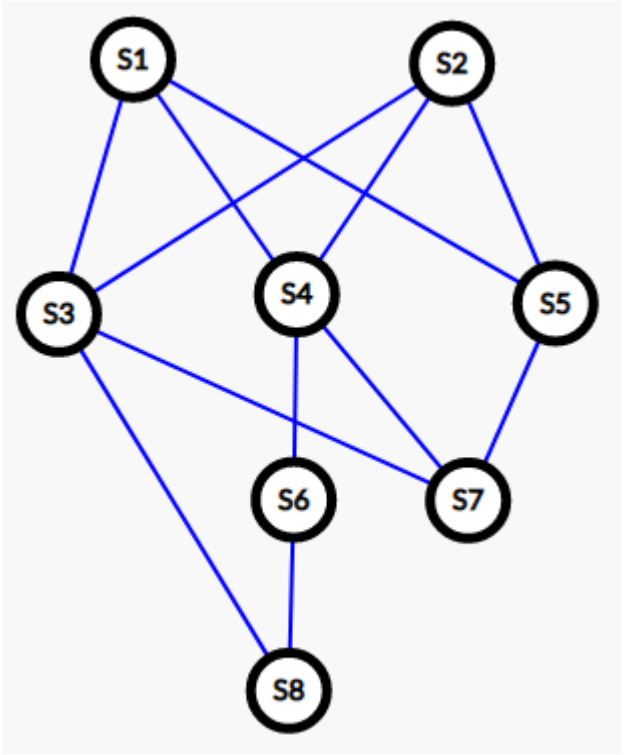
Z : { (S1,S3) (S1,S4) (S1,S5) (S1,S8)
(S2,S3) (S2,S4) (S2,S5) (S2,S7)
(S3,S7) (S3,S8)
(S4,S6) (S4,S7) (S4,S8)
(S5,S7)
(S6,S8)
}

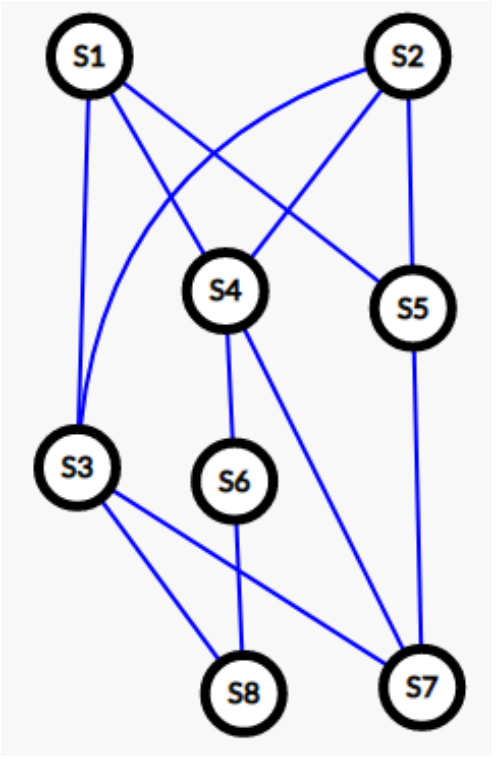
M	Domeniu de definiţie D_S	Domeniu de valori R_S
M1	S1,S2,S7,S8	S3
M2	S1,S7	S5
M3	S3,S4,S8	S1
M4	S3,S4,S5,S7	S2,7
M5	S6	S4,S6,S8

Rezultă graful asociat G', care este organizat pe niveluri eliminând arcele excluse prin tranzitivitate

graful asociat







Sistemul C' echivalent se execută în 4 perioade de timp considerând că avem un număr suficient de procesoare (3 procesoare) sau in 4 perioade avand la dispozitie numai 2 procesoare

Analiza situațiilor de blocare

- Un sistem care este format din sisteme de sarcini combinate paralel $\mathbf{C}=\mathbf{C}_1 \mid \mathbf{C}_2 \mid ... \mid \mathbf{C}_n$

este *blocat* dacă evoluţia sa este suspendată indefinit din cauză că mai multe sisteme de sarcini $\mathbf{C}_i$ sunt active simultan, fiecare utilizând, fără a putea fi întrerupte, resurse necesare altor sisteme din **C**.

- O resursă deţinută de o sarcina neinteruptibila poate fi eliberată numai de sistemul de sarcini care o utilizează la un moment dat.

- Pentru studiul situaţiilor de blocare vom privi sistemul fizic ca o colecţie de resurse R de tip:

  $R = R_1\ R_2 ... R_m$, fiecare tip conţinând:

  $W = W_1\ W_2 ... W_m$ exemplare

  Vectorul $W=(W_1,W_2,...,W_m)$ reprezintă capacitatea sistemului.

Exemplu de resurse R_i:

- unităţi centrale de prelucrare;
- coprocesoare matematice;
- pagini de memorie;
- magistrale de interconectare;
- înregistrări fizice (sectoare, piste, cilindri) la discuri magnetice;
- fişiere de date;
- zone de mesaje;
- tabele de descriere a diferitelor sarcini;
- variabile globale, etc.

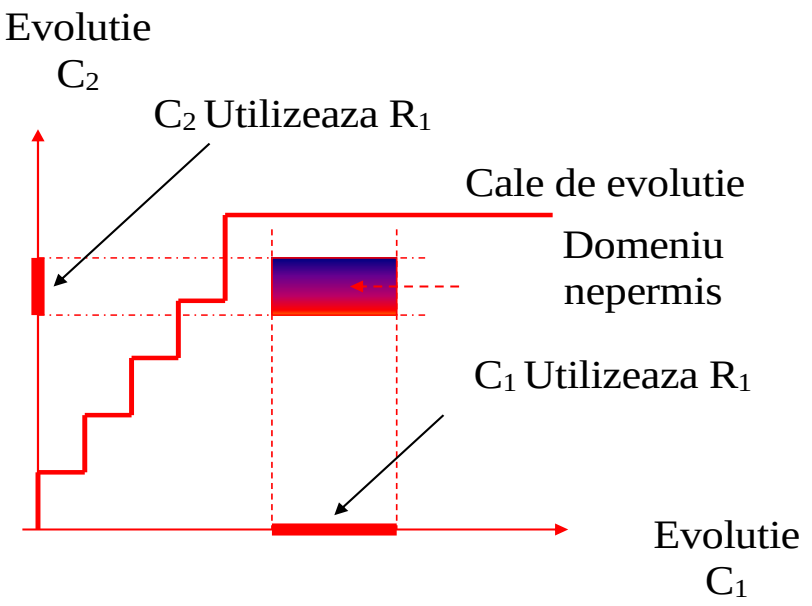
- Să considerăm un exemplu simplu de blocare:

  Fie două sisteme **C**$_1$ şi **C**$_2$ şi câte o resursă de tip R_1 şi R_2.

Evoluţia întregului sistem poate fi reprezentată în spaţiul cu două dimensiuni definit spaţiul de evoluţie.

Evoluţia sistemului **C**=**C**$_1$ | **C**$_2$ este reprezentată de succesiunea de puncte (x_1y_1) (x_2y_2)...(x_ky_k)... în acest spaţiu

Presupunem că originea este în punctul (0,0).
Evoluţia este ireversibilă deci $x_k \leq x_{k+1}$, $y_k \leq y_{k+1}$ şi
Nu orice cale de evoluţie, este realizabilă.
Evoluţia în care **C**$_1$ şi **C**$_2$ asftel sa utilizeaze simultan aceeaşi resursă R_1 este nepermisă
Zona hasurată reprezintă un domeniu nepermis, care conduce la blocarea sistemului.



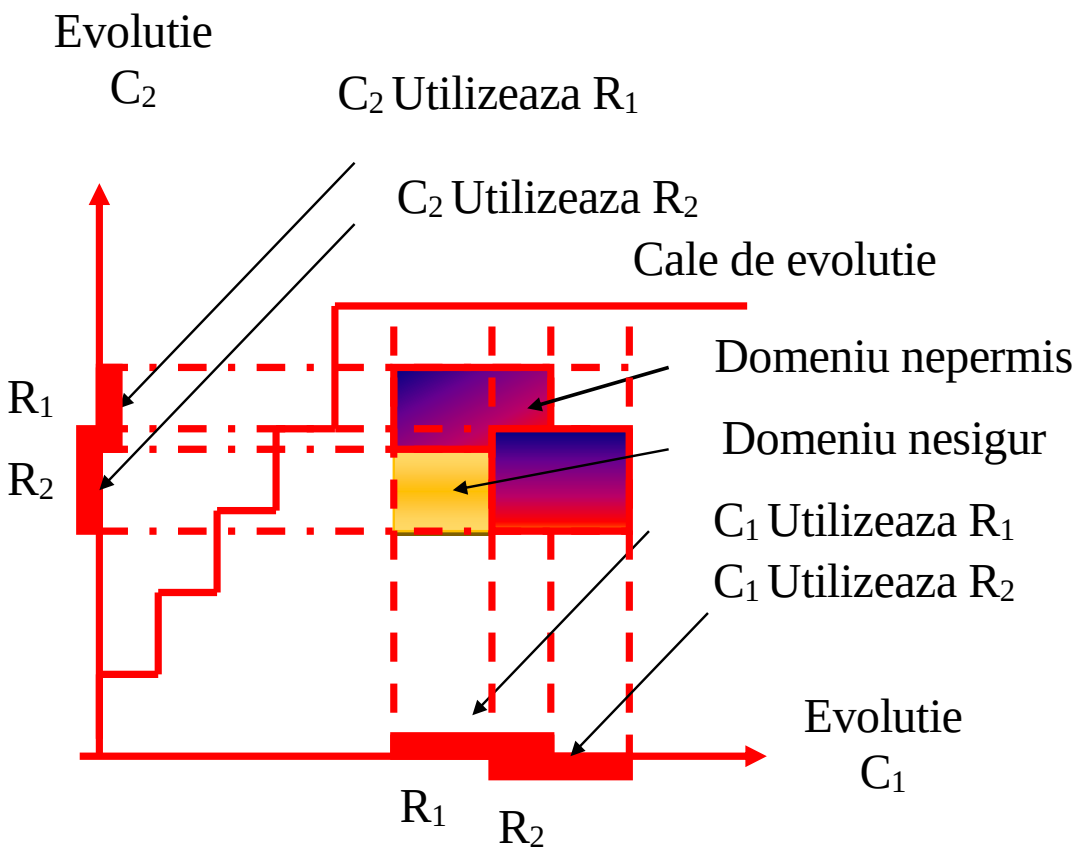
Exemplu de utilizare a doua resurse

Fie utilizarea resurselor R1 şi R2 de către C1 şi C2.

Zona haşurată reprezintă un domeniu nepermis, care conduce la blocarea sistemului.

Zona portocalie reprezintă un domeniu nesigur, care permite în continuare evoluţia sistemului, însă pentru un timp limitat.

Datorită ireversibilităţii procesului de evoluţie se va ajunge în mod sigur la blocarea sistemului.



- Blocarea este definită ca o stare de *aşteptare* circulară nerezolvabilă din cauza condiţiilor de excludere mutuală şi neintreruptibilitate.
- Pentru a exista o cale de evoluţie care conduce la blocare sunt necesare următoarele condiţii:

- **Utilizare exclusivă**: Fiecare sistem de sarcini doreşte
- controlul exclusiv asupra resursei pe care o utilizează;
- **Neîntreruptibilitate**: O resursă nu este eliberată până la terminarea completă a utilizării acesteia;
- **Aşteptare circulară**: Fiecare sistem de sarcini ţine ocupate resurse în timp ce aşteaptă ca altele să elibereze resurse pe care să le preia.

- Relativ la blocare se pun trei probleme:

1. Prevenirea

Una sau mai multe din condiţiile care conduc la blocare sunt înlăturate printr-o proiectare adecvată sau prin restricţii în utillizarea resurselor.

2. Detectarea şi înlăturarea

Sistemul de operare si planificatorul supraveghează evoluţia tuturor sistemelor de sarcini luând măsuri speciale de corecţie a evolutiei la detectarea situaţiilor de blocare.

3. Evitarea

Pe baza unor informaţii preliminare despre cerinţele de resurse, sistemul de operare / planificatorul dirijează evoluţia pentru evitarea zonelor nesigure.

Considerând că fiecare tip de resursă conţine un singur element $w_1=w_2=...=w_n=1$, putem alege o reprezentare sub forma de grafuri, fiecare nod reprezentând o resursă iar arcele reprezinta cererea de resurse.

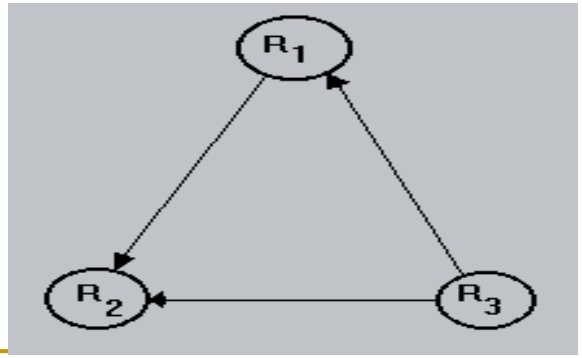
Arcele pot fi etichetate cu necesarul de resurse.

- Arcele sunt orientate şi sunt definite astfel:

  dacă la un moment dat, după evenimentul a_k, sistemul **C** ţine ocupată resursa R_i în timp ce aşteaptă preluarea lui R_j, graful conţine un arc orientat de la nodul R_i la nodul R_j.
- În exemplul urmator numărul de tipuri de resurse este n=3;

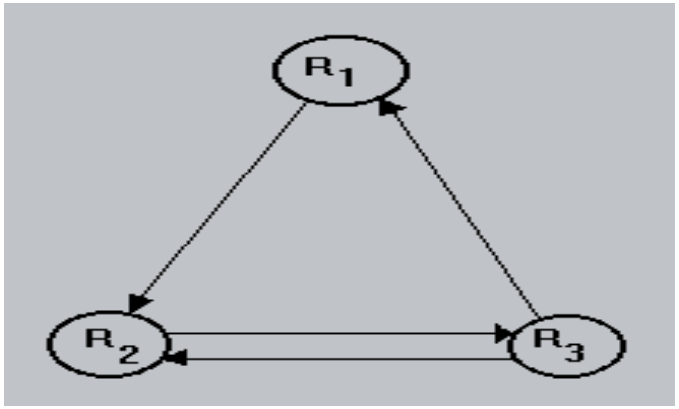
  Un proces utilizează R_1 şi o cere pe R_2 ,

  Alt proces utilizează pe R_3 şi cere pe R_1 şi R_2.



Graf de cereri fara bucle

Dacă cele trei condiţii de blocare sunt posibile (operative) condiţia necesară şi suficientă pentru blocare este existenţa unei bucle (circuit, ciclu) în graful de cereri.
Exemplu , de graf de cereri în care se realizează condiţia de aşteptare circulară cu resurse ocupate.



Graf de cereri cu bucle

- Pentru cazul general în care există mai multe resurse de un anumit tip, fiecare nod reprezintă un tip de resursă.
- Cererile pentru resurse R_i trebuie să nu depăşească, ca număr, w_i.
- Un arc de la R_i la R_j arată că există cel puţin un proces care cere una sau mai multe resurse de tip R_j şi deţine una sau mai multe resurse de tip R_i.
- În cazul acesta o buclă în graful de cereri reprezintă o condiţie necesară dar nu şi suficientă pentru existenţa situaţiei de blocare.

Prevenirea blocării

- Pentru a proiecta un sistem în care posibilitatea blocării este exclusă trebuie să existe certitudinea că în orice moment de timp cel puţin una din condiţiile necesare nu este îndeplinită.
- Prima din cele trei condiţii (excluderea mutuală) este uneori impusă de restricţii fizice privind accesul la resurse nepartajabile şi deci nu poate fi eliminată pentru orice fel de resurse.

Pentru celelalte condiţii vom enumera câteva posibilităţi:

- Dacă o sarcina care utilizează o resursă, cere o altă resursă *trebuie* să o elibereze pe prima şi dacă este necesară în continuare să o ceară din nou împreună cu resursele adiţionale de care are nevoie (***interuptibilitate***).
- Fiecare proces să ceară toate resursele de care are nevoie, putând continua numai după ce le are pe toate (***aşteptare cu resurse***).
- ***Ordonarea liniară*** a tipurilor de resurse pentru toate sarcinile. Dacă un proces utilizează resurse de tip R_i, la un moment dat, el poate cere numai resurse din tipurile următoare lui R_i în acea ordonare,
- exemplu: R_j cu j>i. În acest caz, graful de cereri nu are circuite.

*Observaţie:

- **Prima soluţie** se pretează numai pentru resurse interuptibile, a căror stare poate fi uşor salvată şi restaurată. De exemplu UCP.
- **A doua soluţie** poate deveni costisitoare dacă unele resurse sunt alocate unor sarcini şi sunt neutilizate o perioadă lungă de timp.
- **A treia soluţie** poate fi utilizată numai pentru anumite componente, de exemplu lansarea job-urilor într-un sistem cu multiprogramare prin alocarea memoriei şi a dispozitivelor de I/E într-o ordine fixă.

Detectarea şi înlăturarea blocării

- Pentru cazul în care există o singură resursă de fiecare tip, mecanismul de detectare a blocării constă:
- dintr-o *procedură* care actualizează graful de cereri ori de câte ori apare o cerere, o preluare sau eliberare de resurse şi
- o *procedură* de detectare a circuitelor (buclelor) în acest graf.
- Pentru cazul general, la fiecare moment de timp t, corespunzător secvenţei de execuţie $\alpha = a_1 a_2 ... a_k$, se definesc
- $x_{ij}[k]$ reprezintă numărul de resurse de tip R_j alocate, respectiv şi
- $y_{ij}[k]$ cerute de $\mathbf{C}_i$, după evenimentul a_k din α, unde:

  $\mathbf{C}=\mathbf{C}_1 \mid \mathbf{C}_2 \mid ... \mid \mathbf{C}_n$, $1 \le i \le n$ iar $1 \le j \le m$,

  m fiind numărul de tipuri de resurse

Se obţin astfel:

$$X[k] = \begin{bmatrix} x_1^{(k)} \\ x_2^{(k)} \\ . \\ . \\ x_n^{(k)} \end{bmatrix} \text{ matrice de alocare}$$

$$Y[k] = \begin{bmatrix} y_1^{(k)} \\ y_2^{(k)} \\ . \\ . \\ y_n^{(k)} \end{bmatrix} \text{ matrice de cereri}$$

cu

$X_i[k] = (x_{i1}[k], x_{i2}[k],..., x_{im}[k])$

$Y_i[k] = (y_{i1}[k], y_{i2}[k],..., y_{im}[k]), \quad 1 \le i \le n$

- Starea s_k corespunzătoare secvenţei α după evenimentul a_k este dată de perechea [X[k], Y[k]].

Să notăm cu:

$V[k]=(v_1[k],v_2[k],...,v_m[k])$ vectorul resurselor disponibile.

Elementul $v_j[k] \le w_j$, $1 \le j \le m$, numărul de resurse de tip R_j disponibile la un moment dat după evenimentul a_k este:

$$v_j(k) = w_j - \sum_{i=1}^{n} x_{ij}[k], \quad 1 \le j \le m$$

- Pentru doi vectori X şi Y, relaţia $X \le Y$ este adevarată, dacă şi numai dacă relaţia este adevărată pentru fiecare pereche de elemente corespondente din X şi Y.
- Utilizând informaţia conţinută în starea s_k dată de perechea de matrici [X[k], Y[k]] vom elabora un algoritm de detectare a blocării.

Definiţie:

Fie $\mathbf{C} = \mathbf{C}_1 \mid \mathbf{C}_2 \mid ... \mid \mathbf{C}_n$ unde $\mathbf{C}_i$, $1 \le i \le n$, este de forma

$\mathbf{C}_i = S_i[1]\ S_i[2].....S_i[l_i]$, $l_i \ge 1$.

Fie

$\alpha = a_1 a_2 ... a_k$ o secvenţă de execuţie parţială a lui **C** iar

$\sigma = s_0\ s_1 ... s_k$ secvenţa parţială de stări, corespunzătoare.

Dacă există o mulţime nevidă D de indici ai lui **C** astfel că pentru orice i din D nu este adevărată relaţia:

$Y_i[k] \le$	$V[k]$	$+ \sum_{j \notin D} X_j[k]$
vector de cereri	*vector de resurse*	*vector de resurse deţinute de sarcini ce nu aparţin lui D*

atunci σ este blocată (sau s_k este o stare de blocare).

De asemenea putem spune că orice sistem de sarcini $\mathbf{C}_i$ cu i ε D este blocat.

Exemplu:

Fie $\mathbf{C}_1$ şi $\mathbf{C}_2$ două sisteme de sarcini:

$\mathbf{C}_1 = S_1[1], S_1[2], S_1[3]$ şi $\mathbf{C}_2 = S_2[1], S_2[2]$

care utilizează resursele $R=(R_1,R_2,R_3,R_4)$ cu $w = (3,3,3,3)$, în felul următor :

	C 1			C 2	
	resurse cerute	resurse eliberate		resurse cerute	resurse eliberate
$S_1[1]$	(0,1,2,0)	(0,0,2,0)	$S_2[1]$	(2,0,2,2)	(0,0,0,2)
$S_1[2]$	(1,0,0,1)	(0,0,0,1)	$S_2[2]$	(1,0,1,0)	(3,0,3,0)
$S_1[3]$	(0,0,2,0)	(1,1,2,0)			

$\alpha = SI_1[1]\ SF_1[1] SI_2[1]\ SF_2[1]\ SI_1[2]\ SF_1[2]\ SI_2[2]$ .... secvenţa de execuţie.

	X	Initial	$SI_1[1]$	$SF_1[1]$	$SI_2[1]$	$SF_2[1]$	$SI_1[2]$	$SF_1[2]$	$SI_2[2]$
		R_1,R_2,R_3,R_4	R_1,R_2,R_3,R_4	R_1,R_2,R_3,R_4	R_1,R_2,R_3,R_4	R_1,R_2,R_3,R_4	R_1,R_2,R_3,R_4	R_1,R_2,R_3,R_4	
Alocare	X_1	0 0 0 0	0 1 2 0	0 1 0 0	0 1 0 0	0 1 0 0	1 1 0 1	1 1 0 0	Blocare
X[k]	X_2	0 0 0 0	0 0 0 0	0 0 0 0	2 0 2 2	2 0 2 2	2 0 2 0	2 0 2 0	
Cereri	Y_1	0 1 2 0	0 0 0 0	1 0 0 1	1 0 0 1	1 0 0 1	0 0 0 0	0 0 2 0	
Y[k]	Y_2	2 0 2 2	2 0 2 2	2 0 2 2	0 0 0 0	1 0 1 0	1 0 1 0	1 0 1 0	
V[k]		3 3 3 3	3 2 1 3	3 2 3 3	1 2 1 1	1 2 1 3	0 2 1 2	0 2 1 3	

După evenimentul S1[2], atât Y1[k] ≤ V[k] cât şi Y2[k] ≤ V[k] astfel că P1[3] şi P2[2] nu pot avea loc cu valorile obţinute pentru V[k] înaintea acestor evenimente.

Sistemul s-a blocat deoarece nici C1 nici C2 nu pot continua fără ca unul din ele să fie întrerupt pentru a elibera resursele deţinute.

Algoritm pentru detectarea blocării

Etapele algoritmului sunt:

1°. Iniţializare : $D \leftarrow \{1,2,...,n\}$; $V \leftarrow V(k)$
2°. Caută indicii i din D pentru care $Y_i(k) \leq V$ şi execută pasul 3.
Dacă nu există nici un indice, algoritmul se termină.
3°. $D \leftarrow D \setminus \{i\}$; $V \leftarrow V + X_i(k)$ şi salt la pasul 2.

- Algoritmul se bazează direct pe definiţia de mai sus şi constă din simularea execuţiei sarcinilor până când rămâne un set de sisteme de sarcini $\mathbf{C}_i$ pentru care, în final, resursele disponibile nu sunt suficiente sau mulţimea de indici D devine vidă.
- În cel mai defavorabil caz algoritmul dat analizează la pasul i, întreaga mulţime de sisteme de sarcini din D, în număr de (n-i+1).

Timpul de execuţie al algoritmului este în acest caz proporţional cu

$$m \cdot \sum_{i=1}^{n} (n-i+1) = O(m.n^2)$$

- Printr-o aranjare a structurilor de date şi o reprezentare mai elaborată a stărilor resurselor se poate obţine un algoritm cu timpul de execuţie proporţional cu O(m.n).
- Complexitatea acestui algoritm face ca implementarea lui să fie justificată numai în sisteme în care apariţia unei blocări este intolerabilă.
- În multe sisteme se utilizează o soluţie simplă care se bazează pe limitarea timpului în care sistemul poate sta într-o anumită stare.
- Dacă s-a ajuns la această limită se consideră că s-a realizat o situaţie de blocare şi sistemul este scos din blocare şi trecut într-o anumită stare, predeterminată din care continuă evoluţia.
- Implementarea metodelor de detectare a situaţiilor de "time out" se face atât la nivelul maşinii de bază cât şi la nivelul sistemului de operare.

Evitarea blocării

- Pentru a analiza problema evitării blocării se presupune că o sarcină S constă dintr-o succesiune de paşi.
- Pe parcursul execuţiei unui pas utilizarea resurselor rămâne constantă şi aceasta implică achiziţionarea resurselor necesare pentru acest pas în plus faţă de cele primite la pasul precedent.
- Urmează apoi o perioadă de execuţie în care cerinţele de resurse nu se schimbă.
- În final când se termină execuţia, toate resursele care nu sunt cerute la pasul următor sunt eliberate şi trecute într-o colecţie de resurse disponibile.

- Fie $\sigma = s_0 s_1 ... s_k$ secvenţa de stări corespunzătoare secvenţei de execuţie $\alpha = a_1 ... a_k$.
- Dacă există cel puţin o secvenţa de execuţie completă, validă, care are ca prefix pe α atunci s_k este o stare sigură, altfel starea s_k este o stare nesigură cu pericol de blocare în evoluţia următoare.
- Cu alte cuvinte s_k este *sigură* dacă utilizând resursele disponibile V[k] şi cele care vor fi eliberate de paşii care se execută la momentul respectiv, este posibil să găsim o secvenţă validă de paşi încă neiniţiaţi, pentru sarcinile iniţiate dar neterminate astfel încât toate sarcinile din sistem să poată continua execuţia spre finalizare.
- Starea iniţială în care încă nu s-au alocat resurse, toate fiind disponibile, este intotdeauna o stare sigură.
- Deci există posibilitatea finalizării tuturor sarcinilor.

- Să considerăm în continuare problemele pe care trebuie să le rezolve un supervizor pentru evitarea blocării.
- Să presupunem că sistemul se găseşte în starea s_k sigură şi că există cererile de resurse date de Q[k].
- Supervizorul trebuie să determine dacă orice cerere din Q[k] poate fi satisfacută, $y_{ij}[k] \leq v_j[k]$.
- În general pot fi mai multe astfel de cereri şi deci se pot prevedea diferite *criterii de alocare* pentru a stabili care cerere să fie satisfacută mai întîi, de exemplu:

  FIFO - (First In First Out)-primul venit primul servit

  SJF - (Shortest Job First)- sarcina cea mai scurtă,prima servită;

  ROUND ROBIN - rulare prin rotaţie;

  Priorităţi atribuite sarcinilor;

  etc.

- Pentru a determina dacă o stare s_k este sigură ca urmare a satisfacerii unor cereri de resurse trebuie căutată o secvenţa adecvată de paşi.
- Pentru aceasta se presupune că resursele disponibile sunt completate cu cele alocate sarcinilor în curs de execuţie.
- Aceasta este echivalentă cu presupunerea că primul pas într-o secvenţă analizată nu începe până când toţi paşii în curs de execuţie au toate condiţiile pentru a se finaliza.

Excluderea mutuală
Definiții. Concepte de bază.

- Într-un sistem de sarcini concurente, acestea pot interacţiona în două moduri:
  - indirect, prin disputa pentru aceleaşi resurse;
  - direct, prin transmitere / recepţie de mesaje sau informaţii de stare.

- Când se execută sarcini independente evoluţia acestora poate fi controlată de către supervizor aşa cum am văzut la analiza situaţiilor de blocare.

- Totusi când sarcinile interactionează în mod direct mecanismul de control al evoluţiei lor trebuie să apară în mod explicit în programele care le definesc.

- Interacţiunea directă se efectuează prin mecanisme specifice contextului în care evoluează sistemul de sarcini concurente.

Contextul în care evoluează un sistem de sarcini concurente poate fi:

- centralizat, dacă există o memorie comună accesibilă pentru citire/scriere tuturor sarcinilor, sau
- distribuit dacă nu există memorie comună, ci doar memorie locală accesibilă unei sarcini şi inaccesibilă celorlalte sarcini.

- Excluderea mutuală se ocupă de problemele legate de controlul resurselor reutilizabile (acele resurse a căror stare internă este modificată în timpul utilizării dar care poate fi initializată la atribuirea lor altor sarcini) astfel ca la un moment dat să fie utilizate de o singură sarcină.

- Deoarece sarcinile modifică starea resursei în timpul utilizării, este necesar ca o resursă să fie alocată unei singure sarcini la un moment dat pentru a asigura o execuţie corectă a sarcinilor.
- Pe de altă parte, deoarece o sarcină iniţializează starea unei resurse înainte de a o utiliza, ordinea în care mai multe sarcini o utilizează este indiferentă.
- Astfel orice soluţie pentru excluderea mutuală care implică a priori constrîngeri privind ordinea de execuţie a sarcinilor este neadecvată.
- O soluţie corectă trebuie să se aplice unui sistem de sarcini independente.
- Dacă utilizarea resurselor este sub controlul unui sistem de operare centralizat, la construirea unui sistem de sarcini trebuie să se ţină seama numai de convenţiile prin care se comunică sistemului de operare cererile şi eliberările de resurse.

- Vom considera că excluderea mutuală trebuie rezolvată de însuşi sistemul de sarcini, utilizând *variabile globale* sau variabile de stare şi mesaje pentru comunicaţia între sarcini în funcţie de context: centralizat sau distribuit.

- Sarcinile care trebuie să se execute mutual exclusiv constituie secţiuni critice.

- O soluţie corectă pentru excluderea mutuală trebuie să elimine apariţia unor rezultate diferite sau blocari în tranziţiile sistemului în condiţiile unor timpi de execuţie a sarcinilor diferiţi, pentru diferite procesoare.
- De exemplu: Să considerăm posibilitatea ca două sarcini S_1 şi S_2 care se execută pe procesoare diferite să utilizeze în comun o resursă R.
- S_1 şi S_2 utilizează o variabilă globală care specifică disponibilitatea resursei R.

- S_1 testează variabila, găseşte R disponibilă şi înainte ca S_1 să schimbe valoarea variabilei pentru a arăta că R este ocupată, S_2 testează şi ea variabilă şi găseşte R disponibilă.
- Mai concret S_1 şi S_2 care evoluează într-un context centralizat au primit aceeaşi interpretare, citirea variabilei Semafor, incrementarea valorii citite şi scrierea noii valori.

Fie, de exemplu, secvenţa $SI_1\ SI_2\ SF_1\ SF_2$, deci

1°. S_1 citeşte variabila Semafor;

2°. S_2 citeşte variabila Semafor;

3°. S_1 incrementează şi scrie noua valoare;

4°. S_2 incrementează şi scrie noua valoare.

În urma acestei secvenţe de execuţie, Semafor primeşte valoarea Semafor + 1 în loc de Semafor + 2.

- Rezolvarea cererilor de acces la memorie pentru UCP şi DMA reprezintă un exemplu de excludere mutuală la nivel hardware.
- Scrierea unor texte la o imprimantă comună într-o reţea locală de calculatoare constituie un exemplu de excludere mutuală, la nivel de aplicaţii.
- Accesul la o înregistrare dintr-un fişier pentru a fi citită şi modificată de mai multe sarcini constituie deasemenea un exemplu de excludere mutuala în context distribuit.

- Pentru simplitate vom considera în continuare o singură resursă R care poate fi utilizată de o singură sarcină la un moment dat.

Definiţie:
Fie $\alpha = a_1,a_2,...,a_k$ o secvenţă de execuţie parţială a sistemului de sarcini **C**=(***S***,<). Sarcinile S şi S' sunt mutual exclusive în α dacă şi numai dacă cel mult una din ele este activă după orice prefix a lui α.

Pentru a implementa excluderea mutuală vom considera că o soluţie este corectă dacă cel mult o sarcină este în secţiunea critică la un moment dat şi sunt satisfacute următoarele condiţii adiţionale:

- Nu se face nici o presupunere relativă la activitatea şi numărul de procesoare pe care se execută sistemul de sarcini.

  Desigur, aşa cum s-a arătat şi mai sus, se consideră ca citirea şi scrierea unei variabile de memorie sunt operaţii atomice, indivizibile. Lansarea simultană a unor astfel de operaţii va avea ca efect execuţia lor secvenţială, într-o ordine oarecare, necunoscută apriori.

- Soluţia implementată nu trebuie să fie afectată de vitezele relative ale sarcinilor care se exclud mutual, considerând că aceste viteze sunt diferite de zero.
- O sarcină care operează în afara secţiunii critice nu poate bloca altă sarcină de la intrarea în secţiunea critică.
- Două sarcini care doresc să intre în secţiunea critică nu pot fi într-o buclă de aşteptare reciprocă.

  Deci dacă se eliberează secţiunea critică şi există sarcini în aşteptare, este exclusă posibilitatea ca nici un proces să nu intre în secţiunea critică. În plus, o sarcină care doreşte să intre în secţiunea critică nu poate fi în stare de aşteptare un timp nedefinit.

- Toate sarcinile trebuie tratate la fel, fără priorităţi sau alte privilegii.

Algoritmi de excludere mutuală în mediu centralizat

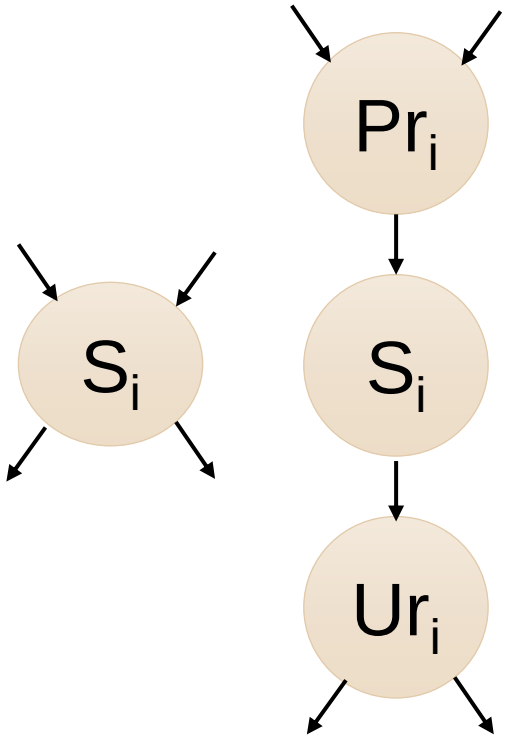
- Complexitatea soluţiei pentru excluderea mutuală depinde de tipurile de operaţii indivizibile care se pot efectua asupra variabilelor globale.
- Astfel dacă se lasă pe seama proiectantului hardware responsabilitatea implementării excluderii mutuale prin operaţii indivizibile mai elaborate, se obţine o soluţie mai simplă pentru excluderea mutuală decât dacă se consideră ca operaţii indivizibile simple citiri şi scrieri în memorie.
- Astfel, sunt calculatoare care au instrucţiuni de tipul Test and Set (Read Modify Write), sau chiar primitive de sincronizare indivizibile, implementate ca operaţii atomice.

- Dacă cel puţin două sarcini independente ale unui sistem **C** trebuie să utilizeze mutual exclusiv resursa R vom modifica **C** pentru a asigura această cerinţă.
- Pentru fiecare S_i care utilizează resursa R vom introduce sarcinile Pr_i şi Ur_i astfel că:

  Pr_i precede imediat sarcina S_i, iar

  Ur_i urmează imediat sarcinii S_i.
- Vom nota cu **C'** sistemul de sarcini extins. Cu toate că S_i rămâne neinterpretat Pr_i şi Ur_i vor fi complet specificate.
- Pr_i va fi astfel proiectat încât să poată fi terminat şi deci să permită iniţierea lui S_i dacă şi numai dacă pentru $\forall\ j \neq i$, Pr_j nu poate fi terminat.
- Funcţia principală a lui Ur_i este de a comunica celorlalte sarcini că S_i a terminat utilizarea lui R.

```
do{
  Entry Section
  Critical Section
  Exit Section
} while (TRUE);
```



- Pentru a proiecta pe Pr_i şi Ur_i astfel ca să nu existe subsecvenţe de execuţie de forma:

$PrF_i \; PrF_j \, UrF_i$

vom considera ca variabile globale:

p - o variabilă de tip întreg.

Q - (Q[0],Q[1],...,Q[n]) - un vector de n+1 variabile de tip întreg.

Iniţial p=0 şi Q[i]=0, $0 \le i \le n$.

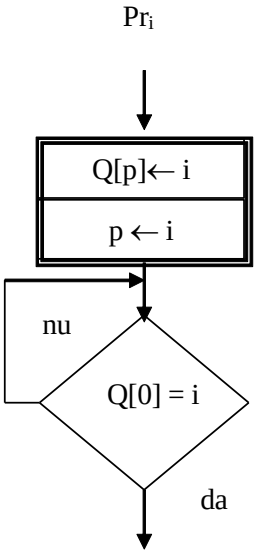
- Funcţia vectorului Q este de a memora într-o ordine FIFO lista sarcinilor ce aşteaptă utilizarea lui R. Acest lucru se realizează prin memorarea indicelui sarcinei care utilizează pe R în Q.
- Astfel, coada de aşteptare la resursa R este organizată ca o simplă listă înlănţuită, memorată în Q.
- *Variabila p* conţine indicele ultimului element din lista (ultimul element intrat în coada Q).
- Condiţia de coadă vidă, considerată ca şi condiţie iniţială este dată de p= 0 şi Q[0] = 0.

Definirea sarcinilor auxiliare Pr si Ur

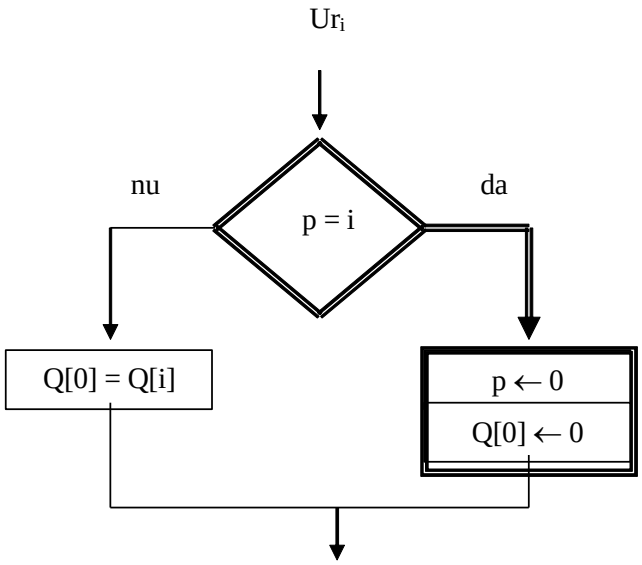
Procesele auxiliare Pr_i şi Ur_i pot fi specificate prin organigramele urmatoare

Sarcina Pr_i arată intenţia lui S_i de a utiliza resursa R, introduce pe S_i în coada Q şi aşteaptă într-o buclă până când S_i ajunge în capul listei Q[0].

Sarcina Ur_i înlătură pe S_i din coadă şi trece următoarea sarcină pentru execuţie în capul listei.
Dacă S_i a fost ultima sarcină introdusă în coadă
(p=i când se execută Ur_i)
atunci coada este vidă şi se trec în zero p şi Q[0].



a) Sarcina Pr_i



b) Sarcina Ur_i

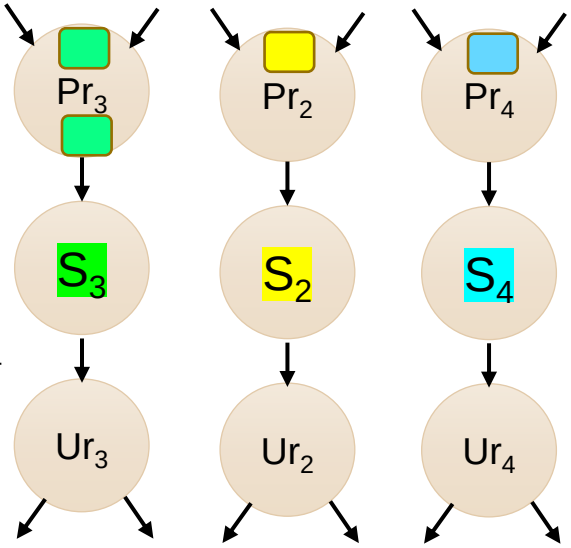
Pentru a putea elimina posibilitatea interferenţelor între Pr_i,Pr_j sau Pr_i,Ur_j ce se execută concurent trebuie ca anumite operaţii asupra variabilelor globale să **fie indivizibile**, cele în chenar dublu.

Fie un sistem de sarcini C=(S,<)

$S=\{S_1,S_2,S_3,S_4,S_5, \ldots S_n\}$

- Se arată modul de reprezentare în coada Q considerând solicitările de a intra în secţiunea critică în ordinea $S_3S_2S_4$.

α= .. PrI_3 PrI_2 PrI_4 PrF_3 SI_3 SF_3 Ur_3 PrF_2 SI_2 SF_2 Ur_2 PrF_4 SI_4 SF_4 Ur_4 ..



p	0				
Q[0]	0				
Q[1]					
Q[2]					
Q[3]					
Q[4]					

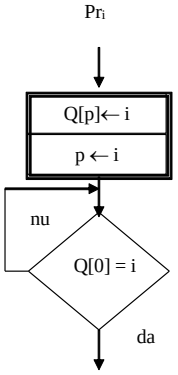
p	0	3			
Q[0]	0	3			
Q[1]					
Q[2]					
Q[3]					
Q[4]					

p	0	3	2		
Q[0]	0	3			
Q[1]					
Q[2]					
Q[3]	2				
Q[4]					

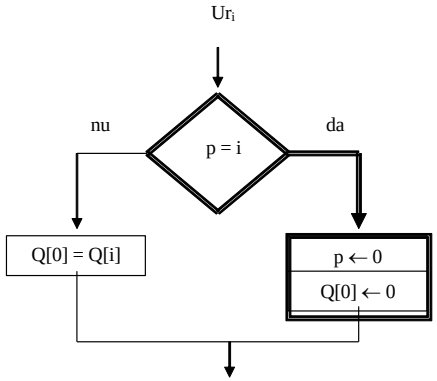
p	0	3	2	4	
Q[0]	0	3			
Q[1]					
Q[2]	4				
Q[3]	2				
Q[4]					

p	0	3	2	4	
Q[0]	0	3 S_3	2		
Q[1]					
Q[2]	4				
Q[3]	2				
Q[4]					

p	0	3	2	4	0
Q[0]	0	3	2 S_2	4	0
Q[1]					
Q[2]	4				
Q[3]	2				
Q[4]					



a) Sarcina Pr_i



b) Sarcina Ur_i

Initial p = 0 şi Q[0] = 0.

Exemplu de interferenţă care conduce la o funcţionare incorectă:

Să presupunem că p=Q[0]=0 şi că Pr_j începe execuţia.

- **Situația 1.** După ce Pr_j se execută Q[0]←j şi înainte de a executa p←j, o sarcină Pr_k cu viteza mai mare în execuţie (pe un procesor mai rapid) execută ambele operaţii **Q[p] ←k ; p ← k**

S_k se va executa imediat, dar cererea pentru Sj s-a pierdut deoarece Q[0] a fost rescris de Pr_k.

Considerând *cele două operaţii indivizibile* **Q[p] ←i ; p ← i**

se elimină astfel de interferenţe.

- **Situația 2.** Să presupunem că p a fost testat de Ur_i şi a fost găsit egal cu i (p=i).

Inainte ca Ur_i să şteargă Q[0] şi p **Q[0] ←0 ; p ← 0** se execută sarcina Pr_k.

Înainte de a se testa egalitatea Q[0]=k în cadrul sarcinii Pr_k, Ur_i se execută mai departe trecând pe Q[0] şi p în zero.

Este evient că Pr_k nu se va executa niciodată, *deci operaţiile implicate de Ur_i trebuie să fie indivizibile.*

- În procedura de realizare a excluderii mutuale arătate anterior **primitiva de aşteptare** Pr_i ocupă procesorul, ceea ce duce la utilizarea ineficientă a acestuia în sisteme în care un număr mare de sarcini intră în competiţie pentru resursa R.
- În acelaşi timp apare o puternică interferenţă la memorie prin accesul la variabilele globale; operaţiile indivizibile fiind destul de lungi.
- Soluţia de excludere mutuală prezentată are marele dezavantaj că necesită execuţia unor operaţii indivizibile complicate, ceea ce de cele mai multe ori nu este posibil prin arhitectura procesoarelor sau chiar dacă este posibil, duce la scăderea performanţelor sistemului.

În continuare vom prezenta o suită de algoritmi, în ordinea dezvoltării lor, care presupun că singurele operaţii indivizibile sunt citirea şi scrierea unei locaţii de memorie.

- Dacă există mai multe cereri de citire sau scriere, ele se vor executa serial, într-o ordine oarecare, nespecificată.
- Prezentarea acestor algoritmi se va face, pe cât posibil, în forma în care au fost publicaţi, desprinderea sarcinilor Pr_i şi Ur_i fiind banală.

Algoritmi de excludere mutuală în mediu centralizat

- **Algoritmul lui Dekker**
- Algoritmul lui Dijkstra
- Algoritmul incorect al lui Hyman
- Algoritmul lui Knuth
- Algoritmul lui De Brujin
- Algoritmul lui Eisenberg şi Mac Quire
- **Algoritmul lui Peterson**
- Algoritmul lui Burns

Dekker si Peterson
IMPORTANT

Algoritmul lui Dekker

- Algoritmul lui Dekker se aplică pentru 2 sarcini S_0 şi S_1 care partajază variabilele:

```
wants_to_enter : array of 2 booleans
turn : integer
wants_to_enter[0] ← false
wants_to_enter[1] ← false
turn ← 0 // or 1
```

Semnificaţia acestor variabile este următoarea:

`wants_to_enter[] ← false` S_i nu doreşte să intre în secţiunea critică.

`wants_to_enter[] ← true` S_i doreşte să intre în secţiunea critică.

`turn = i;` Este rândul lui S_i să intre în secţiunea critică.

- Prin variabilele `wants_to_enter` sarcinile îşi dispută accesul la secţiunea critică.
- Dacă ele doresc simultan să intre în secţiunea critică, vor utiliza variabila comună turn pentru a arbitra această situaţie de simultaneitate.

```
variables
wants_to_enter : array of 2 booleans
turn : integer
wants_to_enter[0] ← false
wants_to_enter[1] ← false
turn ← 0 // or 1
```

S0:

```
wants_to_enter[0] ← true
while wants_to_enter[1] {
  if turn ≠ 0 {
     wants_to_enter[0] ← false
     while turn ≠ 0 {
       // busy wait
     }
     wants_to_enter[0] ← true
   }
}

// critical section
...
turn ← 1
wants_to_enter[0] ← false
// remainder section
```

S1:

```
wants_to_enter[1] ← true
while wants_to_enter[0] {
  if turn ≠ 1 {
     wants_to_enter[1] ← false
     while turn ≠ 1 {
       // busy wait
     }
     wants_to_enter[1] ← true
   }
}

// critical section
...
turn ← 0
wants_to_enter[1] ← false
// remainder section
```

https://en.wikipedia.org/wiki/Dekker%27s_algorithm

Comentarii

- Sarcinile indică intenția de a intra în secțiunea critică care este testată de bucla exterioară while.
- Dacă celălalt proces nu a marcat intenția, secțiunea critică poate fi introdusă în siguranță, indiferent de tura curentă.
- Excluderea reciprocă va fi în continuare garantată, deoarece niciuna dintre sarcini nu poate intra in zona critică înainte de a-și seta indicatorul (ceea ce implică cel puțin o sarcina va intra în bucla while).
- Alternativ, dacă variabila celuilalt proces a fost setată, bucla while este activă și variabila turn va stabili cine are voie să devină critic.
- Sarcinile fără prioritate își vor retrage intenția de a intra în secțiunea critică până când li se va acorda din nou prioritate (bucla interioară while).
- Sarcinile cu prioritate vor intra în secțiunea lor critică.

Comentariu

- Un avantaj al acestui algoritm este că nu necesită instrucțiuni speciale de testare și setare (citire / modificare / scriere atomică) și, prin urmare, este extrem de portabil pe arhitecturi de mașini.
- Un dezavantaj este că este limitat la două sarcni și folosește așteptarea ocupată în loc de suspendarea procesului. (Utilizarea așteptării ocupate sugerează că sarcinile ar trebui să petreacă o cantitate minimă de timp în secțiunea critică.)
- Sistemele de operare moderne oferă primitive de excludere reciprocă, care sunt mai generale și mai flexibile decât algoritmul lui Dekker.
- Cu toate acestea, în absența unei dispute reale între cele două sarcini, intrarea și ieșirea din secțiunea critică sunt extrem de eficiente atunci când este utilizat algoritmul lui Dekker.
- Multe procesoare moderne își execută instrucțiunile ”out of order” Acest algoritm nu va funcționa pe mașinile cu procesoare ”out of order”

Algoritmul lui Peterson

- Algoritmul lui Peterson oferă o descriere algoritmică bună a rezolvării problemei secțiunilor critice și ilustrează unele dintre complexitățile implicate în proiectarea software-ului care răspunde cerințelor de excludere reciprocă, progres și așteptare limitată.
- Intr-un articol cu titlu destul de sugestiv pentru ce o să urmeze, "Miths about the Mutual Exclusion Problem" Peterson a elaborat un algoritm extrem de simplu şi elegant.
- Sunt utilizate doua variabile globale,

  flag - care arată poziţia fiecărui sarcini faţă de secţiunea critică şi turn - care rezolvă situaţiile de simultaneitate.

Algoritmul lui Peterson este:

```
bool flag[2] = {false, false};
int turn;
```

```
S0:      flag[0] = true;
S0_gate: turn = 1;
    while (flag[1] == true && turn == 1)
        {
            // busy wait
        }
        // critical section
        ...
        // end of critical section
        flag[0] = false;
```

```
S1:      flag[1] = true;
S1_gate: turn = 0;
while (flag[0] == true && turn == 0)
        {
            // busy wait
        }
        // critical section
        ...
        // end of critical section
        flag[1] = false;
```

https://en.wikipedia.org/wiki/Peterson%27s_algorithm

Comentariu

- Algoritmul îndeplinește cele trei criterii esențiale pentru a rezolva problema secțiunii critice, cu condiția ca modificările variabilelor turn, flag [0] și flag [1] să se propage imediat și atomic.
- Cele trei criterii sunt excluderea reciprocă, progresul și așteptarea limitată.

Excludere mutuala

- S0 și S1 nu pot fi niciodată în secțiunea critică în același timp:
- Dacă S0 se află în secțiunea critică, atunci flag [0] este adevărat.
  - În plus, fie flag [1] este fals (adică S1 și-a părăsit secțiunea critică), fie turn=0 (adică S1 încearcă acum să intre în secțiunea critică, dar așteaptă),
- Dacă ambele sarcini sunt în secțiunile lor critice, atunci starea trebuie să îndeplinească flag [0] și flag [1] și turn = 0 și turn = 1.
- Nici o stare nu poate satisface atât turn = 0, cât și turn = 1, deci nu poate exista o stare în care ambele sarcini se află în secțiunile lor critice.
- Deoarece turn poate lua una din cele două valori, poate fi înlocuit cu un singur bit, ceea ce înseamnă că algoritmul necesită doar trei biți de memorie.

Progresul: o sarcină nu poate intra imediat din nou în secțiunea critică dacă celălaltă sarcină și-a setat flag-ul pentru a spune că ar dori să intre în secțiunea sa critică.

Așteptarea limitată : în algoritmul lui Peterson, o sarcină nu va aștepta niciodată mai mult de o tură pentru intrarea în secțiunea critică.

Algoritmul lui Peterson pentru mai multe sarcini

- Algoritmul de filtrare generalizează algoritmul lui Peterson la N> 2 procese.
- În loc de un flag boolean, necesită o variabilă întreagă pe sarcină, stocată într-un singur registru atomic cu un scrisă de o singura sarcină și citită de mai multe sarcini (SWMR) și N-1 variabile suplimentare în registre similare.
- Registrele pot fi reprezentate în pseudocod sub formă de tablouri:
- level : array of N integers
- last_to_enter : array of N−1 integers
- Variabilele de nivel iau valori de până la N − 1, fiecare reprezentând o „variabilă de așteptare” distinctă înainte de secțiunea critică.
- Sarcinile avansează de la o variabilă la alta, terminând cu variabila în celula N − 1, care este secțiunea critică. Mai exact, pentru a obține o blocare, sarcina Si va executa

```
i ← Si
for ℓ from 0 to N−1 exclusive
    level[i] ← ℓ
    last_to_enter[ℓ] ← i
    while last_to_enter[ℓ] = i and there exists k ≠ i, such that level[k] ≥ ℓ
        wait
```

Pentru a elibera blocarea la ieșirea din secțiunea critică, sarcina Si stabilește nivelul [i] la -1.

Soluţii hardware de excludere mutuală în mediu centralizat pentru sisteme monoprocesor

- În vederea realizării excluderii mutuale se pot prevedea diferite instrucţiuni sau primitive implementate la nivelul arhitecturii maşinii.
- Există o mare diversitate de astfel de soluţii, iar în continuare vor fi considerate doar câteva exemple mai sugestive.
- Într-un sistem monoprocesor pot exista sarcini concurente care îsi dispută accesul la resurse.
- Apare şi un paralelism efectiv între Unitatea Centrală de prelucrare şi subsistemul de I/E care poate antrena şi alte subansamble, cum ar fi:
  - DMA, canale de I/E, module de memorie,
- Relaţia între UCP şi celelalte subansamble este de tip "master-salve".

- În special datorită sistemului de întreruperi, într-un sistem monoprocesor apare o întreţesere a execuţiei sarcinilor, iar anumite secţiuni ale acestora fiind regiuni critice care trebuie executate în mod exclusiv, fără a fi întrerupte.
- Pentru a realiza aceasta trebuie ca sistemul de intreupere SIntr să poată fi dezactivat / activat sau anumite nivele să fie mascate / demascate.

  Astfel avem:

<început sarcina>;	<început sarcina>;
dezactivare_SIntr;	*mascare_într;*
<secţiune critică>;	<sectiune critică>;
activare_SIntr;	*demascare_într;*
<rest sarcina>;	<rest sarcina>;

- Desigur că ar exista o soluţie pentru excluderea mutuală prin evitarea acesteia, prin eliminarea concurenţei, dar aceasta nu este acceptabilă datorită degradării performanţelor.

Soluţii hardware de excludere mutuală în mediu centralizat pentru sisteme multiprocesor

Pentru sistemele multiprocesor realizarea excluderii mutuale depinde şi mai mult de arhitectura sistemului. În principiu, trebuie realizată execuţia indivizibilă a unor secvenţe de acces la memoria comună.

Dintre soluţiile posibile amintim:

- instrucţiuni atomice de tip XCHG care schimbă între ele două locaţii de memorie sau un registru general şi o locaţie de memorie;
- instrucţiuni atomice de tip TAS (Test And Set) sau RMW (Read Modify Write) care citesc o celulă de memorie, o modifică şi actualizează noua valoare;
- instrucţiuni atomice de tip Lock, Unlock;
- instrucţiuni atomice de incrementare, decrementare;
- instrucţiuni atomice de înlocuire şi adunare.

Excludere mutuală cu instrucțiuni XCHG

- Instrucţiunea XCHG(r,m) interschimbă, ca o operaţie atomică, conţinutul registrului r cu celula de memorie m (semafor).
  - deci r_i este o variabilă locală a sarcinii S_i şi este iniţializată cu 0, iar
  - m este o variabilă globală, comună tuturor sarcinilor care este iniţializată cu 1.
- Ambele variabile pot lua valorile 0, 1.

Algoritmul pentru S_i va fi:

<început sarcina>;

repetă

XCHG(r_i,m);

până_când r_i;

<secţiune critică>;

XCHG(r_i,m);

<rest sarcina>;

- Numai procesul care găseşte m=1 va intra în secţiunea critică şi va trece în zero. La ieşire din secţiunea critică va returna pe m la valoarea 1 pentru a permite celorlalte sarcini să intre în secţiunea critică. (m=1 este o conventie, in unele sisteme se considera m=0)
- Când o sarcina Si intră în secţiunea critică, r_i=1.
- Deci pentru a realiza excluderea mutuală trebuie ca în orice moment să se verifice relaţia:

$$m + \sum_{i=0}^{n-1} r_i = 1,$$

- Microprocesorul x86 include în setul de instrucţiuni o astfel de instrucţiune care prefixată cu LOCK realizează funcţia necesară pentru implementarea excluderii mutuale.

LOCK_Si

```
AST:    MOV     AL,0
        LOCK
        XCHG    AL,m
        TEST    AL,AL
        JZ      AST
```

se intra in executia sarcinii , zona critica

UNLOCK_Si

```
        MOV     m,1
```

LOCK_Sj

```
AST:    MOV     AL,0
        LOCK
        XCHG    AL,m
        TEST    AL,AL
        JZ      AST
```

se intra in executia sarcinii , zona critica

UNLOCK_Sj

```
        MOV     m,1
```

Excluderea mutuală cu instrucțiuni TAS

- Instrucţiunea TAS(m) testează conţinutul lui m şi
  - dacă este 0, m ← 1 şi reîntoarce rezultat adevărat.
  - dacă m este 1, lasă conţinutul nemodificat şi întoarce rezultat fals.
- Variabila globală m este iniţializată cu 0.

Pentru S_i, algoritmul este:

<început sarcina>;

repetă

până_când TAS(m);

<secţiune critică>;

m ← 0;

<rest sarcina>

- Având în vedere că instrucţiunea TAS(m) se execută strict serial, numai sarcina care găseşte m=0 va intra în secţiunea critică, şi va face atribuirea m ← 1.
- La ieşirea din secţiunea critică va face m ← 0 pentru a permite celorlalte sarcini să intre în secţiunea critică.

- Microprocesorul x68000 include în setul de instrucţiuni o instrucţiune TAS indivizibilă, pentru excludere mutuală în structuri multiprocesor.

Excludere mutuală cu instrucțiuni LOCK, UNLOCK

Primitiva LOCK(m)

este echivalentă cu:

{

repetă nimic *cât_timp* m=0;

m ← 0;

}

Primitiva UNLOCK(m)

este echivalentă cu:

{

m ← 1;

}

Pentru familia x86

LOCK_Si

```
AST:    MOV     AL,0
        LOCK
        XCHG    AL,m
        TEST    AL,AL
        JZ      AST
```

se intra in executia sarcinii , zona critica

UNLOCK_Si

```
        MOV     m,1
```

- Iniţializând variabila m cu 1, algoritmul de excludere mutuală este:

<început sarcina>

LOCK(m);

<secţiune critică>;

UNLOCK(m);

<rest sarcini>;

```
LOCK_Si
AST:    MOV     AL,0
        LOCK
        XCHG    AL,m
        TEST    AL,AL
        JZ      AST
```

se intra in executia sarcinii , zona critica

```
UNLOCK_Si
        MOV     m,1
```

- La unele procesoare, această funcţie este realizată de instrucţiuni denumite TSB (Test and Switch Branch).

MICROPROGRAMARE

Noțiuni şi concepte de bază

Organizarea memoriei de control

Suportul fizic pentru păstrarea microprogramelor

Organizarea logică a instrucțiunilor

Codificare verticalã

Codificarea orizontalã

Codificarea minimalã

Codificarea cu control rezidual

Codificarea cu control prin adrese

Codificare mixtã

Implementarea microinstrucțiunilor

NOTIUNI SI CONCEPTE DE BAZĂ



Din punct de vedere structural unităţile de comandă sunt de două tipuri:
-convenţionale în sensul propus de Von Newmann ;
-microprogramate conform conceptului introdus de M.Wilkes.

Microprogramarea este o tehnică de proiectare şi implementare a funcțiilor de control a sistemelor de prelucrare a datelor numerice, ca o secvență de semnale de control ce interpretează static sau dinamic funcțiile de prelucrare a datelor. Semnalele de comandă, necesare la un moment dat pentru controlul primitivelor funcționale sunt organizate într-un cuvânt de control, memorat într-o memorie PROM sau RAM. Structura cuvântului de control este influențată de semnificația atribuită noțiunii de microoperație .

Microoperația (μO) este o primitivă a funcțiilor de prelucrare a datelor, care reprezintă o operație elementară asupra unei primitive funcționale (transfer, acțiune de înscriere sau de incrementare, activare pe magistrală, etc.), ce se desfaşoară de obicei într-o perioadă de timp (perioada de tact sau de ceas).

Microoperația implică existența unui operator, care corespunde unei unități funcționale şi a operanzilor asupra cărora acționează.

Cuvântul de control este pus în corespondență cu noțiunea de microinstrucțiune.

Microinstrucțiunea (μI) este un set de microoperații independente de date, fără conflict de resurse, care se pot executa simultan (pe perioada unei perioade de sincronizare).

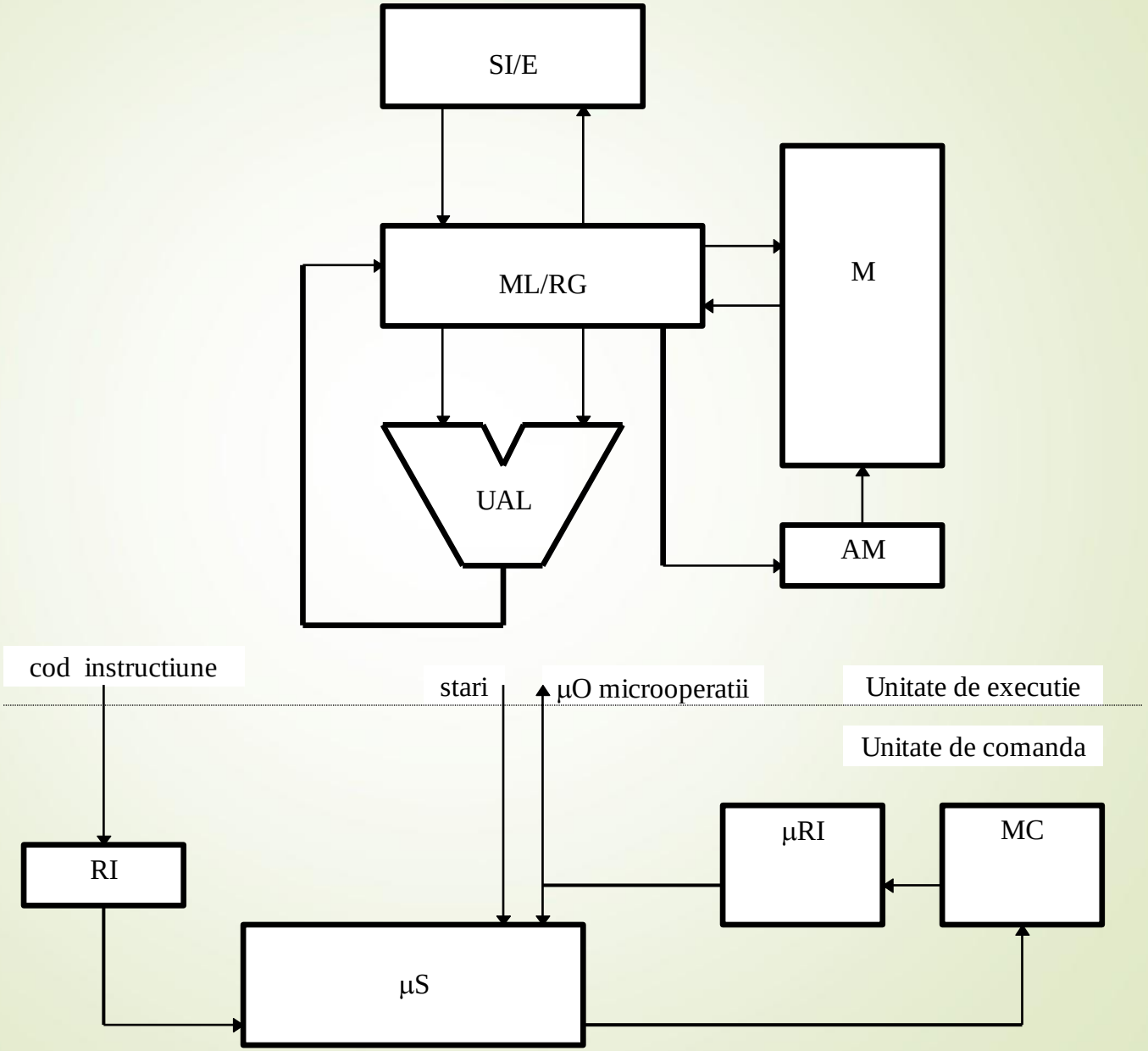
TIPURI DE MICROUNSTRUCTIUNI

- microinstrucțiuni operaționale care controlează primitivele funcționale ale unității de execuție a sistemului numeric, asigurând fluxul de informație şi acțiunile asupra resurselor;
- microinstrucțiune de ramificație (de salt) care inspectează starea primitivelor funcționale şi asigură ramificația în algoritmul de control, constituind suportul pentru implementarea deciziilor.

Prin microprogram se înțelege o secvență de microinstrucțiuni ce implementează un algoritm care descrie :

- citirea interpretarea şi execuția unui set de instrucțiuni maşină;
- primitive ale sistemului de operare;
- primitive ale limbajelor de programare;
- etc.

Structura generală a unui sistem de calcul cu unitate de comandă microprogramată



PRIMITIVE FUNCTIONALE

- **M** - memoria principală a sistemului, în care se păstrează programele ca secvență de instrucțiuni maşină şi datele care se prelucrează ;
- **ML** - memoria locală a sistemului, reprezentată de registrele generale de lucru accesibile sau nu utilizatorului;
- **UAL** - unitatea aritmetică logică ;
- **S I/E** - subsistemul de intrări / ieşiri ;
- **MC** - memoria de control în care se păstrează microprogramul ca secvență de microinstrucțiuni ;
- **μRI** - registrul de microinstrucțiuni, care păstrează microinstrucțiunea curentă ce se execută. Conținutul său specifică toate microoperațiile care se execută în acel moment în unitatea de execuție ;
- **μS** - microsecvențiatorul, unitatea de comandă convențională, elementară, care asigură citirea interpretarea şi execuția microinstrucținilor din memoria de control precum şi înlănțuirea acestora pe baza registrului de instrucțiuni maşină RI şi a stării primitivelor funcționale.



O structură microprogramată se caracterizează prin:

- organizarea memoriei de control ;
- suportul fizic pentru păstrarea microprogramelor
- organizarea logică a microinstrucţiunilor ;
- implementarea microinstrucţiunilor





8.1.1.ORGANIZAREA MEMORIEI DE CONTROL

Dupa criteriu de analiză relația MC față de M:

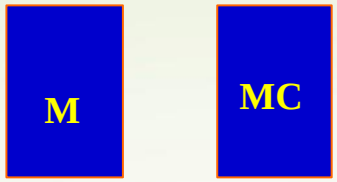
- *memorie de control separată de memoria principală*, atât din punct de vedere fizic cât şi din punct de vedere al adresării logice,
- Raportul dintre viteza de lucru a memoriei de control şi cea a memoriei principale este de circa 10 în favoarea memoriei de control, Vmc >> Vm

- memoria de control este implementată în acelaşi spațiu fizic şi de adresare cu cel al memoriei principale. Ambele au acelaşi ciclu de memorie şi sunt adresate prin aceeaşi logică. Divizarea în memorie de control şi memorie principală se poate face la nivel fizic sau la nivel logic.

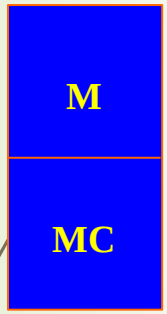
Acest tip de organizare cere ca memoria să fie suficient de rapidă pentru a fi folosită ca MC şi destul de ieftină pentru a fi folosită ca memorie principală.

–memoria de control este implementată separat de memoria principală însă este încărcată din aceasta. Se utilizează un sistem de memorie ierarhică

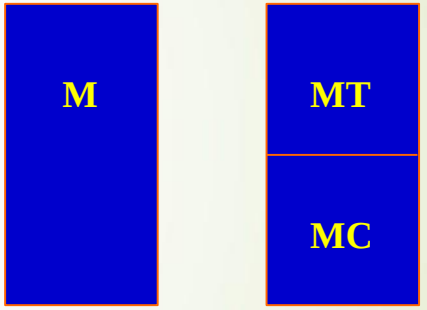
Memoria MC este încărcată din memoria principală prin intermediul memoriei tampon MT.



Organizarea memoriei de control



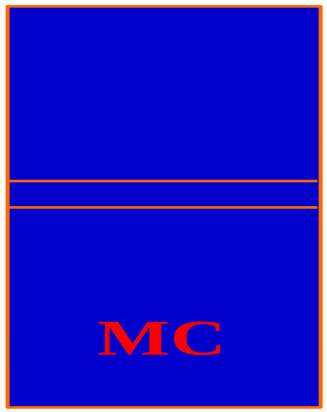
Organizarea MC în cadrul MP



Organizarea MC separată de MP
dar incarcabilă din aceasta

CLASIFICAREA MEMORIEI DE CONTROL MC

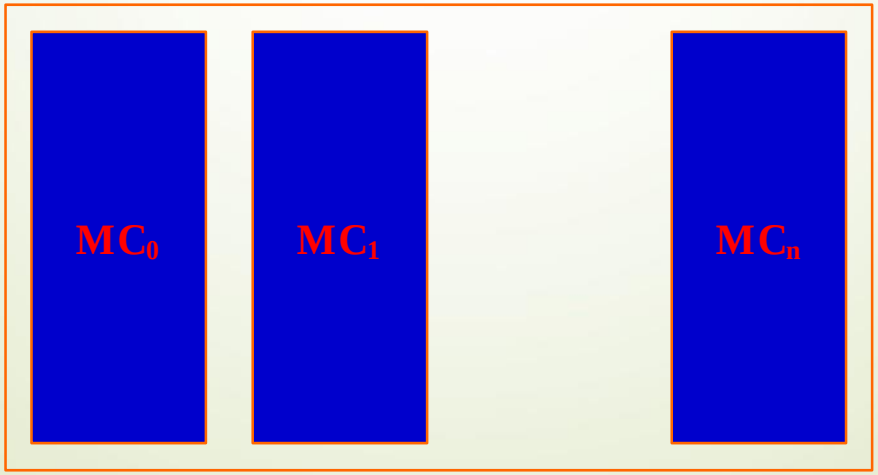
1. ***memorie de control cu o microinstrucțiune pe cuvânt,*** în care fiecărui cuvânt de memorie îi corespunde o singură microinstrucțiune. Citirea unei microinstrucțiuni presupune un singur acces la MC.



2. *memorie de control cu organizare pe pagini*

Unei adrese din memoria de control i se asociază mai multe microinstrucţiuni, din pagini diferite. în acest fel se asigură la nivel de microprogram execuţia unei microinstrucţiuni de tip CASE, care introduce facilitatea de decizii multiple.

O adresă de MC va adresa aceeaşi locaţie în toate paginile mémoriei de control iar vectorul de condiţii de test va activa pagina ce specifică microinstrucţiunea următoare care se va executa.



3. *memorie de control cu organizare pe blocuri*

Pentru acest tip de organizare există două feluri de adrese:

-adrese de microinstrucţiuni din acelasi bloc cu microinstrucţiunea curentă ;

-adrese de blocuri.

Împărţirea memoriei de microprograme în blocuri se face ţinând seama atât de structura microprogramului cât şi de resursele fizice disponibile. O organizare de acest fel conduce, în general, la micşorarea lungimii microinstrucţiunii însă introduce timp suplimentar cu comutarea adreselor de blocuri.



4. *memorie de control divizată*

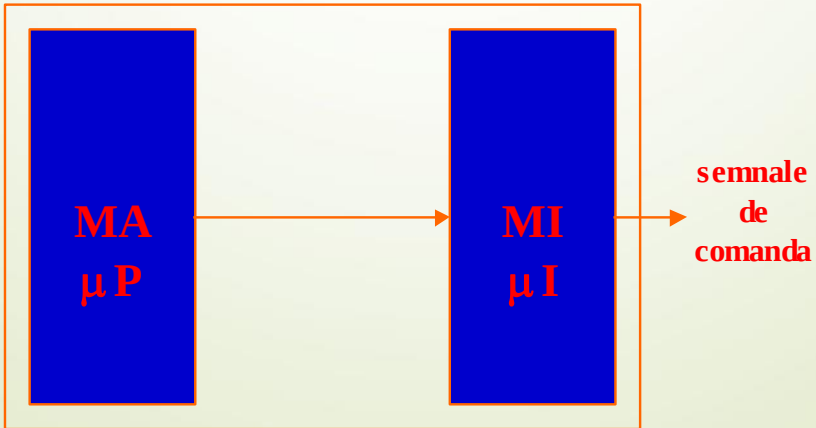
Memoria de microprograme divizată este alcătuită din două unităţi de memorie distincte :

MI - memorie de microinstrucţiuni, care păstrează toate microinstruciunile distincte posibile necesare pentru controlul resurselor fizice ;

MA - memorie de adrese de microinstrucţiuni, care păstrează programul specificat, nu prin microinstrucţiuni ce controlează resursele fizice, ci prin adrese de microinstrucţiuni (adrese pentru memoria MI).

În general numărul de tipuri de microinstrucţiuni distincte este mult mai redus decât microprogramul în sine, ceea ce implică ca numărul de biţi necesari pentru adresarea memoriei MI să fie şi el redus. In acest fel, lungimea cuvântului din memoria MA este mult mai mic decât al memoriei MI, ceea ce conduce la o reducere substanţială a memoriei de control.

În schimb, pentru a executa o microinstrucţiune trebuie făcute două adresări, una la memoria MA şi una la memoria MI ceea ce conduce la un ciclu mai mare de microinstrucţiune.



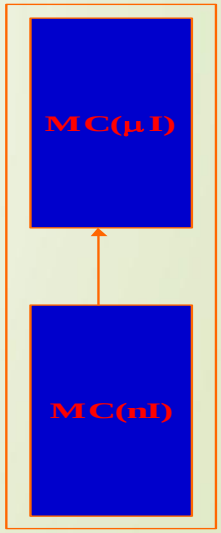
5. *memorie de control structurată pe două niveluri*

La o astfel de organizare mecanismul citirii interpretării şi execuţiei unei instrucţiuni maşină este următorul :

- o instrucţiune maşină este interpretată de un set de microinstrucţiuni rezident în memoria de control MC (μI) ;
- la rândul ei, o microinstrucţiune este interpretată de o secvenţă de nanoinstrucţiuni rezidentă în memoria MC (nI) .

Această tehnică a nanoprogramării este conceptual echivalentă cu microprogramarea, structurarea pe două niveluri oferă o flexibilitate mai mare şi posibiltatea implementării unor structuri de control foarte complexe.

O astfel de organizare întâlnim la calculatorul NANODATA QM1.





SUPORTUL FIZIC PENTRU PASTRAREA MICROPROGRAMELOR

Memoria de microprograme MC este o unitate de memorie de mare viteză care păstrează microprogramele ce se execută.

În ceea ce priveşte suportul fizic, pentru păstrarea microprogramelor, acesta poate fi realizat cu memorii PROM sau cu memorii RAM.

Realizarea memoriei de control :

-cu componente de tip RAM conferă o caracteristică statică

- componentelor de tip RAM oferă o caracteristică dinamică ce permite utilizatorului ca printr-un ansamblu de mijloace software să aibă acces la microprogramul din memoria de control..

Este foarte important în a face deosebire între :

- **maşini microprogramate, şi**
- **maşini microprogramabile**

Prima categorie se referă la modalitatea de implementare a unităţii de comandă în sensul conceptului introdus de Wilkes fără a oferi resursele hardware şi suportul de programe pentru accesul utilizatorului la nivelul microprogramului.

Cea de a două categorie oferă atât resursele hardware cât şi facilităţile software pentru accesul utilizatorului la nivelul microinstrucţiunilor.



ORGANIZAREA LOGICA A INSTRUCTIUNILOR

Un cuvânt din memoria de control specifică un set de microoperații ce constituie componentele primitive ale controlului resurselor sistemului. Organizarea logică a microinstrucțiunilor este influențată de :

- gradul de paralelism între microoperații, ce se doreşte realizat;
- structura maşinii de bază ;
- gradul de codificare sau de flexibilitate dorit ;
- gradul de optimizare al lungimii cuvântului de control .

Presupunând că microprogramele sunt specificate ca o secvență de seturi disjuncte de microoperații, se pot distinge mai multe modalități de codificare a acestora şi anume:

- codificare verticală sau maximală ;
- codificare orizontală sau cu control direct ;
- codificare minimală ;
- codificare cu control rezidual ;
- codificare cu control prin adrese ;
- codificare mixtă.



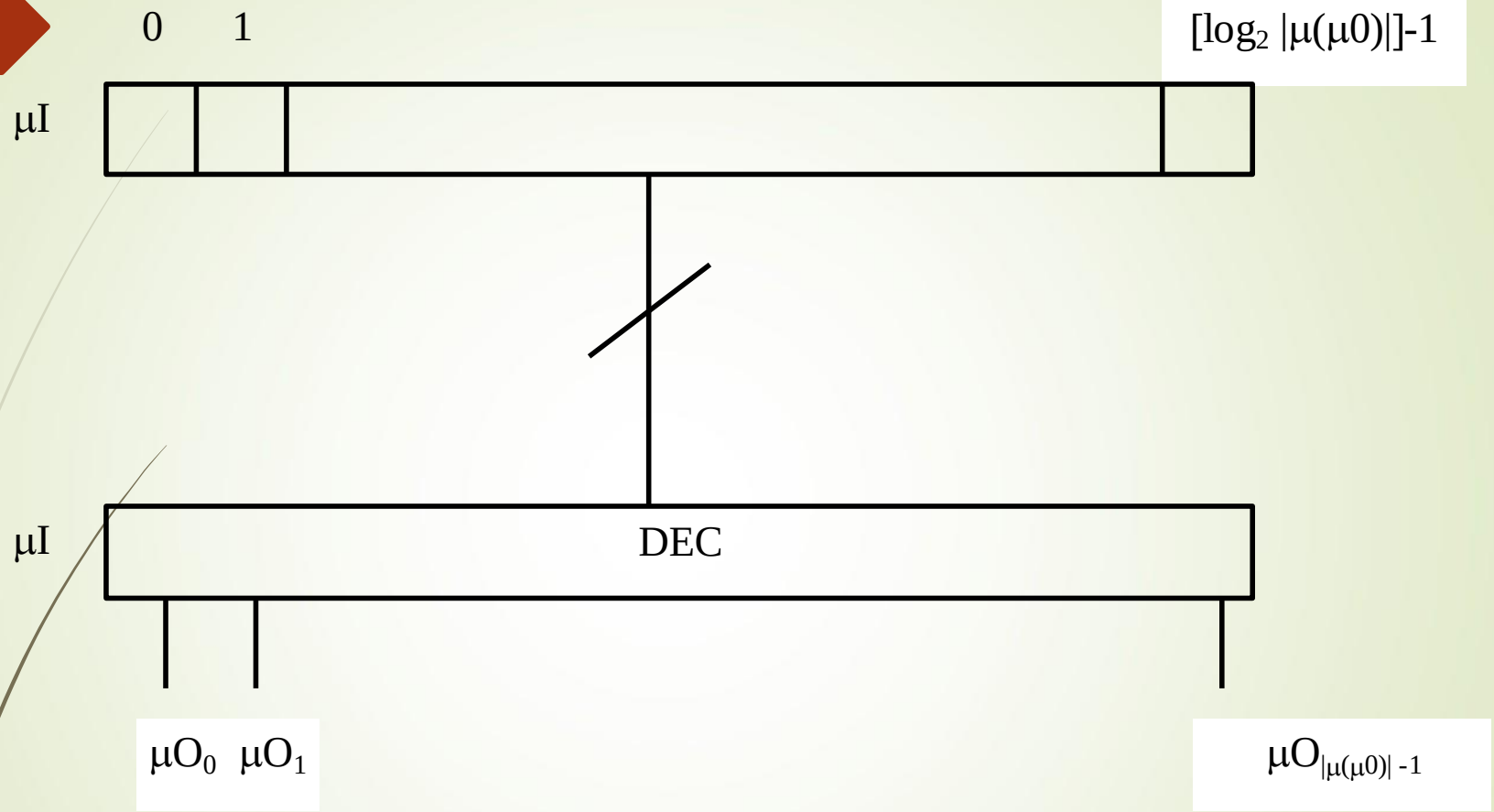
CODIFICARE VERTICALA

Fiecare microinstrucțiune operațională specifică o singură microoperație. Setul de microoperații (MO) necesar pentru controlul primitivelor funcționale se codifică în [$\log_2$ | (MO)] biți, ce constituie lungimea cuvântului din MC.

Această codificare reprezintă un caz extrem, deoarece elimină orice posibilitate de desfăşurare paralelă a operațiilor elementare.

Din punctul de vedere al minimizării cuvântului de control, codificarea verticală implică numărul cel mai mic de biți. Dimensiunea mare a decodificatorului face ca realizarea fizică a acestuia să aibe loc pe mai multe niveluri, ceea ce conduce la introducerea de întârzieri

Un dezavantaj major al codificării maximale îl reprezintă eliminarea controlului paralel asupra resurselor precum şi inflexibilitatea dezvoltării sau completării sistemului în ceea ce priveşte introducerea de noi microoperații. Este aplicabilă numai în sisteme dedicate care au o structură specifică.



Codificarea verticală a microoperațiilor

CODIFICAREA ORIZONTALA

În cadrul acestei codificări, fiecare microoperație din setul (MO) este pusă în corespondență cu un bit din cadrul cuvântului de control. Controlul microoperațiilor se face în mod direct.



Codificarea orizontala

Codificarea orizontală realizează controlul tuturor microoperațiilor paralele, posibile, ce se pot desfăşura în sistem.

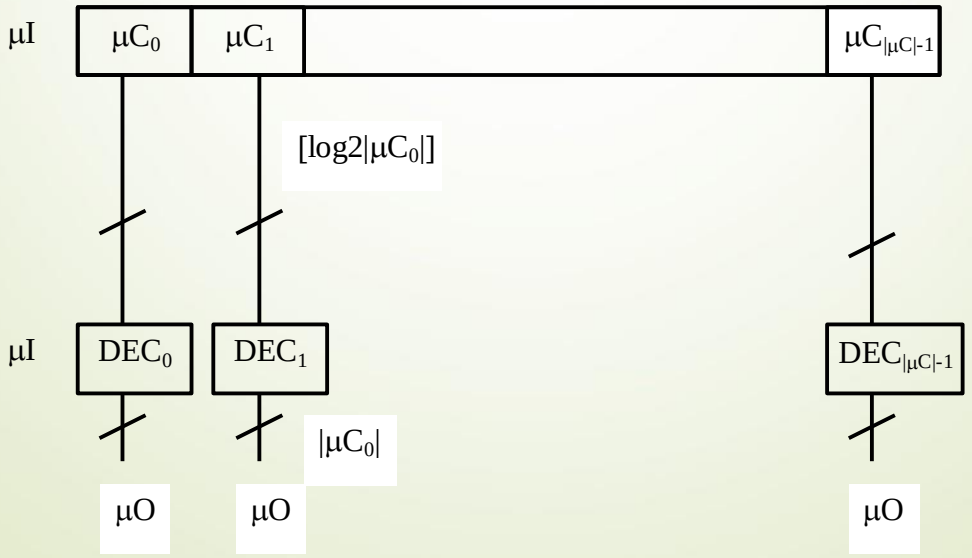
Deşi oferă o flexibilitate mare şi asigură paralelismul maxim, utilizarea acestei codificări este un caz extrem datorită folosirii ineficiente a memoriei de control.

CODIFICAREA MINIMALA

Combină flexibilitatea şi paralelismul potențial oferite de codificarea orizontală cu eficiența codificării verticale.

Ideea de bază este de a grupa în clase de compatibilitate setul de microoperații care se exclud reciproc (μO dintr-o clasă de compatibilitate nu se vor efectua niciodată simultan).

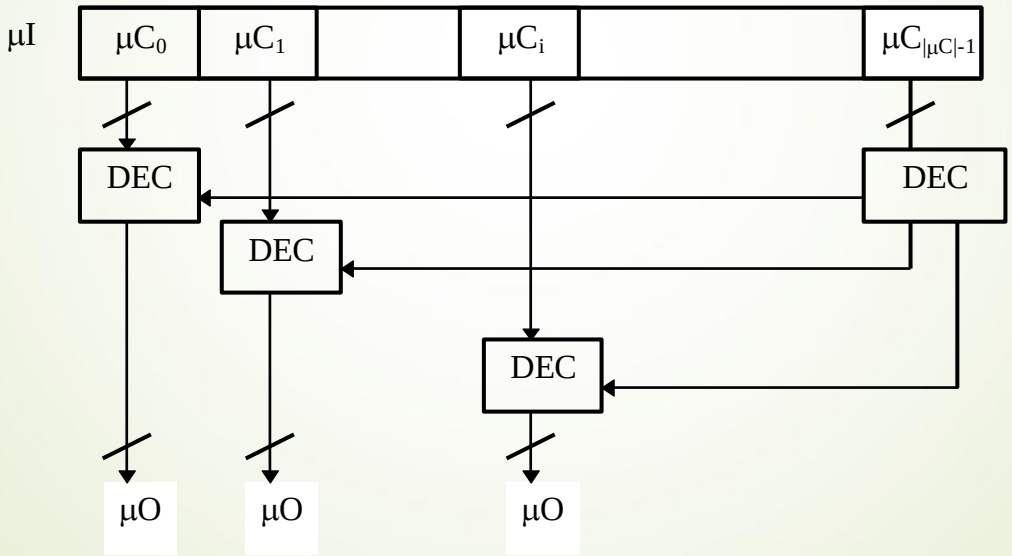
Microinstrucțiunea este împărțită în câmpuri. Un câmp corespunde unei clase de compatibilitate. La nivel de câmpuri se realizează o codificare orizontală iar în cadrul câmpurilor se realizează o codificare verticală.



Pentru un câmp μC_j care codifică $|\mu C_j|$ microoperații sunt necesari $|\log_2(|\mu C_j|+1)|$ biți, deoarece trebuie să se prevadă şi posibilitatea de a nu specifica nici o microoperație din cadrul câmpului.

O variantă a acestei codificări o reprezintă codificarea pe două niveluri sau indirectă.

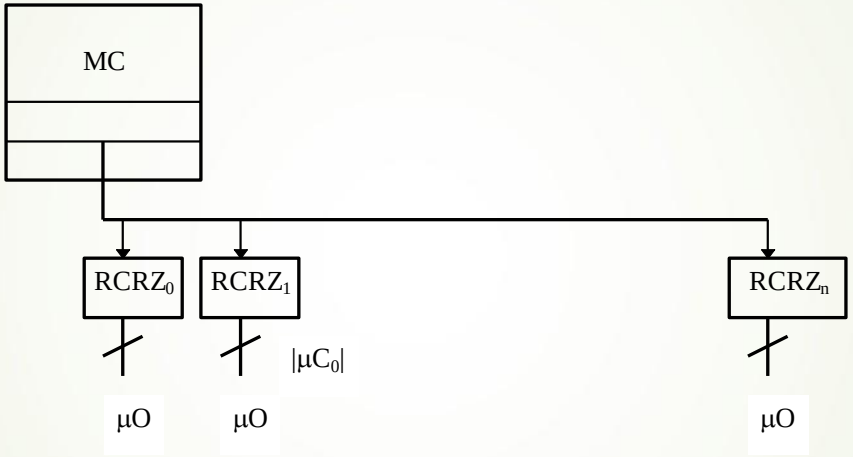
În codificarea pe două niveluri, validarea unor câmpuri depinde de valoarea altui câmp de control din microinstrucțiune.



Codificare minimală pe două niveluri

CODIFICAREA CU CONTROL REZIDUAL

Această metodă de codificare foloseşte pentru controlul primitivelor funcționale registre de control rezidual. Cuvântul de control nu controlează direct resursele ci prin intermediul registrelor de control rezidual încărcate sub acțiunea microinstrucțiunilor



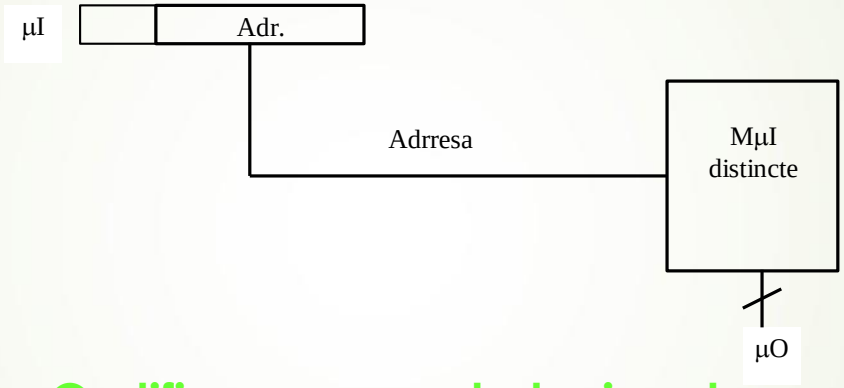
Codificare cu control rezidual

Microinstrucțiunile pot inlocui sau modifica valoarea unuia sau mai multor registre de control. Astfel, se asigură o economie de memorie de control atunci când unele primitive funcționale realizează aceeaşi operație în mod repetat sau când un set de microoperații este activ o perioadă mare de timp, iar alte seturi de microoperații se modifică.

Registrele de control rezidual $RCRZ_j$, care specifică microoperațiile de control al resurselor hardware pot fi manevrate cu ajutorul unor

CODIFICAREA CU CONTROL PRIN ADRESE

O modalitate de implementare a microinstrucțiunilor operaționale este aceea în care se specifică o adresă în cadrul unei memorii, unde sunt memorate toate microinstrucțiunile distincte posibile ce controlează sistemul.



Codificare cu control prin adrese

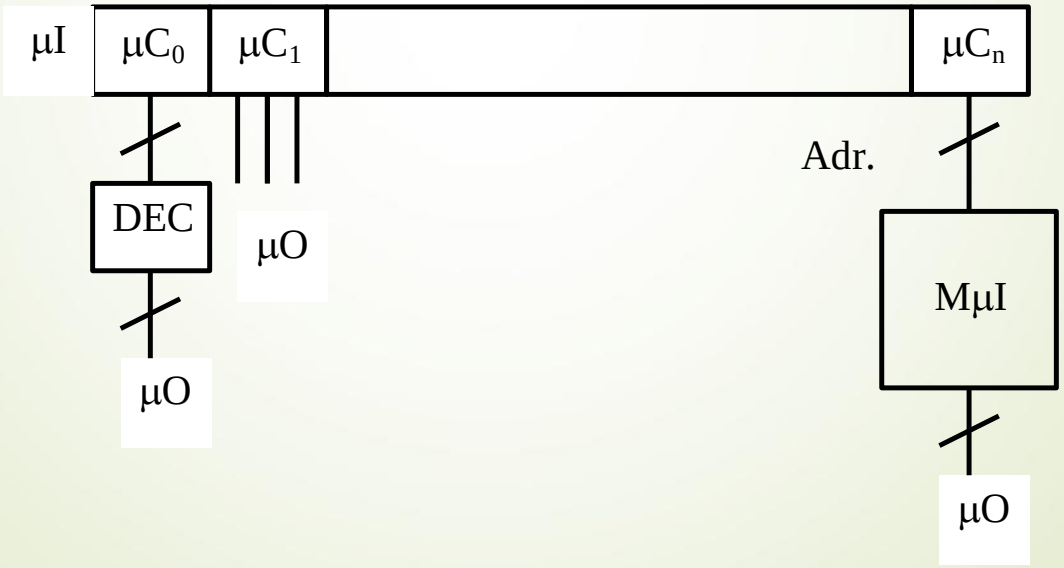
Numărul de microinstrucțiuni distincte nu depinde de numărul de resurse controlate ci de numărul de μO disticte, de mărimea μP şi de numărul de variabile de stare testate.

Memoria care păstrează microinstrucțiunile distincte va avea lungimea cuvântului suficient de mare pentru a controla toate microoperațiile care se pot efectua simultan. Trebuie notat că fiecare μl este memorată o singură dată.

O astfel de implementare, face ca microprogramul să fie format dintr-o secvență de adrese care apelează μl păstrate în memoria de μl.

CODIFICAREA MIXTA

O variantă utilizată mult în practică este aceea în care microinstrucțiunea este împărțită în câmpuri. Unele câmpuri controlează direct microoperațiile (sub formă codificată sau directă) iar altele specifică adrese de memorie ce conține un subset de microinstrucțiuni distincte.



IMPLEMENTAREA INSTRUCTIUNILOR



Microinstrucțiunile sunt citite, interpretate şi executate de către microsecventiator (μS) în acelaşi fel în care o unitate de comandă convențională execută instrucțiuni maşină.

Se poate defini o caracteristică serie-paralel care măsoară cantitatea de suprapunere între faza de execuție a μl curente şi citirea, interpretarea μl următoare.

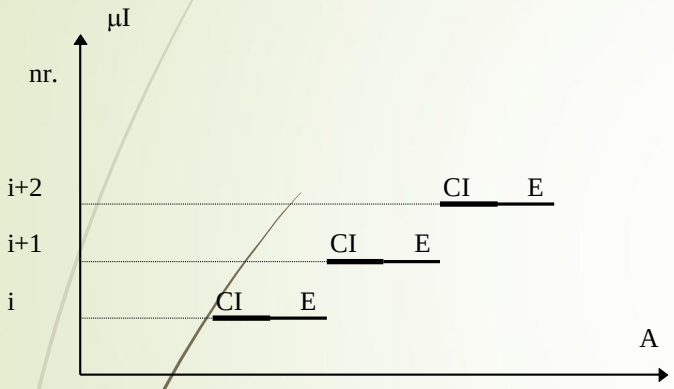
Din punctul de vedere al caracteristicii serie-paralel distingem trei tipuri de implementări :

- implementare serie;
- implementare serie-paralelă ;
- implementare paralelă.

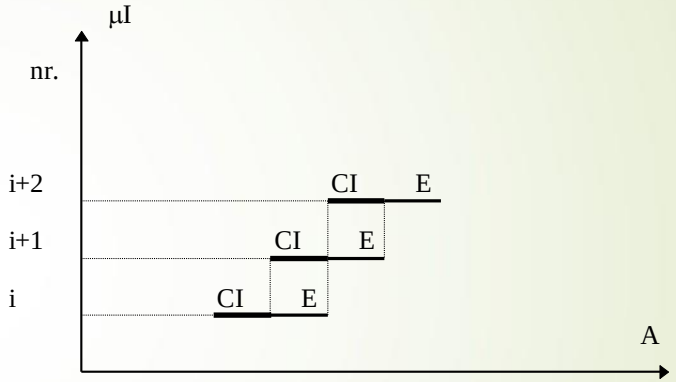
Fie CI-faza de citire interpretare şi E-faza de execuție a unei μl.

În implementarea serie, citirea microinstrucțiunii următoare nu începe decât după ce s-a terminat execuția microinstrucțiunii curente.

În implementarea paralelă, Fig. 9.17, faza de citire a μl următoare se desfăşoară în acelaşi timp cu execuția μl curente.

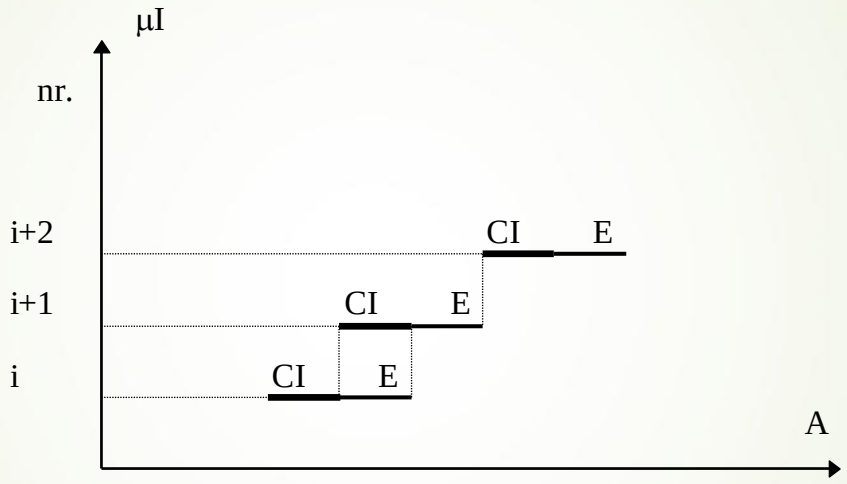


Implementare serie



Implementare paralel

O îmbinare a performanțelor implementării paralele şi a costului redus al implementării serie este realizată de implementarea serie-paralelă a microinstrucțiunilor din figura urmatoare.



Implementare serie-paralel

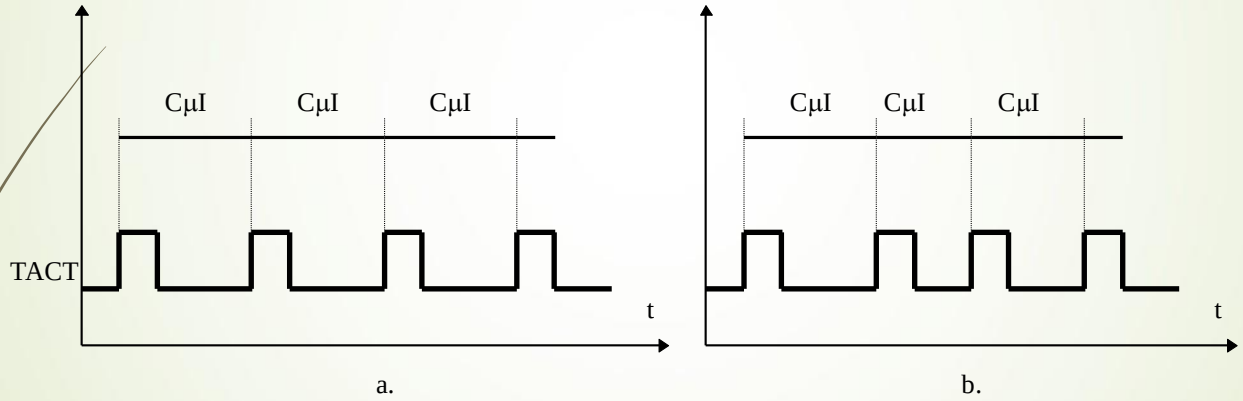
Faza de citire a μl următoare se desfăşoară fie în timpul execuției μl curente, fie după terminarea ei, în funcție de tipul microinstrucțiunii curente.

Succesiunea de microinstrucțini operaționale se desfăşoară prin suprapunerea fazelor de citire şi execuție, iar cele imediat următoare unor ramificații se citesc după terminarea execuției μl curente (de ramificație).

Implementarea serie-paralelă este caracteristică arhitecturilor "pipe line", şi este cea mai folosită.

Referitor la implementarea microinstrucțiunilor se poate defini şi o caracteristică monofază-polifază care se referă la numărul de faze utilizate într-un ciclu de microinstrucțiune.

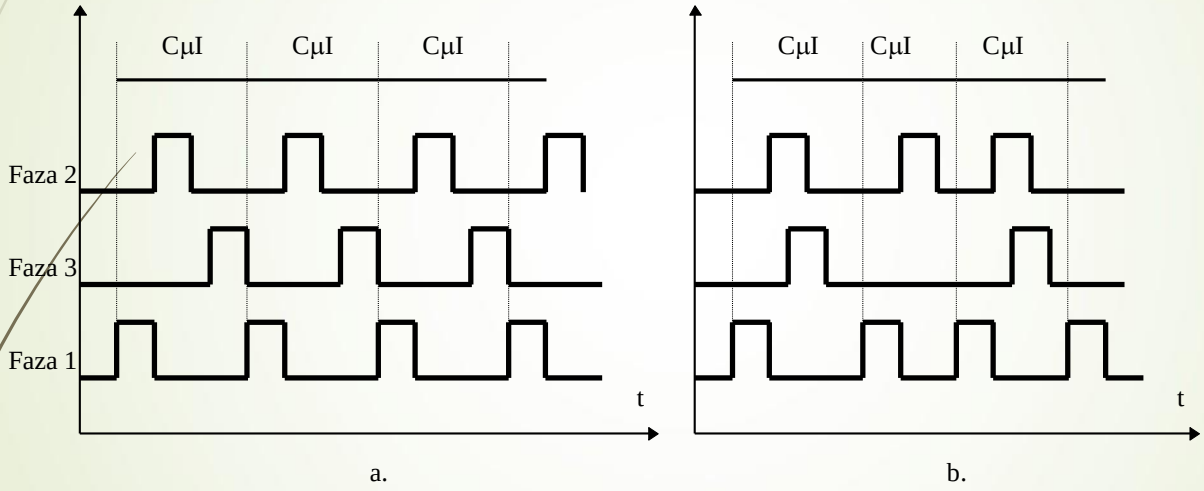
Într-o implementare monofază, microoperațiile sunt generate simultan, toate semnalele de control specificate de microinstrucțiune fiind active în acelaşi timp (pentru cele de tip impuls se ține seama de front).



Implementare monofază

a. ciclul macroinstrucțiunii CμI constant

b. ciclul macroinstrucțiunii CμI variabil



Implementare polifază

a. ciclul microinstructiunii constant

b. ciclul microinstructiunii variabil

Determinarea paralelismului între operații elementare în cadrul structurilor numerice.

- La nivelul unității de comandă a structurii numerice, e necesar să se implementeze controlul concurent al op. elementare.
- Obiectivul problemei de determinare a microoperațiilor paralele constă în a stabili, pe baza:
  - grafului de dependențe de date şi
  - a cererilor de resurse,

  seturile de microoperații ce pot controla resurse diferite ale sistemului simultan.
- Obs: Conceptele si algoritmii prezentati pot fi aplicati şi la execuția task-urilor.

Definitii

- **Def.1** Se numeşte microsubbloc **µSB** un subset maxim al unui set de microoperaţii ce constituie o secvență indivizibilă cu un singur punct de intrare şi un singur punct de ieşire şi conține maximum o microoperație de salt, ca ultimă operație din secvență.
- µSB(µo)=(µB(µo), <) unde
  - µB(µo)= {μo_1, μo_2, μo_3, … , $\mu o_{|\mu B|}$ }
  - < relatie de ordine nereflexiva
- **Def.2 µOd** – microoperatie disponibila- acea microopertaie pentru care toate microoperatiile de care ea era dependenta de date au fost asignate unor microinstrucțiuni.

$$\mu O_d = \{\mu O_j / \forall \mu O_i \angle \mu O_j \Rightarrow \mu O_i \in \mu I\}$$

- **Def. 3** Prin nivel într-un subbloc **μON_k** înțelegem toate microoperatiile independente situate la aceeaşi distanță (adâncime) față de nodul inițial.

Def.4 Setul de operatii disponibile la nivelul k, μOD_k, este construit din multimea de microoperatii disponibile din cadrul asociat microoperatiilor la nivelul k de generare de seturi paralele de microoperatii.

PROP.1. Microoperatiile din seturile de microoperatii disponibile sunt independente.

Presupunem ca nu ar fi independente. Daca avem

$$\mu O_i \in \mu OD_k \qquad \mu O_j \in \mu OD_k$$

si nu sunt independente, atunci fie $\mu O_i < \mu O_j$ sau $\mu O_j < \mu O_i$.

Conform Def 2 $\forall \mu O_l < \mu O_j \qquad \mu O_j \in \mu OD_k$

rezulta ca μO_l a fost asignata (atribuita) unei microinstructiuni. Rezulta ca μO_l nu poate face parte din μOD_k si singura solutie este ca ele sa fie independente.

$|\mu ON|$ - numarul de niveluri dintr-un microsubloc egale cu timpul minim de executie pentru microsublocul respectiv

Fiecare nivel va specifica toate microoperatiile paralele care se desfasoara la un anumit moment de timp.

nmμl- numarul minim de microinstructiuni ce ar putea fi generate intr-un microsubloc

Evaluare estimativa:

- $\boldsymbol{nm\mu I} = \frac{|\mu oB(\mu O)|}{nR}$ - multimea de microoperatii din bloc
- unde:

$$nR - numarul\ de\ resurse\ ale\ sistemului$$

$\mu CR = \emptyset$ conflictul intre resurse

nMμl – numarul maxim de microinstructiuni

nMμl = |MB(μO)|

- toate microoperatiile se executa secvential

nmμl, nMμl sunt cazuri extreme.

nOμl- numarul optim de microinstructiuni

μON= |μON| = cardinalul nivelelor pe care le stabilim

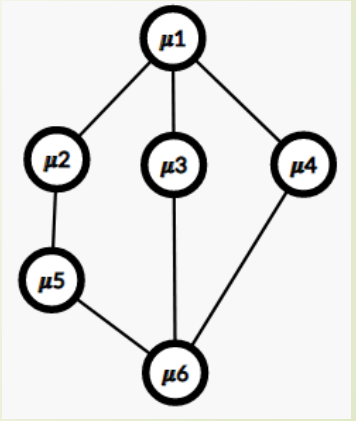
$$\forall \mu ON_j \in \{\mu ON\} \Rightarrow \mu CR(\mu ON_j) = \emptyset$$

- **μPT_I – partitia cea mai timpurie de executie a unei μoperatii.**

  Aceasta este constituita din multimea de niveluri care contin toate μoperatiile independente situate la aceeasi distanta fata de un nod initial in graful de dependente de date.

- $\boldsymbol{\mu PT_l = \{\mu ON | \forall \mu O_i, \mu O_J \in \mu ON \ avem \ DI(\mu O_i) = DI(\mu O_j)\}}$
- unde prin DI am notat adancimea fata de noul initial in graful de dependente de date.
- **μPT_L – partitia cea mai tarzie de executie a unei μoperatii**
- $\boldsymbol{\mu PT_L = \{\mu ON | \forall \mu O_i, \mu O_J \in \mu CM \ avem \ DF(\mu O_i) = DF(\mu O_j)\}}$
- unde prin DF am notat adancimea fata de nivelul final (nodul final)

$$\mu PT_l = \{< \mu O_1 >; < \mu O_2, \mu O_3, \mu O_4 >; < \mu O_5 >; < \mu O_6 >\}$$
$$\mu PT_L = \{< \mu O_1 >; < \mu O_2 >; < \mu O_3, \mu O_4, \mu O_5 >; < \mu O_6 >\}$$



Plecand de la cele doua definitii , se numeste microoperatie critica o microoperatie care apartine aceluiasi subnivel in ambele partitii (μPT_I, μPT_L).

Ex: in cazul anterior avem 2 si 5

Orice modificare (mutare pe alt nivel) pentru o microoperatie critica conduce la un numar suplimentar de momente de timp pentru executie. Pentru celelalte (necritice), le putem pune oriunde intre nivelele date din partitia cea mai timpurie si cea mai tarzie, fara ca pentru executie sa avem necesar un timp suplimentar.

PARTITIA OPERATIILOR ELEMENTARE PE NIVELURI DE EXECUTIE

Partitiile stabilite numai pe baza dependentei de date nu sunt optime, intrucat intervine si conflict de resurse. Pentru a pune in evidenta acest conflict, in cadrul partitiilor se face o grupare dupa tipul de resurse utilizate de fiecare microoperatie.

O micropartitie este egala cu multimea de niveluri.

$$\mu PT = \{\mu ON_1, \mu ON_2, \ldots, \mu ON_{|\mu PT|}\}$$

$$care\ reprezint a\ \ multimea\ de\ niveluri\ in\ care$$

$$\mu ON_j = \{\mu ON_{j1}, \ldots, \mu ON_{j|nR|}\}$$

$$N_{ji} = \{\mu O_h \,|\, \forall \mu O_h \in \mu ON_j \rightarrow \mu O_h\ utilizeaza\ aceeasi\ RES_i\};\ 1 \leq i \leq NR$$

RES_i – resursa i

μON_j- reprezinta nivelul j in cadrul partitiei, care este format din subniveluri care utilizeaza aceeasi resursa

Este bine ca un nivel sa-l impartim in subniveluri . Pentru cele care folosesc aceeasi resursa, intre ele avem conflicte.

Trebuie executate secvential.

- DEF Cererea de resurse QRES este o aplicatie definita pe μB (multimea de resurse din bloc): $\mu oB \times RES \rightarrow N$

  unde $RES = RESS \cup RESD \cup RESB \cup RESULC \cup RESSH.$

  RESS – resursa de tip sursa

  RESD – resursa de tip destinati

  RESB – resursa de tip bus

  ULC – unitati logice combinationale

  SH – shuffler

  $DRES_i$ – resurse de tipul i disponibile

  $QRES_I$ – resurse de tipul i cerute

  $DRES = \{DRES_1, DRES_2, \ldots, DRES_{nR}\}$

  $QRESj \quad \{QRES_{j1}, QRES_2, \ldots, QRES_{j,nR}\}$

  $- vectorul\ de\ cereri\ de\ resurse\ necesare\ microoperatiilor$

  $care\ apartin\ nivelului\ j\ (\mu ON_j)$

  $QRES_{ji} = \sum \mu O_h \ \ sau\ QRES_{ji} = \sum QRES_{\mu O_h},$

  unde $\mu O_h \in \mu ON_j$ care utilizeaza resursa i

- Daca cererea de resurse este mai mare decat disponibilul de resurse, atunci pe nivelul j care foloseste resursa i apare conflict de resurse.

- $\boldsymbol{QRE_{S_{ji}} > DRE_{S_i} \Rightarrow \mu CR(\mu O_{N_{ji}}) = 1}$

Se numeste microinstructiune completa μIC acea microinstructiune la care nu se poate adauga nici o operatie din setul de microoperatii disponibile, fara a se crea conflict de interese.

$$\mu IC: \forall \mu I(\forall \mu O_i \in \mu OD, \mu O_i \notin \mu I \Rightarrow \mu CR(\mu O_i, \mu I) = 1$$

In cazul in care cererea de resurse:

$$QRES_{ji} > DRES_i \Rightarrow \mu IC \subset \mu ON_j; \mu ON_j \backslash \mu IC \neq \emptyset$$

Acest lucru inseamna ca exista microoperatii din nivelul j care vor fi trecute pe nivelele urmatoare datorita conflictului de resurse.

$$QRES_{ji} > DRES_i \Rightarrow \mu IC \subset \mu ON_j; \mu ON_j \backslash \mu IC \neq \emptyset$$

DEFINITII

Intarzierea in ceea ce priveste numarul de niveluri cu care se va muta o microoperatie din nodul curent j spre nodul final din graful de precedenta datorita conflictului de resurse i:

$$nN_{ji} = \begin{cases} 0 & , \ daca \ QRES_{ji} \leq DRES_I \\ \left[\frac{QRES_{ji} - DRES_i}{DRES_i}\right] & , \ daca \ QRES_{ji} > DRES_i \end{cases}$$

Intarzierea maxima pe nivelul j:

$$nN_j = max(nN_{ji}) \ , \ unde \ i = 1,2,\ldots nR$$

Stabilirea unei metodologii de determinare a partitionarilor unui microsubbloc in microinstructiuni complete (care sunt actiunile pe care le fac la un moment dat).

Stabilirea efectului setului de microoperatii disponibile asupra generarii partitiilor.

Pentru a genera o singura microinstructiune completa din μOD,

ar trebui sa existe un criteriu care sa fie dependent numai de celelalte microoperatii din setul disponibil.

Aceasta alegere ar trebui sa fie independenta de microoperatiile ce vor deveni disponibile dupa asignarea microoperatiei respective.

PROPOZITIE: Pentru a alege o microoperatie care apartine setului de microoperatii disponibile k, pentru a o introduce intr-o instructiune μI_C (completa, astfel incat numai o partitie, si anume μPI optima, sa fie generata), trebuie sa existe un criteriu de selectie dependent numai de microoperatiile care apartine setului disponibil dat, si independent de orice microoperatie din setul ce va deveni disponibil. Aceasta implica existenta unui criteriu de ordonare a microoperatiilor din setul disponibil.

JUSTIFICARE: Presupunem ca $\mu CR(\mu OD_k) = 1$, atunci avem conflict de resurse. Nu toate microoperatiile pot fi plasate in aceeasi microinstructiune.

Fie μIC generata de μOD_k.

Faptul ca generam aceasta microinstructiune inseamna ca toate operatiile dependente de aceasta devin disponibile. Vom genera un nou set de microoperatii disponibile:

- $\mu OD_{k+1} = \{\{\mu OD_k \backslash \mu IC\} \cup \{\mu O_d\}_k\}$
  - microoperatii care devin disponibile datorita asignarii acelei instructiuni complete
- $\{\mu O_d\}_k = \{\mu O | \forall \mu O_i \in \mu IC \Rightarrow \mu O_i < \mu O\}$

Fara examinarea efectului unor microoinstructiuni suplimentare asupra dinamicii lui μOD_{k+1} , alegerea unei microoperatii trebuie sa fie independenta de setul $\{\mu O_d\}_k$ rezultat. Trebuie sa existe acest criteriu de ordonare (care nici nu exista).

Orice metoda de ordonare a microoperatiilor din setul μOD_k , bazata pe resursele folosite de fiecare microoperatie si independenta de resursele folosite de celelalte microoperatiil produce o ordonare arbitrara.

Masurarea efectului se poate face pe baza urmatoarelor informatii:

- resursele solicitate:
  - de microoperatia respectiva
  - de microoperatiile dependente de microoperatia analizata
- gradul de dependenta a celorlalte microoperatii fata de cea analizata

Microoperatiile din setul disponibil sunt independente, deci nu avem acces la cele 3 aspecte (influenta uneia asupra celeilalte).

Orice metoda de ordonare va conduce la o alegere arbitrara.

Fiecare microoinstructiune generata produce un singur set de microoperatii disponibile. **Daca doua microinstructiuni complete genereaza acelasi set de microoperatii disponibile, atunci cele doua microinstruinctiuni complete sunt identice.**

- Fie microinstructiunile μIC_i, μIC_j generate din setul disponibil la nivelul k (μOD_k). Presupunem ca $\mu IC_i \neq \mu IC_j$. Fie μOD^i_{k+1} , μOD^j_{k+1} seturile de microoperatii disponibile rezultate la momentul urmator.

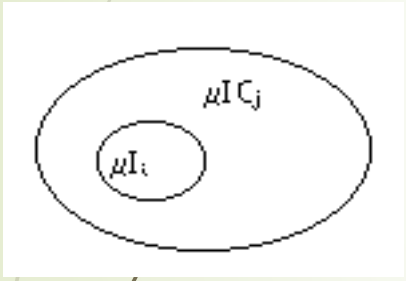
$$\mu OD^i_{k+1} = (\mu OD_k \backslash \mu IC_i) \cup \{\mu O_d\}_{ki}$$
$$\mu OD^j_{k+1} = (\mu OD_k \backslash \mu IC_j) \cup \{\mu O_d\}_{kj}$$
$$\mu IC_i \backslash (\mu IC_i \cap \mu IC_j) \subset \mu OD^i_{k+1}$$
$$\mu IC_j \backslash (\mu IC_i \cap \mu IC_j) \subset \mu OD^j_{k+1}$$

- Daca $\mu OD^i_{k+1} \equiv \mu OD^j_{k+1}$ ar trebui ca:
- $\mu IC_j \backslash (\mu IC_i \cap \mu IC_j) \equiv \mu IC_i \backslash (\mu IC_i \cap \mu IC_j) \Rightarrow \mu IC_i \equiv \mu IC_j$

PROPOZITIE: Partitia unui microsubloc se obtine numai prin generarea de μIC (nu pot lua combinatii mai reduse de microoperatii).

Presupunem ca din setul μOD_k putem genera μI_i si μIC_j .

Alegerea μI_i va produce un set de operatii disponibile μOD_{k+1}^{i}



$\mu IC_j \setminus \mu I_i \neq \emptyset$

$$\mu OD_{k+1}^{i} = \mu OD_{k+1}^{j} \cup (\mu IC_j \setminus \mu I_i)$$

Deoarece $\mu I_i \subset \mu IC_j$ atunci $\mu IC_j \setminus \mu I_i \neq \emptyset$.

$$\mu OD_{k+1}^{i} \subset \mu OD_{k+1}^{j} ,$$

- avem eventual un numar mai mare de subniveluri (nu conduce la solutia optima).

Pe baza observatiei rezulta 2 categorii de algoritmi.

Partitie minima: - prin generarea din setul de microoperatii μOD_k a tuturor μIC posibile si pentru fiecare dintre ele sa se analizeze influenta ce o are asupra generarii setului de microoperatii disponibile urmator, μOD_{k+1}. Acest algoritm are complexitate MP completa.

Partitie euristica: - bazat pe ordonare a microoperatiilor din setul disponibil μOD_k in functie de succesorii in graful de dependente de date. Aceasta metoda nu va genera o partitie optima, insa va fi mult mai rapida in situatii acceptabile.

Calculul limitei inferioare pentru nivelurile de alocare

Presupunem că avem μSB =(μB(μo) , ∠) care are graful asociat G

μB(μo)={ μo_1, $\mu o_{2,...,}$ $\mu o_{|\mu B|}$ } microoperatiile din subblock

P= {P_1, P_2, …..$P_{|P|}$} resursa, elemente de executie

În procesul de calcul al limitei inferioare apar 4 situaţii care trebuie analizate.

CAZUL 1 . Prima valoare a limitei inferioare a înălţimea h a grafului G, unde h = calea cea mai lungă de la un nod rădăcină la un nod terminal.

Valoarea h =valoarea minimă a limitei inferioare (nu s-a ţinut seama de numarul de resurse si de conflictul de resurse).

CAZUL al 2-lea – estimare mai corectă – se analizează şi distribuţia microoperatiilor pe setul de resurse pe care le controleaza.

Fie ρ – această valoare limită.

Pentru calculul lui ρ, graful G va fi împărţit într-un subsistem de grafuri $G_i, 1 \leq i \leq |P|$

Resursele pot fi omogene sau nu.

Împărţirea se face în stabilirea – unor subseturi de sarcini alocate pe resurse, definite astfel:

Gi : $\mu B(\mu o)_{|Pi|}$ = {μo | μo controleaza resursa i}

Subgraf maximal
Microoperatie terminală

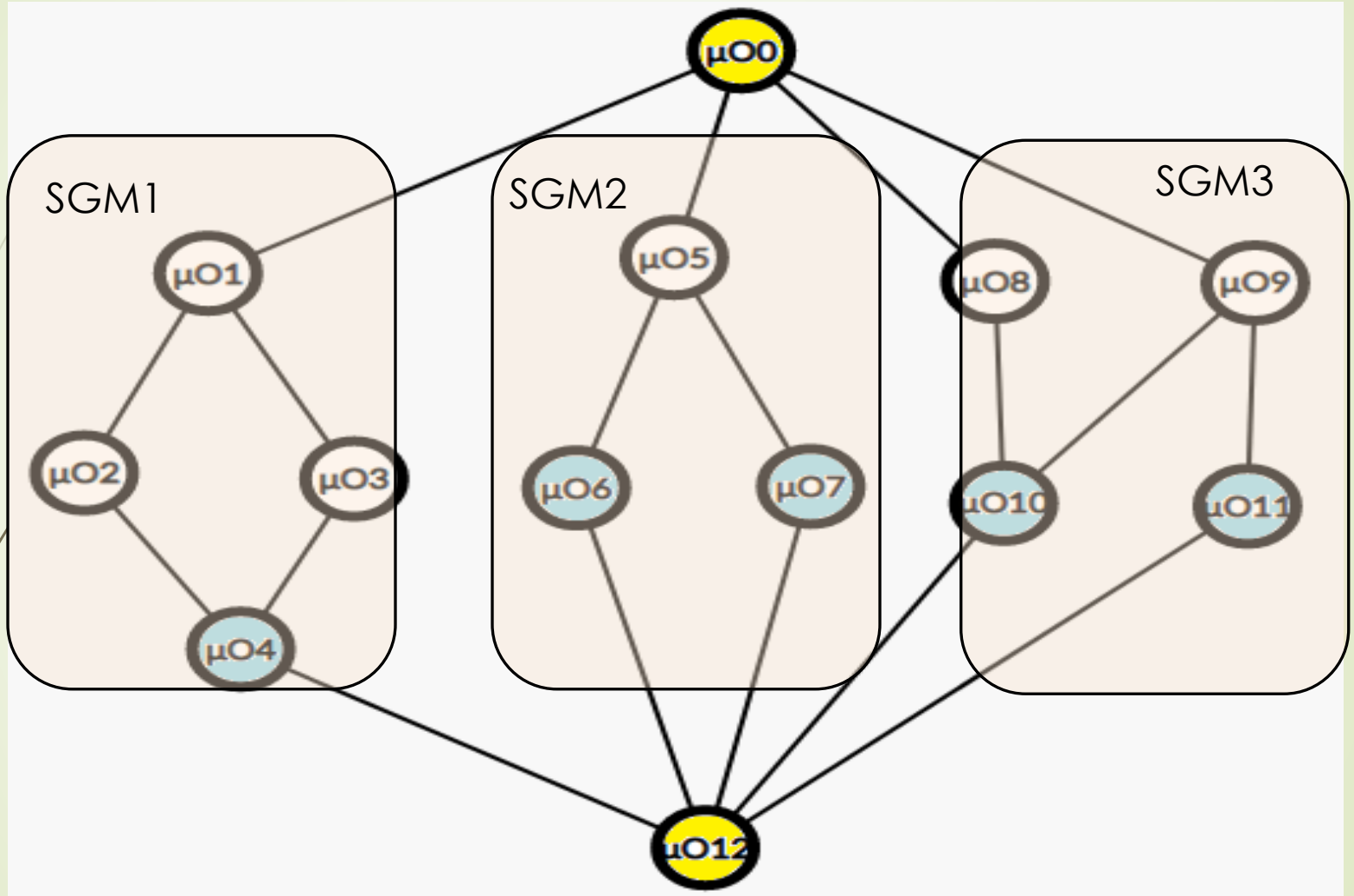
Def. O componentă a unui graf orientat aciclic G_i este un subgraf maximal $SGM_{ij} \subset G_i$ aî pentru orice microperatie μo_k care aparţine lui SGM_{ij}
($\mu o_k \in SGM_{ij}$) există $\mu o_m \in SGM_{ij}$ aî
$\mu o_k \angle \mu o_m$ sau $\mu o_m \angle \mu o_k$
(deci, trebuie să existe o legătură între cele două noduri)

Obs. G_i e considerat ca o combinare paralelă a subgrafurilor maximale.,

$$G_i = || SGM_{ij}$$

Def. Microoperatie terminală. O microoperatie μo a unui graf orientat aciclic e o microoperatie terminală μo_T dacă nu mai există nici o sarcină μo_j aî $\mu o_T < \mu o_j$
Nivelul unei microoperatii μo_i într-un graf G_i orientat e dat de nr. de arce obţinute într-o parcurgere pe drumul maxim între μo_i şi μo_T

Exemplu



Cosiderand ca avem trei resurse P= {P1, P2, P3 }
Sarcini terminale: µo 4, µo 6 , µo 7 , µo 10, µo 11
Nivelul µo1= 2 (sarcinile terminale au nivelul 0)

Calcul ρ

Pentru fiecare microoperatie terminală din SGM_{ij} obţinută prin împărţirea pe resurse a microoperatiilor, vom calcula nivelul din cadrul lui G şi alegem microoperatia terminală cu nivel maxim.

Fie μo_{Tj} aceste microoperatii, iar cu $||\mu o_{Tj}||$ notăm nivelul unei astfel de sarcini în , graful G.

Pentru fiecare subgraf G_i , i=1, ..., $|P|$ calculăm limita inferioară.

ρi= max (ρij} unde 1<= i<=|P|, 1<= j <= |Gi|

ρij= |SGMij| + $||\mu o_{Tj}||$

$\|\mu o_{Tj}\| = \min_k \{\|\mu o_{Tjk}\|\}$ cu 1<=k<=Nμo_T

$|SMG_{ij}|$= nr. de microoperatii din cadrul componentei j a subgrafului G_i

$\|S_{Tj}\|$- nivelul minim al sarcinilor terminale din cadrul componentei j a subgrafului G_i calculat faţă de poziţia în G asociat sistemului de microoperatii asociat µSB.

$|G_i|$=nr. de componente ale subgrafului G_i

$$\rho = \max_i \left\{ \max_j \left\{ |SGM_{ij}| + \min_k \{ \|\mu o_{Tjk}\| \} \right\} \right\} unde, \quad \begin{matrix} 1 \le k \le N_{\mu o_T} \\ 1 \le i \le |P| \\ 1 \le j \le |G_i| \end{matrix}$$

$\mu B(\mu o) = (\mu o_1, \mu o_2, \mu o_3, \mu o_4, \mu o_5, \mu o_6, \mu o_7, \mu o_8, \mu o_9, \mu o_{10}, \mu o_{11}, \mu o_{12})$

$P=\{P_1, P_2, P_3\}$

Presupunem că avem următoarea repartiţie pe resurse.

$\mu o_{P1}=\{ \mu o_1, \mu o_4, \mu o_5, \mu o_7, \mu o_9\}$ G1

$\mu o_{P2}= \{\mu o_2, \mu o_3, \mu o_6, \mu o_8, \mu o_{10}\}$ G2

$\mu o_{P3}=\{ \mu o_{11}, \mu o_{12}\}$ G3

G1:

$SGM_{11} = \{\mu o1, \mu o4, \mu o5, \mu o7, \mu o9\}$

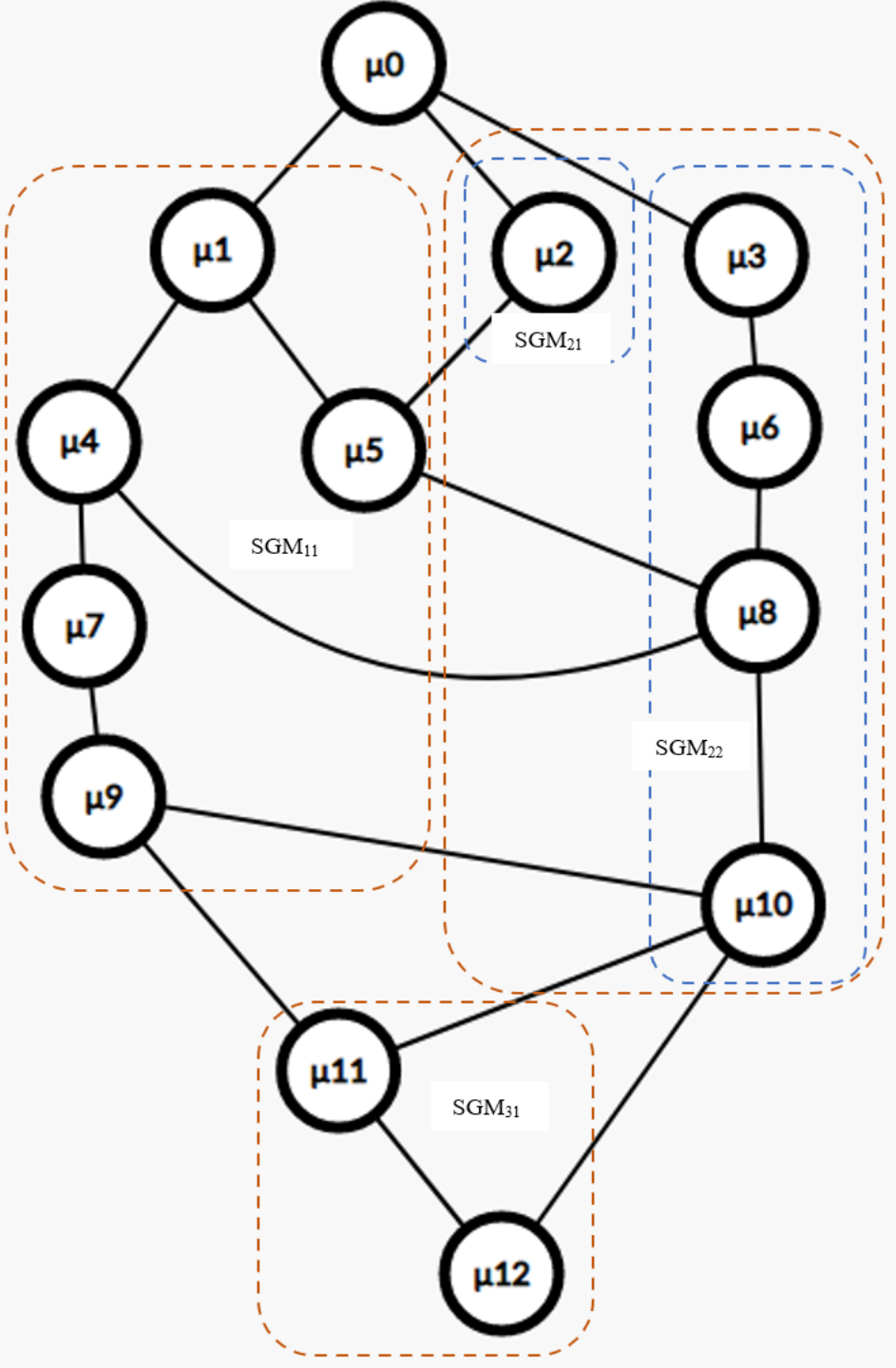
G2:

$SGM_{21} = \{ \mu o2\}$;

$SGM_{22} = \{ \mu o3, \mu o6, \mu o8, \mu o10\}$

G3:

$SGM_{31} = \{ \mu o11, \mu o12\}$



G_1: 2 sarcini terminale μo5, μo9, nivelul lui G_5, calculat în G, pe calea cea mai lungă

$\| \mu o5\|=4$

$\| \mu o9\|=3$

$|SGM_{11}|=5$

$\rho 1 = |SGM_{11}| + \min \{\|\mu o_5\|, \| \mu o_9\|\} = 5 +3=8$

G_2: pentru SGM_{21}, o singură sarcină care este şi terminală

$$\|\mu o_2\| = 5$$

$$\rho_{21} = \left\{\left|\underbrace{SGM_{21}}_{1}\right| + min\|\mu o_2\| = 1 + 5 = 6\right\}$$

pentru SGM_{22},

$$\rho_{22} = \left\{\left|\underbrace{SGM_{22}}_{4}\right| + min\|\mu o_{10}\| = 4 + 2 = 6\right\}$$

$$\Rightarrow \rho_2 = max(\rho_{21}, \rho_{22}) = max(6,6) = 6$$

G_3

SGM_{31} si μo_T=12

$|SGM_{31}|=2$

$|\mu o_{12}|=0$

$$\rho = \max_i \left\{ \max_j \left\{ |SGM_{ij}| + \min_k \{ \|\mu o_{Tjk}\| \} \right\} \right\} unde, \quad \begin{matrix} 1 \le k \le N_{\mu o_T} \\ 1 \le i \le |P| \\ 1 \le j \le |G_i| \end{matrix}$$

ρ3= 2+0=2

Am obţinut 2, 6, 8 $\Rightarrow \rho = 8$

Pe acelaşi exemplu, h=6 (înălţimea maximă a grafului).

Deci, le luăm pe cele cu 8 niveluri sau mai mult.

Justificarea alegerii, în cadrul componentei SGM_{ij}, a sarcinii terminale cu nivel minim în G e aceea că orice parcurgere a unei componente trebuie să se termine într-un astfel de nod.

Nu s-a ţinut cont de conflictul de resurse.

În general, resursele nu sunt distincte, ρ va fi mai mare decât în realitate.

Vom defini pentru fiecare resursa un ρi

$$\rho_i = \max_j \left\{ max \left[\left(\left\lceil |SGM_{ij}| / |P_i| \right\rceil \right), \left(h(SGM_{ij}) + \left\| \mu o_{T_j} \right\| \right) \right] \right\}$$

unde $|Pi=|$ număr de resurse de tip i

$|SGMij|$=nr. de microoeratii din cadrul componentei j,

nr. minim de momente de timp.

Trebuie respectată dependenţa de date.

$h(SGMij)+||\mu oTj||$ dat de dependenţa de date

$\left\lceil \frac{|SGM_{ij}|}{|P_i|} \right\rceil$ - e dat de nr. de resurse pe care le avem

$h(SGM_{ij})$ - nr. de momente de timp necesare pentru execuţia sarcinilor, având în vedere dependenţa de date

CAZUL 3

Limita inferioară $= \mu$, limita inferioară pentru situaţia în care resursele sunt distincte.

- $\mu = \max_i\{|\mu o P_i|\} 1 \leq i \leq |P|$

Fiind o singur resursa, se va fi controlata secvenţial, deci avem $|\mu o P_i|$, fiind $|\mu o P_i|$ microoperatii.

Când resursele nu ar fi fost diferite, am putea considera că

- $\mu = \max_i\{[|\mu o P_i|/|P_i|]\}$

În calculul acestei limite, s-a presupus că nu există dependenţă de date între microoperatiile care controleaza diferite resurse.

CAZUL 4

Se consideră că o microoperatie nu se desfăşoară numai într-o perioadă de timp ca în cazurile anterioare, ci pe durata a mai mulţi cicli.

Notăm cu μMc setul de microoperatii care se desfăşoară pe n_c cicli.

$\Rightarrow$ limita inferioară e influenţată de nr. de cicli şi o posibilitate de calcul ar fi

$z = |\mu M_c| \cdot n_c$ - e o variantă foarte simplistă, numai pentru punerea problemei.

În general, ajungem la $LI = max\{h, \rho, \mu, z\}$

$C^K_{|AD|} \Rightarrow Q$; în funcţie de LI, eliminăm f. mult.

Algoritmul de partitie va merge până la limita inferioară.

Nu se vor gasi soluţii cu valori mai mici decat LI.

Dacă am găsit o soluţie egală cu LI, rezultă că am găsit o soluţie.

ALGORITM CARE CONDUCE LA PARTITIE MINIMA:

- Algoritm de partiționare pe niveluri a setului de micro-operații dintr-un micro-subbloc

$\mu SB = (\mu B(\mu O), <)$ - sistemul de microoperatii pe care trebuie sa-l partitionam

μPT – partitia ce se va genera

μPTA – partitia de microinstructiunea anterioara, cosiderata cea mai buna pana in momentul respectiv

μOD – setul de microoperatii disponibile

μIC_i – microinstructiunea completa generata din setul de microoperatii disponibil curent

nmμI – numarul minim de microinstructiuni ce se poate obtine prin codificarea microoperatiilor din subloc

$\{\mu O_d\}_i$ – multimea de microoperatii ce devin disponibile prin generarea microinstructiunii complete curente

Algoritmul genereaza o partitie optima de μIC, tinand cont de conflictul de resurse si dependenta de date impusa de relatia impusa de ordine partiala.

Algoritm

P1. Initializare

- Se initializeaza partitia $\mu PT = \Phi$, $\mu PTA = \mu B(\mu O)$ - tot microsublocul (deci, daca le luam si le facem sevcential)
- $\mu OD = \{\mu O_j | \forall \mu O_j \in \mu B(\mu O)\ nu\ exista\ \mu O_i < \mu O_j\}$ - deci pentru care nu exista predecesori
- $\mu OND = \{\mu O_i | \mu O_i \in \mu B(\mu O) \backslash \mu OD\}$ - operatii nedisponibile

P2. Daca $\mu OND = \Phi\ si\ |\mu OD| \leq 3$ atunci salt la pas 5

P3. Se genereaza {μIC} – multimea de microoinstructiuni complete

- $\{\mu IC\} = \{\mu IC_j, \ldots, \mu IC_k\}$ - din μOD curent

  daca j ≠ k (s-au generat mai multe μIC)

  atunci salvare $\mu PT_j = \mu PT$

  μOD_j = setul curent μOD

  $\{\mu IC\}_j = \{\mu IC\} \setminus \mu IC_j$

$P4.\ \mu PT = \mu PT \cup \mu IC_j$

$\mu OD = \mu OD \backslash \mu IC_j \cup \{\mu O_d\}_j$

$\mu OND = \mu OND \backslash \{\mu O_d\}_j$

- daca $|\mu OD| \neq 0\ si\ |\mu PT| \leq |\mu PT_j| - 1$ atunci salt pas 2

P5. daca $\mu OD = \Phi$ atunci salt pas 7

P6. Se genereaza μIC din μOD curent.

μPT = μPT U μIC;

μOD = μOD \ μIC;

daca μOD ≠ Φ si |μPT|<|μPTA|-1 atunci pas 4

altfel pas 2

P7. daca | μPT| < | μPTA| atunci μPTA = μPT

altfel daca |μPTA| ≤ nmμI atunci μPTA este optima

STOP.

P8. daca $\{\mu IC\}_j \neq 0$ (cel care a fost salvat la pasul 3)

atunci reface $\mu PT = \mu PT_j$;

$\mu OD = \mu OD_j$;

$j \leftarrow j + 1$; (trec la urmatorul, selecteaza urmatoarea μIC din setul generat)

$\mu IC = \mu IC_j$

$\mu OND = \{\mu O_j | \mu O_j \in \mu B(\mu O) \backslash (\mu PT \cup \mu OD)\}$

salt pas 4

altfel μPTA este optima.

GATA.

- Obs nmμI – Limita inferioara

Algoritm euristic pentru partitionarea unui microsubbloc

Pentru a evita generarea tuturor partitiilor de microinstructiuni complete, folosim un criteriu de ordonare al operatiilor din setul de microoperatii disponibile pa baza numarului de succesori.

Nu duce la solutia optima, dar duce la o solutie acceptabila.

$$psucc(\mu O_i) = \{\mu O_j | \forall \mu O_i, \mu O_j \in \mu B(\mu O),\ cu\ i \neq j,\ avem\ \mu O_i < \mu O_j\}$$

$$psucc(\mu I_i) = \sum_{j=1}^{n} psucc(\mu O_{ij})$$

n = numarul de microoperatii din μI_i

Etapele algoritmului

Pas 1

Se initializeaza partitia $\mu PT = \Phi$, $\mu PTA = \mu B(\mu O)$

$\mu OD = \{\mu O_j | \forall \mu O_j \in \mu B(\mu O)\ nu\ exista\ \mu O_i < \mu O_j\}$ – deci pentru care nu exista predecesori

$\mu OND = \{\mu O_i | \mu O_i \in \mu B(\mu O) \backslash \mu OD\}$ – operatii nedisponibile

pas 2

facem o noua partitie, numai in anumite conditii

daca μOND=Φ (am avansat cu acest test pana la sfarsit) atunci **pas 4**

pas 3.

genereaza $\mu IC = \{\mu IC_j , \ldots, \mu IC_k\}$ micoinstructiuni complete, ordonam acest set in functie de succesori; ordonam astfel incat

$psucc(\mu IC_j) \geq psucc(\mu IC_{j+1}) \geq \ldots \geq psucc(\mu IC_k)$

Dintre doua microinstructiuni , pastram pe nivel pe cea cu cei mai multi succesori, intrucat mutarea ei pe nivelul urmator ar implica mutarea pe nivelul urmator a tuturor succesorilor ei. Deci, ea ar apartine micropartitiei curente.

$\mu PT = \mu PT \cup \mu IC_j$

Reactualizam operatiile disponibile, nedisponibile la momentul respectiv.

$\mu OD = \mu OD \setminus \{\mu O \backslash \mu O \in \mu IC_j\} \cup \{\mu O_d\}_j$

$\mu OND = \mu OND \setminus \{\mu\ O_d\}$

transfer **pas 2**

Pas 4

spatiul disponibil nu exista; gasesc ultima structura de microinstructiuni complete.

Genereaza $\mu IC = \{\mu IC_h, \ldots, \mu IC_l\}$.

Ultima data: iau toate μIC care se genereaza aici (intrucat aici nu mai pot merge pe o cale sau alta).

$\mu PT = \mu PT \cup \mu IC_h$

$\mu OD = \mu OD \setminus \{\mu O \mid \mu O \in \mu IC_h\}$ - nu mai adaug nimic, nemaiavand succesorii (e ultima data).

$h = h+1$ – le iau pe toate, dar din cele generate din spatiul disponibil.

Continuam acest pas pana ce nu mai am nici o μIC.

daca $\mu OD \neq \Phi$ atunci **pas 4**

altfel μPT – partitia generata STOP

Specificarea paralelismului operatiilor elementare

Codificarea minimă a unui set de microinstrucțiuni

Punerea problemei

- În proiectarea unităților de comandă microprogramate, după ce s-a determinat setul de microinstrucțiuni complete ce realizează execuția unui microsubbloc în minimum de paşi prin controlul tuturor operațiilor paralele ce se pot executa în sistem, se pune **problema codificării minime a acestui set de microinstrucțiuni**.
- Problema optimizării numărului de biți **constă în a stabili lungimea cuvântului memoriei de control ce specifică microinstrucțiunile, astfel încât aceasta să fie minimă**, având în vedere asigurarea controlului tuturor microoperațiilor paralele specificate de secvența de microinstrucțiuni ce descrie unitatea de comandă.
- Problema generală a **optimizării numărului de biți este o problemă din clasa "NP complete".** Apartenență la clasa "NP complete" a fost demonstrată de Robertson

μo **compatibile**
μo **incompatibile**

- Fie
  - $\mu B(\mu o)=\{\mu o_1, \mu o_2, ..., \mu o_{[\mu B]}\}$ setul de microoperații distincte din cadrul microsubblocului.
  - $\mu PT=\{\mu IC_1, \mu IC_2, ..., \mu IC_{[\mu PT]}\}$ partiția unui microsubbloc în microinstrucțiuni complete şi
- **Def. 1**
- Două microoperații μo_i şi μo_j sunt **compatibile** dacă pentru orice k, $1 <= k <= [\mu PT]$, dacă $\mu o_i \in \mu IC_k$ atunci $\mu o_j \notin \mu IC_k$.
- **Def. 2**
- Două microoperații $\mu o_i \in \mu B(\mu o)$, $\mu o_j \in \mu B(\mu o)$ sunt **incompatibile** dacă există cel puțin o microinstrucțiune completă μIC astfel încât $\mu o_i \in \mu IC_k$ şi $\mu o_j \in \mu IC_k$.

clasă de compatibilitate

- **Def. 3**
- O **clasă de compatibilitate CC(μo)** este un set (subset) al mulțimii MB(μo) în care oricare două microoperații sunt compatibile între ele.
- $CC(\mu o)=\{\mu o \mid$ pt orice $\mu o_i, \mu o_j \in CC(\mu o)$ avem μo_i compatibilă cu $\mu o_j\}$

- **Def. 4**
- O **clasă de compatibilitate maximă MCC(μo)** este acea clasa de compatibilitate la care nu mai poate fi adăugată nici o microinstrucțiune fără a se pierde compatibilitatea.
- $MCC(\mu o)=\{\mu o_j \mid$ pt orice $\mu o_i \notin MCC(\mu o)$, există $\mu o_j \in MCC(\mu o)$ astfel încât $\mu o_i \in \mu IC_k$ şi $\mu o_j \in \mu IC_k\}$.

În mod analog se defineşte clasa de incompatibilitate maximă MIC(μo).

Costul de implementare a unei clase de compatibilitate

- **Def. 5**
- **Costul de implementare a unei clase de compatibilitate** (măsurat în numărul de biți necesari pentru codificare) este dat de implementarea codificării verticale a microoperațiilor ce compun clasa.
- Cost $CC_i=[\log_2(|CC_i|+1)]$

iar costul total de implementare al cuvântului de control

- $$\text{Cost CC} = \sum_{i=1}^{k} [\log_2(|CC_i|+1)]$$

unde k este numărul de clase de compatibilitate.

- **Obs**
- **O clasă de compatibilitate corespunde unui câmp din cadrul microinstrucțiunii.**

Estimarea costului minim

- Problema codificării minime presupune două faze distincte:
  - enumerarea tuturor claselor de compatibilitate maximă MCC;
  - determinarea unui subset de MCC care să implementeze costul minim pentru codificare.

Costul implementării depinde de

- numărul de MCC necesar pentru acoperirea întregului set de microoperații şi
- numărul de microoperații din fiecare clasă de compatibilitate maximă.

- De notat faptul că un cuvânt din memoria de control, adică o microinstrucțiune completă, este un set de clase incompatibile între microoperațiile componente.
- O clasă de compatibilitate specifică cel mult o microoperație din cadrul unui cuvânt al memoriei de control.

Clase de compatibilitate maximă asociate (AMCC)

- **Def. 6**
- Pentru orice clasă de incompatibilitate IC, clasele de compatibilitate maximă ce conțin un element din IC se numesc **clase de compatibilitate maximă asociate (AMCC) asociate** clasei de incompatibilitate.
- **Prop. 1**
- ***Pentru orice clasă de incompatibilitate maximă MIC reuniunea claselor de compatibilitate maximă asociate acoperă întreg setul de microoperații.***
- **Justificare**
- Fie $MIC(\mu o)=\{\mu o_1, ..., \mu o_{|MIC|}\}$ formată din $|MIC|$ microoperații, unde $|MIC| <= |\mu B|$. Presupunem contrariul şi anume că reuniunea claselor de compatibilitate maximă asociate nu acoperă întreg setul de microoperații. Fie μo_j una din aceste microoperații presupuse neacoperite cu proprietatea că:

$$\mu o_j \in \mu B(\mu o) \text{ dar } \mu o_j \notin MIC(\mu o) \text{ şi } \mu o_j \notin \bigcup_{MIC} AMCC.$$

O clasa de incompatibilitate maximă MIC nu poate acoperi MB(μo).

- **Prop. 2**
- Pentru o clasă de incompatibilitate maximă MIC, **reuniunea oricăror k clase de compatibilitate, k < |MIC|**, nu poate acoperi setul de microoperații μB(μo).

- **Justificare**
- Considerăm MIC(μo)={μo_1, μo_2, ..., $\mu o_{|MIC|}$}. |MIC| este limita inferioară pentru numărul de clase de compatibilitate ce satisface acoperirea, deoarece o clasă de compatibilitate poate acoperi un singur element din MIC.
- Rezultă că orice reuniune de k clase de compatibilitate k < |MIC| nu poate acoperi setul de microoperații.

Clasele MCI si MCC

- **Clasele de incompatibilitate maximale MCI** sunt determinate pe baza grafului de dependență şi a conflictului de resurse şi reprezintă microoperațiile specificate de microinstrucțiunile complete ce descriu microblocul.
- **Clasele MCC sunt determinate din matricea de microoperații** ce indică incompatibilitatea. Aplicarea operatorului SAU între liniile matricii va specifica microoperațiile ce fac parte din clasa de compatibilitate maximă.
- **Calcularea MCC este o binecunoscută problemă în teoria automatelor finite**, existând numeroase metode pentru aceasta soluție. AHO arată că problema calculării MCC este o problema "NP complete".

Prop. 3 partiționare în q câmpuri

- ***Dacă un set de microoperații μB(μo) este partiționat în q câmpuri ale unei microinstrucțiuni, costul minim se va realiza atunci când (q-1) câmpuri specifică câte o microoperație (au un singur bit), iar cel de al q-lea câmp codifică restul de |μB(μo)|-q+1 microoperații.***

Justificare

- Considerăm o partiție arbitrară a LMI biți în q câmpuri. Fie câmpul cu b_{max} biți, câmpul cu lungimea cea mai mare şi fie un oricare alt câmp care are b_i biți.
- Numărul de microoperații ce poate fi codificat de cele două câmpuri este:
- **$NMO = (2^{bmax} - 1) + (2^{bi} - 1)$**
- Facem o modificare în organizarea logică a microinstrucțiunii şi mutăm un bit din câmpul cu b_i biți în câmpul cu b_{max} biți. în acest caz numărul de microoperații ce se pot codifica este:
- **$NMO' = (2^{(bmax+1)} - 1) + (2^{(bi-1)} - 1)$**
- **$NMO' - NMO = 2^{bmax} - 2^{bi} >= 0$ deoarece $b_{max} > b_i$.**
- Deci se pot codifica mai multe operații (microoperații) după modificarea structurii microinstrucțiunii.
- Repetând procesul de mutare a unui bit dintr-un câmp oarecare în câmpul de lungime maximă, numărul de microoperații ce poate fi codificat creşte. Numărul maxim de microoperații ce poate fi codificat rezultă a fi egal cu $(q+2^{(LMI-q+1)}-2)$. Transformând totul în cost se obtine costul minim:
- ***Cost = $q-1+\lfloor \log_2(|\mu B(\mu o)|-q+2) \rfloor$ când (q-1) câmpuri specifică fiecare câte o microoperație, iar ultimul câmp (|μB(μo)|-q+1) microoperații.***

Partiționare a setului MB(μo) microoperații în (q+h) câmpuri

Prop. 4

- ***Nu este posibilă o soluție de partiționare a setului MB(μo) microoperații în (q+h) câmpuri astfel încât costul să fie mai mic decât partiția în q câmpuri.***

Justificare

Fie

- $C = q-1+[\log_2(|\mu B(\mu o)|-q+2)]$ costul minim obținut prin partiția în q câmpuri şi
- $C = q+h-1+[\log_2(|\mu B(\mu o)|-q-h+2)]$ costul minim obținut prin partiția în q+h câmpuri.
- Deosebim 2 cazuri şi anume:
  - 1) $h=1$ şi $|\mu B(\mu o)|-q-1=2^k$
  - 2) $h \neq 1$ sau $|\mu B(\mu o)|-q+1 \neq 2^k$
- în primul caz:
- $C_q=q-1+k+1=q+k$
- $C_{(q+h)}=q+k$
- deci $C_q=C_{(q+h)}$
- în al doilea caz, deoarece :
- $[\log_2(|\mu B(\mu o)|-q+2)]-[\log_2(|\mu B(\mu o)|-q-h+2)] < h$
- rezultă $C_q < C_{(q+h)}$

Obsservatie

Costul minim pentru codificarea setului de microoperații MB(mO) poate să fie mai mare decât cel dat de propoziția 3 adică:

$C \geq q\text{-}1+[\log_2(|\mu B(mO)|\text{-}q+2)]$ când $|MCC|_{max} \leq |\mu B(mO)|\text{-}q$.

într-adevăr costul minim corespunde când:

(q-1) câmpuri specifică fiecare câte o microoperație;

cel de-al q-lea câmp codifică $|\mu B(mO)|\text{-}q+1$ microoperații.

Dacă $|MCC|_{max} \leq |\mu B(mO)|\text{-}q$, rezultă că $|MCC|_{max} < |\mu B(mO)|\text{-}q+1$,

ceea ce ar face ca în ultimul câmp să nu fie toate microoperațiile compatibile.

În acest caz trebuie renunțat la organizarea microinstrucțiunii în (q-1) câmpuri de 1 bit, dar prin aceasta se măreşte şi costul de implementare.

Metodă de minimizare a numărului de biți necesari pentru codificarea microinstrucțiunilor complete

- **1. Se alege clasa de incompatibilitate maximă care are cardinalitatea maximă MIC.**

Fie **$MIC_m \in MIC$** astfel încât $|MIC_m| >= |MIC_j| <$ pentru $j \# m$, $1 <= j <= |MIC|$

Se generează clasele de compatibilitate maximă asociate AMCC clasei MIC.

$AMCC_m$={MCC>= pt orice $\mu o \in MIC_m \exists$ MCC astfel încât $\mu o \in$ MCC}

- **2. Se formează tabela de acoperire modificată prin considerarea numai a claselor de compatibilitate maximă ce aparțin AMCC.**

TAM : $AMCC_m$ x $\mu B(\mu o)$ --> B

Se caută multimea de clase de compatibilitate maximă esențiale $\{MCC_e\}$ inclus în $AMCC_m$ astfel încât există MCC_e unic pentru care : $\mu o_i \in MCC_e$ avem TAM $(MCC_e, \mu o_i)=1$.

Se elimină coloanele corespunzătoare microoperațiilor ce sunt acoperite de clasele esențiale şi cele corespunzătoare microoperațiilor componente ale clasei MIC_m, obținându-se o **tabelă de acoperire redusă** :

TAR : $AMCC_m$ x ($\mu B(\mu o) \setminus MIC_m$) \ $\{MCC_e\}$ --> B

Cont

- **3. Se generează setul soluțiilor de acoperire a microoperațiilor $\{MCC_{ap}\}$: $\{\mu B(\mu o) \setminus MIC_m) \setminus \{MCC_e\}$**

  Fie $SOL=\{SOL_1, SOL_2, ..SOL_p\}$ soluțiile de acoperire în care

  $SOL_j = \wedge / (\{MCC_e\} \cup \{MCC_{ap}\})$

  Se consideră soluția parțială cu cardinalitatea minimă SOL_j.

  Se generează soluția de acoperire a microoperațiilor ce aparțin clasei MIC_m, încă neacoperite: $\{MCC_{am}\}$.

- **4. Se generează setul soluțiilor de acoperire completă:**

  $SOLC_j=SOL_j \wedge \{MCC_{am}\}$.

  Se calculează costul soluției de acoperire completă cu cardinalitatea minimă.

Exemplu

- Considerăm setul de microinstrucțiuni din cadrul microsubblocului :

  $\mu B=\{\mu o_1, \mu o_2, \mu o_3, \mu o_4, \mu o_5, \mu o_6, \mu o_7, \mu o_8, \mu o_9, \mu o_{10}\}$

- Presupunem că pe baza dependenței de date şi a conflictului de resurse între microoperații a rezultat următoarea partiție a microsubblocului :
- $\mu IC_1=\{\mu o_1, \mu o_2, \mu o_4\}$
- $\mu IC_2=\{\mu o_1, \mu o_3, \mu o_5\}$
- $\mu IC_3=\{\mu o_2, \mu o_6, \mu o_8, \mu o_9\}$
- $\mu IC_4=\{\mu o_4, \mu o_5, \mu o_7, \mu o_8\}$
- $\mu IC_5=\{\mu o_3, \mu o_4, \mu o_5, \mu o_6, \mu o_7\}$
- $\mu IC_6=\{\mu o_6, \mu o_9, \mu o_{10}\}$
- $\mu IC_7=\{\mu o_7, \mu o_{10}\}$

$\mu IC_1 = \{\mu o_1, \mu o_2, \mu o_4\}$
$\mu IC_2 = \{\mu o_1, \mu o_3, \mu o_5\}$
$\mu IC_3 = \{\mu o_2, \mu o_6, \mu o_8, \mu o_9\}$
$\mu IC_4 = \{\mu o_4, \mu o_5, \mu o_7, \mu o_8\}$
$\mu IC_5 = \{\mu o_3, \mu o_4, \mu o_5, \mu o_6, \mu o_7\}$
$\mu IC_6 = \{\mu o_6, \mu o_9, \mu o_{10}\}$
$\mu IC_7 = \{\mu o_7, \mu o_{10}\}$

$\mu o \backslash \mu o$	μo_1	μo_2	μo_3	μo_4	μo_5	μo_6	μo_7	μo_8	μo_9	μo_{10}
μo_1										
μo_2										
μo_3										
μo_4										
μo_5										
μo_6										
μo_7										
μo_8										
μo_9										
μo_{10}										

Tabela de incompatibilitate

- $\mu IC_1=\{\mu o_1, \mu o_2, \mu o_4\}$
- $\mu IC_2=\{\mu o_1, \mu o_3, \mu o_5\}$
- $\mu IC_3=\{\mu o_2, \mu o_6, \mu o_8, \mu o_9\}$
- $\mu IC_4=\{\mu o_4, \mu o_5, \mu o_7, \mu o_8\}$
- $\mu IC_5=\{\mu o_3, \mu o_4, \mu o_5, \mu o_6, \mu o_7\}$
- $\mu IC_6=\{\mu o_6, \mu o_9, \mu o_{10}\}$
- $\mu IC_7=\{\mu o_7, \mu o_{10}\}$

$\mu o \backslash \mu o$	μo_1	μo_2	μo_3	μo_4	μo_5	μo_6	μo_7	μo_8	μo_9	μo_{10}
μo_1	x	x	x	x	x					
μo_2	x	x		x		x		x	x	
μo_3	x		x	x	x	x	x			
μo_4	x	x	x	x	x	x	x	x		
μo_5	x		x	x	x	x	x	x		
μo_6		x	x	x	x	x	x	x	x	x
μo_7			x	x	x	x	x	x		x
μo_8		x		x	x	x	x	x	x	
μo_9		x				x		x	x	x
μo_{10}						x	x		x	x

Clasele maximale de incompatibilitate

- $MIC_1=\{\mu o_1, \mu o_2, \mu o_4\}$
- $MIC_2=\{\mu o_1, \mu o_3, \mu o_4, \mu o_5\}$
- $MIC_3=\{\mu o_2, \mu o_4, \mu o_6, \mu o_8\}$
- $MIC_4=\{\mu o_2, \mu o_6, \mu o_8, \mu o_9\}$
- **$MIC_5=\{\mu o_3, \mu o_4, \mu o_5, \mu o_6, \mu o_7\}$**
- **$MIC_6=\{\mu o_4, \mu o_5, \mu o_6, \mu o_7, \mu o_8\}$**
- $MIC_7=\{\mu o_6, \mu o_7, \mu o_{10}\}$
- $MIC_8=\{\mu o_6, \mu o_9, \mu o_{10}\}$

- **MIC_{max} = MIC_5 sau MIC_6**

$\mu o \backslash \mu o$	μo_1	μo_2	μo_3	μo_4	μo_5	μo_6	μo_7	μo_8	μo_9	μo_{10}
μo_1	x	x	x	x	x					
μo_2	x	x		x		x		x	x	
μo_3	x		x	x	x	x	x			
μo_4	x	x	x	x	x	x	x	x		
μo_5	x		x	x	x	x	x	x		
μo_6		x	x	x	x	x	x	x	x	x
μo_7			x	x	x	x	x	x		x
μo_8		x		x	x	x	x	x	x	
μo_9		x				x		x	x	x
μo_{10}						x	x		x	x

Clasele maximale de compatibilitate

- $MCC_1=\{\mu o_1, \mu o_6\}$
- $MCC_2=\{\mu o_1, \mu o_7, \mu o_9\}$
- $MCC_3=\{\mu o_1, \mu o_8, \mu o_{10}\}$
- $MCC_4=\{\mu o_2, \mu o_3, \mu o_{10}\}$
- $MCC_5=\{\mu o_2, \mu o_5, \mu o_{10}\}$
- $MCC_6=\{\mu o_2, \mu o_7\}$
- $MCC_7=\{\mu o_3, \mu o_8, \mu o_{10}\}$
- $MCC_8=\{\mu o_3, \mu o_9\}$
- $MCC_9=\{\mu o_4, \mu o_9\}$
- $MCC_{10}=\{\mu o_4, \mu o_10\}$
- $MCC_{11}=\{\mu o_5, \mu o_9\}$

$\mu o \backslash \mu o$	μo_1	μo_2	μo_3	μo_4	μo_5	μo_6	μo_7	μo_8	μo_9	μo_{10}
μo_1	x	x	x	x	x					
μo_2	x	x		x		x		x	x	
μo_3	x		x	x	x	x	x			
μo_4	x	x	x	x	x	x	x	x		
μo_5	x		x	x	x	x	x	x		
μo_6		x	x	x	x	x	x	x	x	x
μo_7			x	x	x	x	x	x		x
μo_8		x		x	x	x	x	x	x	
μo_9		x				x		x	x	x
μo_{10}						x	x		x	x

Tabela de acoperire a microoperațiilor de către clasele maximale de compatibilitate

- $MCC_1=\{\mu o_1, \mu o_6\}$
- $MCC_2=\{\mu o_1, \mu o_7, \mu o_9\}$
- $MCC_3=\{\mu o_1, \mu o_8, \mu o_{10}\}$
- $MCC_4=\{\mu o_2, \mu o_3, \mu o_{10}\}$
- $MCC_5=\{\mu o_2, \mu o_5, \mu o_{10}\}$
- $MCC_6=\{\mu o_2, \mu o_7\}$
- $MCC_7=\{\mu o_3, \mu o_8, \mu o_{10}\}$
- $MCC_8=\{\mu o_3, \mu o_9\}$
- $MCC_9=\{\mu o_4, \mu o_9\}$
- $MCC_{10}=\{\mu o_4, \mu o_1 0\}$
- $MCC_{11}=\{\mu o_5, \mu o_9\}$

MCC\μo	μo_1	μo_2	μo_3	μo_4	μo_5	μo_6	μo_7	μo_8	μo_9	μo_{10}
MCC_1	x					x				
MCC_2	x						x		x	
MCC_3	x							x		x
MCC_4		x	x							x
MCC_5		x			x					x
MCC_6		x					x			
MCC_7			x					x		x
MCC_8			x						x	
MCC_9				x					x	
MCC_{10}				x						x
MCC_{11}					x				x	

Obs

- Se observă că există două clase maximale de incompatibilitate cu cardinalitate 5 (MIC_5 şi MIC_6).
- Costul minim absolut, conform proprietății 4 este **C = 7**.
- ***Să considerăm $AMCC_6$ - clasele maximale de compatibilitate asociate clasei maximale de incompatibilitate MIC_6 :***

  $AMCC_6$={MCC_1, MCC_2, MCC_3, MCC_5, MCC_6, MCC_7, MCC_9, MCC_{10}, MCC_{11})
- Conform propoziției 1 clasele maximale de compatibilitate din **$AMCC_6$** acoperă toate microoperațiile microsubblocului.

Tabela de acoperire modificată

	μo_1	μo_2	μo_3	μo_4	μo_5	μo_6	μo_7	μo_8	μo_9	μo_{10}
MCC_1	x					**x**				
MCC_2	x						x		x	
MCC_3	x							x		x
MCC_5		x			x					x
MCC_6		x					x			
MCC_7			**x**					x		x
MCC_9				x					x	
MCC_{10}				x						x
MCC_{11}					x				x	

Se observă că MCC_1 şi MCC_7 sunt esențiale şi vor face parte din soluția finală.

Tabela de acoperire redusă

- Considerăm, conform algoritmului, acoperite microoperațiile:
  - incluse în clasele de compatibilitate maximă **MCC_1 şi MCC_7** esențiale şi
  - cele componente ale clasei de incompatibilitate maximă **MIC_6**.
- Astfel, din tabela de acoperire se elimină microoperațiile
  - **μo_1, μo_6, μo_3, μo_8, μo_{10}, respectiv μo_4, μo_5, μo_7.**

MCC \µo	μo_2	μo_9
MCC_2		x
MCC_5	x	
MCC_6	x	
MCC_9		x
MCC_{11}		x

Acoperirea microoperațiilor μo_2 şi μo_9

- Se observă că tabela de acoperire s-a redus substanțial. Acoperirea microoperațiilor μo_2 şi μo_9 se poate face cu ajutorul următoarelor clase maximale de compatibilitate :
- MCC_2, MCC_5
- MCC_2, MCC_6
- MCC_5, MCC_9
- MCC_5, MCC_{11}
- MCC_6, MCC_9
- MCC_6, MCC_{11}

MCC \µo	μo_2	μo_9
MCC_2		x
MCC_5	x	
MCC_6	x	
MCC_9		x
MCC_{11}		x

Setul soluțiilor

- SOL = {SOL_1, SOL_2, SOL_3, SOL_4, SOL_5, SOL_6} astfel :
- SOL = {

  MCC_1 MCC_7 MCC_2 MCC_5 ;

  MCC_1 MCC_7 MCC_2 MCC_6 ;

  MCC_1 MCC_7 MCC_5 MCC_9 ;

  MCC_1 MCC_7 MCC_5 MCC_{11} ;

  MCC_1 MCC_7 MCC_6 MCC_9 ;

  MCC_1 MCC_7 MCC_6 MCC_{11}

  }
- Considerând soluția : **MCC_1 MCC_7 MCC_5 MCC_9 ∈ SOL**, se acoperă toate microoperațiile cu excepția $\mu o_7 \in MIC_6$.
- Pentru acoperirea lui μo_7 se poate lua una din clasele maximale de compatibilitate MCC_2 sau MCC_6.
- $SOLC_{31}$={MCC_1, MCC_7, MCC_5, MCC_9, MCC_2}
- $SOLC_{32}$={MCC_1, MCC_7, MCC_5, MCC_9, MCC_6}

Setul soluțiilor complete

- $SOLC_3 = \{SOLC_{31}, SOC_{32}\}$

unde

- **$SOLC_{31}=\{MCC_1, MCC_7, MCC_5, MCC_9, MCC_2\}$**
- $SOLC_{32}=\{MCC_1, MCC_7, MCC_5, MCC_9, MCC_6\}$

Ambele soluții complete au aceeaşi cardinalitate ce specifică numărul minim de câmpuri necesar pentru codificarea microinstrucțiunilor ce descriu microsubblocul.

Alegând $SOLC_{31}$, rezultă următoarea grupare a microoperațiilor:

MC2 MC7 MC5 MC9 MC1

$\{ \mu o_1 \mu o_7 \mu o_9 ; \mu o_3 \mu o_8 \mu o_{10} ; \mu o_2 \mu o_5 ; \mu o_4 ; \mu o_6 \}$

Microperatiile cuprinse intr-un camp nu mai apar in alte campuri

Sunt mai multe posibilittati

MC2 MC5 MC7 MC9 MC1

$\{ \mu o_1 \mu o_7 \mu o_9 ; \mu o_2 \mu o_5 \mu o_{10} ; \mu o_3 \mu o_8; \mu o_4 ; \mu o_6 \}$

- ce necesită 8 biți pentru codificare.

- **$MCC_1=\{\mu o_1, \mu o_6\}$**
- **$MCC_2=\{\mu o_1, \mu o_7, \mu o_9\}$**
- $MCC_3=\{\mu o_1, \mu o_8, \mu o_{10}\}$
- $MCC_4=\{\mu o_2, \mu o_3, \mu o_{10}\}$
- **$MCC_5=\{\mu o_2, \mu o_5, \mu o_{10}\}$**
- $MCC_6=\{\mu o_2, \mu o_7\}$
- **$MCC_7=\{\mu o_3, \mu o_8, \mu o_{10}\}$**
- $MCC_8=\{\mu o_3, \mu o_9\}$
- **$MCC_9=\{\mu o_4, \mu o_9\}$**
- $MCC_{10}=\{\mu o_4, \mu o_10\}$
- $MCC_{11}=\{\mu o_5, \mu o_9\}$